

Audit RepCore v2

Nutrition, coaching, morpho, physiologie, design et erreurs humaines
16 experts + contrôle du premier audit · 154 points avec leur prompt

6,2/10

6 octobre 2026 · build 1792

Synthèse

En une phrase. RepCore est déjà très riche et solide : 5 951 tests au vert, aucune erreur JS sur 92 écrans ouverts en élève et en coach. Mais pour être vendue, l'app doit encore passer en production ce qui attend dans la PR #75, rattraper les erreurs humaines (bilan sans photos, virgule, séance oubliée) et corriger quelques calculs faux pour certains profils.

Méthode. 16 experts ont testé l'application au banc, en local, avec des élèves et des coaches fictifs, sans toucher à la vraie base. Un 17e a contrôlé point par point ce qui a été appliqué depuis le premier audit. Les doublons entre experts sont signalés par un paragraphe « Recoupement » qui indique le point de référence.

Expert	Domaine	Note
1	Coach en ligne : temps gagné	6/10
2	Morpho et analyse anatomique	6/10
3	Physiologie de l'entraînement	6/10
4	Macronutrition	6,5/10
5	Micronutrition et compléments	6,5/10
6	Base d'aliments	6,5/10
7	Erreurs humaines de l'élève	6/10
8	Erreurs du coach et réparation	5,5/10
9	Bilans et suivi	7/10
10	Intuitivité côté élève	6/10
11	Design et cohérence	6/10
12	Tests exploratoires	7/10
13	Effet waouh	7/10
14	Paramétrage pour le coach	5/10
15	Prêt à vendre	5/10
16	Séance en salle	6,5/10
Moyenne		6,2/10

Ce qui a progressé depuis le premier audit

- **Sur main** : 41 points faits sur 129, 16 partiels. Le cœur du métier est presque entièrement traité (coach, séance, nutrition, synchronisation : 38 sur 40). On le voit au banc : e1RM juste, plancher calorique, Wake Lock, virgule acceptée en séance, ETag entre appareils, messagerie, réponse vocale.
- **Dans la PR #75** (build 1799, pas encore en production) : sauvegarde, veille du serveur, déploiement du Worker, tests bloquants, paywall fermé, annuel remisé, page coach. Une fois fusionnée, une bonne partie des points « non faits » passe en « faits ».
- **Nouveaux défauts nés des corrections** : les messages du coach tombent dans le plafond d'une notification par jour ; les nouvelles lignes « À traiter » sont repliées ; la presse est refusée au-delà de 500 kg ; le tempo est inversé dans la comparaison Motion Lab.
- **Polices** : il n'y a pas eu de gonflement général. L'effet « compressé » vient de quelques éléments trop grands et de 77 styles de titre différents. La réponse : 4 niveaux de titre, et une zone tactile

élargie sans grossir le dessin.

Les 10 chantiers qui comptent le plus

1. **Production** : fusionner la PR #75 avec un contrôle automatique (15.1), puis rendre le paiement Ultime clair (15.2) et sortir Fit Pulse de la base (15.3).
2. **Bilan sans photos, le cas réel** : question à la validation, « Garder quand même », ligne « Bilan corrigé » chez le coach, demande de compléter en un clic (7.1 → 9.2 → 8.1).
3. **Bornes et virgule partout** : « 62,0 » lu 620 kg dans le bilan, la pesée, le profil et les macros (7.4, 12.1, 12.3 à 12.5).
4. **Rien ne se perd** : séance oubliée (7.2), déconnexion (7.6), toast « Annuler » pour le coach (8.2), historique du programme (8.3), suppression et correction avec annulation (7.5, 8.8).
5. **Calculs d'entraînement** : périodisation et décharge dans la séance (3.1), charge suggérée qui suit reps et RIR (3.2), faux plateaux (3.3), fin de repos sonore (16.1).
6. **Calculs de nutrition** : objectif unique chez l'autonome (4.1), cycle ON/OFF (4.2), obésité (4.3), plancher qui compte le sport (4.4), garde-fou TCA sur les micros (5.1).
7. **Morpho** : repère « bras » faux qui classe presque tout le monde en bras courts (2.1), repères par sexe (2.2), relecture après une photo corrigée (2.3), une seule carte morpho (2.5).
8. **Aliments** : synonymes et fautes de frappe (6.1, 6.5), génériques v2 et chaînes (6.3), portions (6.4).
9. **Temps du coach** : noms lisibles et recherche toujours visible (1.1, 1.2), une seule source pour « À traiter » (1.3), réponse au bilan en 3 taps avec ce qui a changé (1.6, 9.1), une page de réglages (14.1).
10. **Waouh et vente** : avant/après dès « Bilan enregistré » (13.4), « Bravo » du coach (13.5), action du jour en haut de l'accueil (10.1), mode visite pour vendre en 2 minutes (15.4).

À savoir sur le banc. L'erreur « reading 'update' » vue par tous les experts vient de l'enregistrement du service worker sans garde (`app/index.html:413`). Elle peut aussi toucher de vrais utilisateurs : le correctif est le point 12.2. Le dossier fictif par défaut ne suivait pas tout le schéma de l'app. Les experts concernés ont rejoué leurs constats avec un dossier corrigé, et les faux résultats (« undefined×8 », « Jamais démarré ») ont été écartés.

Contrôle du premier audit : 41 points faits sur 129

Méthode : `git log 388488d..HEAD` (131 commits, builds 1694 à 1792), puis contrôle de chaque point dans le code actuel (`app/rc-core.1792.js`, `cloudflare/src`, `database.rules.json`, `workflows`). Les points Critiques et Haute ont été vérifiés un par un, au banc quand c'était possible (ports 8970-8975, scripts dans `audit2/17/t*.js`). « Fait » compte aussi les points « faits avec un défaut ».

Bilan en une phrase. Kevin a traité en profondeur ce qui touche le coach, la séance, la nutrition et la synchronisation : agents 1 à 5, 38 points sur 40. Presque rien n'a bougé côté sécurité, serveur, performance, design, gamification, marketing, business, onboarding, suivi des mesures et IA : agents 6 à 16, 3 points faits sur 84.

À lire avant ce tableau. Le contrôle porte sur `main` (build 1792). D'après l'expert 15, la PR #75 (build 1799, 55 lots pas encore en production) contient déjà une partie des points notés « non faits » ici : sauvegarde, veille du serveur, déploiement du Worker, tests bloquants et fermeture du paywall. Les fusionner fait remonter le bilan.

1. Synthèse par agent

Agent du premier audit	Points	Faits	Partiels	Non faits
Benchmark (écarts B1, B5 à B8)	5	1	3	1
1 · Coach en ligne	8	8 (1 avec défaut)	0	0
2 · Préparateur physique	8	7	1	0
3 · Diététicien	8	6	1	1
4 · Athlète en salle / séance	8	8 (2 avec défaut)	0	0
5 · Synchronisation	8	8	0	0
6 · Sécurité	7	0	1	6
7 · Backend / infra	8	0	1	7
8 · Performance front	8	1	2	5
9 · Design UI	8	0	1	7
10 · Gamification	8	0	0	8
11 · Marketing	7	0	1	6
12 · Business	7	0	0	7
13 · QA	7	1	3	3
14 · Onboarding	8	1	0	7
15 · Progrès / Motion Lab	8	0	1	7
16 · IA	8	0	1	7
Total	129	41	16	72

Par priorité : - **Critiques (23)** : 8 faits, 4 partiels, 11 non faits. - **Haute (47)** : 13 faits, 7 partiels, 27 non faits.

2. Points Critiques et Haute

Critiques

N°	Point	État	Preuve	Commentaire
B1	Paieement direct au coach	Partiel	cloudflare/src/paiements-coach.js:121, aa974db	Le code et les tests sont faits, mais tout reste fermé tant que PAIEMENTS_COACH#oui : jamais prouvé en sandbox.
1.1	« À traiter » coupe les bilans, x groupé	Fait	rc-core:27981 (TODO_TOUJOURS_VISIBLES), c863dc1	Testé au banc : avec 9 plateaux, « bilan » et « en retard » restent visibles, et le report s'annule. Les nouvelles lignes, elles, ne sont pas protégées (défaut 3).
2.1	e1RM inversée	Fait	rc-core:54592	Banc : 100x8 à RIR 2 donne 133,3 ; 0 répétition donne 0 ; un single vaut sa charge.
2.2	Charge suggérée qui ignore la prescription	Partiel	rc-core:70259 (chargeSuivante)	Seul un plafond de pas a été ajouté. Banc : 100 kg à RIR 3 donne 105 → 110 → 115 → 120. Le RIR prescrit, les répétitions faites et la décharge sont toujours ignorés.
3.1	Plancher calorique	Fait	rc-core:101820	Banc : 60 appuis sur « -20 » s'arrêtent à 1 461 kcal (plancher 1 397), avec le refus « plancher ».
3.2	Objectif de l'athlète autonome sans effet	Fait	167c030	Banc : maintien 2 023, sèche 1 720 (coefficient 0,85), masse 2 225 (1,10).
4.1	Fin de repos muette, écran verrouillé	Fait avec défaut	rc-core:41979 (Wake Lock), :42028	La notification est silencieuse et posée au début du repos : si l'athlète verrouille lui-même, la fin reste muette (défaut 6).
5.1	Quota plein : séance perdue	Fait	rc-core:3948 (_dossierCourantAPousser)	Le ✓ ne vient qu'après le PUT. Couvert par des tests.
5.2	Course entre appareils sans ETag	Fait	rc-core:5313, scripts/verif/banc-sync.mjs	Le banc à deux appareils tourne avant chaque déploiement.
6.1	Paywall contournable en une écriture	Partiel	rc-core:8445 (role==='coach' donne 'suivi'), database.rules.json:22	455ea98 n'ajoute que des compteurs d'écart. Aucune porte n'est fermée.
6.2	XSS stockée athlète → coach	Non fait	rc-core:110227 (_videoEmbed), :34231	Banc : a.mp4?"> ressort tel quel dans le HTML du coach.
7.1	Cron plafonné vers 180 athlètes	Non fait	cloudflare/src/planif.js:24 (BUDGET=38)	La tenue de livre et les envois perdus après 21 h sont inchangés.
7.2	Personne n'est prévenu si le cron tombe	Non fait	cloudflare/src/index.js (/sante ne lit que la configuration)	Pas de bloc [observability] ni de sonde.
8.1	Le service worker retélécharge tout	Non fait	app/sw.js:222	cache: 'reload' sur tous les ASSETS à chaque build.

N°	Point	État	Preuve	Commentaire
8.2	Pas de minification	Fait	530e1c6, scripts/minifier.mjs	rc-core servi passe de 7,5 Mo à 4,1 Mo. Garde de taille dans firebase.yml.
10.1	Notifications perdues au plafond	Non fait	cloudflare/src/metier.js:84-89	Toujours une par jour, la première arrivée. Aggravé par la messagerie (défaut 1).
11.1	Promesse invité : 2 mois ou 1 mois	Non fait	i/index.html:216, index.html:92 et :848	Le texte dit toujours « premier mois ».
12.1	L'athlète d'un coach tiers ne rapporte rien	Non fait	rc-core:1577	coachQuotaDepasse ne fait qu'alerter. Le paiement coach (B1) est fermé.
12.2	Double abonnement prélevé	Non fait	rc-core:128758 (loadSubscribePage)	Aucun test d'un abonnement actif. onApprove écrase toujours paypalSubscriptionId.
13.1	Tests en CI qui bloquent la livraison	Partiel	firebase.yml (job suite-navigateur), suite.mjs:94	La suite tourne mais ne retient pas le déploiement. cloudflare/test n'est toujours lancé nulle part.
14.1	Pas de première séance pour l'autonome	Non fait	rc-core:46941	PS_PARCOURS_ACTIF=false.
14.2	Code d'invitation perdu	Non fait	rc-core:6664, :12594	Le code reste en mémoire seulement. ouvrirCodeCoach ne pré-remplit pas le champ.
16.1	Proxy IA sur le Worker	Non fait	cloudflare/src/index.js (APPELS)	Aucun appel IA.

Haute

N°	Point	État	Preuve	Commentaire
B5	Recettes et plans de repas	Partiel	a82853c	Les recettes vont du coach au journal. Il n'y a pas de plan de repas réutilisable.
1.2	Messagerie coach ↔ athlète	Fait avec défaut	b11d9ad	Les notifications de message passent sous le plafond d'une par jour (défaut 1).
1.3	Cadence des bilans fixée par le coach	Fait	rc-core:30267 (echeanceBilan)	Même règle dans l'app et dans le Worker. Le coach peut poser 3 questions.
1.4	Réponse vocale au bilan	Fait	rc-core:28406	L'envoi passe par le préreglage Cloudinary non signé (6.5 toujours ouvert).
2.3	Poids du corps, lesté, assisté	Fait	rc-core:69976 (typeCharge)	Banc : TRACTIONS MACHINE ASSISTE et DIPS MACHINE GUIDEE donnent « assiste ».
2.4	Modèle unique, records par répétitions	Fait	rc-core:64980, :70027	Records 1/3/5/8/10/12 répétitions, charge « par main ».

N°	Point	État	Preuve	Commentaire
2.5	Cartographie musculaire	Fait	339fa1c	Option « Compter le travail indirect ».
3.3	Métabolisme Harris par défaut	Fait	rc-core:101948 (mbFormuleDe)	Mifflin dès un IMC de 30. Le libellé dit la formule réellement calculée.
3.4	Macros sur le poids total en obésité	Partiel (annulé)	c5e3456 (build 1778)	Le poids de référence a été retiré volontairement le 05/10. Les alertes « glucides bas » et « total dépassé » restent.
4.2	Minuteur et « Suivant » hors écran	Fait	78de7b2	
4.3	3 à 8 interactions par série	Fait	13077cb	
4.4	« Terminer » sans confirmation	Fait	rc-core:53694	Séries non validées nommées, et le double toucher est ignoré.
4.5	« 82,5 » lu 825	Fait avec défaut	rc-core:52176 (lireCharge)	Banc : « 82,5 » donne 82,5 et « 825 » est refusé. Le plafond de 500 kg refuse une presse lourde (défaut 4).
5.3	Envoi sans base à l'aveugle	Fait	rc-core:3650 (syncUnionSansBase)	
5.4	Envois du coach non journalisés	Fait	rc-core:3903	_enfiler avant tout réseau, plus Background Sync.
6.3	Contenus payants lisibles	Non fait	database.rules.json:435	La boutique est toujours en auth != null.
6.4	Parrainage : fabrique de mois	Non fait	cloudflare/src/metier.js	Ni plafond ni e-mail vérifié. 6b5e34a ne traite que le double comptage.
7.3	Pas de sauvegarde de la RTDB	Non fait	.github/workflows/	Aucun export.
7.4	Worker déployé à la main	Partiel	pages.yml:47 (_pages)	Pages ne publie plus tout le dépôt. Ni tests ni wrangler deploy en CI, et firebase.yml:200 tente encore les fonctions.
7.5	Webhook PayPal sans budget	Non fait	cloudflare/src/metier.js:177	_reste={()=>Infinity}. Le paiement coach ajoute des sous-requêtes au même webhook.
8.3	Écran noir au démarrage	Non fait	app/index.html:638 (opacity:0), :423	qr.js reste synchrone.
8.4	[style*=] du thème clair	Non fait	rc-style.1792.css	10 450 sélecteurs, 1,68 Mo.
8.5	Analyse statique et CI	Partiel	2188680, firebase.yml	types.yml est vert. La suite tourne en CI sans bloquer. Pas d'ESLint.
9.1	Thème clair : contrastes	Non fait		
9.2	Pages publiques i/p/c hors charte	Non fait	i/index.html:45	--vif:#c9f24d toujours là.
9.3	Couleurs non gouvernées	Non fait		

N°	Point	État	Preuve	Commentaire
10.2	Scores de compétition déclarés par le client	Non fait		
10.3	Semaine validée, séances vides	Non fait	rc-core (finishWorkout)	Aucune garde « 0 série ».
10.4	Boucle quotidienne	Non fait		
10.5	Ligues hebdomadaires	Non fait		
11.2	Porte d'entrée B2B	Non fait		
11.3	Preuve sociale	Non fait		
11.4	SEO	Partiel	c10e4d4, index.html:43	Canonical et sitemap sont passés à web.app. Le prix affiche toujours "9.95" et le blog reste orphelin.
12.3	Résiliation et engagement	Non fait	cloudflare/src	Pas d'appel /cancel .
12.4	Renouvellement remboursé, accès ouvert	Non fait	cloudflare/src/paypal.js:464	L'accès n'est fermé que if (rec.premier) .
12.5	Annuel sans remise	Non fait	tarifs.json	an = 12 × mois.
13.2	La suite s'arrête à la première exception	Partiel	93f7936	La fenêtre est fixée (VW/VH), mais les 3 226 ok('...' , (()=>{...})) restent hors d'un try.
13.3	Dates affichées la veille hors d'Europe	Non fait	rc-core:76598	Banc en America/Martinique : _fmtDateCourte('2026-09-29') donne « 28 sept. ».
14.3	Le lien du coach contourne /i/	Non fait	rc-core:127704	Le lien va toujours vers APP_BASE_URL .
14.4	Embarquer 20 athlètes	Non fait	rc-core:33354	Pas d'import ni de programme préparé avant l'inscription.
15.1	Masse grasse synchronisée jetée	Non fait	rc-core (sanFusionSync)	Le plan n'a pas de champ masse grasse.
15.2	Pesée retenue : la dernière du jour	Non fait	android/.../sante/Jours.kt:164 , cloudflare/src/lignes.js:85	
15.3	Tempo Motion Lab inversé	Non fait	app/motion-lab.js:1558	Repris tel quel par la comparaison T5 (défaut 5).
15.4	Écart de poids brut, pas d'objectif	Non fait	rc-core:118956	
16.2	Le brouillon de bilan lit l'athlète	Non fait	rc-core:78306	Aucun champ libre n'est lu.
16.3	Résumé du lundi	Partiel	30fef7e	La vue « Le point du lundi » existe, sans synthèse rédigée.
16.4	Import des programmes du coach	Non fait	rc-core:39343	LEGACY_PDF_IMPORT=false .

3. Régressions et défauts introduits par les corrections

1. Les messages du coach et la réponse vocale tombent dans le plafond d'une notification par jour — Haute · S

Analyse et idée. La messagerie (b11d9ad), la réponse vocale (9b3526f) et l'accueil coaché (71d1c69) passent toutes par `envoyerPush`. Celle-ci n'autorise qu'un push par jour, quel qu'en soit le type (`cloudflare/src/metier.js:84-89`), et la nuit ne garde qu'une place (`push_attente/<uid>.set, :275`). Je l'ai rejoué sous Node : `pushAutorise('message', null, {jour: aujourd'hui}, 14 h)` rend `plafond`, et à 22 h il rend `calme`, puis le message suivant écrase l'attente. Idée : les messages humains (message, coach, réponse de bilan) sortent du plafond, avec un plafond propre et anti-rafale (une notification par fil toutes les 10 min, regroupée). L'attente de nuit devient une liste.

Prompt de mise en place.

Contexte : `cloudflare/src/metier.js`. `pushAutorise(type, prefs, log, t)` refuse tout second push du jour (`push_log/<uid>.jour`) et, en heures calmes, `envoyerPush` écrit `push_attente/<uid>` avec `.set` (une seule place). La messagerie (type 'message' vers le coach, 'coach' vers l'athlète), la réponse au bilan (texte ou vocale) et les relances d'accueil passent par là. Un deuxième message du coach dans la journée, ou une réponse à 22 h suivie d'un défi, n'arrive jamais.

À faire :

1. Introduire `PUSH_HUMAINS = ['message','coach','bilan']` (vérifier les types réellement déposés pour la réponse de bilan et le message). Pour eux, ignorer le plafond quotidien commun et appliquer un plafond propre : au plus 1 push par fil (tag) toutes les 10 minutes (`push_log_humain/<uid>/<tag> = at`, en transaction). Le tag existant ('message-<clé>') sert de regroupement côté appareil.

2. Heures calmes : remplacer `push_attente/<uid>.set` par une file `push_attente/<uid>/<id>` (au plus 5 entrées, les plus récentes). Au réveil de 8 h, envoyer un seul push qui résume (« 2 messages de ton coach, 1 défi ») et vider la file dans la même écriture multi-chemins.

3. Ne rien changer pour les types de jeu (serie, defi, badge...) : ils gardent le plafond d'un par jour.

4. Budget : chaque envoi humain compte dans `fixerBudget` comme aujourd'hui.

Tests (`cloudflare/test/metier.test.mjs` et `messages.test.mjs`) : un défi à 9 h puis un message du coach à 14 h → les deux partent ; deux messages du même fil en 2 min → un seul push ; réponse vocale à 22 h et défi à 23 h → les deux sont résumés à 8 h ; un message à un athlète qui a coupé 'coach' dans ses préférences ne part pas.

Critère de fin : npm test dans `cloudflare/` vert, et le scénario « défi le matin puis message du coach l'après-midi » délivre les deux. Pas de changement d'interface dans l'app ; si un texte change, ne pas agrandir les polices. Si l'app est modifiée : nouveau build (`window.RC_BUILD` dans `app/index.html` et `CACHE` de `app/sw.js`, puis `python3 scripts/versionner_actifs.py`).

2. Bilan validé sans aucune photo et sans question, et la correction ne prévient pas le coach — Haute · S

Analyse et idée. Le rejet des photos « en plongée » (d3fa2dd) jette la photo sans offrir « Garder quand même » (`rc-core:76116`). `bilNext` (`rc-core:75284`) demande confirmation pour un poids manquant, jamais pour des photos manquantes. La modification d'un bilan (039513a) pose `modifieLe`, mais `hasNewBilan` (:30413) ne lit que la réponse : une photo ajoutée après la réponse du coach ne remonte nulle part. Testé au banc : un bilan avec poids et taille seulement est enregistré sans aucune confirmation. Je lui ai ajouté une réponse du coach, puis une photo de face par modification : `hasNewBilan` reste `false`. C'est exactement le cas réel de Kevin (un élève qui valide sans ses photos). Idée : une confirmation « Valider sans tes photos ? » qui propose « Les ajouter plus tard ». Une ligne « Bilan complété » chez le coach quand un bilan déjà répondu reçoit des photos ou des mesures. « Garder quand même » sur une photo rouge.

Prompt de mise en place. Pas de prompt séparé, pour éviter un doublon. Appliquer l'expert 7 (point 1) côté élève, puis l'expert 9 (point 2) côté coach.

3. Les nouvelles lignes « À traiter » passent sous le plafond de huit — Moyenne · S

Analyse et idée. Le correctif 1.1 ne protège que `drapeau`, `bilan` et `overdue` (`TODO_TOUJOURS_VISIBLES`, `rc-core:27981`). Les lignes ajoutées depuis sont empilées APRÈS les signaux d'entraînement (`rc-core:28608-28620`) : « Message sans réponse », « Programme à écrire » (nouvel athlète payant) et « Premier point de l'accueil ». Testé au banc (`todoLignesVisibles`, 9 lignes d'entraînement) : `message` disparaît derrière « + N autres ». Idée : ces trois types rejoignent la liste toujours visible, juste sous les bilans.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`, `renderTodoBlock` (≈28560-28700) et `TODO_TOUJOURS_VISIBLES` (≈27981). Les lignes `'message'`, `'accueil_prog'` et `'accueil_retour'` sont poussées après `grouperLignesEntrainement(...)` et comptent dans `TODO_MAX_LIGNES=8` : avec 8 lignes de plateau, de régression ou de douleur, un message d'athlète sans réponse ou le programme à écrire pour un nouvel inscrit sont repliés.

À faire : ajouter `'message'`, `'accueil_prog'` et `'accueil_retour'` à `TODO_TOUJOURS_VISIBLES` ; les placer avant les signaux d'entraînement dans l'ordre de `rows` (ordre : `drapeau`, `bilan`, `overdue`, `message`, `accueil_prog`, `accueil_retour`, puis le reste) ; vérifier que `window._todoRows`, `dismissTodoRow` et les files gardent les index de l'écran.

Tests dans `app/tests.js` : 10 lignes d'entraînement plus un message et un programme à écrire → les deux sont visibles sans dépliage ; l'ordre des index de `window._todoRows` correspond au DOM.

Critère de fin : suite à 0 échec, nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`), aucune police agrandie.

4. La saisie de charge refuse tout au-dessus de 500 kg (presse à cuisses) — Moyenne · S

Analyse et idée. `lireCharge` borne la saisie à `CHARGE_SAISIE_MAX=500` (`rc-core:52175`). Une presse à 45° chargée à 520 kg est refusée, et la valeur précédente est remise avec un toast. Un athlète avancé ne peut pas noter sa vraie série. La confirmation « 1,5 × le record » suffit déjà comme garde-fou. Idée : un plafond selon le matériel (1 000 kg pour presse, hack et machines guidées, 500 kg ailleurs), et une confirmation plutôt qu'un refus entre 500 et 1 000.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`, `lireCharge(brut,max)` (≈52176), `CHARGE_SAISIE_MAX=500`, appel dans `_woChargeSaisie` (≈52239), `matérielExercice(ex)` existe (pas du matériel).

À faire : `chargeMaxSaisie(ex)` rend 1000 pour les exercices dont `matérielExercice` est `presse/hack/machine guidée` (ou dont le nom contient `PRESSE`, `HACK`, `LEG PRESS`), sinon 500 ; conversion en livres comme aujourd'hui. `_woChargeSaisie` passe ce max. Au-delà de 500 kg sur ces exercices, `rcConfirm` (« 620 kg à la presse ? ») au lieu d'un refus ; `chargeSuspecte` reste en place.

Tests : `lireCharge('520', 1000)` → 520 ; saisie 620 sur `PRESSE A CUISSSES` → confirmation puis enregistrement ; saisie 620 sur `DEVELOPPE COUCHE` → refus ; « 82,5 » toujours 82,5.

Critère de fin : suite à 0 échec, nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`), sans agrandir les polices.

5. La comparaison de deux analyses (T5) affiche un tempo inversé sur les tirages et les extensions à la poulie — Moyenne · S

Analyse et idée. Le point 15.3 n'est pas corrigé : `mlTempo` (`app/motion-lab.js:1558`) compte comme concentrique toute vitesse vers le haut. La nouvelle comparaison (6116478) le reprend tel quel sous le libellé « Tempo : descente, remontée » et « Part de la phase lente ». Sur un tirage vertical, elle compare donc deux phases fausses, avec une tolérance qui leur donne une fausse précision. Idée : un sens

concentrique par exercice (`sensConcentrique(ex)` : haut, bas, ou loin du départ), appliqué à `mlTempo` et aux libellés de la comparaison.

Prompt de mise en place.

```
Contexte : app/motion-lab.js. mlTempo(serie) (~1558) suppose concentrique = vitesse positive (verticale vers le haut, ou axe orienté vers le point le plus loin du départ). Appelants : mlMetriquesBarre (~642), ~2061, ~2239, et la comparaison de deux dates (lignes « Tempo : descente, remontée », « Part de la phase lente », « point dur 0 au bas, 100 au haut »).
```

À faire :

1. `sensConcentrique(nomExercice)` : 'bas' pour les tirages verticaux, les extensions triceps poulie haute, les pulldowns et les crunchs poulie ; 'haut' par défaut pour les poussées verticales, squats et soulevés ; 'inconnu' si le nom ne permet pas de décider. Lire le nom rattaché à la vidéo.
2. `mlTempo` reçoit le sens et inverse le signe de la vitesse pour 'bas'. Pour 'inconnu' en mode chemin, ne pas afficher de tempo (« sens non déterminé : choisis-le »), avec un sélecteur Haut/Bas mémorisé sur l'analyse.
3. Comparaison : libellés « excentrique, concentrique » (pas descente et remontée), point dur exprimé du début à la fin de la phase concentrique ; refus motivé si les deux analyses n'ont pas le même sens. Tests dans `app/tests.js` : trajectoire synthétique de tirage vertical (descente rapide de 1 s, remontée lente de 3 s) → concentrique 1 s, excentrique 3 s ; squat inchangé ; comparaison de deux tirages → ligne « Tempo » juste ; sens inconnu → pas de chiffre.

Critère de fin : `tsc (types.yml)` à 0 erreur, suite à 0 échec, nouveau build (`window.RC_BUILD`, `CACHE` de `app/sw.js`, `python3 scripts/versionner_actifs.py`), sans agrandir les polices.

6. Repos : la notification d'arrière-plan est muette et ne sonne jamais la fin — Moyenne · S

Analyse et idée. Le Wake Lock (c694dfd) règle le cas de l'écran qui s'éteint seul. Mais quand l'athlète verrouille lui-même son téléphone, `_reposNotifier` (rc-core:42028) pose une notification `silent:true` « Fin à 14:32 » au début du repos, et rien ne part à l'échéance. Le rattrapage n'a lieu qu'au retour dans l'app. Idée : au passage en arrière-plan, programmer en page un `setTimeout` jusqu'à `reposFin` qui remplace la notification par une version sonore et vibrante (même tag, `renotify:true`). Les minuteurs d'une page cachée tiennent à la seconde pendant les premières minutes, ce qui couvre un repos.

Prompt de mise en place. Pas de prompt séparé, pour éviter un doublon. Le prompt de l'expert 16 (point 1) couvre ce défaut.

4. Points non faits qui restent importants (déjà détaillés dans le premier audit)

- 6.2 : XSS stockée athlète → coach (`_videoEmbed`, photos de bilan). C'est toujours la faille la plus grave, vérifiée au banc.
- 6.1 : paywall ouvert par une écriture (`status`, `role`, `coachPlan`, `rc_codes`). Seule l'étape de mesure a été faite.
- 12.2 : double abonnement Essentielle + Ultime prélevé en silence.
- 12.4 et 12.3 : remboursement d'un renouvellement sans fermeture d'accès ; résiliation qui n'annule rien chez PayPal.
- 10.1 et 7.6 : plafond d'une notification par jour et attente de nuit à une place. Plus urgent depuis la messagerie (défaut 1).
- 7.1, 7.2, 7.3 : cron plafonné, aucune alerte si le Worker tombe, aucune sauvegarde de la base.
- 7.5 : webhook PayPal sans budget, maintenant chargé en plus du paiement coach.
- 2.2 : charge suggérée qui ignore le RIR et les répétitions prescrits et ne saute pas les décharges. Banc : 100 → 105 → 110 → 115 → 120 kg à RIR 3.
- 14.1 et 14.2 : pas de première séance pour l'autonome, et code d'invitation perdu à l'installation de la PWA.

- 14.3 : le lien d'invitation du coach contourne /i/ et son tuto.
- 11.1 : promesse invité « 2 mois » contredite par « premier mois » (landing, FAQ, /i/).
- 11.4 : prix faux ("9.95") dans les données structurées de la landing.
- 13.3 : dates « AAAA-MM-JJ » affichées la veille aux Antilles et au Québec (vérifié au banc).
- 13.1 : la suite navigateur ne bloque pas le déploiement, et les tests du Worker ne tournent nulle part.
- 8.1 et 8.3 : le service worker retélécharge tout à chaque build, et l'écran reste noir jusqu'à l'exécution de rc-core.
- 6.3 : programmes de la boutique lisibles par tout compte connecté.
- 15.2 et 15.1 : pesée du soir retenue au lieu du matin, et masse grasse des balances jetée.
- 3.6 : la journée en cours comptée comme un écart dans la bande d'observance du coach.
- 3.4 : poids de référence des macros retiré le 05/10 (c5e3456). C'est une décision à reconfirmer : le profil de 120 kg retombe à 2,2 g/kg du poids total, avec des glucides au plancher.
- 10.3 : une séance abandonnée à 0 série compte toujours pour la série et les badges.
- 16.1 : le socle IA (proxy sur le Worker), préalable à 16.2 et 16.4.

Rapports des experts

Chaque expert a testé l'application en local, avec un faux élève et un faux coach, sans toucher à la vraie base. Format de chaque point : deux à trois lignes d'analyse et d'idée, puis le prompt à coller dans Claude Code.

Expert 1 · Coach en ligne : temps gagné — 6/10

Les briques qui font gagner du temps au coach existent et marchent au banc : file de bilans avec « Athlète suivant », brouillon pré-écrit, report avec « Annuler », étiquettes, chrono par dossier, fil privé par athlète, file « Bloc terminé ». Mais à 60 athlètes, l'accueil fait 16 000 px, la recherche est au 14e écran, la liste n'affiche plus les noms et les deux vues de tri se contredisent : le coach cherche avant d'agir.

1. La liste d'athlètes n'affiche plus aucun nom sur téléphone — Priorité : Critique · Effort : S

Analyse et idée. Testé au banc : `.cr-nom` mesure 0 px en 360 et 390 px, 19 px en 430 et 67 px en 768. Les autres éléments de la ligne ne rétrécissent pas : case 17 px, avatar 44, « Ouvrir la fiche » 117, curseur suivi 42, badge 55 (`renderClientRow`, `rc-core:29152-29218`). Résultat : la ligne n'affiche que des initiales et le badge « Nouveau bilan » est coupé. De plus, Ines (douleur urgente) porte le badge vert « Actif », car le badge ignore les signaux. Idée : sur mobile, le nom passe en premier sur toute la largeur. On garde une seule ligne de méta (« dernière séance il y a 2 j · 1 bilan à lire »). Le bouton devient une icône et le curseur suivi descend dans le tiroir. Le badge suit `urgencyScore`.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching), fichier app/rc-core.1792.js (grep -n puis sed -n, ne lis pas tout), CSS app/rc-style.1792.css.
Constat : dans renderClientRow (~ligne 29152), la ligne d'athlète du coach empile case à cocher, avatar 44 px, bouton « Ouvrir la fiche » (.cr-fiche), curseur suivi (_htmlCurseurSuivi) et badge, tous en flex-shrink:0. Le bloc du nom (.cr-nom, flex:1;min-width:0) tombe à 0 px en 360 et 390 px de large, 19 px en 430 et 67 px en 768. Le coach ne voit que des initiales. Le badge « Nouveau bilan » déborde. Le badge vaut « Actif » (vert) pour un athlète qui a une douleur répétée.
À faire :
1. Sous 600 px, la ligne passe sur deux rangées. En haut : avatar + nom complet (ellipsis), avec le badge à droite. En dessous, une méta courte d'une ligne, au plus deux faits parmi « dernière séance il y a N j », « N bilan(s) à lire », « dernier contact il y a N j » (lu dans currentUser.contacts). Calcule la date de la dernière séance par max(date), jamais par sessions[length-1].
2. Sous 600 px, « Ouvrir la fiche » devient une flèche > de même hauteur, sans texte. Toute la ligne reste cliquable. Le curseur SUIVI quitte la ligne et va dans le tiroir d'aperçu (il reste sur la fiche).
3. Le badge suit la même précedence que urgencyScore. Une douleur ou un drapeau donne « Douleur » (rouge) et passe avant « Nouveau bilan ». etatAthlete() et le filet de couleur doivent dire la même chose que le badge (un seul prédicat partagé).
4. Ne change ni les tailles de police ni les cibles. Uniformise avec les variables existantes (--fs-xs, --fs-md).
Cas limites : nom absent (« Profil incomplet »), nom très long, athlète _fromCode (pas de méta séance), thème clair (règles :root[data-theme="clair"] existantes), affichage tableau ≥1025 px inchangé.
Tests à ajouter dans app/tests.js : (a) rendre la liste avec 8 athlètes dans un conteneur de 360 px et vérifier que chaque .cr-nom a une largeur > 120 px ; (b) un athlète avec signauxEntrainement().douleur obtient le badge « Douleur » ; (c) la méta « dernière séance » prend la plus récente même si sessions n'est pas trié.
Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js), puis
```

```
python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.  
Critère de fin : capture en 390 px où les 8 noms sont lisibles en entier, tests verts.
```

2. À 60 athlètes, la recherche est au 14e écran : il faut une recherche toujours visible — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc avec 60 athlètes : la page d'accueil fait 16 068 px. `#ch-search` est à 9 066 px, sous `#ch-vignettes` (index.html:1730), soit environ 7 000 px de grands ronds à deux par ligne. La même population est listée deux fois, ronds puis liste. Le raccourci « / » (rc-core:87540) ne sert pas sur téléphone. Idée : un champ de recherche fixe dans l'en-tête coach, sur tous les onglets. Il cherche par prénom, nom ou étiquette et montre des résultats instantanés avec action directe (Fiche · Bilan · Message · Programme · Nutrition). Les ronds se replient par défaut au-delà de 12 athlètes.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching). Fichiers : app/index.html (#s-coach-home, #ch-mobile-header  
~ligne 1645, #ch-vignettes ~1730, #ch-search ~1789), app/rc-core.1792.js (renderClientList ~25886,  
norm(), raccourci '/' ~87540), app/rc-style.1792.css.  
Constat mesuré (60 athlètes, 390x844) : accueil de 16 068 px, recherche à 9 066 px, plus de 7 000 px  
de vignettes rondes au-dessus. Pour ouvrir un athlète précis, il faut environ 13 écrans de défilement.  
À faire :  
1. Sous #ch-mobile-header, ajoute une barre de recherche fixe (position:sticky), visible sur l'onglet  
Athlètes et dans les écrans coach qui ont un retour vers l'accueil. Même hauteur que l'existant (min-  
height 44 px, police --fs-md) : ne rien agrandir.  
2. La saisie ouvre une liste de résultats en surimpression (8 au plus), avec la même normalisation que  
norm() (accents, casse). Elle trouve le prénom, le nom, l'e-mail et le libellé d'étiquette  
(currentUser.etiquettes / etiquettesAth). Chaque résultat montre le nom, un fait (« bilan à lire », «  
douleur », « dernière séance il y a N j ») et 4 actions directes : Fiche  
(openClientDetail(id,false,true)), Bilan (viewClientBilans puis evoTab('reponses')), Message  
(msgOuvrirFil), Programme (ouvrirSeancesSansBrouillon).  
3. Garde #ch-search plus bas : il filtre la liste comme aujourd'hui, et les deux champs partagent la  
même valeur.  
4. #ch-vignettes est replié par défaut quand il y a plus de 12 athlètes (« Voir les 60 visages »).  
L'état est mémorisé dans localStorage, avec try/catch.  
5. Entrée sur le premier résultat ouvre la fiche. Échap ferme. Le « / » existant cible le nouveau  
champ.  
Erreurs humaines : une faute de frappe d'une lettre doit encore trouver (distance 1 sur le prénom au-  
delà de 4 lettres). Aucun résultat : on propose « Rechercher dans les codes en attente ». Rien ne doit  
se perdre au rafraîchissement périodique de la liste (le champ reste hors de #ch-clients-list).  
Tests dans app/tests.js : fonction pure rechercheAthletes(clients,q,etiquettes) → ids ordonnés (prénom  
exact > début de nom > étiquette > approché) ; « ines » trouve « Inès » ; une faute (« karym ») trouve  
Karim ; une étiquette « Sèche » ramène ses athlètes.  
Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3  
scripts/versionner_actifs.py). Ne pas agrandir les polices.  
Critère de fin : au banc avec 60 athlètes, ouvrir n'importe quel athlète en 2 taps sans défiler.
```

3. « Mes notifications » et « Le point du lundi » se contredisent — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : « Mes notifications » annonce « 5 athlètes · Progression bloquée · Lea, Hugo... », « 2 Nouveaux bilans », « 2 Bilans en retard » et « Bloc terminé · Hugo ». Au même moment, « Le point du lundi » classe Léa, Hugo, Julie et Tom en « Rien à signaler ». La cause : `lundiLigne` (rc-core:27870) ne lit que douleur, plateauMuscle, décrochage et chute. Il ignore bilans, bloc fini, jamais démarré, messages et sous-MEV. Côté notifications, « Progression bloquée » regroupe aussi sousMEV, formeBasse et volumeHaut (`_lignesEntrainement` 26841). Idée : une seule source de vérité pour les deux vues, et un libellé exact par signal.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching), app/rc-core.1792.js.

Constat (banc, 8 athlètes) : renderToDoBlock (~28573) et la vue « Le point du lundi » (lundiLigne ~27870, _lundiLire ~27909, LUNDI_RANGS ~27866) donnent des verdicts opposés. Léa (bilan sans réponse, volume sous le MEV), Hugo (bloc terminé), Julie (bilan en retard) et Tom (inscrit hier, rien fait) sont « Rien à signaler » dans le lundi, mais présents dans les notifications. De plus, la ligne « Progression bloquée » de _lignesEntrainement (~26841) couvre aussi sousMEV, formeBasse et volumeHaut : un athlète simplement sous son minimum de volume est annoncé « Progression bloquée ».

À faire :

1. Crée une fonction PURE signalPrincipal(c,ctx) qui rend {cat, libelle, depuis, cible} par ordre de priorité : drapeau santé > douleur > bilan sans réponse (hasNewBilan) > message sans réponse (msgClientsSansReponse) > bilan en retard (needsAlert) > décharge proposée > charges en baisse > progression bloquée (plateauMuscle seul) > volume sous le minimum (sousMEV) > forme basse > séances écourtées > assiduité en baisse > bloc terminé ou qui se termine (finProgramme) > jamais démarré > rien.
2. lundiLigne l'utilise, et l'ouverture d'une ligne va au bon endroit : bilan → viewClientBilans + evoTab('reponses') ; message → msgOuvrirFil ; bloc → _ouvrirProgfin.
3. Dans _lignesEntrainement, sépare les libellés : « Progression bloquée » (plateauMuscle), « Volume sous le minimum » (sousMEV), « Forme en baisse » (formeBasse). Ce sont les mêmes que dans signalPrincipal.
4. Le point du lundi garde « Rien à signaler » seulement si signalPrincipal rend 'rien'. Pas de score entre athlètes : on trie par rang de signal, puis par ancienneté (comme lundiTrier). Erreur humaine : un signal reporté (isAlertSnoozed) reste reporté dans les deux vues, avec la même échéance.

Tests dans app/tests.js : 6 dossiers (bilan non répondu, bilan en retard, bloc fini, sousMEV seul, jamais démarré, douleur + bilan) : signalPrincipal rend la bonne cat. Pour chaque athlète présent dans une ligne de renderToDoBlock, lundiLigne ne rend jamais 'rien'. Un athlète sousMEV seul n'est jamais libellé « Progression bloquée ».

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : sur le banc à 8 athlètes, plus aucun athlète « Rien à signaler » dans le lundi alors qu'il a une ligne dans « Mes notifications ».

4. Message à plusieurs : la messagerie interne existe, mais « Groupé » ouvre encore N conversations WhatsApp — Priorité : Haute • Effort : M

Analyse et idée. Le fil privé par athlète existe (msgEnvoyer rc-core:27520, msgOuvrirFil 27586). Pourtant « Groupé » (index.html:1763, openWaGroupe 25096) prépare un lien WhatsApp par athlète : 60 athlètes, c'est 60 taps et 60 allers-retours d'application, hors dossier. La barre de sélection (_selMaj 28994 : Étiqueter, Décharge, Programme, Cadence) n'a pas d'action « Message ». « Inactifs » ne propose que « Mail ». Idée : « Envoyer dans l'app » à la sélection ou à une étiquette, en 1 tap après relecture. Chaque fil reçoit le message personnalisé ({prénom}). WhatsApp reste en plan B.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching). Fichiers : app/rc-core.1792.js (openWaGroupe ~25096, _wagPreparer, msgEnvoyer ~27520, msgTexteValide, _selMaj ~28994, bloc « Inactifs » rendu dans #ch-inactifs, tplContexte/modèles ouvrirModeles ~112671), app/index.html (bouton Groupé ~1763).

Constat : le message groupé n'emploie pas la messagerie interne. Il fabrique un lien WhatsApp par athlète (un tap et un changement d'application chacun). Rien n'est tracé dans le fil de l'athlète, et la sélection multiple de la liste n'a pas d'action « Message ».

À faire :

1. Dans la feuille « Message groupé », ajoute en premier le bouton « Envoyer dans l'app (N) ». Il appelle msgEnvoyer pour chaque destinataire coché, au plus 4 en parallèle, avec le texte personnalisé (« Salut {prénom}, » + texte, via le moteur de modèles existant, jamais le prénom d'un autre). « Préparer les envois WhatsApp » reste en second bouton (plan B).
2. Avant l'envoi, un écran de relecture montre le nombre de destinataires, le message tel que le recevra le premier, et la liste des prénoms. Envoi après « Envoyer à N athlètes ». Au-delà de 20 destinataires, il faut retaper le nombre.
3. Pendant 10 s après l'envoi, « Annuler l'envoi » retire les messages déjà postés (suppression de l'id dans messages/<coach>/<athlète>). Ensuite, le compte rendu affiche « 57 envoyés, 3 en échec : Réessayer ». Les échecs (hors ligne, 403) sont listés par prénom et peuvent être relancés.
4. Ajoute « Message » à la barre de sélection (_selMaj) et à chaque ligne groupée de « Mes notifications » (bilans en retard, inactifs) : la feuille s'ouvre avec ces athlètes cochés et le texte

du signal (`_waCorpsGroupe`). Ajoute aussi « Cocher l'étiquette » (déjà là via `_htmlCocherEtiquette`) en tête.

5. Chaque envoi appelle `noterContact(id)`.

Erreurs humaines : double tap sur Envoyer (un seul lot), réseau coupé au milieu (aucun doublon au renvoi : clé d'idempotence par lot + athlète), texte vide (refus), athlète détaché entre-temps (403, compté en échec sans bloquer les autres).

Tests dans `app/tests.js` : fonction pure `planEnvoiGroupe(selection, texte, prenom) → messages personnalisés, chacun avec son prénom ; idempotence (même clé de lot, pas de second envoi) ; échecs isolés ; plafond et confirmation au-delà de 20.`

Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Critère de fin : au banc (réseau simulé par un faux fetch), 8 athlètes reçoivent leur message en 3 taps, et l'annulation retire tout dans les 10 s.

5. Boîte de messages : 60 requêtes en série à l'ouverture, et « Aucun athlète rattaché » dès que le réseau flanche — Priorité : Haute · Effort : M

Analyse et idée. `msgChargerFils` (`rc-core:27494`) attend une requête par athlète, l'une après l'autre, avec `limitToLast=50` (`MSG_PAGE 27421`). À 60 athlètes : 60 allers-retours et environ 3 000 messages relus pour afficher une liste de résumés. C'est relancé depuis l'accueil toutes les 10 min (27679, 27693). Testé au banc avec 8 athlètes et le réseau coupé : l'écran Messages affiche « Aucun athlète rattaché pour l'instant. », car un fil en échec est simplement sauté (27504). Idée : un index `messagesIndex/<coach>/<athlète>` (dernier message, non lus), un seul appel, avec un état d'erreur honnête.

Prompt de mise en place.

Dépôt `RepCore (/home/user/coaching)`. Fichiers : `app/rc-core.1792.js` (`msgChargerPage ~27478`, `msgChargerFils ~27494`, `_rendreFils ~27566`, `msgEnvoyer ~27520`, `_msgMarquerLus`), `database.rules.json`, `cloudflare/` si l'écriture de l'index passe par le Worker.

Constat : la liste des fils lance une requête REST par athlète, séquentielle (await dans la boucle), chacune avec `limitToLast=50`, juste pour en tirer le dernier message et le nombre de non-lus. À 60 athlètes, la boîte met plusieurs secondes et le signal « message sans réponse » de l'accueil dépend de ce cache de 10 min. Si les requêtes échouent (hors ligne, jeton absent), les fils sont sautés et l'écran dit « Aucun athlète rattaché pour l'instant ».

À faire :

1. À chaque `msgEnvoyer` et à chaque marquage « lu », écris un résumé dans `messagesIndex/<cléCoach>/<cléAthlète> : {dernierTexte (80 car.), de, at, nonLusCoach, nonLusAthlete}`. Mets à jour `database.rules.json` : lecture par le coach et l'athlète concernés seulement, écriture par l'un des deux, validation des types et des tailles.
2. `msgChargerFils` lit cet index en UN appel. En repli, pour les fils sans index (anciens), on lance au plus 6 requêtes en parallèle avec `limitToLast=1`.
3. `_rendreFils` distingue trois cas : « Chargement... », « Pas de réseau : la liste s'affichera une fois connecté (Réessayer) » et « Aucun athlète rattaché » (seulement si `getClientes()` est vide). En hors ligne, on affiche le dernier index connu (`localStorage`, `try/catch`), marqué « hors ligne ».
4. Le compteur de non-lus de l'accueil lit l'index : le cache de 10 min tombe à 60 s.

Erreurs humaines : athlète détaché (fil retiré de l'index au détachement), deux appareils du coach (l'index est le même), message supprimé (index recalculé depuis la dernière page).

Tests dans `app/tests.js` : 60 athlètes simulés avec un faux fetch qui compte les appels → au plus 1 appel quand l'index existe, au plus 6 requêtes simultanées en repli ; fetch qui échoue → le texte « Pas de réseau » s'affiche, pas « Aucun athlète » ; l'index est mis à jour par `msgEnvoyer`. Tests de règles dans `cloudflare/test` si le dépôt en a pour `database.rules.json`.

Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Critère de fin : boîte de 60 fils affichée en un appel ; avec le réseau coupé, le message dit la vraie raison.

6. Répondre à un bilan : un tap de trop, et rien sur l'entraînement à côté — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : notification (1), puis « Nouveau bilan à checker » (2), qui ouvre l'Évolution sur l'onglet **Mesures** (`viewClientBilans rc-core:33787`, ne pose pas `evoTab('reponses')`), puis « Réponses » (3), puis « Envoyer » (4). Sur l'écran de réponse, rien ne rappelle que Karim déclare une douleur 5/6 au développé couché sur ses 3 dernières séances : il faut revenir à la fiche. Après la réponse au bilan 2, le bilan 1 reste « sans réponse » et n'est jamais proposé (`hasNewBilan` ne lit que le dernier). Idée : ouvrir directement sur Réponses, avec un bandeau de contexte en tête (signaux, dernière séance, dernier poids, mot de l'athlète), et proposer les anciens bilans restés sans réponse.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching), app/rc-core.1792.js : viewClientBilans (~33787), _majBoutonBilan (~32701), evoTab, blocReponseBilan (~78462), _proposerBilanSuivant (~27140), hasNewBilan / dernierBilan (~30400), bilansSansReponse.
```

Constat (banc) : le bouton « Nouveau bilan à checker » de la fiche ouvre l'écran Évolution sur « Mesures ». Le coach doit encore toucher « Réponses ». L'écran de réponse ne montre ni les signaux d'entraînement (douleur, décrochage), ni la date de la dernière séance, ni « Son mot ». Après une réponse au dernier bilan, un bilan plus ancien resté sans réponse n'est ni signalé ni proposé.

À faire :

- `viewClientBilans(opts)` : quand `hasNewBilan(c)` est vrai, ou quand l'appel vient de la file de bilans ou du bouton « Nouveau bilan à checker », ouvre sur `evoTab('reponses')`, avec le bilan non répondu le plus récent déplié et le curseur à la fin du brouillon. « Voir le dernier bilan » (sans nouveauté) garde Mesures.
- En tête du volet Réponses, ajoute un bandeau d'une ou deux lignes en `--fs-xs`, sans nouveau composant : le signal principal (s'il existe), « dernière séance il y a N j (nom) », « poids 61,4 kg le 5 oct. ». Chaque élément est un lien vers la section de la fiche (même cible que `LUNDI_CIBLE`).
- Après `saveReponseBilan`, si `bilansSansReponse(c) > 0`, affiche « 1 bilan plus ancien sans réponse : y répondre / le marquer traité » avant « Athlète suivant → ». « Marquer traité » pose un indicateur (`b.traité=true`) sans texte envoyé, réversible.

Erreurs humaines : réponse envoyée au mauvais athlète (le nom de l'athlète est rappelé sur le bouton : « Envoyer à Karim ») ; envoi involontaire du brouillon pré-écrit sans l'avoir touché (premier envoi d'un brouillon non modifié : on demande une confirmation courte, et le réglage « ne plus demander » est mémorisé) ; brouillon tapé conservé (`rbBrouillon`) si le coach quitte l'écran.

Tests dans `app/tests.js` : `viewClientBilans` sur un athlète avec bilan non répondu → onglet actif 'reponses' ; bandeau avec la douleur quand `signauxEntrainement(c).douleur` ; après réponse au dernier bilan avec un ancien sans réponse, la proposition apparaît ; « marquer traité » retire l'ancien de `bilansSansReponse`.

Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Critère de fin : depuis « Mes notifications », un bilan se répond en 3 taps et le coach voit la douleur sans quitter l'écran.

7. Ajuster la nutrition : les cibles sont à 3 600 px et chaque « -20 » part à l'athlète — Priorité : Moyenne · Effort : M

Analyse et idée. Testé au banc sur l'onglet Nutrition de Léa : le total calorique et ses boutons sont à 3 578-3 611 px, et « Enregistrer et transmettre » à 3 839 px. Ils viennent après le type de diète, la phase, le calculateur et la liste de 40 sports. Pour -100 kcal, il faut 5 taps « -20 ». Chacun écrit et pousse le dossier à l'athlète (`tbkDelta rc-core:103603` → `CLOUD.pushOne`), et le bouton « Enregistrer et transmettre » existe aussi : deux modèles d'enregistrement sur le même écran. Idée : une carte « Cibles du jour » en tête de l'onglet. Elle affiche ON/OFF (kcal, P, G, L), des pas réglables ($\pm 50/\pm 100$) et une seule transmission avec « Annuler ». Le calculateur se replie dessous.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching), app/rc-core.1792.js : renderCoachNutriSection, tbkDelta (~103603), appliquerDeltaKcal, athDelta (~91872), le bouton « Enregistrer et transmettre à l'athlète
```

», l'historique des modifications (« Historique des modifications »).

Constat (banc, 390 px) : sur l'onglet Nutrition de la fiche, le total calorique est à environ 3 600 px du haut, après les choix de diète, la phase, le calculateur de besoins et la liste des sports. Chaque appui sur -20 ou +20 enregistre et pousse le dossier de l'athlète (CLOUD.pushOne), alors qu'un bouton « Enregistrer et transmettre » existe plus bas. 5 appuis pour -100 kcal donnent 5 écritures distantes. À faire :

1. En tête de l'onglet Nutrition (juste après « Message à ... »), ajoute une carte compacte « Cibles » : kcal, P, G, L pour ON et OFF, sur deux lignes de chiffres, dans le style des cartes existantes (pas de police plus grande).
2. Boutons de pas -100 / -50 / +50 / +100 (le pas par défaut est un réglage coach, voir le point 9). Ils modifient un brouillon local, avec l'écart affiché (« -100 kcal par rapport à l'envoyé »).
3. Un seul bouton « Transmettre à <prénom> » écrit et pousse une fois. Ensuite, « Annuler » pendant 10 s restaure les cibles précédentes (entrée d'historique « annulé »).
4. Les -20/+20 du tableau détaillé passent par le même brouillon : plus aucun push par appui.
5. Le calculateur détaillé (facteurs, sports, méthodes) reste disponible dessous, replié par défaut une fois des cibles posées. L'état est mémorisé par coach.

Erreurs humaines : quitter l'onglet avec un brouillon non transmis → bandeau « Modifs non transmises : Transmettre / Jeter » ; passage sous le plancher (plancherAthlete) → confirmation explicite ; double tap sur Transmettre (un seul envoi).

Tests dans app/tests.js : 5 appuis -20 puis Transmettre → un seul pushOne simulé ; Annuler restaure les macros exactes ; quitter avec un brouillon déclenche l'avertissement ; l'historique garde l'entrée.

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : depuis la fiche, -100 kcal transmis en 3 taps sans défiler, avec une seule écriture distante.

8. Préparer le bloc suivant : pas de « Reconduire », et des échauffements avant les exercices — Priorité : Moyenne · Effort : M

Analyse et idée. Testé au banc avec Hugo (bloc terminé) : la feuille « Bilan du bloc » propose « Assigner le bloc suivant ». Seuls les programmes de la bibliothèque y figurent : « Ta bibliothèque de programmes est vide » (_htmlSuiteBilanBloc rc-core:28235). Pour reconduire son propre bloc, il faut passer par « Modifier le bloc » puis les semaines, ou enregistrer le programme dans la bibliothèque puis l'assigner, soit 6 à 8 taps. Il est impossible de préparer le bloc suivant à l'avance. Dans l'éditeur d'une séance, le premier champ « Séries » est à 1 670 px, sous les suggestions d'échauffement et de retour au calme. Idée : « Reconduire ce bloc » et « Préparer le bloc suivant (démarré le ...) » dans la feuille, et des exercices en premier.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching), app/rc-core.1792.js : _htmlSuiteBilanBloc (~28235), progfinAssigner, _ouvrirProgfin, programmeDe (~71202), finProgramme (~28181), écran s-coach-sessions (ouvrirSeancesSansBrouillon ~38646, PUBLIER), éditeur de séance s-coach-program (protocoles échauffement / retour au calme rendus avant « EXERCICES »).

Constat (banc) : à la fin d'un bloc, la feuille « Bilan du bloc » ne propose que des programmes de la bibliothèque (« Ta bibliothèque de programmes est vide » si elle l'est). Reconduire le programme de l'athlète oblige à passer par « Modifier le bloc » ou par « Enregistrer dans mes programmes » puis l'assignation. Il n'existe pas de bloc suivant programmé à l'avance. Dans l'éditeur de séance, les cartes de protocoles (~1 000 px) précèdent la liste des exercices.

À faire :

1. Dans _htmlSuiteBilanBloc, ajoute en tête « Reconduire ce bloc ». Il copie sessions_config, pose programme.debut au lundi suivant et garde les semaines (modifiables), le tout en brouillon dans _coachEditClient, puis on publie avec PUBLIER comme aujourd'hui.
2. Ajoute « Préparer le bloc suivant ». Il crée programmeSuivant {debut, semaines, sessions_config} sur le dossier, éditable dans le même écran (onglet « Bloc suivant »). Au lundi de début, il devient le programme courant : bascule au chargement côté athlète et côté coach, journalisée. Ligne « Bloc qui se termine » : si programmeSuivant existe, la ligne dit « bloc suivant prêt, démarre le ... » et passe en gris.
3. Éditeur de séance : la liste EXERCICES vient en premier. Échauffement et retour au calme passent dessous, repliés en une ligne (« Échauffement : Jour lourd, force · changer »).

Erreurs humaines : le bloc suivant est modifiable et supprimable jusqu'à sa date de début ; un bloc courant modifié après la préparation du suivant ne touche pas le suivant ; si le coach quitte s-coach-

sessions avec des modifications non publiées, un avertissement s'affiche (« Publier / Jeter / Rester »).

Tests dans app/tests.js : Reconduire → debut = lundi suivant et sessions_config identique (copie profonde) ; programmeSuivant devient courant au lundi prévu (horloge simulée), pas avant ; finProgramme n'émet plus « Bloc terminé » quand le suivant a démarré ; dans l'éditeur, le premier input visible est le nom de la séance puis les séries du premier exercice.

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : depuis la ligne « Bloc terminé », le bloc est reconduit en 3 taps + PUBLIER.

9. Paramétrer plus : il manque une page « Mes réglages de coaching » et un accueil réglable — Priorité : Moyenne · Effort : M

Analyse et idée. L'onglet Profil du coach (testé au banc) ne contient que la vitrine, la marque, PayPal, la disponibilité et l'apparence. Il n'y a aucune valeur par défaut de travail : cadence de bilan à l'arrivée, pas de calories, signature de réponse, programme ou modèle de départ des nouveaux, seuils des signaux. Les seuils se règlent ligne par ligne avec ⚙. L'accueil empile une vingtaine de blocs (arrivées par mois, kit de la semaine, suggestion de formule « Tu suis 7 athlètes... » alors que « 8 avec suivi » s'affiche au-dessus, tunnel, charges articulaires...) que le coach ne peut ni masquer ni ordonner. Idée : une page « Réglages de coaching » et un « Personnaliser l'accueil ».

Recoupement. La page « Réglages de coaching » est détaillée par l'expert 14 (point 1), qui fait référence : garder ici « Personnaliser l'accueil » et l'alignement du nombre d'athlètes.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching). Fichiers : app/index.html (onglet #ct-profil, panneau #ct-dashboard et ses blocs), app/rc-core.1792.js (coachTab ~125490, loadCoachHome, réglages ⚙ des lignes de notifications, setBilanFreq / selCadence ~30343, QC_BILAN_DEFAULTS ~112625, ouvrirFormulesReponse, msgTemplates), database.rules.json (nouveau champ du dossier coach).

Constat : aucun réglage de travail n'existe au niveau du coach. Chaque valeur se règle athlète par athlète ou signal par signal, et l'accueil impose environ 20 blocs dans un ordre fixe (16 068 px à 60 athlètes). Le bloc de formule annonce « Tu suis 7 athlètes » alors que la liste compte « 8 avec suivi » (règles de comptage différentes, à aligner).

À faire :

1. Dans l'onglet Profil, ajoute une entrée « Réglages de coaching » vers un écran simple, rangé dans currentUser.reglagesCoach (un seul objet). Cadence de bilan donnée à tout nouvel athlète (hebdo / 2 sem.), modèle de programme et formule de réponse par défaut, pas de calories (20/50/100), signature ajoutée aux réponses, heures calmes des notifications, seuils par défaut des signaux (repris des ⚙ existants, qui restent là pour un athlète précis).
2. Les nouveaux codes et athlètes héritent de ces défauts à leur création. Les dossiers existants ne changent pas, sauf « Appliquer à tous » avec confirmation et liste des athlètes concernés.
3. En bas de l'accueil, ajoute « Personnaliser l'accueil » : une liste des blocs avec un interrupteur et ▲▼, rangée dans reglagesCoach.accueil. « Mes notifications » et la recherche ne se masquent pas. Un retour aux réglages d'origine est disponible.
4. Aligne le comptage « Tu suis N athlètes » sur celui de la liste « avec suivi » (une seule fonction de comptage).

Erreurs humaines : chaque réglage montre sa valeur d'origine (« par défaut : 2 semaines ») ; « Réinitialiser » par ligne ; aucun réglage n'efface de donnée d'athlète.

Tests dans app/tests.js : un nouveau code créé après réglage hérite de la cadence ; « Appliquer à tous » ne touche que les dossiers confirmés ; un bloc masqué n'est plus rendu, et la recherche ne peut pas être masquée ; le nombre « Tu suis » est égal au nombre « avec suivi ».

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : un coach règle une fois sa cadence, son pas de calories et sa signature, et masque 5 blocs d'accueil ; les deux nombres d'athlètes concordent.

10. Infos clés de la fiche : poids sans date, aucune date de dernière séance en tête — Priorité : Basse · Effort : S

Analyse et idée. Testé au banc sur la fiche de Léa : la tête affiche « 61.4kg POIDS » sans date ni source. L'onglet Nutrition dit « dernière pesée 62 kg », et Données « 61,4 kg pesé au bilan du 5 oct. ». Le nombre de séances (19) n'a pas de date de dernière séance ; il faut descendre dans « Récapitulatif des séances ». Idée : sous les compteurs de tête, une ligne de faits datés : « dernière séance : sam. 4 oct. · pesée : 62 kg le 5 oct. · dernier contact : il y a 3 j ».

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching), app/rc-core.1792.js : openClientDetail (~33090), remplissage de #ccd-weight, #ccd-sessions, #ccd-streak ; currentUser.contacts (dernier contact, ~28280) ; weightLog et bilans de l'athlète.
Constat (banc) : la tête de fiche affiche un poids sans date (celui du dernier bilan), alors que l'onglet Nutrition en cite un autre (dernière pesée du journal). Le nombre de séances n'a pas de date de la dernière. Pour répondre à « il s'est entraîné quand ? », il faut défiler jusqu'au récapitulatif.
À faire :
1. Fonction PURE faitsCles(c,coach,maintenant) → {derniereSeance:{date,nom}, pesee:{kg,date,source:'journal' | 'bilan'}, contact:{date}}. Elle prend la plus récente de weightLog et des poids de bilan, avec max(date) partout (jamais l'ordre du tableau).
2. Sous les quatre compteurs de tête, ajoute une ligne en --fs-xs : « Dernière séance : sam. 4 oct. · Pesée : 62 kg le 5 oct. · Contact : il y a 3 j ». Chaque fait est un lien vers sa section (#ccd-sessions-recap, courbe de poids, fil de messages).
3. #ccd-weight reprend le même poids que la ligne (une seule source), avec la date en title.
Cas limites : aucune séance (« pas encore de séance »), aucune pesée, athlète _fromCode (ligne masquée), poids masqué pour l'athlète (masquerPoids n'affecte pas l'affichage coach, mais garde l'avertissement existant).
Tests dans app/tests.js : sessions non triées → derniereSeance juste ; pesée du journal plus récente que le bilan → source 'journal' ; même valeur dans #ccd-weight et l'onglet Nutrition.
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : sans défiler, la tête de fiche répond à « dernier poids ? dernière séance ? dernier échange ? ».
```

Expert 2 · Morpho et analyse anatomique — 6/10

L'analyse photo (anat*) est d'un niveau rare dans une app grand public : repères de Leva (1996) et ANSUR II par sexe, z-scores honnêtes, notes de confiance par point, modèles squat, soulevé et développé sourcés, et la règle G7 qui ne fait descendre que la consigne vers l'élève. En revanche, le moteur « axes et profils » au ruban, celui que le coach lit en premier, part d'un repère faux pour les bras et d'un calibrage qui mélange hommes et femmes. La morpho est aussi éclatée sur quatre onglets qui ne se parlent pas, et une photo ou une mesure corrigée n'est jamais relue.

1. Le repère « bras » classe presque tout le monde en « bras courts » — Priorité : Critique · Effort : S

Analyse et idée. La consigne dit « de la pointe de l'épaule au poignet » (rc-core.1792.js:15113), c'est-à-dire acromion → styloïde, environ $0,34 \times$ la taille. Le repère `RATIO_BRAS_REF=0.45` (l.15003) correspond pourtant à une mesure jusqu'au bout des doigts. Testé au banc avec des mesures réalistes (Léa, 165 cm, 55 cm ; Tom, 180 cm, 60 cm) : les deux sortent « A3 bas, 33 % ». Le coach voit alors « Bras courts pour la taille » sur chaque fiche, et P5 « le pousseur » s'allume dès que A4 est bas. Il faut remplacer ce repère par un repère par sexe de même définition, et détecter la mesure prise jusqu'aux doigts.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. La mesure « deb-bras » est définie partout comme acromion → poignet (MORPHO_MESURES l.15113, ANAT_BIAIS_SEGMENTS l.58957, questionsMorpho l.17489 « de l'acromion à la styloïde »). Mais `RATIO_BRAS_REF=0.45` (l.15003), et les fiches P4 et P5 de MORPHO_PROFILS (« au-delà de ~48 % », « sous ~42 % ») supposent une mesure jusqu'au bout des doigts. Acromion → styloïde vaut environ $0,34 \times$ la taille (ANSUR II : acromion-radiale + radiale-stylien). Résultat : tout athlète bien mesuré sort A3 « bas », questionsMorpho affiche « Bras courts pour la taille », et P4 ne peut jamais sortir.

À faire :

1. Remplacer `RATIO_BRAS_REF` par un repère PAR SEXE, recalculé sur ANSUR II avec la même méthode et le même commentaire de source qu'ANAT_LARGEURS (l.58814) : (acromion-radiale + radiale-stylien) / stature, moyenne et écart-type par sexe. Ordre de grandeur attendu : 0,34. La marge vaut 1,5 écart-type, et jamais moins que l'erreur du ruban. Adapter `ratioBras`, `_morphoBrut('A3')`, `morphoAxes` (`repereTexte`), `questionsMorpho` et les textes `signatureTexte` de P4 et P5 aux nouveaux pourcentages.
 2. Garde-fou contre l'erreur humaine la plus probable : un rapport deb-bras / taille > 0,40 donne motif 'doigts' et non une position. `questionsMorpho` pose alors : « Mesuré jusqu'au bout des doigts ? La mesure s'arrête à l'os du poignet. » Aucun profil n'est calculé dessus.
 3. Si le sexe est absent, aucune position A3 ne sort, et « il manque le sexe » s'ajoute à `aMesurer`. Tests dans `app/tests.js` : les fixtures M2 (deb-bras 78 à 90 pour 178 ou 180 cm) ne sont pas anatomiques, il faut les recalculer sur 58 à 62 cm. Ajouter : homme de 180 cm avec 60 cm → A3 neutre ; femme de 165 cm avec 55 cm → neutre ; 80 cm pour 180 cm → motif 'doigts', aucune question « bras courts » ; P4 atteignable avec 66 cm pour 180 cm.
- Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`), puis `python3 scripts/versionner_actifs.py`. Ne pas agrandir les polices.
- Finir quand : `node app/tests.js` (ou la commande de test du dépôt) passe et qu'une fiche réaliste n'affiche plus « Bras courts ».

2. Calibrage des axes sans distinction de sexe, alors que les repères par sexe sont déjà dans le code — Priorité : Haute · Effort : M

Analyse et idée. `morphoCalibrage` (l.15453) prend la médiane de tous les athlètes du coach, hommes et femmes confondus, pour A2, A4, A5 et A6. Testé au banc avec un coach suivi par 10 femmes : un homme exactement dans la moyenne ANSUR (épaules/bassin 1,51) sort « A5 haut, charpente large ». Dans une base surtout masculine, la même erreur range chaque femme en P6 « charpente étroite ». Le commentaire affirme qu'« aucun repère publié » n'existe, alors qu'ANAT_LARGEURS (1,513 H / 1,344 F) et ANAT_REF (cuisse/jambe et bras/avant-bras de Leva, centre à centre comme MediaPipe) sont déjà dans le fichier. Il faut s'en servir par sexe, et ne garder le calibrage du coach qu'en ajustement, séparé par sexe. Le repère entrejambe 0,46 (l.15002) est aussi trop bas : l'adulte moyen tourne autour de 0,48, ce qui pousse environ un homme sur quatre vers « jambes longues ».

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. `morphoCalibrage` (l.15453) et `morphoAxes` (l.15692) calent A2, A4, A5 et A6 (et A2photo, A4photo) sur la médiane des athlètes du coach, sans distinguer les sexes. Or le fichier contient déjà des repères par sexe : ANAT_LARGEURS (l.58814, biacromial / bicrétal ANSUR II : H 1,513 ± 0,090, F 1,344 ± 0,110), ANAT_REF (l.58617, de Leva 1996, centre à centre : cuisse/jambe H 0,2425/0,2493, F 0,2124/0,2492 ; bras/avant-bras H 0,1618/0,1545, F 0,1586/0,1523) et ANAT_DISP pour les dispersions.

À faire :

1. A5 (épaules / bassin au mètre) : repère par sexe tiré d'ANAT_LARGEURS, marge = $1,5 \times$ rapport_et. A2photo et A4photo (rapports MediaPipe centre à centre) : repère de Leva par sexe, marge tirée d'ANAT_DISP et de l'erreur photo (ANAT_ERR). A6 : garder le repère homme > 1,65 m, et en ajouter un pour la femme (tables de « frame size » : fine < 14 cm, épaisse > 16,5 cm environ, à sourcer dans le commentaire).
2. Le calibrage du coach reste, mais PAR SEXE (`morphoCalibrage(athletes, sexe)`), et seulement pour A2 et A4 au mètre, dont la définition (entrejambe – creux du genou) n'a pas d'équivalent publié. Sous MORPHO_CALIB_MIN athlètes du même sexe, rien.
3. A1 : repère par sexe sur la hauteur d'entrejambe ANSUR II (crotch height / stature, environ 0,48 H et 0,475 F, à recalculer et sourcer comme ANAT_LARGEURS), marge 1,5 écart-type. Préciser la consigne de deb-entrejambe : « livre serré à l'entrejambe, tranche contre le mur : du sol au-dessus du livre ». Le pubis est plus haut de plusieurs centimètres. Marquer les bilans saisis avec la nouvelle consigne (deb-entrejambe-`proto:'livre'`) pour ne pas mélanger les deux protocoles dans le calibrage.
4. Sexe absent : pas de position sur A1, A3, A5, A6, et « il manque le sexe » dans `aMesurer`.

Tests dans app/tests.js : population de 10 femmes ANSUR-moyennes, homme moyen → A5 neutre (aujourd'hui « haut ») ; l'inverse ; homme de 180 cm avec entrejambe à 86 cm → A1 neutre ; calibrage par sexe sous 8 → absent.

Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Fini quand : les tests passent et qu'aucun axe osseux ne prend de position sans repère du même sexe.

3. Une photo ou une mesure corrigée n'est jamais relue — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : après remplacement de la photo de face du bilan de départ et correction de la hauteur de rotule (l'échelle), anatARefaire rend toujours false, parce qu'il ne compare que la date du bilan (l.62181), et morphoInitialePeutEtre rend false sur une analyse gelée. De plus, _bilEnregistrerModif (l.75074) n'appelle pas morphoInitialePeutEtre. C'est exactement le cas réel de Kevin : un élève qui ajoute ses photos après coup n'est pas analysé avant le bilan suivant, et une échelle fausse reste figée. Il faut une empreinte des entrées (photos et rotule), et relancer l'analyse quand elle change.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Trois analyses dépendent du bilan de départ : morphoAnat (anatAnalyser, l.62256, relancée seulement si anatARefaire l.62181, qui compare a.bilan à la date du premier bilan), morphoInitiale (lot 8, morphoInitialePeutEtre l.17144, jamais relancée une fois 'gelee') et morphoPhoto (lireMorphoPhoto). La correction d'un bilan passe par modifierBilan / _bilEnregistrerModif (l.75035 et l.75074), qui garde la même date et n'appelle pas morphoInitialePeutEtre. Remplacer la photo de face ou corriger deb-rotule ne relance donc rien. Un élève qui ajoute après coup les photos oubliées n'est pas analysé.

À faire :

1. Fonction pure empreinteMorpho(bilan) : hash court des sources photo face, dos et profil (url ou clé de photoBilanRef) + deb-rotule + deb-height. Elle est stockée dans morphoAnat.empreinte et dans morphoInitiale.empreinte.
2. anatARefaire : vrai si l'empreinte stockée diffère de celle du premier bilan. Garder les points posés à la main seulement si la photo n'a pas changé : si elle a changé, les jeter et le dire au coach (« photo remplacée par l'élève le JJ/MM : analyse refaite »).
3. morphoInitialeARefaire : vrai aussi quand l'état est 'gelee' mais que l'empreinte du bilan de référence (photoRef.bilan) a changé. Une analyse gelée sur une échelle corrigée n'est plus gelée.
4. _bilEnregistrerModif : appeler morphoInitialePeutEtre(currentUser) après l'écriture, comme saveBilanFinal (l.79065).
5. Côté coach, une ligne discrète dans l'en-tête de l'analyse : « Analyse du JJ/MM, sur la photo du JJ/MM ». Pas de nouveau bouton.

Cas limites : bilan modifié sans changement de photo ni de rotule → aucune relance ; photo retirée → état 'attente' avec la raison ; hors ligne → relance au prochain affichage de l'onglet Données.

Tests dans app/tests.js : dossier avec morphoAnat et morphoInitiale gelée ; on change deb-photo-face → anatARefaire et morphoInitialeARefaire vrais ; on change deb-weight seul → faux ; _bilEnregistrerModif sur un bilan de départ qui reçoit ses 3 photos → morphoInitialePeutEtre appelée (espion).

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Fini quand : corriger photo ou rotule dans « Historique des bilans → Modifier » fait réapparaître une analyse à jour dans la fiche coach.

4. Photo « rouge » : aucun plan B, l'élève reste bloqué — Priorité : Haute · Effort : S

Analyse et idée. Une photo jugée rouge par le contrôle MediaPipe n'est pas gardée du tout (l.76116), sans option pour passer outre. Or le moteur échoue sur des photos valables : peau sombre en faible lumière, tenue ample, morphologie atypique, prothèse. L'élève finit alors par valider sans photos, ce qui est le cas réel cité par Kevin. Il faut proposer, au deuxième refus, « Garder quand même : ton coach vérifiera », et marquer la photo rouge-forcee pour que les analyses de mesure l'ignorent.

Recoupement. Le bouton « Garder quand même » est déjà demandé par l'expert 7 (point 1) : ici, seul s'ajoute le marquage « rouge-forcée » que les analyses morpho ignorent.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Au chargement d'une photo de bilan (compressImage, vers l.76100), _bilControlderPhoto applique photoControle (l.75565). En rouge, la photo n'est pas gardée et l'élève ne peut que recommencer (l.76116). Si le moteur se trompe sur une photo correcte, l'élève est bloqué, et il valide son bilan sans photos.

À faire :

1. Compter les refus rouges par vue pendant le bilan (_bilRefus[key]). Au 2e refus de la même vue, sous le verdict, ajouter un lien texte (pas un gros bouton) : « Garder quand même, ton coach vérifiera ». Il garde la photo avec bilData[key+'-ctl']='rouge-force|'+codes.
2. _htmlPhotoVerdict affiche pour 'rouge-force' : « Gardée à ta demande : ton coach la vérifiera. »
3. Côté coach : la miniature porte un liseré orange et la raison. anatAnalyser, morphoAnalyserInitiale et lireMorphoPhoto ne prennent JAMAIS de centimètres sur une photo 'rouge-force' (raison « photo gardée malgré le contrôle »), mais la montrent pour le suivi visuel. Le coach garde la main avec les boutons existants (Relancer, Ajuster les points).
4. Au moment de valider, si une vue manque, rappeler en une phrase « Photo de profil manquante : tu pourras l'ajouter depuis Historique des bilans → Modifier », pour que le plan B soit connu.

Tests dans app/tests.js : deux contrôles rouges → le lien apparaît ; clic → bilData garde la photo avec '-ctl' 'rouge-force' ; morphoInitialeDe sur une photo 'rouge-force' → état 'attente' avec la raison ; un seul rouge → pas de lien.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les cibles.

Fini quand : un élève ne peut plus être bloqué par le contrôle et le coach sait quelles photos ont été forcées.

5. La morpho est éclatée sur quatre onglets et noyée dans la nutrition — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc sur la fiche de Léa. Les profils composés s'affichent dans l'onglet NUTRITION, sous la micronutrition (ccd-micro, l.17722). La revue d'exercices est dans Entraînement (ccd-revue-morpho), l'analyse photo dans Données (ccd-anat), les amplitudes dans Données et répétées dans Nutrition, et les longueurs figées seulement dans l'écran Amplitudes. Chaque profil déroule 7 paragraphes (environ 4 000 caractères pour 3 profils). Il faut une seule carte « Morpho » dans Données : trois lignes de synthèse (profils, réglages à appliquer, mesure manquante) et le détail replié.

Prompt de mise en place.

Contexte : RepCore, app/index.html et app/rc-core.1792.js. La morpho d'un athlète est dispersée dans la fiche coach : _htmlQuestionsMorpho (profils, « à regarder », amplitudes) est injecté dans #ccd-micro, onglet Nutrition (renderCoachMicroSection l.17715) ; la revue d'exercices dans #ccd-revue-morpho (Entraînement, renderRevueMorphoCoach l.16417) ; l'analyse photo dans #ccd-anat (Données) ; les amplitudes dans #ccd-amplitudes (Données), affichées une seconde fois dans Nutrition ; les longueurs figées (_htmlMorphoInitiale) seulement dans l'écran s-coach-amplitudes.

À faire, sans ajouter d'étape au coach :

1. Retirer la morpho de renderCoachMicroSection. Créer dans l'onglet Données, juste avant #ccd-anat, une section « Morpho » qui commence par une SYNTHÈSE de 3 lignes au plus : (a) les profils retenus, en noms courts séparés par « · » ; (b) les réglages actionnables (morphoReglages + aménagements de la revue), un par ligne et chiffrés (« Squat : cale 2,5 cm ») ; (c) la mesure manquante qui débloquent le plus (aMesurer[0]).
2. Tout le reste (mécanique, privilégier, accent, piège, signature, axes, amplitudes, longueurs figées) dans un <details> par profil, replié par défaut. La revue d'exercices reste dans Entraînement, réduite à sa liste avec un lien « voir la morpho » qui ouvre Données sur la section.
3. Une seule source pour les amplitudes : la section Données. Supprimer le doublon.
4. Uniformiser les titres de section sur le composant .cc-sect existant, sans changer les tailles de police.

Cas limites : aucun profil et rien à mesurer → la section ne s'affiche pas (comportement actuel). Athlète mineur → la synthèse dit « axes osseux non présentés avant 18 ans ». Athlète côté élève : rien ne s'affiche (garde G7 inchangée, test existant).

Tests dans app/tests.js : le HTML de #ccd-micro ne contient plus « Profils composés » ; la synthèse fait 3 lignes au plus ; les profils sont dans des <details> sans attribut open ; les assertions G7

existantes passent toujours.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Fini quand : le coach trouve toute la morpho d'un athlète au même endroit, lisible en 5 secondes.

6. Les 12 longueurs saisies par l'élève, sans aucun contrôle immédiat — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : dans le questionnaire de départ, « 770 » à l'entrejambe (des millimètres) et « 95 » aux épaules sont acceptés sans un mot (bLongueurs, l.75989, oninput brut). Ils ne ressortent qu'au coach, plus tard, en « à révéfier ». Trois mesures sont aussi impossibles à prendre seul (largeur d'épaules « de dos », bassin, thorax entre deux livres). Il faut montrer l'ordre de grandeur attendu dès la saisie (selon la taille déjà donnée), proposer « 77,0 cm ? » en un geste quand l'élève a tapé des millimètres, et ne laisser visibles que 4 mesures clés (rotule, entrejambe, bras, poignet). Les autres vont dans « Pour aller plus loin, à deux ».

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js, bLongueurs (l.75989), appelée à l'étape « Mensurations initiales » du questionnaire de départ. Les 12 MORPHO_MESURES s'y affichent à plat, avec un oninput qui écrit bilData sans contrôle. mesureMorpho (l.15164) et coherencesMorpho (l.15207) savent déjà qu'une valeur est aberrante, mais seul le coach le voit, plus tard.

À faire :

1. Fonction pure attenduMesure(cle, tailleCm, sexe) → {min,max} : fraction de la taille par mesure (entrejambe 0,44–0,52 ; bras acromion→poignet 0,30–0,38 ; rotule 0,25–0,31 ; creux du genou 0,23–0,29 ; avant-bras 0,13–0,17 ; envergure 0,95–1,10 ; pied 0,13–0,17), bornes MORPHO_MESURES pour les tours.
2. Sous chaque champ, après saisie, une ligne en text-dim : « attendu pour 165 cm : 72 à 86 cm ». Hors fourchette : ligne orange, sans bloquer. Si valeur / 10 tombe dans la fourchette : « 77,0 cm ? », un tap remplace la valeur. Mêmes contrôles pour les incohérences (genou > entrejambe, avant-bras > bras).
3. Visibles d'emblée : deb-rotule (elle donne l'échelle des photos), deb-entrejambe, deb-bras, deb-poignet. Les 8 autres vont dans un <details> « Pour aller plus loin (à faire à deux) », avec « à deux » sur épaules, bassin et thorax.

4. La taille vient de bilData['deb-height'] si elle est saisie à cette étape, sinon de currentUser. Erreurs humaines : virgule ou point acceptés, champ vidé = suppression, valeur collée avec « cm ».

Tests dans app/tests.js : attenduMesure('deb-entrejambe',165) contient 77 et pas 770 ; la suggestion ×0,1 apparaît pour 770 ; les 4 champs clés sont hors du <details> ; aucune nouvelle étape dans DEB_STEPS.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les champs.

Fini quand : une saisie en millimètres ou hors norme est vue et corrigée par l'élève avant l'envoi.

7. La revue d'exercices applique la règle d'une variante à une autre, et propose des variantes absurdes — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : pour Léa (P10), la revue affiche sous « SOULEVE DE TERRE » la consigne « Soulevé sumo : prudence... ». revueMorpho (l.16333) apparie sur le seul schéma, même quand la spécificité vaut 0 et que l'aménagement vise une variante précise (sumo, front squat, développé nuque). Côté coach, « Bras courts » propose aussi « FESSIER A LA MACHINE DE TRACTION, ISO LATERAL..., MUSCLE UP » : _variantesSchema (l.15061) prend les premiers noms par ordre alphabétique, et un exercice de fessiers est classé « tirage-vertical » (l.77116).

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. revueMorpho (l.16317) relie un exercice du programme à un aménagement de profil (MORPHO_PROFILS, champ amener) par le seul schéma moteur. Un aménagement qui nomme une VARIANTE (« Soulevé sumo », « Front squat », « Développé nuque », « Squat pieds serrés profond », « Dips profonds ») s'affiche donc aussi sur les autres variantes du schéma (ex. « Soulevé sumo : prudence » sous un soulevé conventionnel). _variantesSchema (l.15058) rend les N premiers noms de _SCHEMA_INDEX par ordre alphabétique, d'où « FESSIER A LA MACHINE DE TRACTION, MUSCLE UP » proposés

en variantes de tirage. « FESSIER A LA MACHINE DE TRACTION » est classé tirage-vertical dans la table des schémas (l.77116) alors qu'il est aussi dans FESSIERS (l.34773).

À faire :

1. Ajouter aux aménagements qui visent une variante un champ `variante:/regex/` (sumo, front, nuque, serr, dips...). revueMorpho et morphoPourExercice ne les retiennent que si le nom de l'exercice matche. Les aménagements génériques (sans `variante`) gardent le comportement actuel.
 2. _variantesSchema(schema, n, opts) : exclure les exercices de la même famille que l'exercice courant ; exclure ceux dont le groupe musculaire principal diffère de celui du schéma (le fessier à la machine de traction) ; préférer une liste courte et choisie par profil (champ `variantes:[...]` dans chaque aménagement : « Développé haltères », « Presse convergente »...), et ne retomber sur le catalogue qu'à défaut, trié par fréquence d'usage et non par ordre alphabétique. Exclure les mouvements avancés (muscle-up) des suggestions automatiques.
 3. Corriger le classement de « FESSIER A LA MACHINE DE TRACTION » (extension de hanche, pas tirage vertical).
- Tests dans app/tests.js : profil P10 et programme avec « SOULEVE DE TERRE » → aucune ligne « Soulevé sumo » ; avec « SOULEVE DE TERRE SUMO » → la ligne sort ; _variantesSchema('tirage-vertical') ne contient ni FESSIER ni MUSCLE UP ; schemaDe('FESSIER A LA MACHINE DE TRACTION') ≠ 'tirage-vertical'. Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
- Fini quand : chaque ligne de la revue concerne réellement l'exercice affiché.

8. Tests d'amplitude unilatéraux et asymétries en double : il manque l'essentiel du bilan morpho — Priorité : Moyenne · Effort : M

Analyse et idée. La hanche est classée « butée précoce » dès que l'arrêt est noté « net », quel que soit l'angle, même à 135° (l.15772). Les « rotations » annoncées dans le titre ne sont jamais saisies, et cheville et hanche n'ont qu'un côté. Au banc, l'épaule G 6 cm / D 9 cm n'est pas signalée. Enfin, deux moteurs d'asymétrie coexistent avec des règles différentes : _morphoAsymetries (1 cm absolu, moyenne : « 1,5 cm ») et asymetries (3 % relatif, dernier bilan : « +1,6 cm »). Les deux s'affichent sur la même fiche, sans tenir compte de la main dominante, alors qu'un bras dominant plus gros de 1 à 2 cm est banal. L'idée : un seul moteur, des tests G/D, la latéralité, et la butée qui ne compte que sous un angle seuil.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. MORPHO_TESTS (l.15229) : cheville (un seul côté), hanche « flexion et rotations » (un seul angle de flexion, aucune rotation saisie), épaule G/D, flexion avant. morphoAxes A8 (l.15772) : f.hanche.position='haut' dès que butee==='nette', même à 135°. Deux moteurs d'asymétrie : _morphoAsymetries (l.15632, MORPHO_ASYNC_CM=1 cm absolu, moyenne de 3 bilans) et asymetries/asymetrieSite (l.86380, seuils relatifs ASYM_PAIRES, dernier bilan). Les deux s'affichent sur la fiche coach avec des chiffres différents (ex. 1,5 et 1,6 cm), et aucun ne connaît la main dominante.

À faire :

1. Un seul moteur : _morphoAsymetries délègue à asymetries() (seuils relatifs) et reprend son chiffre. Une seule phrase dans toute l'app.
 2. Latéralité : une question au questionnaire de départ (« Tu es droitier, gaucher ? », deux boutons, dans l'étape existante). Si le côté fort est le côté dominant, le seuil bras/avant-bras passe à 5 % et le signal reste informatif (pas d'entrée dans « Pourquoi cet athlète est ici »). Côté non dominant plus fort, ou membres inférieurs : seuil actuel.
 3. Tests d'amplitude G/D : cheville et hanche reçoivent les champs g et d (le format 'paire' existe déjà pour l'épaule). Les anciens relevés à une valeur restent lisibles (valeur = les deux côtés, marquée « côté non précisé »). Hanche : ajouter rotation interne assise (en degrés, au téléphone posé sur le tibia, à expliquer dans le protocole), pour G et D.
 4. Asymétrie fonctionnelle : écart G/D ≥ 4 cm au genou au mur, $\geq 10^\circ$ en rotation interne de hanche, ≥ 5 cm à l'épaule → ligne dans A8 « asymétrie de mobilité », profil P14 possible avec la source 'test'.
 5. A8 hanche : position 'haut' seulement si butée nette ET flexion $< 115^\circ$ (constante nommée, avec commentaire de source). Butée nette à 130° → neutre, texte « arrêt net en fin d'amplitude normale ».
- Tests dans app/tests.js : hanche {deg:135,butee:'nette'} → pas de P10 ; {deg:105,butee:'nette'} → P10 ; cheville g 12 / d 6 → asymétrie de mobilité ; une seule phrase d'asymétrie sur la fiche ; droitier avec bras droit +1,5 cm sur 3 bilans → pas de signal de priorité.
- Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les champs.

Finis quand : la fiche ne montre plus deux chiffres d'asymétrie, et une hanche normale n'est plus classée « butée précoce ».

9. Les leviers photo ne servent ni aux profils ni à l'éditeur de séance — Priorité : Moyenne · Effort : L

Analyse et idée. L'analyse photo calcule la cuisse, la jambe, le tronc, l'humérus et l'avant-bras, ainsi que l'angle de buste au squat, la prise au développé en cm et le levier au soulevé (`anatLeviers`, `anatSquatModele`, `anatDeveloppeModele`, l.60317-60533). Mais `morphoAxes` ne les lit jamais : A1, nommé « Fémur / tronc », est en fait entrejambe / taille. L'encart « Proportions » de l'éditeur d'exercice (`_htmlMorphoExercice`, l.16591) ne montre que les 4 réglages au ruban. Par ailleurs, le biais moteur n'est appris que pour le bras, l'avant-bras et la rotule (l.58956), pas pour la cuisse, alors que le point « hanche » de `MediaPipe` est le moins fiable. L'effet « waouh » serait de poser une consigne chiffrée par exercice : « Prise : index à 52 cm », « Squat : barre basse, cale 1,5 cm ».

Prompt de mise en place.

Contexte : `RepCore`, `app/rc-core.1792.js`. Deux moteurs morpho ne se parlent pas. (a) `morphoAxes` / `morphoProfils` (l.15692, l.16162) au ruban : A1 « Levier fémoral, Fémur / tronc » est calculé comme entrejambe / taille. (b) L'analyse photo `anat` (`anatMesures` l.59651, `anatLeviers` l.60533, `anatSquatModele`, `anatSouleveModele`, `anatDeveloppeModele`) mesure cuisse, jambe, tronc, humérus et avant-bras, et calcule l'angle de buste au squat, la prise au développé et le levier au soulevé, comparés à de Leva du même sexe. `morphoInitiale` (lot 8) gèle aussi fémur, tibia et tronc en cm, mais ces valeurs ne sont relues nulle part. L'encart de l'éditeur (`_htmlMorphoExercice` l.16591, `morphoPourExercice` l.16563) n'utilise que `morphoReglages`.

À faire :

1. `_morphoBrut` : quand `morphoAnat` porte des mesures de confiance A ou B (`anatConfiance`), alimenter une facette « photo » de A1 (cuisse / tronc, repère de Leva par sexe), de A2 (cuisse / jambe) et de A4 (humérus / avant-bras). Garder la règle « le mètre prime, la photo confirme ou suspend » (`MORPHO_DESACCORD`). Renommer A1 « Jambes / taille » tant qu'il n'a que le ruban.
2. Ajouter 'cuisse' à `ANAT_BIAIS_SEGMENTS`, apprise contre (deb-entrejambe – deb-rotule) quand les deux existent ; sans calibration, la marge de la cuisse prend le biais par défaut × 2 (le point hanche est le moins fiable).
3. `morphoPourExercice` : pour les schémas squat, charnière-hanche et poussée-horizontale, ajouter la consigne chiffrée issue des modèles `anat` (prise index à index et position des bagues ; barre haute ou basse et cale ; départ surélevé), avec sa source en une ligne. Un bouton existant (`anatEnvoyerConsigne`) la pose en `reglageCoach`. Rien d'autre ne descend vers l'élève (G7).
4. Aucune donnée photo → comportement actuel.

Tests dans `app/tests.js` : un dossier avec `morphoAnat` de confiance B et cuisse longue → A1 a une facette photo, et P1 peut sortir sans entrejambe ; un désaccord ruban/photo opposé → A1 suspendu ; `morphoPourExercice('DEVELOPPE COUCHE')` rend une prise en cm quand biacromial et bras existent ; aucune phrase osseuse dans le HTML élève.

Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Finis quand : le coach qui pose un développé couché voit la prise conseillée en cm pour CET athlète, avec sa source.

10. L'illustration « abducteurs » montre les adducteurs — Priorité : Basse · Effort : S

Analyse et idée. `app/img/muscles/abducteurs.webp` colore l'intérieur des cuisses, comme `adducteurs.webp`. Les abducteurs (moyen fessier, petit fessier, TFL) sont pourtant sur la face latérale de la hanche. La vignette est affichée par l'écran Volume (`_volIllus`, l.86743) : l'élève qui fait de l'abduction voit le mauvais muscle travailler. Il faut refaire la vignette sur la même planche (vue postéro-latérale, moyen fessier et TFL colorés).

Prompt de mise en place.

Contexte : RepCore. app/img/muscles/abducteurs.webp (128×128) colore la face interne des cuisses, c'est-à-dire les adducteurs, comme app/img/muscles/adducteurs.webp. Les abducteurs de hanche sont le moyen fessier, le petit fessier et le tenseur du fascia lata (face latérale et postéro-latérale de la hanche). L'image est servie par _volIllus (app/rc-core.1792.js l.86743, ensemble VOL_ILLUS).
 À faire : produire une nouvelle abducteurs.webp dans le même style que fessiers.webp (planche anatomique grise, muscle teinté), en vue postéro-latérale ou de 3/4 arrière, moyen fessier et TFL colorés, le grand fessier en gris. Même taille (128×128), même compression, même teinte que les autres vignettes. Ne pas toucher aux autres images.
 Test dans app/tests.js : l'image existe, fait 128×128, et son hash diffère de celui d'adducteurs.webp (garde-fou contre une copie).
 Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py pour versionner l'actif). Ne pas agrandir les polices.
 Fini quand : la carte Volume « Abducteurs » montre la face latérale de la hanche.

Expert 3 · Physiologie de l'entraînement — 6/10

Le socle a nettement progressé depuis le 29/09 : e1RM juste (Epley sur reps + RIR, chargePourReps), un seul modèle charge-répétitions, types de charge (poids du corps, lesté, assisté), records par répétitions, cycle qui ne touche plus le volume, décharge qui baisse réellement le volume dans la grille. Ce qui manque le plus : la périodisation du coach n'arrive pas dans la séance de l'athlète, la charge suggérée pousse toujours vers l'échec, et le détecteur de plateau voit des « reculs » dans une double progression réussie.

1. La périodisation et la décharge du bloc n'arrivent jamais dans la séance — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc : un bloc de 6 semaines, décharge posée sur la semaine en cours (appliquerDecharge). getSemaineEffective annonce 2 séries à RIR 3, mais launchWorkout reçoit sessions_config[idx] brut (rc-core.1792.js:47334, :48326) : la séance affiche 4 séries, « visé : 2 », avec seulement le bandeau DÉCHARGE. Les écarts de semaine (+1 série, RIR qui descend) restent donc décoratifs. Autres problèmes : la décharge par créneau (case du coach, ou bouton « Alléger la semaine » de l'athlète, :14294) ne baisse rien, ne s'efface jamais, et affiche « voulue par ton coach » même quand c'est l'athlète qui l'a demandée. Enfin, getPrevPerf (:70358) prend une séance de décharge comme référence. L'idée : une seule fonction « séance du jour » qui applique la semaine effective, et une décharge datée qui s'efface seule.

Prompt de mise en place.

Contexte : dépôt RepCore, fichier app/rc-core.1792.js (~130 000 lignes : grep -n puis sed -n, ne pas tout lire). Le coach périodise via programme.ecarts (poserEcartSemaine, _creneauModule ~71280) et la décharge de bloc (appliquerDecharge ~14262 : facteurSeries 0,6 et RIR +1). getSemaineEffective (~71237) calcule bien la semaine, mais l'athlète lance sa séance avec le gabarit brut : _aperçuOuSeance (~47331) et commencerDepuisAperçu (~48313) passent currentUser.sessions_config[idx] à demarrerSeance → launchWorkout (~48470). Constat au banc : semaine de décharge = 2 séries RIR 3 dans getSemaineEffective, 4 séries RIR 2 dans woState.exercices.

À faire :

1. Crée seanceDuJour(user, idx, date) PURE : si getSemaineEffective(user,date) rend une semaine, renvoie une COPIE de creneaux[idx] (séries, rir, exercices de l'écart), sinon une copie de sessions_config[idx]. Utilise-la dans _aperçuOuSeance, commencerDepuisAperçu, l'aperçu (_renderAperçu) et la fiche imprimée, pour que l'aperçu, la séance et la grille coach disent la même chose. Ne modifie jamais sessions_config.
2. Décharge par créneau (branche « sans bloc » d'appliquerDecharge et coachToggleDeload ~39008) : remplace le booléen permanent par deloadJusqu'à dimanche 23:59 de la semaine (ms). launchWorkout applique alors à la copie de travail les mêmes règles que le bloc (séries × DECHARGE_FACTEUR_SERIES arrondi, min 1 ; RIR prescrit + DECHARGE_RIR_PLUS, plafonné à 5). Migration en lecture : un ancien deload:true sans date vaut jusqu'à la fin de la semaine où il est lu pour la première fois, puis s'efface. La case du coach reste décochable à tout moment.

3. Bandeau (~51622) : « Semaine allégée voulue par ton coach » seulement si le coach l'a posée ; « Tu as allégé ta semaine » si c'est l'athlète (garde l'origine dans le champ). Dans le journal, garde `_journalSeance('dispo_decharge')`.

4. `getPrevPerf` (~70358), `prevSeries`, `_prevSeriePoidsCorps` : saute les séances `sess.deload===true`. La suggestion après une décharge repart de la dernière séance normale (avec la décote de reprise si elle a plus de 28 jours).

5. Pendant une séance de décharge, la charge suggérée ne monte pas : on prend `min(charge de référence, suggestion)`.

Erreurs humaines à couvrir : le coach coche puis décoche dans la même minute (aucun effet résiduel) ; l'athlète touche deux fois « Alléger la semaine » (une seule décharge) ; un bloc expiré (`indexSemaineBloc` null) retombe sur le gabarit ; un créneau inactif dans l'écart.

Tests à ajouter dans `app/tests.js` (lancés par `chargerTests()`) : semaine de décharge de bloc → `seanceDuJour` donne `séries = round(4×0,6) = 2` et RIR 3 ; écart +1 série en S2 → 5 séries le jour J ; `deloadJusqua` passé → séance normale ; `getPrevPerf` ignore une séance `deload` ; ancien `deload:true` migré et effacé la semaine suivante.

Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` de `app/sw.js`), puis `python3 scripts/versionner_actifs.py`. Aucune police ni cible tactile agrandie.

Critère de fin : `chargerTests()` vert ; au lancement d'une semaine de décharge, l'écran de séance montre le nombre de séries et le RIR de la grille du coach.

2. La charge suggérée ignore les répétitions et le RIR visé, et l'arrondi des haltères fait exploser les sauts — Priorité : Critique · Effort : M

Analyse et idée. Le point 2 du premier audit n'a été traité qu'en partie : `chargeSuivante` (:70259) applique encore le seul multiplicateur du RIR fait (`SUG_MULTIPLICATEURS`), plafonné en kilos. Testé au banc, avec une consigne de 4 × 8-10 à RIR 2 : - semaine passée en haltères, 20 kg × 7, 6, 5, 5 à RIR 1 (sous la fourchette) : l'app propose 22 kg ; - dans la séance, séries faites à RIR 3 : 22 → 24 → 28 → 32 kg ; - élévations latérales : 8 → 10 → 12 → 14 kg, car l'arrondi « vers le haut » au pas de 2 kg (`arrondiCharge`, :69850) transforme +2,5 % en +25 % ; - RIR non saisi : multiplicateur 1, donc la charge ne monte jamais, même en haut de fourchette.

L'idée : garder la grille de Kevin, mais l'appliquer à l'écart entre le RIR fait et le RIR visé, et seulement quand les répétitions sont au rendez-vous (double progression).

Recoupement. Point 2.2 du premier audit, encore ouvert selon l'expert 17 : ce prompt le remplace.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `chargeSuivante(charge,rir,contrepoids,decote,exNom)` (~70259) multiplie la dernière charge par `multiplicateurRir(rir)` (`SUG_MULTIPLICATEURS` ~70072, « la formule du coach » : on la GARDE) puis arrondit. Elle est appelée pour la 1re série (`_blocExo` ~51931, à partir de `getPrevPerf`) et d'une série à l'autre (`renderSets` ~52974), et en ~129625. Ni les répétitions faites, ni la fourchette (`fourchetteReps`), ni le RIR prescrit (`_rirPrescrit` ~68617) n'entrent. Mesures au banc : 20 kg haltères ×7,6,5,5 @RIR1 pour 4×8-10 @RIR2 → 22 kg ; série à série @RIR3 : 22→24→28→32 ; élévations 8→10→12→14 ; RIR vide → jamais de hausse.

À faire, en une fonction `PURE progressionCharge({charge, repsFaites, rirFait, reps (texte de consigne), rirCible, contrepoids, decote, ex, user}) → {kg, repsVisees, raison}` :

- Écart d'intensité : $d = \text{rirFait} - \text{rirCible}$ (`rirCible = _rirPrescrit(ex)`, sinon 2 ; 'echec' = 0). On applique la grille de Kevin à d ($d \leq 0 \rightarrow \times 1$, $d = 1 \rightarrow \times 1,025\ldots$; échec alors que la cible est $\geq 1 \rightarrow \times 0,95$). Un RIR fait égal au RIR visé ne fait donc plus monter la charge.
- Double progression : avec une fourchette a-b, pas de hausse tant que `repsFaites < b` à `rirFait ≥ rirCible`. On monte quand toutes les séries de travail atteignent b, et on vise alors a. Sous a (ex. 5 reps pour 8-10), on garde la charge (ou on baisse si `rirFait < rirCible`) et la raison dit « vise 8 reps ». RIR non saisi : la règle des répétitions suffit (b atteint partout → +1 pas).
- Arrondi : au pas du matériel (`arrondiSuggestion`), mais une hausse plus petite que la moitié du pas ne fait pas monter la charge : on garde la charge, et `repsVisees = reps faites + 1`. Plafond relatif en plus du plafond en kg : jamais plus de +10 % d'une séance à l'autre, ni de +5 % d'une série à la suivante.
- Série à série (`renderSets` ~52950) : la série suivante ne monte que si la série précédente a été faite au haut de la fourchette avec $d \geq 2$. Sinon même charge, ou -1 pas si `rirFait < rirCible - 1`.
- Contrepoids : même logique, inversée (on baisse l'assistance). Poids du corps sans lest : `repsSuivantes` garde son rôle, borné au haut de la fourchette ; au-delà, la raison propose « ajoute un

test ou une variante plus dure ».

6. Le texte sous la suggestion (~52027-52040) affiche raison : « 20 kg × 7-6-5-5 à RIR 1, sous ta fourchette 8-10 → on garde 20 kg, vise 8 ».

Garde decote, cycle, consigne coach (_cons.kg) et programmation % 1RM (s.rpeCible) inchangés.

Tests app/tests.js (chargerTests()) : 20×(7,6,5,5)@1, 8-10@2 → 20 ; 60×(10,10,10)@2, 8-10@2 → 62,5 (barre) ; 8 kg haltères ×15@3, 12-15@2 → 8 kg + repsVisees ou 10 seulement si 15 partout ; série à série 4×10@3 avec cible 3 → charge constante ; RIR vide et 10/10/10 pour 8-10 → hausse d'un pas ; aucune hausse > 10 % en 1 000 cas aléatoires. Adapte les tests de chargeSuivante (~22615-22680) : multiplicateurRir reste testé tel quel.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : suite verte ; au banc, le scénario 20 kg ×7-6-5-5 propose 20 kg et dit pourquoi.

3. Le détecteur de plateau voit un recul dans une double progression réussie — Priorité : Critique · Effort : S

Analyse et idée. perfExercice (:54637) passe de l'e1RM au « volume de série » (charge × reps) dès que reps + RIR dépasse 12. Le cas courant en hypertrophie (10-15 reps à RIR 2-3) bascule donc d'une métrique à l'autre. Testé au banc : - 60 kg × 10 → 15 puis 65 kg × 10 → 13, à RIR 3 : l'e1RM réel passe de 96 à 100 (nouveau meilleur), mais _calculEtat rend « regression » (845 < 0,95 × 900). Le motif « Recul » de scoreFatigue en tire +25 points. - 10-12 reps à RIR 2 : la série est coupée à chaque changement de métrique, et l'état reste « insuffisant » pour toujours. - Une série lourde suivie de séries allégées (top set + back-off) : le score prend le max de deux unités différentes (700 contre 120). - Sans RIR saisi : 100 × 8 → 100 × 10 rend « ralentissement », car maxGlobal vaut -Infinity.

L'idée : une seule métrique, l'e1RM, en marquant la fiabilité au lieu de changer d'unité.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. perfExercice (~54637) choisit par série 'e1RM' si e1rmFiable(r,rir) (reps+RIR ≤ PERF_REPS_MAX_E1RM=12), sinon 'volume-serie' = w×r, puis prend le MAX des deux unités ; _calculEtat (~54716) coupe la série à chaque changement de métrique. Mesures au banc :

(a) 3×(60×10,12,14,15) puis 65×10,11,12,13 @RIR3 → 'regression' alors que l'e1RM passe de 96 à 100 ;
(b) 60×10/11/12 @RIR2 → 'insuffisant' en permanence (métrique qui alterne) ;
(c) 100×5@1 + 70×10@3 → score 700 'volume-serie' une semaine, 120 'e1RM' la suivante ;
(d) RIR vides, 100×8→100×10 → 'ralentissement' (maxGlobal = -Infinity car toutes les séances sont sansRirDominant).

À faire :

1. perfExercice : score = max des e1rm(w, r, rir) sur les séries faites (chargeEffective, reps > 0), toujours en kg. Ajoute fiable = (meilleure série) e1rmFiable(r,rir) et plafonne les reps potentielles à 20 dans le calcul (au-delà, la série compte avec 20). metrique vaut toujours 'e1RM' ; garde le champ pour compatibilité ; « volume-serie » ne sert plus qu'à l'affichage si besoin.

2. _calculEtat : plus de coupure de série par métrique. Une séance « non fiable » compte pour n et T, mais ne peut fixer le record que s'il n'existe aucune séance fiable dans la fenêtre.

3. Quand toutes les séances sont sansRirDominant, on ne rend plus -Infinity : on compare les e1RM calculés avec RIR 0 (borne basse cohérente d'une séance à l'autre), et le résultat porte sansRir:true pour que l'écran dise « estimation sans RIR ».

4. Vérifie les consommateurs : scoreFatigue (~14123, motif 'regression'), _listeEtats, musclesEnPlateau, recordAPorteeExo, l'écran plateaux.

Tests app/tests.js (chargerTests()) : scénarios (a) → 'progression' ; (b) → verdict rendu (pas 'insuffisant') après 4 séances sur 21 jours ; (c) → score = e1RM de la série lourde ; (d) → 'progression' ; une vraie baisse (100×8@2 → 90×8@2 répété) reste 'regression'.

Convention : nouveau build (RC_BUILD dans app/index.html, CACHE dans app/sw.js, python3 scripts/versionner_actifs.py). Pas de police agrandie.

Critère de fin : suite verte et plus aucune occurrence de comparaison entre un e1RM et un w×r.

4. Douleur, blessure et grossesse ne freinent pas la progression proposée — Priorité : Haute · Effort : S

Analyse et idée. La carte « Ça t'a fait mal deux fois » (:53330) agit dans la séance (−20 %), mais la séance suivante repart de `getPrevPerf` × multiplicateur. Exemple : une série à 100 kg à RIR 3 avec une gêne de 5 donne 107,5 kg la fois suivante. Une contrainte déclarée (`NIVEAUX_CONTRAINTE`) ne bloque pas non plus la hausse. Pour une athlète enceinte, `grossesseSuspend` coupe la nutrition et la fatigue (:14079), mais ni la suggestion ni `recordAPortee` (:88180) : on lui propose encore de battre des records. L'idée : un « frein de progression » unique, expliqué en une ligne.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. La suggestion de charge (_blocExo ~51905-51960, chargeSuivante ou sa remplaçante) et le record à portée (recordAPortee ~88180, recordAPorteeExo ~88129) ne lisent ni la gêne de la séance précédente (s.pain, échelle 1-6, DLR_SEANCE_SEUIL=4 ~53330), ni le choix « J'allège » (woState.douleurChoix), ni les contraintes actives (NIVEAUX_CONTRAINTE / NIVEAUX_AVERTISSANTS ~76505-76543), ni la grossesse (grossesseSuspend ~90666 : 'enceinte' | 'allaitement' ; etatGrossesse pour le post-partum).  
Crée freinProgression(user, exNom, prev) PURE → null | {niveau:'maintien'|'baisse', rirMin, raison} :  
- gêne ≥ 4 sur une série de cet exercice lors de la dernière séance, ou « J'allège » choisi → 'maintien' (pas de hausse) et raison « Gêne la dernière fois : on garde la charge » ; gêne ≥ 4 sur deux séances de suite → 'baisse' (−10 %) ;  
- contrainte avertissante sur la zone de l'exercice (réutilise le rapprochement zone ↔ exercice déjà fait par _blocContrainteSeance) → 'maintien' ;  
- grossesse ou allaitement → 'maintien' sur tout, rirMin 2, pas de record à portée, pas de célébration de record de charge ; le post-partum garde les règles normales mais affiche une fois « Reprise post-partum : valide avec ton coach ou ta sage-femme » ;  
- drapeau rouge actif (drapeauQuelconqueActif) → rien n'est proposé (comportement actuel de l'écran d'arrêt).  
Applique-le dans _blocExo (après la décote, avant l'arrondi), dans le pré-remplissage série à série (renderSets ~52950) et dans recordAPortee (renvoie null si un frein est actif). La raison s'affiche sous la suggestion, sur une ligne, sans nouvel encart.  
Erreurs humaines : gêne saisie par erreur puis corrigée dans la séance (on lit la valeur finale) ; exercice remplacé en cours de séance (clé exKey) ; grossesse déclarée puis retirée (le frein disparaît immédiatement).  
Tests app/tests.js (chargerTests()) : 100 kg@RIR3 avec pain '5' → suggestion 100 ; deux séances avec pain ≥ 4 → 90 ; grossesse 'enceinte' → recordAPortee null et aucune hausse ; post-partum → hausse normale.  
Convention : nouveau build (RC_BUILD dans app/index.html, CACHE dans app/sw.js, python3 scripts/versionner_actifs.py). Pas de police agrandie.  
Critère de fin : suite verte ; au banc, une gêne de 5 notée la semaine passée affiche « on garde la charge ».
```

5. Trois indices de récupération qui se contredisent, et le sommeil, la VFC et le cycle sans effet sur la charge du jour — Priorité : Haute · Effort : M

Analyse et idée. Trois calculs indépendants coexistent : - `disponibilite` (:73732) : sommeil comparé à la médiane personnelle, douleur, écart de RIR, ratio de charge (ACWR) ; - `scoreRecuperation` (:116957) : sommeil sous un seuil absolu, stress, FC de repos et VFC ; - `scoreFatigue` (:14123), qui déclenche la décharge : il ne lit ni le sommeil ni la VFC.

Le coach et l'athlète peuvent donc recevoir trois verdicts différents le même jour. Et l'état orange se contente de retirer une série, alors que la règle « Règles, c'est dur » baisse la charge de 20 % (`getCycleFactor`, :47497). L'idée : un seul feu de récupération, qui règle le RIR visé du jour (autorégulation à la Helms et Zourdos) au lieu d'amputer arbitrairement.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. Trois indices : disponibilite(user,dateISO) (~73732, DISPO_POIDS sommeil 40 / douleur 25 / rir 20 / charge 15, sommeil contre la médiane perso),
```

scoreRecuperation(user) (~116957 : forme, sommeil < MICRO_SOMMEIL_SEUIL absolu, stress du bilan, volume > MRV, FC repos/VFC via recupCardio), scoreFatigue(user) (~14123, FATIGUE_POIDS, qui déclenche evaluerPropositionDecharge). Le cycle déclaré applique ×0,80 ou ×0,95 sur la charge (getCycleFactor ~47497).

À faire, sans ajouter aucune saisie pour l'athlète :

1. Fais de disponibilite() la seule source de l'état du jour : ajoute-lui une entrée 'cardio' (recupCardio : FC de repos haute ou VFC basse contre sa propre base de 28 j ; poids 15, les autres sont renormalisés) et le cycle déclaré jl_difficile comme entrée de ressenti. scoreRecuperation lit le sommeil par _dispoSommeil (médiane perso) : un seul jugement du sommeil dans l'app.
2. Effet sur la séance : 'orange' → rirCible + 1 pour toutes les suggestions du jour (via _rirPrescrit du jour ou un paramètre de la fonction de progression) et pas de record à portée. 'rouge' → la proposition actuelle (alléger la semaine) est conservée. Le cycle déclaré « c'est dur » passe de ×0,80 à « RIR +1 et charge gardée » (même mécanisme que l'orange), « ça va » ne change plus rien ; garde l'option d'en faire une préférence coach.
3. scoreFatigue ajoute un motif 'recuperation' quand disponibilite a été orange ou rouge ≥ 3 jours sur 7 (poids 15), pour que la proposition de décharge et le feu du jour racontent la même histoire.
4. Fiche coach : une ligne unique « Récupération : orange depuis 3 jours (sommeil -25 % sous sa médiane) » au lieu de trois blocs ; supprime l'affichage redondant de _htmlRecuperationCoach si la ligne le couvre.

Erreurs humaines : nuit non synchronisée (entrée absente, poids redistribué, jamais 0) ; montre portée une seule nuit ; sommeil saisi deux fois le même jour (on garde la dernière saisie).

Tests app/tests.js (chargerTests()) : deux nuits à -30 % → orange et rirCible+1 sur la suggestion ; VFC -20 % sous la base sur 3 mesures → entrée cardio < 100 ; jl_difficile → charge inchangée et RIR +1 ; 3 jours orange → motif 'recuperation' dans scoreFatigue.

Convention : nouveau build (RC_BUILD dans app/index.html, CACHE dans app/sw.js, python3 scripts/versionner_actifs.py). Pas de police agrandie, pas de nouvel écran.

Critère de fin : suite verte ; un seul verdict de récupération par jour, visible pareil chez l'athlète et chez le coach.

6. Aucun profil d'entraînement : le débutant de 16 ans et le senior de 65 ans ont les mêmes seuils — Priorité : Moyenne · Effort : M

Analyse et idée. Aucune variable d'ancienneté ou d'âge n'entre dans le moteur. On trouve MIN_JOURS = 21 et JOURS_PLATEAU = 28 (:54239), une seule table REPERES_VOLUME (:34558), les mêmes multiplicateurs et la même décote de reprise. Un débutant qui stagne 10 jours en progression linéaire n'est pas signalé. Un avancé qui bat un record par mois est déclaré « en plateau ». Un senior entraîné repart à -10 % après 8 semaines. Un ado de 16 ans (AGE_MINIMUM, :12620) peut viser l'échec et les records. L'idée : un niveau déduit automatiquement, que le coach peut corriger d'un toucher.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Constantes globales : MIN_SEANCES/MIN_JOURS/JOURS_PLATEAU (~54239), REPERES_VOLUME (~34558, ajusté par athlète via la boucle de retour par muscle), SUG_DECOTES/decoteReprise (~70212), le plafond du pas (PAS_PLAFOND_KG ~70245). Aucune ne dépend de l'âge (_ageUtilisateur) ni de l'ancienneté.

Crée profilEntrainement(user) PURE → {niveau:'debutant'|'intermediaire'|'avance', age, jeune:boolean, senior:boolean, source:'auto'|'coach'} :

- niveau auto : moins de 6 mois d'historique de séances, OU e1RM squat/développé couché/soulevé sous des ratios de poids de corps simples (par sexe : par ex. DC < 0,8×PDC homme, < 0,5×PDC femme) → débutant ; plus de 3 ans d'historique ET ratios élevés → avancé ; sinon intermédiaire.

user.niveauEntrainement posé par le coach l'emporte (source 'coach'). Pas de question nouvelle à l'athlète.

- jeune = âge < 18 ; senior = âge ≥ 60.

Branche-le :

1. Plateau : JOURS_PLATEAU 14 / 28 / 49 et MIN_JOURS 14 / 21 / 28 selon le niveau.
2. Progression : débutant → la double progression peut monter dès que le haut de fourchette est atteint sur la DERNIÈRE série ; avancé → plafond relatif +5 % par séance.
3. Volume : repères ×0,8 pour débutant et senior au départ (la boucle de retour existante les ajuste ensuite) ; jamais sous le MEV de la table pour les muscles prioritaires.
4. Reprise : senior → décote -10 % dès 21 jours et -20 % dès 42 jours.
5. Jeune : rirMin 1 (pas d'échec proposé), pas de record à portée sur les mouvements avec une charge > 1×PDC, texte neutre.
6. Fiche coach : une ligne « Profil : intermédiaire (auto) · modifier » qui ouvre un choix à 3

boutons.

Tests app/tests.js (chargerTests()) : historique de 2 mois → débutant ; coach pose 'avance' → avancé ; âge 16 → jeune et pas d'échec proposé ; senior → decoteReprise(30) = 0,9 ; avancé avec record il y a 35 jours → pas 'plateau'.

Convention : nouveau build (RC_BUILD dans app/index.html, CACHE dans app/sw.js, python3 scripts/versionner_actifs.py). Pas de police agrandie.

Critère de fin : suite verte ; les seuils lus par _calculEtat et la suggestion viennent tous de profilEntrainement.

7. Effet waouh : « pourquoi cette charge » et la prédiction de fin de bloc — Priorité : Moyenne · Effort : M

Analyse et idée. L'app calcule déjà la pente d'e1RM (tendanceLineaire, :88118), le volume par muscle et le RIR, mais elle ne relie jamais ces trois mesures pour le coach. La suggestion dit seulement « Basée sur ton dernier mardi (20 kg à RIR 1) ». Testé au banc : la phrase ne mentionne ni les répétitions ni la cible, et le record à portée calcule une troisième charge de son côté. L'idée : - côté athlète, une phrase de décision ; - côté coach, une carte par exercice clé : tendance, date projetée d'un objectif et réponse au volume, comme chez RP, mais lisible.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Briques existantes : tendanceLineaire (~88118), _rapElrm, recordAPorteeExo (~88129), etatExercice (~54706), le volume hebdomadaire par muscle (serieDure ~70556 et le cache de volume), REPERES_VOLUME. Le texte de suggestion est dans _blocExo (~52027-52045).

À faire :

1. Athlète, sous la charge proposée (même taille de police qu'aujourd'hui, une ligne) : la raison rendue par la fonction de progression (voir le lot double progression), par ex. « 60 × 12-12-11 à RIR 2 → +2,5 kg, vise 10 ». Le record à portée doit utiliser la MÊME fonction (plus de troisième calcul) ; s'ils divergent, on n'affiche que la suggestion.

2. Coach, fiche athlète, onglet Progression : pour les 3 exercices principaux (les plus fréquents en primaire), une carte compacte : e1RM des 6 dernières semaines (sparkline existante si elle existe, sinon chiffres), pente en kg/semaine, « au rythme actuel : 100 kg × 5 vers le 14 décembre » (projection linéaire bornée à 12 semaines, cachée si la pente ≤ 0 ou si moins de 4 points), et le volume hebdomadaire moyen du muscle sur la même période avec sa zone MEV/MAV/MRV.

3. Réponse au volume (« dose-réponse ») : quand on a ≥ 8 semaines, compare la pente d'e1RM des semaines à volume haut et à volume bas du muscle et écris une phrase prudente : « Ses pectoraux ont mieux progressé autour de 14 séries que de 20 » (seulement si l'écart de pente dépasse 2× le bruit, sinon rien).

Rien de nouveau à saisir ; rien ne s'affiche quand les données manquent.

Tests app/tests.js (chargerTests()) : projection correcte sur une série linéaire synthétique ; aucune projection avec une pente négative ; la phrase de dose-réponse n'apparaît pas sur des données plates ; la raison de la suggestion contient les reps faites et la cible.

Convention : nouveau build (RC_BUILD dans app/index.html, CACHE dans app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ; réutiliser les composants de carte existants.

Critère de fin : suite verte ; au banc, la fiche coach de Léa montre une carte de projection pour SQUAT.

8. Séries d'approche : des singles pour les séries lourdes, et une rampe complète sur chaque exercice — Priorité : Basse · Effort : S

Analyse et idée. Testé au banc : seriesApproche(200, '1') (soulevé de terre) donne 80 × 1, 120 × 1, 150 × 1, 180 × 1, à cause de la règle « jamais plus que R » (:70139). Pour 140 kg × 3 au squat, on obtient 3 répétitions dès 40 %. La pratique va dans l'autre sens : 40 % × 5, 60 % × 3, 75 % × 2, 85-90 % × 1. Autre défaut : la rampe complète s'affiche aussi sur le 3e exercice d'un muscle déjà chaud, ou sur des élévations à 10 kg. L'idée : un plancher de répétitions sur les premiers paliers, et une seule série « d'acclimatation » quand le muscle a déjà travaillé dans la séance.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `seriesApproche(chargeTravail, repsPrescrites, ex)` (~70139), `repsApproche(pct, R)` (~70195) : plafond 10/6/4/2 selon le palier, et jamais plus que R. `_htmlMonteeCharge` (~70170) est appelée pour chaque exercice (~52051).

À faire :

1. `repsApproche` : `reps = clamp(plafond(pct), min = plancher(pct))`, où le plancher vaut 5 jusqu'à 50 %, 3 jusqu'à 65 %, 2 jusqu'à 80 %, 1 au-delà ; R ne borne plus que le dernier palier ($\leq R$). Résultat attendu : 200 kg $\times$ 1 $\rightarrow$ 80 $\times$ 5, 120 $\times$ 3, 150 $\times$ 2, 180 $\times$ 1 ; 100 kg $\times$ 12 $\rightarrow$ 50 $\times$ 10, 70 $\times$ 6, 85 $\times$ 2 (inchangé).
2. Muscle déjà échauffé : si un exercice précédent de la séance a le même muscle primaire (`resoudreMusclesLecture`) avec au moins une série faite, un seul palier à $\sim 70 \% \times \min(R, 5)$. Isolation (`schemaDe 'isolation*'`) sous 15 kg : un seul palier à 50 %.
3. Rien ne change dans les données : la rampe reste un pense-bête (aucune série enregistrée).

Tests `app/tests.js` (`chargerTests()`) : les deux exemples du point 1 ; 2e exercice de pectoraux après le développé $\rightarrow$ 1 palier ; contrepoids et isométrie $\rightarrow$ [] (inchangé).

Convention : nouveau build (RC_BUILD dans `app/index.html`, CACHE dans `app/sw.js`, python3 `scripts/versionner_actifs.py`). Pas de police agrandie.

Critère de fin : suite verte.

9. Reprise après plus de 16 semaines et poids du corps historique : deux trous d'estimation — Priorité : Basse · Effort : S

Analyse et idée. Au-delà de 112 jours, `decoteReprise` rend null (:70212) : l'athlète lit « reprends à une charge que tu tiens facilement pour 10 » et ne reçoit aucun chiffre, alors que son ancien e1RM est connu. Par ailleurs, `chargeEffective` applique le poids du corps ACTUEL à tout l'historique (:69998, `poidsCorpsActuel`). Après une sèche de 10 kg, toutes les anciennes tractions sont recalculées avec le nouveau poids, et la progression réelle disparaît. L'idée : proposer un chiffre de départ prudent, et lire le poids du corps à la date de la séance.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `decoteReprise` (~70212 ; `SUG_JOURS_ABANDON=112`) rend null au-delà, et `_blocExo` (~52027) affiche un texte sans charge. `chargeEffective(set, ex, user)` (~69998) appelle `poidsCorpsActuel(user)` (~85180) pour toutes les séances, quelle que soit leur date.

À faire :

1. Au-delà de 112 jours : charge de départ = `chargePourReps(0,7  $\times$  meilleur e1RM des 3 dernières séances fiables de l'exercice, bas de fourchette, RIR 3)`, arrondi vers le bas au pas du matériel, avec la raison « Reprise après N mois : charge de départ prudente, ajuste à la 2e série ». Sans e1RM connu, garde le texte actuel.
2. Crée `poidsCorpsAu(user, dateMs)` : la pesée (`weightLog`, puis bilans) la plus proche dans les ± 21 jours, sinon `poidsCorpsActuel`. `chargeEffective` reçoit la date de la séance (paramètre optionnel ; sans date, comportement actuel) ; `perfExercice`, `recordsExercice` et `e1rmRecordsDeSeance` la passent.

Tests `app/tests.js` (`chargerTests()`) : 150 jours sans squat, ancien e1RM 120 $\rightarrow$ départ $\approx 0,7 \times 120$ ramené à 8 reps @RIR3, arrondi bas ; tractions à 80 kg de PDC en janvier puis 70 kg en mars, mêmes reps $\rightarrow$ e1RM de janvier calculé sur 80 kg.

Convention : nouveau build (RC_BUILD dans `app/index.html`, CACHE dans `app/sw.js`, python3 `scripts/versionner_actifs.py`). Pas de police agrandie.

Critère de fin : suite verte.

Expert 4 · Macronutrition — 6,5/10

Ce qui est déjà très bon : la chaîne métabolisme de base $\rightarrow$ NAF $\rightarrow$ sport en MET nets $\rightarrow$ coefficient est juste, au kcal près, sur mes 4 profils de poids normal. Les corrections du premier audit sont en place : l'objectif de l'athlète agit, le plancher est remis côté athlète, Mifflin s'applique dès un IMC de 30, l'alerte NAF/métier existe et la sèche demande une confirmation. Ce qui manque le plus : l'athlète autonome a deux réglages qui se contredisent (« Ta phase » et « Mon objectif »). Le cycle ON/OFF se dérègle à 6 séances. Et le calcul « au poids du corps » donne des cibles intenables dès l'obésité.

Recalcul fait en exécutant les vraies fonctions dans la page (`cibleTableur`, `ciblesAthlete`, `besoinsProposes`, `plancherEffectif`, `_tbJournees`, build 1792), plancher appliqué côté athlète :

Profil	MB	NAF	Sport/j	Dépense	Cible	P / L / G (g)	Verdict
A. H 80 kg, 180 cm, 30 ans, bureau, 4 × 60 min, sèche 0,85	1 858 (Harris)	1,2	183	2 413	2 051	192 / 64 / 177	juste (2,4 g/kg = 192 g, 37 % des kcal)
B. H 120 kg, 175 cm, 40 ans (IMC 39), 3 × 60, sèche 0,85	2 099 (Mifflin)	1,2	206	2 725	2 316 (= plancher)	288 / 96 / 75	50 % des kcal en protéines, glucides très bas
B'. même profil avec tours de taille et de cou (MM 77,8 kg)	2 050 (Katch)	1,2	206	2 666	2 266	288 / 96 / 63	3,7 g/kg de masse maigre
C. F 50 kg, 155 cm, 25 ans, 5 × 75 min + 4 h de course rapide, sèche 0,85	1 297	1,2	479	2 035	1 730	120 / 40 / 223	juste ; à 0,70 (coach) : 1 425 kcal acceptés sans alerte, DE ≈ 26 kcal/kg MM
D. F 60 kg, 165 cm, 25 ans, 3 ×, prise de masse 1,10	1 411	1,2	103	1 796	1 976	132 / 48 / 254	juste
E. F 95 kg, 160 cm, 45 ans (IMC 37), tours mesurés (MM 44,5 kg), sèche	1 331 (Katch)	1,2	163	1 760	1 496	228 / 76 / 0	61 % des kcal en protéines, 0 g de glucides, total servi +100 kcal

Conclusion sur le dernier commit (« calcul sur le poids du corps ») : il est juste pour A, C et D (IMC < 30), et faux pour B et E. Pour B et E, c'est la règle retirée le 05/10 qui donnait le bon ordre de grandeur.

1. Athlète autonome : « Ta phase » et « Mon objectif » se contredisent, et la sèche devient inerte — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc (athlète sans coach) avec une phase « Prise de masse » choisie sur l'accueil : un clic sur « Sèche » dans Nutrition ne change rien (2 527 kcal avant comme après), parce que `cibleTableur` donne la priorité à `phaseCourante` (rc-core:102165). Dans l'autre sens, un objectif « Sèche » posé sans phase ne déclenche jamais l'ajustement automatique (`ajustementPropose : if(!p) return null`, l.116477). Si l'athlète accepte un ajustement, `origine:'ajustement'` verrouille sa carte sur « Le total vient de ton coach », alors qu'il n'a pas de coach. La carte recalcule à chaque pesée (1 957 kcal) pendant que les anneaux gardent la cible enregistrée (2 100). Enfin, l'accueil affirme « Ça ne change ni tes séances ni tes macros » (l.119323), ce qui est faux. Idée : un seul réglage pour l'athlète sans coach. Choisir l'objectif pose aussi la phase, une seule écriture met la carte et les anneaux à jour, et on n'affiche aucune mention « coach » sans coach.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js (~130 k lignes : grep -n puis sed -n, ne lis pas tout). Chez un athlète SANS coach (pas de coachEmailKey ni de coachId), deux réglages coexistent :
- user.phase, posée par la carte « Ta phase » de l'accueil (confirmerPhase → changerPhase, ~l.115581 et 119341) ;
- nutrition.perso.objectif, posé par « Mon objectif » dans Nutrition (athObjectif, ~l.91639).
cibleTableur (~l.102165) prend phaseCourante(u) en priorité. Résultat mesuré au banc : avec une phase « masse », le bouton « Sèche » ne change rien ; avec un objectif seul, ajustementPropose (~l.116467) et le point de la semaine rendent null (aucune phase). Après appliquerAjustement (~l.115903), macros.origine='ajustement' fait que ciblesPoseesParCoach rend vrai : la carte affiche « Le total vient de ton coach ».
Travail :
1. Une source de vérité pour l'athlète sans coach : athObjectif(k) appelle aussi changerPhase(currentUser,k,null,'athlete'), avec les mêmes refus (TCA, grossesse, REDS), et confirmerPhase(t) écrit aussi nutrition.perso.objectif=t quand t appartient à ATH_OBJECTIFS. Au chargement, migration douce : si les deux divergent, la phase la plus récente gagne et on recopie (une
```

seule fois, sans toast).

2. Corriger le texte de `_htmlChoixPhase` : « Ça change ta cible calorique et tes protéines » pour l'athlète sans coach.
3. `ciblesPoseesParCoach` : l'origine 'ajustement' ne verrouille QUE si l'athlète a un coach. Sinon, elle se lit comme 'athlete'.
4. `_htmlCiblesAthlete` : si le dossier n'a pas de coach, remplacer « Ton coach voit ces cibles... » (l.91939) par rien, ou par « Tes cibles suivent ton poids : elles se mettent à jour à chaque pesée ». Quand la cible recalculée diffère de `nutrition.macros` de plus de 20 kcal, écrire la nouvelle cible (`_athEcrireCibles`) au chargement de l'écran pour que la carte et les anneaux montrent le même chiffre. Sur un appareil d'athlète suivi par un coach, ne rien écrire.

Cas limites : phase 'recomp' ou 'peak' (absentes d'ATH_OBJECTIFS) → l'objectif reste 'maintien' et le bouton actif le dit ; double clic sur « Sèche » → une seule confirmation (garde booléenne pendant le `rcConfirm`) ; hors ligne → `saveUser local`, aucune erreur.

Tests à ajouter dans `app/tests.js` (style `ok('...',(()=>{...}))()` de la suite nutrition) :

- athlète sans coach avec phase 'masse' : `athObjectif('seche')` (`rcConfirm` simulé à vrai) → `phaseCourante().type==='seche'` et `ciblesAthlete(u).coef===0.85` ;
- objectif 'seche' seul + pesées stagnantes sur 35 jours : `ajustementPropose(u)` n'est pas null ;
- après appliquerAjustement sans coach : `ciblesPoseesParCoach(u)===false`, et le HTML de la carte ne contient pas « coach » ;
- avec coach : comportement inchangé (la phase du coach gagne).

Lance la suite (`scripts/verif/README.md`, puis `node scripts/verif/suite.mjs <url>`). Nouveau build : `window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`. Ne touche à aucune taille de police.

Critère de fin : chez l'athlète sans coach, un seul geste fixe l'objectif, et la carte, les anneaux, l'ajustement et le point de la semaine lisent tous ce même objectif.

2. Cycle ON/OFF à 6 séances : –90 % de glucides le jour OFF, puis la sèche est annulée par le plancher — Priorité : Critique · Effort : S

Analyse et idée. `cycleGlucides` (rc-core:101713) compense sur la semaine avec $x_{off} = 0,15 \times n_{on} / n_{off}$. Avec 6 créneaux, cela donne –90 % : le jour OFF tombe à 20 g de glucides (H 80 kg), 11 g (F 55 kg) et 12 g (H 120 kg). Côté coach, ce jour OFF vaut 968 kcal pour la femme de 55 kg, sans aucune alerte dans « Journées ». Côté athlète, `_relevePlancher` relève le jour OFF puis reporte le même relèvement sur le jour ON (`lift`, l.103978). La moyenne de la semaine monte alors de 2 128 à 2 463 kcal (+335 kcal/j, soit plus de déficit), et de 1 347 à 1 593 kcal pour la femme de 55 kg. Idée : plafonner la baisse du jour OFF à 30 % et réduire la hausse du jour ON pour garder la moyenne. Quand le plancher mord, le relèvement se fait sans report sur le jour ON, et un plancher qui mord un jour donné s'affiche.

Prompt de mise en place.

Contexte : `RepCore`, `app/rc-core.1792.js`. `cycleGlucides(user,g)` (l.101705) calcule $g_{on}=g \times 1,15$ et $g_{off}=g \times (1 - 0,15 \times n_{on} / n_{off})$. Avec 6 créneaux actifs, $g_{off}=g \times 0,10$. `_tbJournees` (l.103963) et `besoinsProposes` (l.102505) relèvent la journée OFF au plancher puis ajoutent le même lift au jour ON : la moyenne hebdomadaire dépasse alors la cible. Mesures au banc : H 80 kg sèche 0,85, 6 créneaux → OFF 20 g de glucides ; avec le plancher, moyenne 2 463 au lieu de 2 128 kcal.

Travail :

1. Constante `CYCLE_GLUC_OFF_MAX=0.30`. Dans `cycleGlucides` : $x_{off} = \min(CYCLE_GLUC \times n_{on} / n_{off}, CYCLE_GLUC_OFF_MAX)$, puis $x_{on} = x_{off} \times n_{off} / n_{on}$ ($\leq CYCLE_GLUC$), pour que $(n_{on} \times g_{on} + n_{off} \times g_{off}) / 7 = g$ à ± 1 g. Renvoyer `pctOn` et `pctOff` réels. Les libellés « +15 % / -x % » de `besoinsProposes` et du tableau « Journées » lisent ces valeurs.
2. Relèvement au plancher dans `_tbJournees` et `besoinsProposes` : relever chaque jour indépendamment (OFF au plancher, ON inchangé s'il est déjà au-dessus). Ne reporter le lift sur ON que si l'écart ON-OFF devient négatif. Renvoyer `moyenneServie` et l'écart à la cible ; si l'écart dépasse 50 kcal, l'afficher (« le plancher du jour de repos ajoute X kcal par jour en moyenne »).
3. Côté coach (`appliquer=false`) : sous la ligne « Jour OFF » du tableau Journées, ajouter une alerte rouge quand `j.off.kcal < plancherAthlete(c)`, avec le chiffre du plancher. Une ligne, classe existante `.mac-alerte`, sans nouvelle taille de police.

Cas limites : $n_{on}=0$ ou 7 → pas de cycle (inchangé) ; $g=0$ → $g_{on}=g_{off}=0$; dossier en saisie manuelle → rien ne bouge.

Tests à ajouter dans `app/tests.js` :

- 6 créneaux, $g=196$: $g_{off} \geq 0,70 \times 196$ et moyenne hebdomadaire à ± 1 g de 196 ;
- 3, 4 et 5 créneaux : moyenne à ± 1 g ;
- H 80 kg, 180 cm, 30 ans, 6 créneaux, sèche, plancher appliqué : moyenne servie $\leq$ cible + 50 kcal ;

- côté coach, F 55 kg, coef 0,75, 6 créneaux : le HTML du tableau contient l'alerte de plancher du jour OFF.
Adapter l'assertion qui fige cycleGlucides (~tests.js:43273) en expliquant le changement. Lancer la suite, puis nouveau build (window.RC_BUILD dans index.html, CACHE dans sw.js, python3 scripts/versionner_actifs.py). Pas d'agrandissement de police.
Critère de fin : aucun jour OFF sous 70 % des glucides moyens, et la moyenne de la semaine servie égale la cible, plancher compris, à 50 kcal près.

3. Obésité : le « g/kg de poids du corps » sert 288 g de protéines et 0 à 75 g de glucides — Priorité : Haute · Effort : M

Analyse et idée. poidsMacros renvoie désormais toujours le poids total (rc-core:101775, commit c5e3456), et le test tests.js:21767 fige 120 kg pour un IMC de 39. Au banc, cela donne H 120 kg : P 288 g (50 % des kcal), G 75 g. F 95 kg : P 228 g (61 %), G 0 g, et le total servi dépasse la cible de 100 kcal. Le plan alimentaire suit : sources de glucides à « 0 g », 188 g de jambon sec par repas, environ 550 kcal au-delà de la cible (planAlertes). La journée « tenue » (protéines $\geq$ 95 %) devient inatteignable, et la confirmation de sèche ne dit rien de tout cela. Idée, dans le sens de la décision de Kevin : garder le poids du corps par défaut, mais à partir d'un IMC de 30, proposer le poids ajusté, écrit dans la case elle-même (« $2,4 \times 87 \text{ kg ajustés} = 210 \text{ g}$ »), et réversible d'un geste par le coach.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Le 05/10 (build 1778), Kevin a décidé que les g/kg s'appliquent au POIDS DU CORPS : poidsMacros(u) (~l.101775) renvoie toujours {type:'total'}. Le reproche de Kevin portait sur un calcul que l'écran ne montrait PAS à l'endroit du chiffre. On respecte ce principe : le poids total reste le défaut et TOUT écart est écrit dans la case.
Problème mesuré : H 120 kg / 175 cm, sèche 2,4 g/kg $\rightarrow$ 288 g de protéines (50 % des kcal), 75 g de glucides ; F 95 kg / 160 cm $\rightarrow$ 228 g, 0 g de glucides, total servi +100 kcal.
Travail :
1. Nouveau réglage par dossier nutrition.tableur.baseGkg $\in$ {'total','ajuste'}. Sans réglage : 'total' si IMC < 30, sinon 'ajuste' (IMC calculé comme _imcPourFormule). Poids ajusté = $25 \times \text{taille}^2 + 0,25 \times (\text{poids} - 25 \times \text{taille}^2)$.
2. poidsMacros(u) lit ce réglage et renvoie {kg, type:'total'|'ajuste', lib}. libPoidsMacros affiche « calculé sur 87 kg (poids ajusté) ».
3. Afficher la base DANS la cellule des protéines et des lipides : sur la carte de l'athlète (_htmlCiblesAthlete) et dans le tableau MACRONUTRIMENTS du coach (« 2,4 g/kg $\times$ 87 kg ajustés »), plus un sélecteur coach « Base des g/kg : poids du corps | poids ajusté » à côté du g/kg. Ce sont les mêmes composants et les mêmes tailles qu'aujourd'hui.
4. athObjectif('seche') : si glucidesBas ou depasse>0 après calcul, le rcConfirm le dit en une phrase (« glucides à 75 g/j : séances moins énergiques »).
Cas limites : taille inconnue $\rightarrow$ total ; IMC exactement 30 $\rightarrow$ ajusté ; coach qui force 'total' sur un IMC de 35 $\rightarrow$ respecté, et le libellé le dit ; besoinEau lit aussi poidsMacros : vérifier qu'il reste plausible.
Tests à ajouter dans app/tests.js :
- remplacer l'assertion « IMC 39 : 120 kg » (~tests.js:21767) par : sans réglage, poidsMacros(gros).type==='ajuste' et kg $\approx$ 87,4 ; avec baseGkg='total', kg===120 ;
- H 80 kg / 180 cm : p===192 (inchangé) ; F 50 kg : p===120 (inchangé) ;
- H 120 kg / 175 cm sèche : $190 \leq p \leq 230$ et $g \geq 150$;
- le HTML de la carte contient « ajusté » quand la base est ajustée, et ne le contient pas sinon.
Lancer la suite, puis nouveau build (window.RC_BUILD dans index.html, CACHE dans sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir de police.
Critère de fin : aucune cible protéique au-delà de 40 % des kcal pour un IMC $\geq$ 30 sans choix explicite du coach, et la base de calcul est toujours lisible à côté du chiffre.

4. Plancher sans tours de mesure : la dépense sportive est oubliée, et le gros gabarit est bloqué — Priorité : Haute · Effort : S

Analyse et idée. Sans masse maigre, plancherEffectif prend $\max(1\ 200/1\ 500 ; 22 \times \text{poids})$ (rc-core:116755) et ignore la dépense d'entraînement, alors qu'il la compte avec les tours de mesure.

Petite femme très active (profil C) : le coach peut poser un coefficient de 0,70, soit 1 425 kcal, sans aucune alerte. La disponibilité énergétique tombe pourtant à environ 26 kcal/kg de masse maigre (zone RED-S). Pour le profil B, $22 \times 120 = 2\,640$ kcal, plafonné à 85 % de la dépense : le plancher égale la sèche à 0,85. Testé au banc, 30 clics sur –20 sont tous refusés, et les coefficients 0,80 à 0,70 n'ont aucun effet. Idée : quand les tours manquent, estimer la masse maigre (formule de Boer, sexe, poids, taille) et appliquer partout la même règle de disponibilité.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js, plancherEffectif(user,brut) (~l.116722). Avec une masse
maigre mesurée (masseMaigreDuBilan), le plancher proportionnel vaut  $30 \times \text{MM} + \text{dépense d'entraînement}$ 
(kcalSportParJour(user).jour). Sans masse maigre, il vaut  $22 \times \text{poids total}$ , sans la dépense
d'entraînement. Deux défauts mesurés :
- F 50 kg / 155 cm, 5 × 75 min de musculation + 4 h de course rapide (479 kcal/j de sport) : plancher
1 200, une cible coach à 1 425 kcal passe sans alerte ( $\text{DE} \approx 26 \text{ kcal/kg MM}$ ) ;
- H 120 kg / 175 cm :  $22 \times 120 = 2\,640$ , plafonné à 85 % de la dépense = la cible de sèche ; l'athlète
ne peut ni descendre de 20 kcal ni choisir 0,80.
Travail :
1. masseMaigreEstimee(user) : Boer ( $\text{H} : 0,407 \times \text{P} + 0,267 \times \text{T} - 19,2$  ;  $\text{F} : 0,252 \times \text{P} + 0,473 \times \text{T} - 48,3$ ),
renvoyée avec source:'estimee'. masseMaigreDuBilan reste prioritaire.
2. plancherEffectif :  $\text{parMM} = 30 \times (\text{MM mesurée sinon estimée}) + \text{\_dexo}$ . Garder  $\text{parPoids} = 22 \times \text{poids}$ 
UNIQUEMENT quand la taille est inconnue. règle vaut 'masse_maigre' ou 'masse_maigre_estimee'.
libelleReglePlancher dit « estimée sur ta taille et ton poids ».
3. Ne pas toucher à la majoration TCA ni au plafond à 85 % de la dépense, ni à l'absolu 1 200/1 500.
Cas limites : taille absente → ancien comportement ; masse maigre estimée ≤ 0 → ignorée ; profil aTCA
→ majoration inchangée.
Tests à ajouter dans app/tests.js :
- profil F 50 kg ci-dessus sans tours : plancherEffectif(u).kcal entre 1 550 et 1 650 et
cibleTableur(u,{coef:0.70,appliquerPlancher:false}).sousPlancher===true ;
- H 120 kg / 175 cm, 3 créneaux, sèche 0,85 : un athDelta(-1) côté athlète est accepté (le plancher
est sous la cible) ;
- H 80 kg / 180 cm : plancher dans ±60 kcal de l'ancienne valeur (non-régression) ;
- les assertions existantes qui figent 22 kcal/kg (grep KCAL_PLANCHER_PAR_KG dans tests.js) : à
adapter, avec la raison.
Lancer la suite, puis nouveau build (window.RC_BUILD dans index.html, CACHE dans sw.js, python3
scripts/versionner_actifs.py). Ne pas agrandir de police.
Critère de fin : avec ou sans tours de mesure, le plancher suit la même règle de disponibilité
énergétique, dépense sportive comprise.
```

5. Ajustement accepté : la fiche du coach bascule en saisie manuelle et le plan ne suit plus — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc (Léa suivie, cibles posées par le tableau). L'athlète accepte un ajustement, appliquerAjustement écrit macros[jour] et origine:'ajustement' (rc-core:115913). Ensuite saisieManuelle(c) vaut vrai : la fiche du coach passe en saisie manuelle (dix champs, menus g/kg figés). Et planCiblesJour continue de recalculer depuis le tableau (l.94689) : le plan du jour OFF annonce 1 707 kcal pendant que les anneaux visent 1 650. Il existe déjà un réglage partagé fait pour ce cas : le ±20 (tableur.ajust). Idée : passer l'ajustement automatique par ce réglage partagé (en kcal, sur le jour visé), pour que le tableau, le plan, la carte et les anneaux bougent ensemble.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js. appliquerAjustement(retour) (~l.115903) écrit directement
nutrition.macros[a.jour] et pose macros.origine='ajustement'. Conséquences mesurées au banc :
- saisieManuelle(c) (~l.102949) rend vrai pour cette origine : la fiche du coach passe en saisie
manuelle et _tbReconcilier / _tbEcrireCibles cessent de recalculer ;
- planCiblesJour (~l.94676) relit cibleTableur : le plan diffère des anneaux (jour OFF 1 707 contre 1
650 kcal).
Le ±20 partagé existe déjà (nutrition.tableur.ajust, appliquerDeltaKcal ~l.91361, deltaKcalPartage),
et cibleTableur l'applique pour tout le monde.
Travail :
```

1. Ajouter à `tableur.ajust` un champ 'auto' (kcal par jour, signé) et, si besoin, un 'jour' ('on'|'off'|'tous'). `deltaKcalPartage` additionne `coach+athlete+auto`. `_tbJournées` applique le delta auto au seul jour visé quand il est renseigné.
2. `appliquerAjustement` : au lieu d'écrire `macros[jour]`, incrémenter `tableur.ajust.auto` de `a.kcalDelta`, puis réécrire les cibles par le chemin habituel (`_athEcrireCibles` chez l'athlète sans coach, `_tbJournées + origine` inchangée sinon). Garder `_ajustJournaliser` et le verrou de quinzaine.
3. `libelleAjustKcal` : afficher « Ajustement automatique : -210 kcal les jours de repos » des deux côtés, avec un bouton « Annuler » chez le coach qui remet auto à 0 (tracé dans `ajustHisto`).
4. Migration : un dossier qui porte déjà origine 'ajustement' → origine reprise à 'tableur' si `tableur.naf` est posé (sinon 'athlete'), et l'écart `macros-moins-calcul` reporté dans `ajust.auto`, une seule fois.

Cas limites : saisie manuelle explicite (`nut.manuel===true`) → l'ajustement décale les grammes comme `appliquerDeltaKcal` le fait déjà ; plancher et `aTCA` : mêmes gardes qu'aujourd'hui ; deux appareils hors ligne → `syncFusion` garde le max de `ajust.maj`.

Tests à ajouter dans `app/tests.js` :

- athlète suivi en grille, ajustement 'baisse' appliqué : `saisieManuelle(c)===false`, `planCiblesJour(c,false).kcal === ciblesEnVigueur(c, un jour OFF).kcal` ;
- migration d'un dossier 'ajustement' : origine reprise, kcal identiques avant et après ;
- annulation coach : `ajust.auto===0` et les cibles reviennent au calcul.

Lancer la suite, puis nouveau build (`window.RC_BUILD` dans `index.html`, `CACHE` dans `sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir de police.

Critère de fin : après un ajustement, le tableau du coach, le plan, la carte et les anneaux affichent le même total, et la fiche reste en automatique.

6. « Point de départ proposé » à côté de « Mon objectif » : deux propositions différentes sur le même écran — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc (athlète sans coach, diète flexible, sans cibles). La carte « Mon objectif » propose 2 724 kcal par jour. Juste dessous, « Point de départ proposé » (`besoinsProposes`, `rc-core:92361`) propose 2 872 kcal ON et 2 612 OFF, à 1,8 g/kg et « objectif à 100 % de la dépense », sans tenir compte de l'objectif. Deux boutons d'enregistrement mènent à deux résultats différents : l'athlète hésite, et c'est exactement ce que Kevin veut éviter. Idée : en diète flexible, la carte « Mon objectif » suffit. Le point de départ ne s'affiche que dans les diètes qui n'ont pas cette carte, et il lit le même objectif.

Prompt de mise en place.

Contexte : `RepCore`, `app/rc-core.1792.js`. Dans le rendu de la diète flexible (~l.93140), l'écran affiche `_htmlCiblesAthlete` (carte « Mon objectif », moteur `cibleTableur`) PUIS `_htmlDepartAthlete` (~l.92359, moteur `besoinsProposes`). `besoinsProposes` ignore `nutrition.perso.objectif` (il lit `typePhase`) et propose des journées ON/OFF à 100 % de la dépense et à 1,8 g/kg. Au banc : 2 724 kcal sur la carte, contre 2 872/2 612 sur le point de départ.

Travail :

1. Ne plus rendre `_htmlDepartAthlete` quand `_htmlCiblesAthlete` rend sa forme réglable (athlète sans cibles du coach). Le garder pour les autres diètes.
2. Là où il reste, lui faire lire la même phase que `cibleTableur` : passer `{objectif:perso.objectif}` et, dans `besoinsProposes`, utiliser l'objectif de l'athlète comme phase quand `typePhase(user)` est null (même règle que `cibleTableur` ~l.102165).
3. Le premier geste sur « Mon objectif » enregistre les cibles (c'est déjà le cas) : ajouter sous la carte une ligne courte, « Choisis ton objectif : tes cibles s'enregistrent tout de suite », visible tant que `nutrition.macros` est vide (classe `.rc-obj-note` existante).

Tests à ajouter dans `app/tests.js` :

- athlète sans coach, flexible, sans macros : le HTML de l'écran contient « Mon objectif » et pas « Point de départ proposé » ;
- `besoinsProposes(u)` avec `perso.objectif='seche'` et sans phase : `delta < 0` et `gParKg === 2.4`.

Lancer la suite, puis nouveau build (`window.RC_BUILD` dans `index.html`, `CACHE` dans `sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir de police.

Critère de fin : l'athlète autonome ne voit qu'une proposition chiffrée, qui suit l'objectif choisi.

7. Observance : la journée en cours est toujours un « écart » (point 6 du premier audit non appliqué) — Priorité : Moyenne · Effort : S

Analyse et idée. `htmlObservanceCoach` passe `fin = aujourd'hui` à `observance14` (`rc-core:105924-105927`), qui classe en écart tout jour saisi qui n'est pas à $\pm 7\%$. Testé : un petit-déjeuner de 500 kcal noté le matin donne `{etat:'ecart', ecartKcal:-1500}`, avec la phrase « un seul écart, le mardi ». `grep en_cours|incomplet` sur la bande : aucune occurrence. Le coach qui ouvre la fiche en journée voit donc une case rouge fabriquée, et risque de relancer l'athlète à tort. Idée : le jour en cours passe à l'état « en cours », et une saisie manifestement partielle passe à l'état « incomplet ».

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js, observance14(log,cibles,finISO) (~l.105865), motifEcart, phraseObservance et htmlObservanceCoach (~l.105920). Le jour === localISODate(new Date()) est compté comme 'ecart' dès qu'une seule saisie existe.
Travail :
1. Dans observance14 : le jour égal à la date du jour (passer `aujourd'hui` en 4e paramètre, défaut localISODate(new Date())) prend l'état 'en_cours'. Il ne compte ni dans tenus, ni dans ecarts, ni dans les moyennes.
2. Nouvel état 'incomplet' : jour passé dont kcal < 50 % de la cible ET qui ne porte qu'un repas distinct (entries[].repas). Il n'est pas compté comme écart, et phraseObservance dit « (n saisies partielles) ».
3. CSS dans le fichier rc-style : .obs-en_cours (hachuré, même taille que les autres cases) et .obs-incomplet. Ajouter les deux entrées à la légende existante, sans agrandir quoi que ce soit.
4. cibleTenueJour, la série de l'assiette et adherence() ne changent pas.
Tests à ajouter dans app/tests.js (suite N8, ~l.52516) :
- une entrée de 500 kcal aujourd'hui, cible 2 000 : dernier jour etat==='en_cours', ecarts===0 ;
- hier, un seul repas à 600/2 000 : 'incomplet' ;
- hier, 3 repas à 1 400/2 000 : 'ecart' ;
- les assertions N8 existantes restent vertes (elles passent un finISO passé).
Lancer la suite, puis nouveau build (window.RC_BUILD dans index.html, CACHE dans sw.js, python3 scripts/versionner_actifs.py).
Critère de fin : aucune case rouge pour la journée en cours ni pour une saisie manifestement partielle.
```

8. Cibles recalculées en silence à l'ouverture de la fiche du coach — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : le coach ouvre simplement la fiche de Léa, et `_tbReconcilier` (`rc-core:104740`) réécrit et pousse ses cibles (1 650/1 500 → 1 987/1 707 kcal sur mon jeu de données), sans toast pour le coach ni mot pour l'athlète. L'historique (`_histoNoter`) garde la trace, mais personne ne la voit. Le principe voulu par Kevin (les cibles suivent la grille et le poids) est bon. Ce qui manque, c'est la transparence et un retour arrière. Idée : quand la réconciliation bouge le total de plus de 30 kcal, afficher un bandeau d'une ligne en tête de l'onglet nutrition (« Cibles mises à jour : poids 66 → 64 kg, -40 kcal · Annuler »), et la même phrase sur la carte de l'athlète.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js. renderCoachNutriSection(c) (~l.104725) appelle _tbReconcilier(c) (~l.104689) : si la grille du coach est posée (ou origine 'tableur'), _tbEcrireCibles recalcule et l'appel pousse les nouvelles macros, sans rien dire.
Travail :
1. _tbReconcilier renvoie {avant, apres, cause}, où cause ∈ {'poids','reglage','bilan'} : comparer poidsNutritionnel au poids mémorisé dans nutrition.tableur.dernierCalcul {kg, kcalOn, kcalOff, date}, à mettre à jour à chaque écriture.
2. Si |apres.on.kcal - avant.on.kcal| > 30 : bandeau d'une ligne en tête de #ccd-nutrition (composant de bannière existant, même taille de texte) : « Cibles mises à jour (poids 66 → 64 kg) : 1 650 → 1 610 kcal. » Le bouton « Annuler » remet les macros d'avant avec origine 'histo' (mécanisme existant, build 1408).
3. Côté athlète, une ligne dans .rc-obj-ajust : « Tes cibles ont suivi ta pesée : -40 kcal. » Elle
```

reste 7 jours.

Cas limites : saisie manuelle → rien (inchangé) ; plusieurs ouvertures de suite → un seul bandeau tant que les chiffres ne bougent plus ; hors ligne → bandeau quand même, l'envoi suit la file existante.

Tests à ajouter dans app/tests.js :

- dossier en grille, pesée -2 kg : _tbReconcilier renvoie cause 'poids' et le HTML de la fiche contient « Cibles mises à jour » ;

- « Annuler » : macros identiques à avant, origine 'histo', et la réouverture suivante ne réécrit rien ;

- écart ≤ 30 kcal : pas de bandeau.

Lancer la suite, puis nouveau build (window.RC_BUILD dans index.html, CACHE dans sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir de police.

Critère de fin : aucune cible ne change sans qu'au moins une phrase lisible par le coach et par l'athlète le dise, avec un retour arrière en un geste.

9. Plan alimentaire modèle : les aliments fixes dépassent déjà la cible chez les petits gabarits — Priorité : Basse · Effort : M

Analyse et idée. planDepuisModele pose les mêmes aliments fixes quel que soit le gabarit (seuls les œufs et les amandes suivent le poids, rc-core:94175). Mesuré : pour F 50 kg en sèche, le squelette couvre déjà 56 g de lipides pour une cible de 40 g, et le total du plan dépasse la cible d'environ 150 kcal (10 %). Pour H 120 kg, il couvre 202 g de glucides pour une cible de 94 g. planAlertes le signale, mais c'est ensuite au coach de corriger ligne par ligne. Idée : un bouton « Ajuster le squelette à la cible ». Il réduit en proportion les aliments fixes qui portent la macro en excès, au pas de leur unité, et le coach valide d'un geste.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js, plan alimentaire du coach : planDepuisModele (~l.94175), planCouverture (~l.94441), planRestant (~l.94491), planAlertes (~l.94824), renderPlanCoach (~l.95732). Quand `restant[macro] < 0`, les sources libres affichent 0 et une alerte texte s'affiche, sans autre aide.

Travail :

1. Fonction PURE `planReduireSquelette(plan, cibles, chercher)` : pour chaque macro négative, réduire les quantités des lignes fixes (pas les notes, ni les fruits, ni les compléments) qui apportent le plus de cette macro, dans l'ordre décroissant de contribution, jusqu'à `restant ≥ 0`. Arrondir au pas de l'unité (`planUniteItem` : 1 œuf, 5 g, 10 mL). Ne jamais descendre sous 50 % de la quantité d'origine ; au-delà, garder l'alerte. Renvoyer `{plan, changements:[{nom, avant, apres}]}`.

2. Dans l'alerte existante de `renderPlanCoach`, un bouton « Ajuster le squelette » (bouton secondaire existant) qui montre la liste des changements dans un `rcConfirm` puis enregistre (`savePlanCoach`).

3. Fiche imprimable et vue athlète : aucune modification de mise en page.

Tests à ajouter dans app/tests.js :

- F 50 kg / 155 cm sèche, modèle `seche_F` : après réduction, `restant.l ≥ 0` et `kcal du plan ≤ cible × 1,05` ;

- aucune ligne sous 50 % de sa quantité ;

- un plan sans excès revient inchangé.

Lancer la suite, puis nouveau build (window.RC_BUILD dans index.html, CACHE dans sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir de police.

Critère de fin : sur les modèles fournis, un clic ramène le plan à ± 5 % de la cible chez une femme de 50 kg comme chez un homme de 120 kg (avec le point 3 appliqué).

Expert 5 · Micronutrition et compléments — 6,5/10

La base est rigoureuse et rare dans ce marché : huit micronutriments Ciquel reportés au prorata dans chaque entrée, repères Anses par sexe, « inconnu n'est pas zéro », signal persistant sur deux semaines, questions au coach plutôt que des diagnostics, compléments notés A/B/C avec leurs interactions. Ce qui manque le plus, c'est que tout cela atteigne les bonnes personnes sans risque. Les athlètes en diète stricte (le défaut) ne voient rien. Les aliments conseillés ne s'affichent pas, et quand ils s'affichent, la base propose du foie cru à une végétarienne. Enfin, le garde-fou TCA laisse passer huit pourcentages.

1. Le bloc « Micronutriments » contourne le garde-fou TCA et se lit comme un mur de texte — Priorité : Critique · Effort : S

Analyse et idée. Testé au banc : avec `tcaRisque=true`, le signal se tait (`_htmlSignalMicro` teste `aTCA`, rc-core:92957). En revanche, `_htmlCouvertureMicro` (rc-core:14485, appelé à 100694) affiche quand même « Fer : 36 % de la référence... » et sept autres lignes, alors que la doctrine écrite à 92942 dit l'inverse. Le bloc fait 8 phrases, avec une mention de fiabilité même à 97 %, et le disclaimer apparaît trois fois sur le même écran. Il faut appliquer la même garde au bloc et remplacer les phrases par 8 mini-barres neutres sur une seule carte.

Prompt de mise en place.

```
Contexte : RepCore (app/rc-core.1792.js). Sur l'écran nutrition de l'athlète,
_htmlCouvertureMicro(user) (≈ ligne 14485, appelée dans _renderFjDaySummary ≈ 100694) affiche huit
phrases « X : NN % de la référence sur 7 jours ». _htmlSignalMicro (≈ 92954) se tait si aTCA(u) est
vrai, mais _htmlCouvertureMicro ne le teste pas : une athlète avec user.tcaRisque===true voit quand
même les huit pourcentages.

À faire :
1. Dans _htmlCouvertureMicro, retourner '' si aTCA(_dossier(user)) est vrai (try/catch comme dans
_htmlSignalMicro). Le coach garde son accès via risquesMicro, sans changement.
2. Refondre le rendu sans changer le calcul (couvertureMicro reste la source) : une carte «
Micronutriments · 7 jours » avec 8 lignes compactes « libellé · mini-barre neutre · NN % ». Ordre FIXE
de MICRO_REFS, aucune couleur d'alerte (gris/texte, barre plafonnée visuellement à 100 %, la valeur
réelle reste écrite). La mention de fiabilité n'apparaît que si partDocumentee < 0.90, en petit sous
la ligne (« calculé sur 69 % du journal »). La mention « sexe non renseigné » apparaît une seule fois
en pied de carte, pas sur chaque ligne.
3. Un seul MICRO_DISCLAIMER par écran : si _htmlSignalMicro est rendu, la carte de couverture ne
répète pas le sien.
4. Réutiliser les tokens existants (--fs-xs, --fs-2xs, --surface-1, --border, --r-3). Ne pas agrandir
les polices ni les cibles. Le bloc doit être PLUS COURT qu'aujourd'hui en hauteur à 390 px.
Cas limites : sexe inconnu ; moins de 7 jours journalisés (rien n'est rendu) ; nutriment sans donnée
(ligne absente) ; tcaRisque passé de true à false (le bloc revient).
Tests à ajouter dans app/tests.js : aTCA vrai → _htmlCouvertureMicro rend '' ; partDocumentee 0,97 →
aucune mention de fiabilité ; 0,69 → mention présente ; un seul disclaimer quand signal et carte
coexistent ; aucune couleur var(--red*) ni var(--amber) dans le HTML de la carte.
Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js), puis
python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.
Critère de fin : tests verts ; au banc, une athlète TCA ne voit aucun pourcentage de micronutriment ;
une athlète standard voit 8 barres sur une carte moins haute qu'avant.
```

2. Les « 3 aliments riches » ne s'affichent pas, et la base propose du foie cru à une végétarienne — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc (Léa, végétarienne déclarée, B12 à 27 % deux semaines de suite). La carte affiche la phrase sans aucun aliment, même après 6 s, parce que `_ciquaLDB` n'est pas chargé au rendu (« non chargé » relevé, rc-core:96486). Une fois la base chargée, `alimentsRichesEn` (rc-core:14714) renvoie « Foie, oie, cru », « Foie, canard, cru » pour le fer et « Rognon, porc » pour la B12. Les évictions (`evictionDe`, 106405), le régime végétarien et les aliments crus à cuire ne sont pas filtrés. Il faut charger la base puis repeindre la carte, filtrer le régime, les évictions et le cru, et pour la B12 chez une végétarienne, donner une phrase dédiée au lieu d'une liste.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. _htmlSignalMicro (≈ 92954) appelle alimentsRichesEn(u,cle,3) (≈
14714), qui lit _ciquaLDB. À l'ouverture de l'écran nutrition, _ciquaLDB n'est pas chargé (chargement
paresseux, _loadCiqua ≈ 96486) : la carte affiche la phrase sans aliment. Une fois la base chargée,
la fonction propose « Foie, oie, cru », « Foie, canard, cru », « Rognon, porc, cuit », y compris à une
athlète déclarée végétarienne (_microVege, ≈ 14806) et sans tenir compte de « Ce que je ne mange pas »
(evictionDe, ≈ 106405).
```

À faire :

1. Si un signal existe et que `_ciquaDB` est vide, lancer `_loadCiqua()` puis repeindre uniquement la carte du signal (lui donner un id, remplacer son `outerHTML`). Aucun spinner ; hors ligne, la phrase reste seule, comme aujourd'hui.
2. Dans `alimentsRichesEn`, sur la BASE seulement (pas sur ce que l'athlète a déjà journalisé) : exclure tout aliment pour lequel `evictionDe(user,f)` n'est pas null ; si `_microVege(user)`, exclure les groupes « viandes, œufs, poissons et assimilés » et les noms contenant viande, abats, foie, rognon, poisson, fruits de mer, etc. (liste en constante `MICRO_MOTS_ANIMAUX`) ; exclure les fiches dont le nom contient « cru » / « crue » quand une fiche « cuit » du même aliment existe, ou à défaut tout abats cru.
3. Cas B12 + végétarien/végétalien : ne pas lister d'aliments. Afficher la phrase « Chez les végétaliens, la B12 ne vient pas des végétaux : c'est un sujet à voir avec ton coach ou un professionnel de santé. » Rester dans le registre « apports », jamais « carence ».
4. Garder la règle « les siens d'abord » et la mention « que tu manges déjà ».

Cas limites : base indisponible (`ciquaIndisponible`) ; aucune proposition après filtrage (la phrase reste seule) ; éviction par groupe ou par motif ; athlète qui journalise elle-même du foie (il reste proposable, car c'est le sien).

Tests dans `app/tests.js` : végétarienne + fer → aucun aliment du groupe viandes/poissons ; éviction « foie » → aucun foie ; B12 + végétarienne → liste vide et phrase dédiée ; aliment cru d'abats jamais proposé depuis la base ; aTCA → rien (non-régression).

Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, CACHE de `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Critère de fin : au banc, la carte du signal montre trois aliments plausibles en moins de 3 s après ouverture, et jamais de foie cru ni de viande chez une végétarienne.

3. Diète stricte (le défaut) : zéro micronutriment, alors que le plan du coach porte déjà les aliments Ciqua — Priorité : Haute · Effort : M

Analyse et idée. `DIETE_DEFAULT='strict'` (rc-core:91191). Testé au banc : la même Léa passée en diète stricte n'a ni carte Micronutriments ni signal. Côté coach, `risquesMicro` lit un journal vide. Les lignes du plan portent pourtant un identifiant Ciqua et une quantité (`_planAlimentLigne`, rc-core:≈94260). Il faut un encart « Micronutriments du plan » dans l'éditeur du coach (`renderPlanCoach`, 95732) : huit barres calculées sur une journée type du plan, mises à jour à chaque modification. C'est le geste d'un diététicien, et il ne demande aucune saisie de plus.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. Le coach compose un plan alimentaire (`ouvrirPlanCoach` ≈ 94919, `renderPlanCoach` ≈ 95732). Chaque ligne du squelette porte `item.ciqua (id)` et une quantité, résolu par `_planAlimentLigne` (≈ 94260). Les catalogues de sources (`planCatalogue(p,'p'|'c')`) sont des choix équivalents. En diète stricte (défaut, `typeDiete`), l'athlète ne voit aucun micronutriment, et le coach non plus.

À faire :

1. Fonction `PURE microPlan(plan, user, chercher) → {cle: {apport, ref, part, partDocumentee}}` pour chaque clé de `MICRO_REFS`. Elle somme les lignes du squelette (`quantité × valeur/100`). Pour chaque catalogue de sources, elle prend la MOYENNE des aliments du catalogue à la portion prévue. Même règle qu'ailleurs : un aliment sans valeur sort de l'apport ET du total (`partDocumentee`), il n'est jamais compté zéro. Elle réutilise `refMicro(user,cle)` pour le repère (sexe de l'ATHLÈTE, pas du coach).
2. Dans `renderPlanCoach`, sous les totaux macros : un encart repliable fermé par défaut, « Micronutriments du plan (journée type) », avec les 8 mini-barres neutres du point « bloc micro » si elles existent, sinon une ligne texte par nutriment. En tête, UNE ligne de synthèse : « Sous 70 % : fer, B12 » (ou « Tous au-dessus de 70 % »). Recalcul à chaque modification de ligne.
3. Pour chaque nutriment sous 70 %, un lien discret « Voir 3 aliments » qui réutilise `alimentsRichesEn` (filtré par le régime et les évictions de l'athlète).
4. Rien n'est montré à l'athlète dans ce lot.

Cas limites : Ciqua pas encore chargé (`charger` puis `repeindre`) ; ligne libre sans ciqua (ignorée et comptée dans « non documenté ») ; plan vide (encart absent) ; sexe inconnu (repère le plus élevé, comme `refMicro`).

Tests dans `app/tests.js` : plan de 3 lignes connues → apports exacts ; ligne sans fer → exclue de l'apport et `partDocumentee < 1` ; catalogue de 2 sources → moyenne ; athlète F vs H → repère fer différent.

Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, CACHE de `app/sw.js`, puis `python3`

scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : au banc, en coach, composer un plan riz-pâtes-poulet pour une athlète affiche « Sous 70 % : fer... » sans aucun clic supplémentaire pour la synthèse.

4. Fiche coach « Signaux faibles » : 9 blocs, 6 fois la même question, et des coquilles — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc (coach, fiche de Léa) : neuf entrées, dont « Un bilan sanguin récent existe-t-il ? » six fois. Le fer apparaît dans trois entrées distinctes (sèche + cycle, couverture fer, végétarienne). On lit aussi « Couverture vitamine b12 », « 3 autres nutriments est dans le même cas » (rc-core:14774) et « Il déclare boire » pour une femme (14929). Il faut regrouper par nutriment, avec une ligne par nutriment qui cumule ses motifs, une seule question « bilan sanguin » en pied si au moins un nutriment est concerné, et corriger les accords.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. risquesMicro(user) (≈ 14823) produit une liste {cle, lib, motif, question}, rendue telle quelle par renderCoachMicroSection (≈ 17715). Sur une athlète végétarienne en sèche longue avec cycle suivi et apports bas, le coach lit 9 blocs, dont 6 finissent par « Un bilan sanguin récent existe-t-il ? », et le fer apparaît dans 3 blocs séparés.

À faire (sans changer les règles de déclenchement) :

1. Ajouter une fonction PURE regrouperRisquesMicro(liste) qui fusionne par nutriment : 'fer' + 'couverture_fe' + la partie fer de 'b12_fer_vegetal' → un groupe « Fer » ; 'couverture_b12' + la partie B12 de 'b12_fer_vegetal' → « Vitamine B12 ». Chaque groupe = titre + motifs courts à puces + UNE question. Les entrées non nutritionnelles (stagnation, sopk, vitamineD, hydratation, magnesium/sommeil, traitement, recuperation) restent telles quelles.
2. La question « Un bilan sanguin récent existe-t-il ? » n'apparaît qu'UNE fois, en pied de la section, suivie des nutriments concernés (« ferritine, B12, zinc »).
3. Trier : groupes à plusieurs motifs d'abord, puis le reste.
4. Coquilles : libellés avec majuscule d'origine (ne pas faire toLowerCase sur « Vitamine B12 », écrire « vitamine B12 ») ; « N autres nutriments sont » (pluriel) ; motif hydratation accordé au sexe (« Elle déclare » / « Il déclare » / « Déclare » si inconnu), comme le fait déjà _htmlRecuperationCoach (« elle ou il »).

Cas limites : un seul motif (pas de regroupement visible) ; sexe inconnu ; liste vide (section cachée, inchangé).

Tests dans app/tests.js : la fixture végétarienne + sèche + cycle + apports bas donne au plus 6 blocs et une seule occurrence de « bilan sanguin récent » ; « vitamine B12 » avec le B majuscule ; « 3 autres nutriments sont » ; « Elle déclare » pour gender 'F'.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : au banc, la section se lit sans défiler sur un écran de 844 px pour la fixture ci-dessus.

5. Repères fixes : ni l'âge, ni la ménopause, ni l'intensité des règles, ni la grossesse ne comptent — Priorité : Moyenne · Effort : S

Analyse et idée. refMicro (rc-core:14415) ne lit que le sexe. Le fer est donc à 16 mg pour toutes les femmes, alors que l'Anses donne 11 mg pour des pertes faibles ou modérées et après la ménopause. Résultat : une femme ménopausée de 58 ans se voit signaler du fer bas à tort. L'app connaît pourtant déjà confCycle().intensiteRegles (90544), menopauseDeclaree (89944) et etatGrossesse (90660). Le calcium vaut 1 000 mg avant 25 ans. Pendant une grossesse, les repères changent (folates, iode, fer) : il faut se taire.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. refMicro(user,cle) (≈ 14415) choisit MICRO_REFS[cle].H ou .F selon le sexe seul. Le commentaire au-dessus documente le choix « fer F = 16 mg, borne prudente (pertes menstruelles élevées) ». Résultat : une femme ménopausée ou aux règles légères est comparée à 16 mg et

reçoit des signaux fer bas à tort (signalMicro, risquesMicro).

À faire, toujours avec la seule source Anses citée dans le commentaire :

1. Fer, femme : 11 mg si menopauseDeclaree(user) ou statut aménorrhée/contraception continue déclaré (cycleSansCycle), ou si confCycle(user).intensiteRegles est renseignée et ≠ 'difficile' ; 16 mg si 'difficile' ou si rien n'est renseigné (prudence conservée).
 2. Calcium : 1000 mg si l'âge (via la date de naissance, fonction d'âge existante) est < 25 ans, 950 mg sinon.
 3. Grossesse ou allaitement déclaré (etatGrossesse / grossesseSuspend) : couvertureMicro, signalMicro et la partie couverture de risquesMicro rendent null ou ne produisent rien, et la carte dit une seule fois « Repères suspendus pendant la grossesse : ton suivi médical prime. »
 4. refMicro rend en plus {motifRef} (« après la ménopause », « règles abondantes », « moins de 25 ans »). La phrase coach le cite entre parenthèses.
 5. Mettre à jour le commentaire de doctrine (sources, date de consultation).
- Cas limites : âge inconnu (950) ; sexe inconnu (max des deux, inchangé) ; intensité non renseignée (16) ; ménopause et règles déclarées en même temps (ménopause prime).
- Tests dans app/tests.js : femme ménopausée → 11 ; règles 'supportable' → 11 ; 'difficile' → 16 ; non renseigné → 16 ; 22 ans → calcium 1000 ; grossesse → couvertureMicro null et signalMicro null.
- Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
- Critère de fin : tests verts, et aucun signal fer chez une femme ménopausée dont les apports sont à 12 mg/j.

6. Compléments : il manque B12, fer, électrolytes, nitrates et bicarbonate, ainsi que les interactions du fer — Priorité : Moyenne · Effort : S

Analyse et idée. SUPPLEMENTS_LIST (rc-core:106098) compte 15 fiches. Il n'y a ni B12 (incontournable chez le végétalien), ni fer, ni électrolytes, ni nitrates de betterave ou bicarbonate (deux des cinq compléments jugés solides par le consensus du CIO de 2018), ni vitamine C ou calcium. Les interactions ne visent que les doublons d'acides aminés et le magnésium/ZMA. Le fer pris avec du calcium, du zinc, un multivitamine, ou avec le café et le thé déjà journalisés dans l'écran caféine, n'est jamais signalé. Il faut ajouter ces fiches avec leur niveau de preuve et ces interactions, dans le même format que l'existant.

Prompt de mise en place.

Contexte : app/rc-core.1792.js, SUPPLEMENTS_LIST (≈ 106098) : fiches {nom, dose, unite, momentsRecommandes, preuve A/B/C, note, formes?, interactions[{avec,message}], dureeAvantJugement?}. _suppInteractions (≈ 106230) signale deux produits actifs qui partagent un moment. _suppNorm doit rester injectif (un test le vérifie).

À faire :

1. Ajouter les fiches suivantes, dans le registre prudent de l'existant (jamais « carence », jamais de dose conseillée hors fourchette usuelle, « à voir avec un professionnel de santé » quand c'est médical) :
 - Vitamine B12 (preuve A chez le végétalien ; note : « Indispensable en alimentation végétalienne, à voir avec un professionnel de santé pour la forme et la dose. »)
 - Fer (preuve B ; note : « Uniquement après un bilan sanguin (ferritine). Avec de la vitamine C, loin du café, du thé et du calcium. » ; momentsRecommandes ['jeun','matin'])
 - Électrolytes / boisson d'effort (B ; intra)
 - Nitrates (jus de betterave) (A ; avant-entraînement, 2 à 3 h avant)
 - Bicarbonate de sodium (A ; avant-entraînement ; note sur les troubles digestifs, à fractionner)
 - Caséine (B ; coucher)
 - Vitamine C (C), Calcium (C)
 2. Interactions (déclarées d'un seul côté, comme aujourd'hui) : Fer↔Calcium, Fer↔Magnésium, Fer↔ZMA, Fer↔Multivitamines (« espace-les de deux heures ») ; Fer↔Vitamine C, avec un message POSITIF (« bonne association ») et un style neutre, pas ambre ; Calcium↔ZMA.
 3. Interaction transverse simple : si un complément « Fer » actif a un moment qui coïncide avec une prise de caféine notée le même jour (caffeineLog, entrées avec time), une seule ligne dans le cadre « Au même moment » : « Fer et café au même moment : le café réduit l'absorption du fer. »
 4. Vérifier que _suppNorm reste distinct pour les 23 fiches, et mettre à jour les icônes (SUPP_ICONES / img/complements.webp) avec une case existante générique si aucune ne correspond.
- Tests dans app/tests.js : unicité de _suppNorm ; Fer + Calcium au même moment → message ; à des

moments différents → rien ; Fer + Vitamine C → message positif ; `_suppFiche('Vitamine B12')` trouvée. Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` de `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices. Critère de fin : au banc, ajouter Fer + Magnésium au coucher affiche l'avertissement, et la liste d'ajout propose B12 et Nitrates.

7. Le coach ne voit pas si les compléments sont pris, et la whey cochée n'entre pas au journal — Priorité : Moyenne · Effort : S

Analyse et idée. Les prises cochées sont stockées dans `nutrition.supprises` (35 jours) et synchronisées. Pourtant, « Le coach ne voit pas la coche » (`rc-core:107918`). Pour la créatine ou la bêta-alanine (`dureeAvantJugement:4`), la régularité est justement ce que le coach doit juger. Testé au banc : cocher « Whey 30 g » n'ajoute rien aux protéines du jour. Il faut afficher côté coach « Pris 12 j sur 14 » sur chaque ligne de complément, et côté athlète, quand on coche un complément qui a un équivalent Ciqua (whey, caséine), proposer une fois « Ajouter au journal ? ».

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `nutrition.supprises = {'AAAA-MM-JJ': ['<id>@<moment>', ...]}` (≈ 107905, `supprisesDuJour`, `_supprisesEcrire`, `basculerPriseSupp`, `prendreToutSupp`). Les données remontent au coach avec le dossier, mais son écran (`#ccd-supplements`) n'en montre rien. Les fiches Whey/Caséine existent aussi comme aliments Ciqua locaux (ids -1 à -5 dans `app/data/ciqua.json`), mais cocher la prise n'ajoute rien au journal.

À faire :

1. Fonction `PURE observanceSupp(user, suppId, finISO, n=14) → {jours, pris}` : `jours` = jours où le produit était actif ET où l'athlète a coché au moins une prise de quoi que ce soit (sinon on ne sait pas : jour exclu, jamais compté comme oubli) ; `pris` = jours où ce produit a au moins une clé cochée.
 2. Côté coach, sur chaque ligne de complément, une mention discrète « pris 12 j / 14 » (ou rien si `jours < 4`). Pour une fiche avec `dureeAvantJugement`, ajouter « régularité : N semaines ». Aucune couleur d'alerte.
 3. Côté athlète, quand `basculerPriseSupp` passe à « pris » pour un complément dont `_suppFiche` est Whey ou Caséine, et seulement la première fois par jour : un toast avec action « Ajouter 30 g au journal » qui crée l'entrée Ciqua locale (`quantité = dosage_quantity` si l'unité est g, sinon 30 g) au repas correspondant au moment. L'annulation de la coche ne retire PAS l'aliment, mais le toast propose « Annuler » pendant 5 s (même mécanisme que l'annulation d'ajout existante).
- Cas limites : produit désactivé en cours de période ; double tap sur la coche (pas de double entrée au journal) ; diète stricte (pas de journal : pas de proposition) ; consentement santé refusé (inchangé). Tests dans `app/tests.js` : `observanceSupp` ignore les jours sans aucune coche ; 12/14 correct ; un double basculement ne crée qu'une entrée ; diète stricte → aucune proposition.
- Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` de `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.
- Critère de fin : au banc, le coach lit « Créatine · pris 12 j / 14 » sur la fiche de l'athlète sans ouvrir aucun menu.

8. Oméga-3 et vitamine D invisibles : un repère « poisson gras de la semaine » coûte une ligne — Priorité : Moyenne · Effort : S (repère) / M (extraction Ciqua)

Analyse et idée. Le convertisseur n'extrait que 8 micronutriments (`COLS_MICRO`, `scripts/ciqua_convert.py:225`). Les folates ne couvrent que 908 fiches sur 3 495. La vitamine D, l'EPA et le DHA, que Ciqua publie, sont absents. Côté vitamine D, l'app se limite à une question saisonnière au coach. Le plus simple et le plus parlant est un repère PNNS, « Poisson gras : 1 sur 2 cette semaine », détecté par le nom dans le journal (saumon, sardine, maquereau, hareng, truite, anchois). L'extraction de la vitamine D et de l'EPA+DHA peut venir ensuite.

Prompt de mise en place.

Contexte : app/rc-core.1792.js (journal nutrition, _renderFjDaySummary ≈ 100694 ; bloc micro _htmlCouvertureMicro ≈ 14485) et scripts/ciqual_convert.py (COLS_MICRO ≈ 225). Aucun suivi des oméga-3 ni de la vitamine D par l'alimentation.

Étape 1 (S) :

1. Fonction PURE poissonGrasSemaine(user, finISO) → {portions, jours} : compte les entrées des 7 derniers jours dont le nom normalisé (_microNorm) contient l'un de MICRO_POISSONS_GRAS = ['saumon', 'sardine', 'maquereau', 'hareng', 'truite', 'anchois', 'thon rouge'] et dont la quantité est ≥ 80 g (une portion ; 2 portions si ≥ 200 g). Exclure les sandwichs et plats où le poisson n'est qu'un ingrédient (groupe « entrées et plats composés » : compter alors ½ portion).

2. Une ligne dans la carte Micronutriments : « Poisson gras : N / 2 cette semaine (repère PNNS) ». Elle est masquée si _microVege(user) est vrai ; dans ce cas, et seulement si aucun complément oméga-3 n'est actif, on affiche à la place « Oméga-3 végétaux : noix, colza, lin ». Même garde aTCA que la carte.

3. Côté coach, dans risquesMicro, la règle vitamineD (≈ 14870) ajoute au motif « poisson gras : N portion(s) sur 7 jours » quand le journal est renseigné.

Étape 2 (M), dans le même lot si la source Ciqua est disponible :

4. Ajouter à COLS_MICRO les colonnes Ciqua Vitamine D (µg/100 g) et AG 20:5 EPA + AG 22:6 DHA (g/100 g), sous les clés 'vd' et 'o3' (somme EPA+DHA), en vérifiant les indices sur l'en-tête réel du fichier, et régénérer app/data/ciqua.json. Ajouter 'vd' (Anses : 15 µg) à MICRO_REFS. 'o3' (repère Anses : 500 mg/j EPA+DHA) reste en dehors de MICRO_REFS pour ne pas déclencher signalMicro, et sert seulement à afficher le repère.

Cas limites : produit OFF (pas de champ : non compté, jamais zéro) ; végétalien ; 0 entrée sur 7 jours (rien).

Tests dans app/tests.js : 2 entrées saumon 120 g → 2 ; sandwich thon → ½ ; végétarienne → ligne poisson absente ; aTCA → rien.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : au banc, journaliser une sardine affiche « Poisson gras : 1 / 2 cette semaine ».

9. Sel : la tuile présente 3,8 g comme un objectif à remplir alors que le sel de cuisson n'est jamais compté — Priorité : Basse · Effort : S

Analyse et idée. Testé au banc : la tuile affiche « Sel 1,1 / 3,8 g » comme une macro à atteindre (TUILES_NUT, rc-core:92846 ; cible issue de _selPourGrille, 92911). Mais les aliments Ciqua de base sont « sans sel ajouté », si bien que le journal sous-estime toujours le sel. On lit aussi « 3.8 g » avec un point dans « Tes cibles » et « 1,1 » avec une virgule ailleurs. Il faut présenter le sel comme une fourchette (cible du moteur → 5 g, repère OMS, ou plafond du moteur si les séances sont longues), rappeler en une ligne que la cuisine n'est pas comptée, et ajouter un bouton « + pincée » (0,5 g).

Prompt de mise en place.

Contexte : app/rc-core.1792.js. La tuile Sel (TUILES_NUT ≈ 92846, htmlGrilleTuilesNut, _selPourGrille ≈ 92911) affiche « total / cible » comme pour les protéines. La cible vient du moteur sodium (cibleSodiumJour), souvent au plancher 1500 mg Na = 3,8 g de sel. Les aliments Ciqua « sans sel ajouté » ne portent pas le sel de cuisson : le total est toujours sous-estimé. « 3.8 » s'affiche avec un point dans les cibles.

À faire :

1. Tuile Sel : afficher « 1,1 g · repère 3,8–5 g » (borne haute = max(5, cible) plafonnée à sodiumMgToSaltG(SODIUM_CONFIG.plafondAthleteMg) ; si ceiling.motif==='MEDICAL', borne haute = cible). Barre neutre ; ambre seulement au-dessus de la borne haute, jamais en dessous de la borne basse.

2. Sous la ligne sel du journal (_htmlSelSemaine ≈ 66149), ajouter : « Le sel ajouté en cuisine n'est pas compté. » et un petit bouton « + pincée de sel » qui ajoute l'entrée « Sel iodé, pincée » (0,5 g de sel, 0 kcal, iode depuis la fiche Ciqua du sel iodé si présente) au dernier repas saisi du jour. Annulable comme un ajout normal.

3. Formatage : toutes les valeurs de sel en virgule décimale (fonction de formatage existante).

Cas limites : plafond médical (hypertension déclarée) ; aucun aliment saisi (pas de bouton) ; double tap sur « + pincée » (deux pincées assumées, l'annulation existe).

Tests dans app/tests.js : la tuile ne contient jamais « 3.8 » ; motif MEDICAL → pas de fourchette au-delà de la cible ; « + pincée » ajoute 0,5 g et 0 kcal.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, puis python3

scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : au banc, la tuile ne se lit plus comme « il te manque 2,7 g de sel ».

10. Eau : une erreur de saisie devient définitive après un rechargement, et la veille est impossible à corriger — Priorité : Basse · Effort : S

Analyse et idée. « Annuler » s'appuie sur `_eauAjouts`, une liste qui ne vit que le temps de la session (rc-core:65975). Après un rechargement ou une mise en arrière-plan sur Android, un « +500 ml » tapé par erreur ne peut plus être retiré. `_htmlEauSuivi` (66016) ne connaît que la date du jour, si bien qu'on ne peut rien noter pour hier. Il faut que le chiffre bu soit modifiable (appui sur « 1,5 L bus » → champ numérique) et que le suivi suive le jour affiché par le journal.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Suivi de l'eau : nutrition.eau[dateISO] = ml (≈ 65963), `_htmlEauSuivi(u,on)` (≈ 66016) ne lit que `localISODate(new Date())`, ajouterEau / annulerEau s'appuient sur `_eauAjouts` (≈ 65975), une liste en mémoire perdue au rechargement. Erreur humaine réelle : un « +500 ml » de trop ne se corrige plus après un rechargement, et l'eau d'hier ne se note pas.

À faire :

- `_htmlEauSuivi(u,on,dateISO)` suit la date affichée par le journal (`_fjDate`) ; `_renderFjDaySummary` la passe. Pas de saisie dans le futur.
- Rendre le libellé « X L bus » tapable : il ouvre un petit champ numérique en litres (virgule acceptée, borné 0–10, même borne que `EAU_SAISIE_MAX_ML`), avec Valider / Annuler, qui écrit la valeur exacte. « Annuler » (dernier ajout) reste pour le geste rapide de la session.
- Côté coach, la moyenne sur 7 jours (`eauMoyenne7j`) reste inchangée.

Cas limites : saisie vide (pas d'écriture) ; « 1,5 » et « 1.5 » ; valeur > 10 L (refus avec message) ; 0 (efface le jour : absent, pas zéro, comme la règle existante).

Tests dans app/tests.js : écrire 1,5 L sur la veille ; valeur 12 refusée ; 0 supprime la clé du jour ; aucune écriture pour une date future.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les boutons.

Critère de fin : au banc, après rechargement, on corrige 2,5 L en 2 L en deux gestes, et on note l'eau d'hier depuis la flèche ←.

Expert 6 · Base d'aliments — 6,5/10

La base est saine et complète : c'est la table Ciquel 2025 de l'Anses en entier (3 484 aliments, version du 3/11/2025, publiée le 19/11/2025), plus 11 génériques sportifs RepCore, sans aucun doublon d'id ni de nom. Au banc, 85 % des 226 aliments courants testés sortent dans les 3 premiers résultats. Ce qui manque le plus, c'est la langue de l'athlète : 20 aliments présents dans la base renvoient « Aucun résultat » parce qu'ils ne portent pas leur nom courant (crème fraîche, lait d'amande, thon en boîte, coca, big mac, nutella...). Aucune faute de frappe n'est tolérée, et les portions manquent justement sur les produits sportifs.

Méthode (testé au banc, port 8860-8862). J'ai lancé la vraie fonction `onFjSearch` dans la page sur 226 requêtes réalistes (14 familles) et 31 requêtes « maladroites », puis lu le DOM (`#fj-results-list`). Un aliment compte comme trouvé si la bonne fiche est dans les 3 premiers résultats. Fichiers : `audit2/06/cas.py` (liste et critères), `res.json` (résultats bruts), `portions.json`.

Famille	Trouvé (top 3)	Famille	Trouvé (top 3)
Petit-déjeuner	26/32	Fruits	14/14
Féculents	21/22	Légumes	15/15
Viandes	25/26	Snacks, huiles	11/12
Poissons	13/14	Sauces	5/5

Famille	Trouvé (top 3)	Famille	Trouvé (top 3)
Laitiers	16/19	Produits sportifs	10/18
Boissons	5/8	Fast-food, chaînes	13/20
Plats préparés	10/13	Boulangerie	8/8

Total : 192/226 (85,0 %), dont 178 classés en 1^{re} position (78,8 %). Les 34 échecs se répartissent ainsi : - **20 aliments présents en base mais 0 résultat** (problème de vocabulaire) : corn flakes, nutella, lait d'amande, lait de soja, granola, thon en boîte, crème fraîche, barre de céréales, coca, coca zéro, red bull, big mac, whopper, frites mcdo, tenders, california roll, pad thaï, boisson protéinée, skyr protéiné, protéine vegan. - **5 mal classés** (la bonne fiche est au-delà du 3^e rang) : pain, spaghetti, nems, jambon blanc, chèvre. - **9 vraiment absents** : cottage cheese, créatine, BCAA, gainer, pâte à tartiner protéinée, pudding protéiné, tacos français, poke bowl, KFC.

Sur les requêtes maladroites : fautes de frappe 4/12, synonymes 11/15, casse et ordre des mots 4/4.

1. La recherche ne parle pas la langue de l'athlète (20 aliments en base répondent « Aucun résultat ») — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc : « crème fraîche » renvoie 0 résultat alors que « Crème 30% MG, épaisse » existe. Même chose pour « lait d'amande » (la fiche s'appelle « Boisson à l'amande »), « thon en boîte » (« appertisé »), « coca » (« Cola »), « nutella », « corn flakes », « red bull », « big mac », « pdt », « chocolatine » et « yogourt ». Le filtre `_fjContientTous` (rc-core.1792.js:96417) exige tous les mots tels quels, et il n'existe aucune table de synonymes. Je propose un champ `a` (alias) généré dans `ciqual.json` et consulté par le filtre, pour que le nom courant mène à la fiche officielle.

Prompt de mise en place.

Contexte : RepCore, recherche d'aliments. Les noms Ciqual sont des noms de laboratoire (« Boisson à l'amande », « Crème 30% MG, épaisse », « Thon, au naturel, appertisé »), et l'athlète tape « lait d'amande », « crème fraîche », « thon en boîte ». Le filtre `_fjContientTous` (app/rc-core.1792.js~96417) cherche chaque mot dans `f.s` et ne trouve rien. Mesuré : 20 aliments PRÉSENTS en base renvoient « Aucun résultat ».

À faire :

- Créer `scripts/alias_aliments.tsv` (UTF-8, une ligne par alias : `alias<TAB>id_ciqual[,id2...]` <TAB>commentaire). Au moins 250 alias, dont obligatoirement : crème fraîche→Crème 30% MG épaisse et semi-épaisse et légère ; lait d'amande / lait de soja / lait d'avoine→Boisson à l'amande / au soja / à l'avoine (aliment moyen d'abord) ; thon en boîte / thon conserve→Thon au naturel appertisé égoutté ; coca / coca-cola / soda→Cola sucré ; coca zéro / coca light→Cola avec édulcorants ; red bull / monster / boisson énergétique→Boisson énergisante ; nutella→Pâte à tartiner chocolat et noisette ; corn flakes→Pétales de maïs nature ; granola→Muesli croustillant ; barre de céréales→Barre céréalière ; big mac / whopper / burger→Hamburger et Cheeseburger de restauration rapide ; frites mcdo / frites fast-food→Frites ; tenders / nuggets→Nuggets de volaille ; california / sushi→Sushi ou maki ; pad thaï→Nouilles aux crevettes sautées ; pdt / patate / patates→Pomme de terre ; chocolatine→Pain au chocolat ; yogourt / yoghourt→Yaourt ; jambon blanc / jambon de Paris→Jambon cuit supérieur et choix ; steak / steak haché→Boeuf steak haché 5 %, 10 %, 15 % ; blanc de poulet / filet de poulet→Poulet filet ; skyr protéiné→Skyr nature ; boisson protéinée→Spécialité laitière riche en protéines ; protéine vegan→Protéine végétale (id -4).
- Dans `scripts/ciqual_convert.py` (modes `xlsx` ET `--depuis-json`) : lire ce TSV, poser sur chaque fiche visée un champ `"a": ["lait d amande", ...]` DÉJÀ normalisé comme `f.s` (minuscules, sans accents, æ→oe). Refuser la génération (exit 1) si un id du TSV n'existe pas dans la table : un alias orphelin est une recherche cassée.
- Dans `_fjContientTous`, ou dans un nouveau helper `_fjTexteRecherche(f)` qui rend `f.s + ' | ' + (f.a|| []).join(' | ')`, faire porter le filtre ET `_classerAliments` sur ce texte. Un alias qui correspond EXACTEMENT à la requête vaut le bonus « le nom EST la requête » (+200). Appliquer le même helper dans `onFjSearch` (~99543), `onPlanSearch` (~94987), `_rendreResultatsIngredient` (recettes, ~97425) et la recherche coach (~95012) : une seule fonction décide.
- Côté affichage, ne rien changer : on montre le nom Ciqual. Sous le nom, en `var(--fs-2xs)` et couleur `var(--sub)`, ajouter « trouvé via « lait d'amande » » seulement quand c'est l'alias qui a fait

correspondre (sinon l'athlète ne comprend pas pourquoi « Boisson à l'amande » sort).

Cas limites : un alias de deux mots ne doit pas faire remonter des fiches qui n'ont qu'un des deux mots. Un alias trop court (moins de 3 lettres) n'est accepté que s'il est listé exprès (« pdt »). Un alias ne doit pas masquer les aliments perso ni ceux du coach, qui restent devant.

Tests (app/tests.js, bloc existant de la recherche Cigual, près de la ligne ~21709) : avec la table chargée, chacune des 20 requêtes « crème fraîche, lait d'amande, lait de soja, thon en boîte, coca, coca zero, red bull, nutella, corn flakes, granola, barre de cereales, big mac, whopper, frites mcdo, tenders, california roll, pad thai, pdt, chocolatine, yogourt » rend au moins 1 résultat, et la fiche attendue est dans les 3 premiers. Ajouter une non-régression : « riz », « poulet », « oeuf » gardent la même fiche en 1^{re} position.

Fin : la suite passe. Nouveau numéro de build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, et CIQUAL_VERSION de sw.js puisque cigual.json change), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

2. Le classement met des faux amis en tête (pain perdu, chevreuil, eau minérale) — Priorité : Haute • Effort : M

Analyse et idée. Testé au banc : « pain » sort « Pain perdu, Pain panini, Pain pita », et la baguette n'arrive qu'en 10^e position (capture 06/pain.png). « nems » sort « Salade verte, sans assaisonnement » puis des eaux minérales, parce que « nem » est contenu dans « assaisonnement ». « chèvre » sort « Chevreuil, Chevreau » ; « pizza » sort le fromage râpé ; « beurre » sort le beurre de cacahuète ; « banane » la plantain ; « kebab » la sauce ; « spaghetti » la courge ; « poulet cuit » le cœur de poulet. Cause : un filtre par sous-chaîne (rc-core:96417) et aucune notion d'aliment de référence ni d'habitude de l'athlète dans _classerAliments (96429). Je propose une correspondance en début de mot, une fiche de référence par mot de tête et un bonus pour ce que l'athlète mange souvent.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js, _fjContientTous (~96417) et _classerAliments (~96429). Mesuré au banc sur la vraie recherche : « pain »→Pain perdu en 1er, la baguette en 10e ; « nems »→Salade verte (le mot « assaisonnement » contient « nem ») ; « chevre »→Chevreuil ; « pizza »→Mélange de fromages râpés ; « beurre »→Beurre de cacahuète ; « banane »→Banane plantain ; « kebab »→Sauce kebab ; « spaghetti »→Courge spaghetti ; « poulet cuit »→Coeur de poulet ; « oeufs »→Oeufs au lait.

À faire :

1. Filtre : un mot de la requête doit correspondre au DÉBUT d'un mot du nom (regex \b + mot), et non plus n'importe où dans la chaîne. Garder le pluriel (_fjFormes). Exception : un mot de 6 lettres ou plus peut encore correspondre en sous-chaîne, pour les mots composés (« chou-fleur »), mais avec un malus de -40.
2. Fiches de référence : dans scripts/cigual_convert.py, un TSV scripts/references_aliments.tsv (mot_de_tete<TAB>id) d'environ 120 lignes, par exemple pain→7001 Pain blanc (baguette), banane→13005, beurre→Beurre doux, pizza→Pizza au fromage ou margherita, kebab→Sandwich grec ou kebab, spaghetti→Pâtes sèches standard cuites, chevre→Fromage de chèvre lactique affiné, nems→Nem aliment moyen, jambon→Jambon cuit supérieur, poulet→Poulet filet cru, oeuf→Oeuf cru, riz→Riz blanc cuit. Le script pose "r":1 sur ces fiches. Dans _classerAliments, +150 quand f.r et que le premier mot de la requête est le mot de tête.
3. Habitudes : +80 si l'id est dans les favoris (_fjFavs), +60 s'il est dans usageFoods avec n ≥ FJ_FREQUENT_MIN, +30 s'il est dans recentFoods. L'athlète qui mange toujours du « Riz basmati cuit » doit le voir en premier quand il tape « riz ». Fonction PURE _bonusHabitue(f,user), testable.
4. Malus « bruit » : -60 pour les eaux minérales de marque (86 fiches « Eau minérale X »), les fiches « prélevé à la Martinique » (79) et le groupe « aliments infantiles » (39), SAUF si la requête contient le mot distinctif (« volvic », « martinique », « bébé »/« infantile »).

Cas limites : le tri doit rester stable, et la recherche plan coach (onPlanSearch), les recettes et les aliments perso utilisent le même classement. Une requête vide de résultats après le filtre en début de mot retombe sur l'ancien filtre par sous-chaîne, pour ne jamais rendre moins qu'avant.

Tests (app/tests.js) : avec Cigual chargée, la 1^{re} position vaut, pour pain : un Pain blanc ou de tradition ; nems : un Nem ; chevre : un fromage de chèvre ; pizza : une Pizza ; beurre : un Beurre (pas de cacahuète) ; banane : Banane chair crue ; kebab : le Sandwich kebab ; spaghetti : des Pâtes ; poulet cuit : une viande de poulet rôtie ou cuite. _bonusHabitue : un favori passe devant la fiche de référence. Les 192 cas déjà réussis du lot audit (riz, oeuf, whey, skyr, steak haché...) ne régressent pas.

Fin : suite verte. Nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js,

3. Lot « RepCore génériques v2 » : 9 vrais manques, et les chaînes de restauration — Priorité : Haute · Effort : M

Analyse et idée. C'est bien le Ciqua 2025 complet (3 484 fiches, contre environ 3 186 en 2020), et les 11 génériques sportifs (ids -1 à -11, scripts/ciqual_convert.py:145) sont en place. Restent absents : cottage cheese, gainer, pâte à tartiner protéinée, pudding protéiné, créatine et BCAA (à 0 kcal, mais attendus), tacos français, poke bowl, le poulet frit façon KFC, et les produits phares des chaînes (Big Mac, McFlurry, wrap KFC, menu sushi). Je propose un lot d'environ 150 fiches vérifiées, avec une source par ligne, généré par un script reproductible et sans toucher aux ids existants. Le poids ajouté est d'environ 30 Ko brut, soit environ 7 Ko après gzip, pour une base qui pèse aujourd'hui 876 Ko brut et 168 Ko après gzip.

Prompt de mise en place.

Contexte : RepCore. app/data/ciqual.json = Ciqua 2025 (3 484 fiches, id>0) + 11 génériques RepCore (ids -1 à -11, src:'repcore', liste GENERIQUES dans scripts/ciqual_convert.py ~145). L'athlète musculation ne trouve pas, même hors ligne : cottage cheese, gainer, pâte à tartiner protéinée, pudding/crème dessert protéinée, créatine, BCAA, tacos français, poke bowl, poulet frit pané type fast-food, ni les produits phares des chaînes.

À faire :

- Sortir les génériques du code Python vers scripts/repcore_generiques.csv (UTF-8, séparateur ;), colonnes : id;nom;groupe;k;p;c;l;f;e;sel;portion_lib;portion_g;alias;source;date_source. Reprendre les ids -1 à -11 À L'IDENTIQUE (des plans de coach les référencent : ne jamais renuméroter ni réutiliser un id). Les nouveaux ids partent de -12, et un id supprimé est réservé à vie (colonne commentée « retiré »).
- Ajouter environ 140 lignes en 4 lots.
 - Sport : whey aromatisée chocolat et vanille, gainer, BCAA (0 kcal), créatine (0 kcal), pâte à tartiner protéinée, crème dessert protéinée, pudding protéiné, barre protéinée type « crispy », cookie protéiné, pancake protéiné (préparation), porridge protéiné, eau de coco, boisson de récupération, compote à boire.
 - Laitiers et marché : cottage cheese, fromage blanc 0 % « protéiné », yaourt grec 0 %, lait sans lactose, lait d'avoine barista.
 - Chaînes (le nom commence par le générique, la marque n'est qu'un alias) : « Burger double fromage, restauration rapide (type Big Mac) », Whopper, McChicken, nuggets x6, frites portion moyenne, McFlurry, poulet frit pané (type KFC), wrap, tacos français poulet, kebab assiette, poke bowl saumon et poulet, plateau sushi x8, pizza 4 fromages.
 - Supermarché : salade composée poulet, wrap poulet, sandwich triangle, bowl riz et poulet.
- Règles de valeurs : (a) priorité à une fiche Ciqua proche, documentée (« = Ciqua 25600 ») ; (b) sinon USDA FoodData Central (domaine public CC0), avec le numéro FDC ; (c) sinon la moyenne d'au moins 3 étiquettes françaises lues sur Open Food Facts (codes EAN dans la colonne source), ou le tableau nutritionnel publié par la chaîne en France, avec sa date. Aucune valeur sans source.
- Le script vérifie et REFUSE (exit 1) : $|k - (4p + 4c + 9l + 2f)| > 15 \%$; $p + c + l + f + e > 105 \text{ g}$; un id en double ; un nom ou un alias déjà pris par une fiche Ciqua ; une source vide. Il écrit ensuite le JSON trié et stable (même entrée, même sortie, octet pour octet), app/data/ciqual.version (date), et affiche le delta (ajouts, modifications, retraits).
- App : afficher sous le nom d'une fiche src:'repcore' la mention « Valeur moyenne RepCore » (var(--fs-2xs), var(--sub)), et compléter la ligne « Source : Anses... » en bas de #s-food-search par « + aliments génériques RepCore (sources par fiche) ».

Cas limites : un plan qui référence un id négatif retiré continue de s'afficher grâce à ciqualRef (_planCiqua ~94253) : le vérifier. Les valeurs à 0 kcal (créatine, BCAA) ne doivent pas déclencher l'estimation 4/4/9.

Tests (app/tests.js) : toutes les fiches id<0 ont src:'repcore' et k, p, c, l non nuls ; cohérence 4/4/9 à 15 % près ; ids -1 à -11 inchangés ; « cottage », « gainer », « creatine », « bcaa », « pate a tartiner proteinee », « tacos », « poke », « kfc », « big mac » rendent la fiche attendue dans les 3 premiers. Ajouter un test Node dans scripts/verif/ qui relance le script deux fois et compare les sorties (reproductibilité).

Fin : python3 scripts/ciqual_convert.py --depuis-json régénère tout. Suite verte. Nouveau build

(window.RC_BUILD dans app/index.html, CACHE et CIQUAL_VERSION de app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

4. Les portions sont fausses ou absentes là où l'athlète en a le plus besoin — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc (unitesPour) : la whey n'a pas de « dose », le skyr pas de « pot », la barre protéinée pas de « 1 barre » ni le gel de « 1 gel », car le groupe « produits pour sportifs » n'a aucune unité. À l'inverse, le lait se voit proposer « pot = 125 g » sans « verre », et l'emmental « pot 125 g » plus « paume 120 g » : c'est le cas des 174 fiches de fromage, lait et crème sur les 356 produits laitiers (rc-core:96685). Croissant, burger, tranche de jambon, biscotte, galette de riz et tortilla n'ont aucune unité ; 24 aliments seulement ont la leur (96641). Je propose de porter les portions dans les données, avec un champ `u` généré, plutôt que dans le code.

Prompt de mise en place.

Contexte : RepCore, saisie en unités naturelles. FJ_UNITES_ALIMENT (app/rc-core.1792.js ~96641) ne couvre que 24 ids. FJ_UNITES_GROUPE (~96685) donne « pot 125 g » à TOUT le groupe « produits laitiers » : le lait, l'emmental, le parmesan et la crème reçoivent un pot, et le lait n'a pas de verre. Le groupe « produits pour sportifs » n'a rien : la whey se saisit en grammes, sans dose.

À faire :

- scripts/portions_aliments.tsv : id_ou_motif<TAB>cle<TAB>libelle<TAB>pluriel<TAB>g<TAB>source. Environ 200 lignes, avec un motif par regex sur f.s accepté pour couvrir une famille. Exemples : whey et isolat→dose 30 g ; caséine→dose 30 g ; skyr→pot 150 g ; yaourt→pot 125 g ; fromage blanc→pot 100 g et cuillère à soupe 30 g ; petit-suisse→pot 60 g ; lait et boissons végétales→verre 250 ml et bol 300 ml ; fromages à pâte dure→portion 30 g et tranche 20 g ; fromage râpé→poignée 20 g ; jambon cuit→tranche 45 g ; blanc de poulet ou de dinde en tranche→tranche 30 g ; steak haché→steak 100 g et 125 g ; filet de poulet→filet 130 g ; barre protéinée→barre 60 g ; barre céréalière→barre 25 g ; gel→gel 35 g ; croissant→pièce 55 g ; pain au chocolat→pièce 65 g ; baguette-1/4 de baguette 60 g ; biscotte→pièce 8 g ; galette de riz→pièce 9 g ; tortilla→pièce 60 g ; hamburger et cheeseburger→pièce ; pizza→part 1/4 et pizza entière ; riz, pâtes et semoule cuits→assiette 200 g ; flocons d'avoine→cuillère à soupe 10 g et bol 60 g ; beurre de cacahuète→cuillère à soupe 15 g ; banane, pomme, œuf comme aujourd'hui. Pour la source, prendre le recueil « Portions alimentaires » SU.VI.MAX / Inserm, les portions types de l'étude INCA3 (Anses), ou la portion de l'étiquette la plus vendue.
- ciqual_convert.py écrit "u":[{"c":"dose","l":"dose","p":"doses","g":30}] sur chaque fiche visée (les ids prennent le pas sur les motifs), et refuse un $g \leq 0$ ou > 1500 .
- unitesPour(aliment) lit d'abord aliment.u, puis FJ_UNITES_ALIMENT (gardé en repli tant que l'ancien JSON peut être en cache), puis les groupes. Retirer « pot » du groupe « produits laitiers » et le laisser au seul aliment ou motif. Garder « g » toujours en premier. Les repères paume, poing et pouce ne s'affichent plus pour les fromages (portionsPourGroupe : exclure les motifs fromage, beurre et crème).
- Préselection : quand une fiche a une unité u, le sélecteur s'ouvre sur cette unité avec $n = 1$. Exception : si l'athlète a déjà saisi cet aliment (fjDerniereQty), on garde sa dernière quantité. Cela retire une étape sur les aliments les plus saisis (whey, œuf, skyr).
Cas limites : l'aller-retour unité-g-unité ne dérive pas (qty reste la seule vérité, comme aujourd'hui) ; une portion fabricant OFF garde la priorité sur un produit de marque ; une demi-unité est acceptée.

Tests (app/tests.js, bloc « Critère 5 » ~30183) : whey -2→contient dose 30 ; skyr -6→pot 150 ; lait demi-écrémé→verre, mais pas pot ; emmental→pas de pot ni de paume ; jambon cuit supérieur→tranche ; croissant→pièce ; un aliment sans u ni groupe→uniquement g.

Fin : suite verte. Nouveau build (window.RC_BUILD dans app/index.html, CACHE et CIQUAL_VERSION de app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

5. Zéro tolérance aux fautes : « pouelt », « yahourt », « saumond » ne renvoient rien — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : 4 requêtes avec faute sur 12 aboutissent. « pouelt », « yahourt », « yaourth », « cacahouète », « spaghetti » et « saumond » renvoient 0 résultat, et « poulé » sort « Poule ».

`_fjNorm` (96421) ne fait qu'ôter les accents. Je propose un second passage tolérant, uniquement quand le premier ne trouve rien, avec un bandeau « Résultats pour « poulet » » (un geste au lieu de retaper).

Prompt de mise en place.

Contexte : RepCore, recherche d'aliments (`onFjSearch` ~99543, `onPlanSearch` ~94987, recettes ~97425 dans `app/rc-core.1792.js`). Mesuré : « pouelt », « yahourt », « yaourth », « cacahouete », « spaghetti », « saumond » → 0 résultat ; « poulé » → Poule avant Poulet.

À faire :

1. Fonction PURE `_fjPhonétique(mot)` appliquée à la requête ET au vocabulaire : h muet supprimé (yahourt→yaourt), ou→u devant voyelle (cacahouete→cacahuete), ph→f, gh→g, double consonne simplifiée, d ou t final muet supprimé (saumond→saumon).

2. Vocabulaire : à la première recherche, construire une fois un Set des mots distincts de f.s et des alias (environ 6 000 mots), en mémoire seulement.

3. Si le filtre normal rend 0 fiche, corriger chaque mot de la requête vers le mot du vocabulaire le plus proche : Damerau-Levenshtein ≤ 1 pour 4 à 6 lettres, ≤ 2 au-delà, égalité départagée par la fréquence du mot dans la table. Relancer la recherche avec la requête corrigée, et afficher en tête « Résultats pour « poulet » · Rechercher « pouelt » quand même » (lien, `var(--fs-xs)`).

4. Ne jamais corriger un mot qui a déjà des résultats, ni un mot de moins de 4 lettres. Ne jamais corriger silencieusement : le bandeau est obligatoire.

Tests (`app/tests.js`) : la correction trouve les 6 cas ci-dessus (1re fiche attendue : Poulet, Yaourt, Cacahuète, Pâtes ou Spaghetti, Saumon) ; « riz » et « oeuf » ne déclenchent aucune correction ; « xqzw » rend « Aucun résultat » sans plantage ; le calcul du vocabulaire prend moins de 50 ms sur la table (mesuré avec `performance.now`).

Fin : suite verte. Nouveau build (`window.RC_BUILD` dans `app/index.html`, CACHE de `app/sw.js`), puis `python3 scripts/versionner_actifs.py`. Ne pas agrandir les polices.

6. Produits scannés : perdus hors ligne, introuvables par leur nom, impossibles à épingler — Priorité : Moyenne · Effort : M

Analyse et idée. Hors ligne, le bouton « Scanner » disparaît (`rc-core:98364`) et `offParEAN` refuse (99021) ; rien ne garde le code pour plus tard, alors que la salle est souvent sans réseau. Un produit de marque déjà mangé, par exemple la whey de l'athlète scannée hier, n'est ni favorable (99529) ni retrouvable en tapant « whey » : il ne revient que dans « Récents », sur une recherche vide. Je propose un carnet local « Mes produits » des produits scannés et une file de scans hors ligne.

Prompt de mise en place.

Contexte : RepCore, Open Food Facts (`app/rc-core.1792.js` : `offParEAN` ~99018, `scanTraiterCode` ~99342, `selectOffFood` ~99513, `_renderFjRecent` ~98377, bouton Scanner masqué hors ligne ~98364). Un produit OFF n'est ni favori ni cherchable par nom, et le scan hors ligne est impossible.

À faire :

1. Carnet « Mes produits » : à chaque ENREGISTREMENT d'un produit OFF au journal, ranger dans `currentUser.nutrition.produits` (au plus 40, du plus récent au plus ancien) {ean, nom, marque, portion, p, c, l, k, f, sel, vu:date, src:'off'}. Ce n'est pas une copie de la base OFF : ce sont les produits que l'athlète a lui-même consommés, avec l'attribution OFF affichée sur la fiche. Écrire l'attribution en commentaire (licence ODbL 1.0 et DbCL).

2. Recherche : ces produits passent par le même filtre que les aliments perso (`_fjContientTous` sur le nom normalisé + marque), dans une section « Mes produits » juste après « Mes aliments ». Ils peuvent être épinglés : la clé de favori 'off:'+ean est acceptée par `_fjFavs` et `toggleFavFood`.

3. Scan hors ligne : le bouton Scanner reste visible. Hors ligne, le code lu est cherché d'abord dans « Mes produits » (succès immédiat) ; sinon il est mis en file (`nutrition.scansEnAttente`, au plus 10, {ean, date, repas, qty}) avec le message « Produit mis de côté : il sera complété au retour du réseau ». Au retour en ligne (événement online, et au démarrage), `offParEAN` pour chaque code, puis un toast « 2 produits scannés à confirmer » qui ouvre la fiche pré-remplie : l'athlète valide la quantité. Rien n'est ajouté au journal sans confirmation.

4. Erreurs humaines : même code scanné deux fois en 10 s → un seul ajout ; un produit supprimé du carnet garde ses entrées de journal ; un code inconnu d'OFF au retour du réseau → « Créer cet aliment à la main » (le chemin existe déjà).

Tests (`app/tests.js`) : produit enregistré-présent dans « Mes produits » et trouvé par « whey » ; épinglage 'off:...' qui survit à un rechargement ; `navigator.onLine=false` + EAN connu du carnet→fiche ouverte sans réseau ; EAN inconnu hors ligne-mis en file ; retour en ligne simulé (`offParEAN` bouchonné)→toast de confirmation, et aucune écriture au journal avant validation ; dédoublement du

double scan.

Fin : suite verte. Nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

7. Une ligne de résultat qui montre les protéines, pas le groupe Ciqual — Priorité : Moyenne · Effort : S

Analyse et idée. `_fjResultHtml` (98576) affiche les kcal et le groupe Ciqual, parfois trompeur : « Pain perdu · produits laitiers », « Flocons d'avoine · produits sucrés ». Le pratiquant de musculation choisit pourtant sur les protéines, et le plan coach les montre déjà (rc-core:95028). Par ailleurs, 685 noms dépassent 60 caractères (« Chayote ou christophine ou chouchoy à peau blanche... prélevée à la Martinique »). Je propose « 165 kcal · P 31 · G 0 · L 3,6 » à la place du groupe, et des noms tronqués à 2 lignes, le nom complet restant sur la fiche.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js, `_fjResultHtml` (~98576), `_htmlPersoResult`, `_htmlCoachResult`, `_offResultHtml` (~99494). La 2e ligne d'un résultat affiche « 287 kcal/100g · produits céréaliers ». Le groupe Ciqual est souvent hors de propos (Pain perdu classé « produits laitiers ») et ne sert pas à choisir.

À faire :

1. 2e ligne unifiée pour les 4 sources (table, perso, coach, marque) : « 287 kcal · P 9 · G 56 · L 1,2 » pour 100 g, nombres à la française. La protéine est en gras (font-weight 700, même taille). Valeur absente → « - », jamais 0. Garder le badge VALEUR INDISPONIBLE et la mention « kcal estimées » (`k_calc`). Même composant pour toutes les listes : une seule fonction `_htmlLigneMacros(f)`.
2. Nom : limité à 2 lignes (display:-webkit-box; -webkit-line-clamp:2; overflow:hidden), titre complet dans l'attribut title et sur la fiche s-food-add. Ne PAS changer les tailles de police.
3. Le groupe Ciqual n'apparaît plus que sur la fiche détaillée.

Cas limites : lecteur d'écran (aria-label complet sur la ligne), largeur 320 px sans débordement horizontal, thème clair et sombre.

Tests (app/tests.js) : `_htmlLigneMacros({k:287,p:9,c:56,l:1.2})` contient « P 9 » et « 287 kcal » ; une valeur manquante affiche « - » ; une fiche `k_calc` garde sa mention ; capture au banc à 320 px sans scroll horizontal.

Fin : suite verte. Nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

8. Qualité des fiches : 19 aliments sans énergie, 5 sans aucune macro, 70 sans glucides — Priorité : Basse · Effort : S

Analyse et idée. Mesuré dans `ciqual.json` : 81 fiches sans `k`. Pour 62 d'entre elles, l'énergie est estimée (`k_calc`) ; 19 restent sans valeur, comme « Brochette de bœuf, crue », « Poireau, cuit » ou « Tomate marmande, crue ». 70 fiches n'ont pas de glucides, 29 pas de protéines, 5 aucune macro (chayottes de Martinique, vanille). Une brochette crue ou un poireau cuit sont saisis par des athlètes et comptent alors « - » kcal. Je propose de compléter ces fiches depuis une fiche Ciqual sœur explicitement désignée, avec la mention « valeur reprise de... », et de masquer de la recherche les 5 fiches vides.

Prompt de mise en place.

Contexte : RepCore, `scripts/ciqua_convert.py` et `app/data/ciqua.json`. Mesuré : 19 fiches sans énergie et non calculables (au moins une macro manque), dont « Brochette de bœuf, crue », « Brochette de volaille, crue », « Poireau, cuit », « Tomate marmande, crue », « Sauté d'agneau au curry, préemballé » ; 5 fiches sans aucune macro (ids 20806, 20807, 20820, 20831, 11057).

À faire :

1. `scripts/ciqua_soeurs.tsv` : `id_incomplet<TAB>id_soeur<TAB>justification` (ex. Poireau cuit → Poireau bouilli/cuit à l'eau ; Tomate marmande crue → Tomate ronde crue ; Brochette de bœuf crue → Bœuf à griller (aliment moyen)). Le script recopie SEULEMENT les champs absents (jamais une valeur Anses présente), pose "`k_calc`":true si `k` est recalculé, et "`repris`":id_soeur.
2. Les fiches sans aucune macro reçoivent "`cache`":1. La recherche les ignore (filtre dans

```

_fjContientTous ou en amont), mais _planCiqua les résout toujours, pour qu'un plan ancien ne casse pas.
3. Fiche aliment : si f.repris, afficher « Valeurs complétées depuis « <nom de la sœur> » (Ciqua) » en var(--fs-2xs).
4. Le script affiche le compte final : fiches sans k, sans p, sans c, sans l.
Tests (app/tests.js) : aucune fiche visible en recherche n'a k == null ; les 5 fiches cachées ne sortent pas sur « chayote » mais _planCiqua(20806) les rend ; une fiche « repris » garde ses valeurs Anses d'origine.
Fin : suite verte. Nouveau build (window.RC_BUILD dans app/index.html, CACHE et CIQUAL_VERSION de app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

```

Stratégie de complétion (synthèse pour Kevin). - Sources et licences. - Socle : Ciqua 2025 (Anses), sous Licence Ouverte Etalab 2.0, avec attribution. C'est déjà le cas. - Génériques et chaînes : USDA FoodData Central, dans le domaine public (CC0), complété par les tableaux publiés par les chaînes en France, datés. - Produits de marque : Open Food Facts en ligne (ODbL 1.0, avec attribution), plus le carnet local des produits que l'athlète a lui-même consommés (point 6). N'embarquer un extrait OFF hors ligne que dans un fichier séparé, publié lui aussi sous ODbL. - **Format.** Un seul `ciqua.json` généré, avec les champs ajoutés `a` (alias), `u` (portions), `r` (fiche de référence) et `src`. Les sources éditables sont des TSV/CSV relus par un humain, et un script unique, reproductible et bloquant en cas d'incohérence. - **Poids.** Environ +60 Ko brut (+15 Ko après gzip) pour 150 fiches, 250 alias et 200 portions, sur 876 Ko aujourd'hui. Cela reste compatible avec le préchargement hors ligne du service worker (CIQUAL_VERSION).

Expert 7 · Erreurs humaines de l'élève — 6/10

Beaucoup de garde-fous sont déjà très solides. Le brouillon de bilan survit au rechargement et au changement de compte, « Valider sans ton poids ? » est demandé, un double tap ne crée qu'un seul bilan, la fin de séance est confirmée et liste les séries non cochées, la charge « 82,5 » est bien lue, la pesée du jour compare avec la veille, et un bilan envoyé peut être modifié. Le manque principal tient à quatre choses : les erreurs qui partent dans le dossier n'ont presque jamais de retour arrière (une séance ou un bilan ne se supprime pas, une séance ne se corrige pas), le coach n'est pas prévenu quand l'élève répare, et plusieurs pertes se font en silence (séance oubliée effacée après 24 h, déconnexion qui efface tout).

1. Bilan validé sans photos : aucune question au moment de valider, et une photo « à reprendre » est refusée sans recours — Priorité : Critique · Effort : S

Analyse et idée. Testé au banc : bilan coaching avec le poids seul, « Valider » à l'étape Photos → enregistré directement, puis écran « Bilan enregistré, ton coach est prévenu ». `bilNext` (`rc-core.1792.js:75284`) ne vérifie que le poids. Pire : une photo jugée rouge par le contrôle de pose (`loadBilPhoto :76085`, « Personne n'est reconnu », pieds coupés...) n'est pas gardée et rien ne permet de la garder quand même. Dans une petite salle de bain, l'élève ne peut que valider sans photos : c'est exactement le cas de Kevin. L'idée : demander une seule fois, à la validation, avec trois choix, et proposer « Garder quand même » sur un verdict rouge.

Recoupement. Point de référence pour le bilan validé sans photos (cas réel). Les experts 2 (point 4), 8 (point 1), 9 (point 2), 13 (point 4) et 17 (défaut 2) y renvoient : appliquer ce prompt en premier, les autres le complètent sans le refaire.

Prompt de mise en place.

```

Contexte : app/rc-core.1792.js, questionnaire de bilan. bilNext (~l.75284) valide sans regarder les photos ; seul le poids déclenche rcConfirm('Valider sans ton poids ?'). loadBilPhoto (~l.76085) jette la photo quand photoControle (~l.75565) rend 'rouge', sans recours. Les clés photo sont <prefixe>-photo-face|back|side (prefixe 'bil' ou 'deb', _bilPhotoPrefixe ~l.74718).

```

grossesseSuspend(currentUser) rend les photos facultatives (BIL_STEPS, étape photos ~l.78755). rcConfirm3 existe (~l.47968).

Erreurs humaines à couvrir : l'élève oublie les photos, n'a pas le temps de les prendre, ou voit une photo refusée à tort par le moteur (lumière, petite pièce, pieds coupés).

À faire :

1. Fonction PURE photosBilanManquantes(bilType,bilData,user) : renvoie la liste des vues absentes parmi face/back/side. Elle renvoie [] quand grossesseSuspend(user) est vrai, quand on modifie un bilan déjà envoyé (_bilEdition), ou quand aucune des 3 vues n'est attendue.
 2. Dans bilNext, juste après le garde-fou du poids : si la liste n'est pas vide, rcConfirm3('Il manque tes photos', 'Ton coach s'en sert pour voir ce que la balance ne montre pas.', 'Ajouter mes photos', 'Envoyer sans, je les ajoute plus tard', 'Annuler').
 - « Ajouter » : reste sur l'étape Photos (bilStep = dernière étape) et fait défiler jusqu'à la première carte manquante.
 - « Envoyer sans » : enregistre normalement et pose bi.photosAVenir=[vues manquantes] sur le bilan enregistré (dans saveBilanFinal, ~l.78888).
 - « Annuler » ou toucher le voile : ne fait rien.Une seule question en tout : pas de seconde confirmation quand le poids manque aussi. Fusionne les deux messages en un seul texte dans ce cas.
 3. Dans loadBilPhoto, sur un verdict rouge : garde l'affichage du verdict et ajoute un petit bouton texte « Garder quand même », sans agrandir les polices. Il range la photo comme une orange, avec bilData[key+'-ctl']='orange|force','+codes. Ajoute PHOTO_CTL.MSG.force='Gardée malgré l'avertissement.'. Le coach voit donc la réserve.
 4. Tant que bi.photosAVenir n'est pas vide : sur l'accueil athlète, une ligne discrète dans la carte du prochain bilan, « Photos du bilan n°N à ajouter → », qui ouvre modifierBilan(id) directement sur l'étape Photos (nouveau paramètre optionnel étape). Le tableau se vide à mesure que _bilEnregistrerModif (~l.75074) reçoit les vues manquantes.
 5. Tests dans app/tests.js (testExercices) :
 - photosBilanManquantes : bilan complet → [] ; bilan sans photos → 3 vues ; grossesse → [] ; _bilEdition posé → [].
 - bilNext sur la dernière étape, sans photos, rcConfirm3 simulé à null : aucun bilan ajouté. Simulé à 'mid' : un bilan avec photosAVenir de longueur 3.
 - « Garder quand même » : bilData[key] est rempli et le ctl commence par 'orange|force'.
 6. Convention du dépôt : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js), puis python3 scripts/versionner_actifs.py. N'agrandis aucune police ni aucune cible tactile.
- Critère de fin : chargerTests() à 0 échec. Au banc, valider un bilan sans photos affiche une seule question. « Envoyer sans » fait apparaître la ligne « Photos à ajouter » sur l'accueil, et ajouter les photos la fait disparaître.

2. Séance oubliée : effacée sans un mot au bout de 24 h, ou enregistrée le lendemain à la mauvaise date avec « 20 h » de durée — Priorité : Critique · Effort : S

Analyse et idée. _woLoadSnap (:47774) supprime l'instantané dès que startTime a plus de 24 h, quel que soit le nombre de séries validées. Testé au banc : 4 séries validées, app fermée sans « Terminer », réouverture 30 h plus tard → plus de carte de reprise, instantané supprimé, aucune séance, aucun message. Réouverture au bout de 20 h → « Séance gelée, reprendre maintenant », puis « Terminer » enregistre la séance à la date du jour, avec « 20 h » de durée (duration:1200). Le coach voit soit un trou, soit une séance de 20 h datée du mauvais jour, et marquerJourSeance met aussi le mauvais jour en jour d'entraînement. L'idée : ne jamais jeter des séries validées, et proposer « Enregistrer telle quelle » en datant la séance de sa dernière série.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. _woLoadSnap (~l.47774) supprime rc_wo_state quand Date.now()-startTime>24 h. La carte d'accueil #clh-resume-workout (index.html ~l.3626, remplie dans loadClientHome ~l.44558) ne propose que « Reprendre maintenant ». finishWorkout (~l.53790) date la séance avec Date.now() et calcule duration depuis woState.startTime. Chaque série validée porte tValid (toggleSet ~l.53517).

Erreurs humaines à couvrir : oublier d'appuyer sur « Terminer », fermer l'app ou se faire tuer l'app par le téléphone, reprendre le lendemain matin.

À faire :

1. Fonction `PURE seanceOubliee(snap,maintenant)` : renvoie null s'il n'y a aucune série validée. Sinon {series, debut:min(tValid), fin:max(tValid), heuresDepuis:(maintenant-fin)/3600e3}.
2. `_woLoadSnap` : ne supprime plus en silence un instantané qui a des séries validées. Au-delà de 24 h (ou de 6 h sans activité), il reste lisible en mode « oubliée », borné à 7 jours. Au-delà de 7 jours, on l'enregistre automatiquement comme séance partielle (voir 3) au lieu de l'effacer.
3. Nouvelle fonction `enregistrerSeanceOubliee()` : reprend l'instantané dans `woState` et appelle `finishWorkout` avec une option {date: fin, duree: round((fin-debut)/60000)}. Elle ajoute 3 min pour la dernière série et plafonne à 240 min. `complete` vaut false si moins de la moitié des séries est faite, comme `_woTerminerDepuisNav`. `finishWorkout` accepte cette option sans changer son comportement par défaut. `marquerJourSeance` utilise donc la bonne date.
4. Carte d'accueil, quand `seanceOubliee(...).heuresDepuis > 6` : texte « Séance de <jour> non terminée · N séries » et deux boutons de même taille que l'existant : « L'enregistrer » (`enregistrerSeanceOubliee`, toast « Séance du <jour> enregistrée ») et « Reprendre ». Sous 6 h, la carte actuelle ne change pas.
5. `woResumeAndGo` après plus de 6 h sans activité : la séance reprise garde la date de la première série, et la durée ne compte pas le trou. Enregistre `woState.pauseMs` ou, plus simplement, calcule la durée entre la première et la dernière `tValid`.
6. Tests dans `app/tests.js` (`testExercices`) :
 - `seanceOubliee` : instantané sans série validée → null ; 3 séries validées → fin = dernière `tValid`.
 - `_woLoadSnap` avec `startTime` vieux de 30 h et 4 séries validées : `rc_wo_state` n'est PAS supprimé.
 - `enregistrerSeanceOubliee` : la séance enregistrée est datée de la dernière série (hier) et dure ≤ 240 min.
7. Convention du dépôt : nouveau build (`window.RC_BUILD`, `CACHE` de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne grossis aucune police ni aucun bouton. Critère de fin : `chargerTests()` à 0 échec. Au banc, une séance commencée 30 h plus tôt avec 4 séries validées se retrouve dans l'historique à sa date, en un seul toucher.

3. Corriger un bilan déjà répondu ne prévient pas le coach, et l'écran de fin ne propose pas d'ajouter les photos — Priorité : Haute · Effort : S

Analyse et idée. Le retour arrière existe (« Historique des bilans → Modifier », :75015), mais testé au banc : bilan répondu « il me manque tes photos », l'élève ajoute deux photos, toast « ton coach verra la correction ». Pourtant `hasNewBilan` (:30413) reste faux, parce que `bilanRepondu` est vrai et que `modifieLe` ne sert qu'à afficher « · modifié » (:75018, :78664). Le coach ne voit rien. Et l'écran « Bilan enregistré » n'a ni « Corriger » ni « Ajouter mes photos » : l'historique est rangé sur l'écran de choix du bilan. L'idée : une correction faite après la réponse rallume le bilan chez le coach, et l'écran de fin donne le bon raccourci.

Recoupement. Le signal « bilan corrigé » côté coach est détaillé par l'expert 9 (point 2), qui fait référence : ne garder ici que les raccourcis de l'écran de fin (« Ajouter mes photos », « Corriger ce bilan »).

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `_bilEnregistrerModif` (~l.75074) appelle `appliquerModifBilan` (~l.75054), qui pose `b.modifieLe`. `hasNewBilan` (~l.30413) ne regarde que `bilanRepondu(dernierBilan)`. `bilansSansReponse` (~l.78073) compte la charge du coach. L'écran de restitution est ouvert par `ouvrirRestitutionBilan` (~l.86298).

Erreurs humaines à couvrir : l'élève corrige un poids faux ou ajoute ses photos APRÈS la réponse du coach ; l'élève ne trouve pas où corriger.

À faire :

1. `appliquerModifBilan` : quand des clés changent ET que `bilanRepondu(b)` est vrai, pose `b.correctionApresReponse={le:maintenant, cles:[...changées], photos:true|false}`. Ne touche pas `reponseCoach`.
2. Fonction `PURE bilanCorrigeNonVu(b)` : vrai si `b.correctionApresReponse` existe et que le coach ne l'a pas encore lue (`b.correctionVueLe < correctionApresReponse.le`, ou absent). `hasNewBilan(c)` devient vrai aussi quand `bilanCorrigeNonVu` porte sur n'importe quel bilan des 60 derniers jours, pas seulement le

dernier. La même condition alimente le badge et la ligne « À traiter ». Libellé : « Photos ajoutées au bilan n°N » ou « Bilan n°N corrigé ».

3. Côté coach, ouvrir ce bilan (renderReponsesBilans ~l.78594) pose `correctionVueLe=Date.now()` et sauvegarde. Les champs changés ressortent par un léger soulignement, sans changer les tailles.

4. Écran de restitution (ouvrirRestitutionBilan) : si le bilan qu'on vient d'envoyer porte `photosAVenir` (voir le point « bilan sans photos ») ou n'a aucune photo, bouton secondaire « Ajouter mes photos » → `modifierBilan(id)` sur l'étape Photos. Sinon, un lien texte discret « Corriger ce bilan ».

5. Le toast « ton coach verra la correction » devient « Ton coach est prévenu de la correction » seulement quand le point 2 s'applique. Sinon le message actuel reste.

6. Tests dans `app/tests.js` (`testNotifs`) :

- bilan répondu + `appliquerModifBilan` qui ajoute `bil-photo-face` → `hasNewBilan` vrai ;
- `correctionVueLe` posé après → `hasNewBilan` faux ;
- modification sans changement → aucun champ `correctionApresReponse` ;
- bilan non répondu modifié → le comportement actuel ne change pas.

7. Convention du dépôt : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne grossis aucune police.

Critère de fin : `chargerTests()` à 0 échec. Au banc : athlète, le coach répond, l'élève ajoute ses photos ; côté coach, l'athlète remonte dans « À traiter » avec « Photos ajoutées ».

4. Poids du bilan : « 62,0 » devient 620 kg, accepté sans rien demander — Priorité : Haute · Effort : S

Analyse et idée. Le correctif de la virgule (`lireCharge`, :52176) ne couvre que les charges de séance. Le poids du bilan (`bQ`, :75381) et les 12 mensurations et longueurs (:75976, :75997) restent des `type="number"`. Testé au banc (Chromium fr-FR, `keyboard.type('62,0')`) : valeur « 620 », bilan enregistré « Poids 620 kg », profil `weight:'620'`, écran de fin « ENREGISTRÉ 620 kg », alors que la pesée du jour refuserait 620 et demanderait confirmation sur l'écart (`_enregistrerPesee`, :118825). L'idée : une seule lecture décimale pour tous les champs du bilan, et le même contrôle d'écart que la pesée.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `lireCharge` (~l.52176) lit « 82,5 » dans un champ texte `inputmode="decimal"`. Le bilan utilise encore `<input type="number">` : `bQ` (~l.75381, poids et taille), `bBodySchema` (~l.75976, 12 mensurations), `bLongueurs` (~l.75997). En Chromium fr-FR, « 62,0 » donne valeur '620'. `_enregistrerPesee` (~l.118825) borne déjà 25–300 kg (`PESEE_MIN/MAX`) et confirme au-delà de `PESEE_ECART_CONFIRM` par rapport à la dernière pesée.

Erreurs humaines à couvrir : virgule du clavier français, chiffre de trop (620), cm saisis en mm (850 au lieu de 85), poids saisi dans la case taille.

À faire :

1. Fonction PURE `lireDecimal(brut,{min,max,pas})` sur le modèle de `lireCharge` : elle accepte « 62,0 », « 62.0 » et « 62 », et rend null pour le reste. `lireCharge` peut s'appuyer dessus sans changer son comportement.
2. `bQ`, `bBodySchema` et `bLongueurs` passent en `type="text" inputmode="decimal" autocomplete="off"`. Garde les ids, les `oninput` et le style tels quels (aucune taille changée). `bMesureSaisie` stocke la valeur normalisée (point décimal).
3. Bornes : poids `PESEE_MIN..PESEE_MAX`, taille 100..250, mensurations 10..250 cm, longueurs bornes de `MORPHO_MESURES` s'il y en a. Hors bornes : la case passe en bordure orange et une ligne d'aide s'affiche sous le schéma, par exemple « 620 kg ? Vérifie la virgule. ». Pas de toast à chaque frappe.
4. Dans `bilNext`, avant d'enregistrer : un poids qui s'écarte de plus de 10 % de la dernière référence (poidsNutritionnel ou dernier bilan), ou une mensuration de plus de 25 %, déclenche UNE question qui liste les valeurs suspectes : « 620 kg (dernière : 63,5) – Corriger / C'est juste ». « Corriger » ramène à l'étape Mensurations, case en surbrillance.
5. Tests dans `app/tests.js` (`testExercices`) :
 - `lireDecimal('62,0')===62` ; `lireDecimal('62.5')===62.5` ; `lireDecimal('6a')===null` ;
 - le rendu de `BIL_STEPS[0]` et `DEB_STEPS[0]` ne contient plus de `type="number"` ;
 - `bilNext` avec `bil-weight '620'` et une pesée à 63,5 : `rcConfirm` appelé, aucun bilan ajouté si la réponse est false.
6. Convention du dépôt : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). N'agrandis ni les cases ni les polices.

5. Rien ne se supprime ni ne se corrige après coup : séance fausse, séance en double, bilan envoyé deux fois — Priorité : Haute · Effort : M

Analyse et idée. Le détail d'une séance (ouvrirSeanceHistorique, :65112) est en lecture seule. Aucune séance ni aucun bilan ne peut être retiré, alors que les pierres tombales sont prêtes et respectées à la fusion (marquerSupprime :6482, lues à :3728 et :6399), mais branchées seulement pour les vidéos (:110750). Testé au banc : un second bilan coaching envoyé 20 h après le premier part sans question, et les deux restent à vie chez le coach. Une charge fausse (100 au lieu de 10) reste dans les records et sert à la suggestion suivante. L'idée : « Corriger » et « Supprimer, avec annulation » sur la séance, « Supprimer ce doublon » sur un bilan récent, et une question avant un bilan trop rapproché.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. marquerSupprime(user,'sessions'|'bilans',cle) (~l.6482), avec cleSuppressionSeance et cleSuppressionBilan (~l.6505), pose une pierre tombale que la fusion respecte déjà (~l.3728, ~l.6399). Aucune UI ne l'appelle pour les séances ni pour les bilans. Le détail d'une séance passée : ouvrirSeanceHistorique (~l.65112), écran s-seance-detail. L'historique des bilans : ouvrirHistoriqueBilans (~l.75015). openBilan (~l.75107).

Erreurs humaines à couvrir : mauvaise charge ou mauvaises répétitions saisies et validées, mauvais exercice, séance enregistrée deux fois (deux appareils, test), bilan renvoyé parce que l'élève croyait que le premier n'était pas parti.

À faire :

1. Séance passée, dans s-seance-detail, un lien texte discret « Corriger » sous les exercices : édition en place des charges et répétitions des séries validées (champs texte lus par lireCharge), renommage d'un exercice par la liste des exercices existante (pas de saisie libre). On recalcule ensuite sets, volume, les caches (_viderCachePlateau, _viderCacheSignaux, _viderCacheVolume) et majBadges, puis sess.corrigeLe=Date.now(). Limite : séances des 14 derniers jours.
2. « Supprimer la séance » dans le même menu : rcConfirm, puis retrait, puis marquerSupprime('sessions',cleSuppressionSeance(s)), puis saveUser. Un bandeau « Séance supprimée · Annuler » reste 8 s (même gabarit que _htmlDernierAjout ~l.98315). « Annuler » remet la séance et retire la pierre tombale.
3. Bilans, dans ouvrirHistoriqueBilans : « Supprimer » seulement pour un bilan de moins de 48 h que le coach n'a pas encore répondu (sinon, Modifier seulement). Même mécanique : marquerSupprime('bilans',cleSuppressionBilan(b)), avec annulation 8 s.
4. openBilan('coaching') : si un bilan coaching a été envoyé il y a moins de 5 jours, rcConfirm3('Tu as envoyé un bilan le <date>', '', 'Le corriger', 'En faire un nouveau', 'Annuler'). « Le corriger » → modifierBilan(id).
5. Côté coach : une séance corrigée porte « · corrigée » à côté de sa date. Une suppression n'envoie rien.
6. Tests dans app/tests.js (testExercices) :
 - suppression puis fusion (_mergeUser) avec un serveur qui a encore la séance → elle ne revient pas ;
 - « Annuler » → elle revient et la pierre tombale est retirée ;
 - correction d'une charge 100 → 10 : volume et recordsExercice recalculés ;
 - openBilan avec un bilan de la veille : rcConfirm3 appelé.
7. Convention du dépôt : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Pas de nouveau gros bouton : des liens texte de la taille existante. Critère de fin : chargerTests() à 0 échec. Au banc : supprimer une séance puis annuler laisse l'historique intact, et un second bilan le lendemain déclenche la question.

6. Déconnexion (ou changement de téléphone) : séance en cours, brouillon de bilan et envois en attente partent derrière un simple « Se déconnecter ? » — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : séance en pause avec 4 séries validées, brouillon de bilan (« Finir plus tard »), pastille « 1 envoi en attente ». `logout` (:122768) demande « Se déconnecter ? » sans aucun détail, puis `silentLogout` et `_comptesRemiseAZero` (:8066) effacent `rc_wo_state` et `rc_bil_draft`. Un élève qui se déconnecte pour passer sur son nouveau téléphone perd sa séance, son bilan commencé et des photos peut-être pas encore envoyées. L'idée : la même question, mais qui dit ce qui sera perdu, avec un plan B en un geste.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. logout (~l.122768) : rcConfirm('Se déconnecter ?'), puis silentLogout (supprime rc_wo_state), puis _comptesRemiseAZero (~l.8066, supprime woState, bilData et le brouillon). CLOUD.enAttenteDeSync() donne la file d'envoi (pastille _majIndicAttente ~l.7531). _woLoadSnap, _woSeriesValidees (~l.47800), _bilLoadDraft et _bilDraftRempli existent. photosBilanARenvoyer(user) (~l.69092) liste les photos pas encore envoyées.
```

Erreurs humaines à couvrir : se déconnecter pour changer de téléphone, prêter son téléphone, se déconnecter « pour réparer » un souci.

À faire :

1. Fonction PURE `pertesALaDeconnexion({snap,draft,file,photos})` → liste de phrases courtes : « une séance en cours (4 séries) », « un bilan commencé (3 réponses) », « 2 envois pas encore partis », « 2 photos de bilan pas encore envoyées ». Liste vide = rien à perdre.
 2. `logout` : si la liste est vide, la question actuelle ne change pas (aucune friction en plus). Sinon `rcConfirm3('Avant de te déconnecter', liste, 'Enregistrer et me déconnecter', 'Me déconnecter quand même', 'Annuler')`.
 - « Enregistrer et me déconnecter » : séance en cours → `enregistrerSeanceOubliee()` (voir le point « séance oubliée ») ou `finishWorkout(true)`. Puis `CLOUD.viderFile()` et `photosBilanRenvoyer`, avec 8 s d'attente au plus et un indicateur sur le bouton. Le brouillon de bilan, lui, ne part pas au cloud : ne pas l'effacer à la déconnexion (il porte déjà l'adresse de son propriétaire, `_bilDraftAMoi` le protège des autres comptes), et le dire (« ton bilan commencé t'attendra sur ce téléphone »). Puis la déconnexion.
 - En cas d'échec réseau : `toast` « Pas de réseau : reste connecté jusqu'au prochain envoi », et on ne se déconnecte pas.
 3. Tests dans `app/tests.js` (`testNotifs`) :
 - `pertesALaDeconnexion` sans rien → [] ;
 - avec un instantané (2 séries) et une file de 1 → 2 phrases ;
 - `logout` avec `rcConfirm3` simulé à null : `rc_wo_state` est toujours là.
 4. Convention du dépôt : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne grossis aucune police.
- Critère de fin : `chargerTests()` à 0 échec. Au banc, la déconnexion avec une séance en pause énumère ce qui sera perdu, et « Annuler » ne touche à rien.

7. Journal alimentaire : le × supprime au premier toucher sans annulation, et une quantité ne se corrige pas — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : le × d'une ligne (`deleteFoodEntry`, :100655 ; bouton :100761) retire l'aliment tout de suite, sans question ni « Annuler ». Il mesure 20 × 20 px, collé au ≠ des équivalences. Toucher la ligne ne fait rien : pour corriger 150 g en 105 g, il faut supprimer puis tout ressaisir. `saveFoodEntry` (:100408) accepte jusqu'à 9 999 g sans regarder les portions habituelles (1 000 g d'huile passent). L'idée : réutiliser le bandeau « Annuler » qui existe déjà pour les ajouts, et ouvrir la ligne au toucher pour corriger la quantité, le repas ou le jour.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. deleteFoodEntry(date,id) (~l.100655) supprime sans trace. Le bandeau d'annulation des ajouts : _fjDernierAjout, _htmlDernierAjout (~l.98315) et annulerDernierAjout
```

(~l.98182), qui sait déjà remettre une entrée « à sa place ». Rendu des lignes : `_renderFjDaySummary` (~l.100661, boutons `≠` et `×` ~l.100759). Écran d'ajout : `fja-qty` (index.html ~l.4946), `setFjaQty`, `saveFoodEntry` (~l.100408). `_fjDerniereQty(id)` (~l.96596) donne la dernière quantité de cet aliment.

Erreurs humaines à couvrir : `×` touché au lieu de `≠`, mauvaise quantité, mauvais repas, aliment saisi sur le mauvais jour, zéro de trop.

À faire :

1. `deleteFoodEntry` passe par le même mécanisme que l'annulation : `_fjDernierAjout={date,ids:[],retirees:[{index,entree}],suppression:true}`. Le bandeau dit « Aliment retiré · Annuler » pendant 8 s, ou jusqu'au prochain ajout. Pas de `rcConfirm` : l'annulation suffit et n'ajoute aucune étape.
 2. Toucher le nom d'une ligne ouvre la feuille d'ajout pré-remplie (aliment, quantité, repas), avec un sélecteur « Jour » (aujourd'hui / hier / avant-hier) et le bouton « Mettre à jour ». L'entrée est remplacée au même index (même id). Elle passe sur un autre jour si le jour a changé. Annulable par le même bandeau.
 3. `saveFoodEntry` : si `qty > 4 × _fjDerniereQty(id)` et `qty > 300 g`, ou si l'entrée seule dépasse 1 500 kcal, une question « 1 000 g d'huile d'olive (9 000 kcal) ? », Corriger / C'est juste.
 4. Garde la taille actuelle des boutons `≠` et `×` (contrainte de Kevin). Écarte-les seulement de 8 px pour limiter le toucher croisé.
 5. Tests dans `app/tests.js` (`testExercices`) :
 - `deleteFoodEntry` puis `annulerDernierAjout` → l'entrée revient au même index ;
 - modification de quantité → même id, kcal recalculées ;
 - passage d'une entrée d'aujourd'hui à hier → absente d'aujourd'hui, présente hier ;
 - `qty 1000` sur un aliment dont la dernière quantité est 10 → `rcConfirm` appelé.
 6. Convention du dépôt : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). N'agrandis pas les polices.
- Critère de fin : `chargerTests()` à 0 échec. Au banc, `×` puis « Annuler » restaure la ligne, et toucher une ligne permet de passer de 150 g à 105 g en deux gestes.

Expert 8 · Erreurs du coach et réparation — 5,5/10

Plusieurs filets solides existent déjà : l'historique des cibles de macros, avec « Remettre en place » et « Annuler », 5 versions restaurables du programme, la modification d'un bilan par l'athlète, la suppression d'un dossier où il faut retaper le prénom, et le refus de publier le brouillon d'un autre athlète. Il manque l'essentiel côté support : rien ne prévient le coach quand un bilan est complété après sa réponse, aucun geste n'est annulable par un toast, les modèles et plans alimentaires n'ont pas de corbeille, les homonymes ne sont pas distingués, et Kevin n'a aucun outil pour réparer le dossier d'un élève sans toucher à la base.

1. Bilan incomplet : le coach ne peut pas demander de le compléter, et ne voit pas quand il l'est — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc : bilan déjà répondu, puis complété par l'athlète une heure plus tard (`modifieLe` et photo ajoutés). Résultat : `hasNewBilan = false` et aucune notification, la ligne de l'athlète ne change pas (`hasNewBilan`, `rc-core:30413`, ne lit que `reponseCoach`). Le coach n'a pas non plus de bouton « Demander les photos » : `demandesMesure` (`rc-core:57267`) ne couvre que les mensurations, et dit à l'athlète « Tu la saisis à ton prochain bilan » (57430). L'idée : un bouton « Demander de compléter ce bilan » sous chaque bilan côté coach. Côté athlète, une carte d'accueil ouvre `modifierBilan(id)` directement à l'étape des photos. Le bilan complété redevient « à relire » chez le coach.

Recoupement. La partie « le coach voit que le bilan a été complété » est celle de l'expert 9 (point 2) : ne pas la doubler. Ce point apporte le bouton « Lui demander de compléter » et la carte d'accueil de l'élève.

Prompt de mise en place.

Contexte : `RepCore`, `app/rc-core.1792.js` (énorme : `grep -n puis sed -n`, ne lis jamais tout). Cas réel : un élève valide son bilan sans photos, et il a fallu bricoler la base pour qu'il puisse y revenir. L'athlète peut maintenant corriger un bilan envoyé (`modifierBilan` / `appliquerModifBilan` /

_bilEnregistrerModif, vers la ligne 75035). Mais le coach ne peut pas le lui DEMANDER, et quand l'athlète corrige un bilan déjà répondu, le coach n'en sait rien (hasNewBilan, ligne ~30413, ne regarde que bilanRepondu).

À faire :

1. Modèle de données sur le bilan : `b.aCompleter = {vues:['face','back','side']} (sous-ensemble), mesures:[clés], note:'', le:ts, par:coachId}`. Fonction PURE `manquesBilan(b)` qui rend les vues photo absentes (via `photoBilanExiste`) et les mesures vides parmi les champs « bil-* » attendus.
 2. Côté coach, dans `blocReponseBilan` (ligne ~78462), si `manquesBilan(b)` n'est pas vide : une ligne « Il manque : photo de face, de dos » avec UN bouton « Lui demander de compléter ». Un clic pose `b.aCompleter`, met à jour `updatedAt`, `DB.set` puis `CLOUD.pushOne`, et dépose `deposerEvenement({type:'bilan_a_completer',...})` comme le fait `saveReponseBilan`. Un second clic affiche « Demandé le ... · Retirer la demande ». Même patron visuel que `_htmlCcdManque` (ligne ~57390).
 3. Côté athlète : une carte en haut de l'accueil, « Ton coach te demande de compléter ton bilan du 4 octobre : photos de face et de dos ». Le bouton « Compléter » appelle `modifierBilan(id)` puis saute directement à l'étape des photos (calcule `bilStep` d'après les étapes réelles, n'invente pas d'index fixe).
 4. `_bilEnregistrerModif` : si les manques demandés sont comblés, supprimer `b.aCompleter`. Dans tous les cas où un bilan DÉJÀ RÉPONDU change, poser `b.completeLe = Date.now()`.
 5. `hasNewBilan` / `bilansSansReponse` / la ligne « Nouveaux bilans à lire » : un bilan est aussi « à relire » si `b.completeLe > b.reponseDate`. Libellé : « Léa a complété son bilan du 4 oct. (photos ajoutées) ». La réponse du coach n'est PAS effacée.
 6. Erreurs humaines à couvrir : demande sur le mauvais bilan (bouton Retirer), athlète hors ligne (la carte reste jusqu'à l'envoi), athlète qui ajoute une seule des deux photos (la demande reste, réduite à la vue manquante), bilan de départ (« Bilan d'inscription ») traité pareil, double clic (idempotent).
- Tests dans `app/tests.js` : `manquesBilan` sur trois cas ; `hasNewBilan` devient vrai quand `completeLe > reponseDate` ; consommation partielle de `aCompleter` ; retirer la demande ; aucune demande possible sur un athlète `_fromCode`.
- Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`), puis `python3 scripts/versionner_actifs.py`. N'agrandis aucune police ni aucune cible : réutilise les classes `ccd-manque` existantes.
- Finis quand : au banc, un coach demande les photos en un clic, l'athlète les ajoute en deux gestes depuis l'accueil, le bilan remonte en « à relire » chez le coach, et la suite de tests passe.

2. Aucun geste du coach n'est annulable : il faut un toast « Annuler » commun — Priorité : Critique · Effort : M

Analyse et idée. `toast()` (`rc-core:11359`) n'affiche que du texte, sans bouton, et rien dans l'app ne propose « Annuler » juste après un geste. Testé au banc : « Retirer cet élève du suivi » écrit tout de suite, désactive son code et envoie le retrait au serveur (`rc-core:33614`). Pour revenir en arrière, il faut lui créer un nouveau code. Même chose pour la suppression d'un modèle (`36816`), d'un plan alimentaire (`95895`) ou d'une recette. L'idée : une seule fonction `rcAnnulable(message, appliquer, defaire)` qui affiche un toast avec « Annuler » pendant 8 s. Pour les gestes qui touchent le serveur, l'écriture distante n'attend pas : « Annuler » rejoue l'état d'avant.

Prompt de mise en place.

Contexte : `RepCore`, `app/rc-core.1792.js` (`grep -n puis sed -n`). Le toast (fonction `toast`, ligne ~11359) n'affiche que du texte. Aucun geste du coach ne se défait : retirer un élève du suivi (`confirmDeleteClient` ~33614), supprimer un modèle (`deleteCoachProgTemplate` ~36816), supprimer le plan alimentaire (`supprimerPlanCoach` ~95895), supprimer une recette (`supprimerRecette` ~97371), reporter une correction de modèle (`reporterPropagation` ~38511). Le seul « Annuler » existant est `histoAnnuler` (`nutrition`, ~103933) : c'est le patron à suivre.

À faire :

1. Créer `rcAnnulable({message, defaire, duree=8000})` : réutilise l'élément `#toast` et ajoute un bouton texte « Annuler » (classe existante `rb-lien` ou `btn-sm`, sans agrandir la police). Un appui appelle `defaire()`, puis `toast('Annulé')`. Un seul annulable à la fois : un nouveau toast ferme le précédent sans rien défaire. Lecteur d'écran : `role="status"`, et le bouton reste atteignable au clavier.
2. `defaire()` reconstruit l'état d'avant à partir d'un instantané pris AVANT l'écriture : copie profonde du nœud athlète ou de `currentUser.coachPrograms[idx]`. Puis il réécrit par le chemin normal (`DB.set`, `CLOUD.pushOne`, `saveUser`).

3. Brancher sur :

- `confirmDeleteClient` : instantané de `coachId`, `coachName`, `coachEmailKey`, `coachCode` et des codes désactivés. `defaire()` réactive les codes (`_majActifDistant(token,true)`) et rattache l'athlète. Si le serveur refuse, le dire en clair.
- `deleteCoachProgTemplate` : remettre le modèle à son index, avec ses versions et son état de vente.
- `supprimerPlanCoach`, `supprimerRecette` : remettre l'objet.
- `reporterPropagation` : voir le point 6 de l'audit (annulation groupée), mais l'appel à `rcAnnulable` se fait ici.

4. La confirmation reste pour les gestes lourds, mais elle NOMME l'objet : « Retirer Léa Moreau de ton suivi ? », « Supprimer « PPL » ? 3 athlètes l'utilisent (leur programme ne change pas) ».

5. Cas limites : appui sur Annuler hors ligne (on défait en local, la file de renvoi rejoue), changement d'écran pendant les 8 s (le toast survit), changement de compte (l'annulable est oublié dans `_comptesRemiseAZero`), double appui sur Annuler (idempotent).

Tests dans `app/tests.js` : `rcAnnulable` appelle `defaire` une seule fois ; retirer puis annuler rend le même `coachEmailKey` et les codes actifs ; supprimer un modèle puis annuler le remet au même index avec le même id ; le texte de confirmation contient le nom complet.

Convention : nouveau build (`window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne grossis ni les polices ni les cibles.

Fini quand : au banc, chacun des cinq gestes se défait en un appui dans les 8 s et laisse la base comme avant.

3. L'historique du programme perd la version d'origine après 5 retouches — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : 6 publications successives (un simple renommage à chaque fois). L'historique ne contient plus que « Retouche 1 à 5 », et la version d'origine de Léa est perdue (`contientOriginal: false`). En cause : `_SESSIONS_HISTORY_MAX=5` (`rc-core:38055`), sans dédoublonnage, et chaque PUBLIER pousse une entrée (39228). Le coach qui retouche 15 fois par semaine efface donc vite la version d'avant le mauvais modèle. L'idée : garder des jalons. Une entrée par jour au plus pour les retouches, une entrée « Avant « PPL » » à chaque application de modèle ou report, épinglée et conservée, 15 entrées au total, et l'étiquette « ce qui a changé » dans la liste.

Prompt de mise en place.

Contexte : `RepCore`, `app/rc-core.1792.js`. `_pushSessionsHistory` (~38056) garde 5 copies de `sessions_config`, sans dédoublonnage. Elle est appelée à chaque PUBLIER (`saveCoachSessions` ~39228), à chaque application de modèle (`_assignerModele` ~38123) et à chaque report de correction (`reporterPropagation` ~38511). Mesuré au banc : 6 publications effacent la version d'origine. L'écran « Versions précédentes » (`renderSessionsHistory` ~38752) sait déjà restaurer et copier.

À faire :

1. `_pushSessionsHistory(a, motif)` avec `motif` ∈ {`'publication'`, `'modele:<nom>'`, `'report:<nom>'`, `'restauration'`}. Chaque entrée porte {`ts`, `motif`, `sessions_config`}.
2. Règles de conservation, en fonction `PURE elaguerHistoriqueSeances(liste, maintenant)` :
 - ne pas empiler une entrée identique à la précédente (comparaison JSON de `sessions_config`) ;
 - entrées `'publication'` : au plus une par jour calendaire, on garde la plus ancienne du jour (l'état d'avant la session de travail) ;
 - entrées `'modele:/'report:'` : toujours gardées, au plus 10 ;
 - plafond total 15. Au-delà, on retire d'abord les publications les plus anciennes.
3. Dans la liste, afficher le motif : « Avant le modèle « PPL » », « Avant tes retouches du 6 oct. », plus le résumé existant (`resumeVersionSeances`). Ajouter « Ce qui change si tu restaures » en réutilisant le calcul de différence du modèle (`opsPourAthlete / _c4Lib0p`) pour lister les séances qui bougent.
4. Restaurer pousse d'abord l'état courant avec le motif `'restauration'` : restaurer par erreur se défait aussi.
5. Taille : vérifier que 15 copies restent sous le budget du dossier. Écarter `photo` et `photo2` des copies, comme `_ajouterModeleDepuis`.

Tests dans `app/tests.js` : 6 publications le même jour donnent 1 entrée ; modèle puis 20 publications sur 20 jours, l'entrée « avant modèle » est toujours là ; pas de doublon identique ; restaurer crée une entrée `'restauration'`.

Convention : nouveau build (`RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Pas de police agrandie.

Finis quand : au banc, après un modèle appliqué par erreur et dix retouches, le coach retrouve « Avant le modèle » et le restaure en deux gestes.

4. Modèles et plans alimentaires : pas de corbeille, pas d'historique — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : supprimer le modèle « PPL », utilisé par 3 athlètes, demande seulement « Supprimer ce programme ? », sans nom ni nombre d'utilisateurs. La suppression part avec ses 3 versions (p.versions sert seulement au calcul des différences, rc-core:38327) et coupe le lien de propagation, alors que la fiche affiche encore « Programme : PPL ». Le plan alimentaire n'a aucun historique : savePlanCoach (95877) écrase l'ancien et supprimerPlanCoach le détruit. Pourtant les macros ont déjà historique, « Remettre en place » et « Annuler » (103887-103950). L'idée : une corbeille de 30 jours pour les modèles, plans et recettes, et le patron des macros appliqué au plan alimentaire.

Recoupement. Les modèles de plan alimentaire sont créés par l'expert 14 (point 7) : la corbeille couvre aussi ces modèles.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. La nutrition a le bon patron : histoCibles / _histoNoter / histoRemettre / histoAnnuler (~103800-103950), avec l'historique des cibles, la remise en place et l'annulation. Les autres objets n'ont rien :

- deleteCoachProgTemplate (~36816) fait un splice définitif. Le modèle part avec p.versions, et les athlètes gardent assignedProgramId vers un id disparu.
- savePlanCoach (~95877) écrase c.nutrition.plan sans rien garder, et supprimerPlanCoach (~95895) le détruit.
- supprimerRecette (~97371) est définitif.

À faire :

1. Corbeille du coach : currentUser.corbeille = [{type:'modele'|'plan'|'recette', id, nom, le, donnees, athleteId?}]. On garde 30 jours et 20 éléments au plus, purge à chaque écriture. Ces trois suppressions y déposent l'objet entier.
 2. Dans « Mes programmes », une ligne discrète « Corbeille (n) » en bas de liste. Elle ouvre une feuille avec nom, date et « Restaurer ». Restaurer un modèle lui redonne le MÊME id : les athlètes qui pointaient vers lui retrouvent la propagation.
 3. La confirmation de suppression d'un modèle nomme le modèle et compte ses utilisateurs (utilisateursModele) : « Supprimer « PPL » ? 3 athlètes l'utilisent : leur programme ne change pas, mais tu ne pourras plus leur reporter une correction. Il reste 30 jours dans la corbeille. »
 4. Plan alimentaire : c.nutrition.planHisto, au plus 5 entrées {le, par, plan}, alimenté par savePlanCoach (si le plan change, comparaison JSON) et par supprimerPlanCoach. Sur l'écran du plan (renderPlanCoach), un bouton « Versions précédentes (n) » sur le modèle de « Historique des modifications » des macros : remettre en place, puis « Annuler » juste après.
 5. Afficher sur la fiche « Programme : PPL (modèle supprimé) » quand assignedProgramId ne correspond plus à aucun modèle.
 6. Cas limites : restaurer un modèle qui porte le même nom qu'un modèle créé entre-temps (suffixer « (restauré) »), corbeille synchronisée entre deux appareils (fusion par id), quota local plein (ne pas garder les photos de séance dans la corbeille).
- Tests dans app/tests.js : supprimer puis restaurer garde l'id et versions ; purge à 30 jours ; planHisto ne double pas un plan identique ; restaurer un plan puis annuler rend l'état d'avant ; message de confirmation avec le nombre d'utilisateurs.
- Convention : nouveau build (RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne grossis pas les polices ni les cibles.
- Finis quand : au banc, un modèle supprimé se restaure en deux gestes avec sa propagation intacte, et un plan alimentaire écrasé revient à sa version précédente.

5. Deux « Léa » : rien n'empêche de travailler sur la mauvaise — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc avec Léa Martin et Léa Moreau. Les titres disent « LÉA », « LÉA : SÉANCES » et « LÉA : ÉVOLUTION » (rc-core:33791 et 38722). La confirmation du retrait dit « Retirer cet

élève de ton suivi ? » (33626), sans nom. Pour « Copier vers → Autre athlète », il faut taper un numéro dans une liste en texte : « Écris le numéro » (37349). L'idée : un nom court qui distingue les homonymes (« Léa Mo. ») dans tous les titres et toutes les confirmations, avec les initiales colorées de la liste, et un choix du destinataire par boutons plutôt que par numéro.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Les écrans du coach sont titrés avec le seul prénom : evo-title (~33791, « c.fname+ ' : Évolution' »), csm-title (~38722), le titre de la fiche. Les confirmations ne nomment pas toujours l'athlète (confirmDeleteClient ~33626 : « Retirer cet élève de ton suivi ? »). copierSeanceVersAthlete (~37339) fait choisir le destinataire en tapant un numéro (rcSaisie « Écris le numéro »). Avec deux homonymes, le coach se trompe d'athlète sans aucun signal.

À faire :

1. Fonction PURE nomCourtAthlete(a, tous) : le prénom seul s'il est unique dans le portefeuille. Sinon prénom + début du nom jusqu'à ce que ça distingue (« Léa Ma. » / « Léa Mo. »), et l'e-mail en dernier recours.
 2. L'utiliser dans les titres de s-coach-client, s-coach-sessions, s-coach-bilan-evo, s-coach-plan, la file des bilans, et chaque rcConfirm ou rcSaisie qui agit sur un athlète (grep « cet athlète », « cet élève »). Pas de police ni de titre agrandis : même composant, texte plus précis.
 3. Dans la barre de titre de l'écran Séances et de l'écran Plan, ajouter la pastille d'initiales déjà utilisée dans la liste (même couleur, même taille), pour qu'on reconnaisse l'athlète d'un coup d'œil.
 4. copierSeanceVersAthlete : remplacer la saisie de numéro par une feuille de boutons (un par athlète, nomCourtAthlete plus pastille), puis une rangée de 7 jours. Garder la confirmation de remplacement qui existe.
 5. confirmDeleteClient et supprimerAthleteDefinitivement : le nom complet en tête de la question. Tests dans app/tests.js : nomCourtAthlete sur prénoms uniques, homonymes, homonymes complets (e-mail), accents ; le texte de confirmDeleteClient contient le nom ; copierSeanceVersAthlete n'appelle plus rcSaisie pour le destinataire.
- Convention : nouveau build (RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne grossis rien.
- Fini quand : au banc, avec deux Léa, chaque écran et chaque confirmation dit de quelle Léa il s'agit.

6. Reporter une correction de modèle chez N athlètes ne se défait pas d'un coup — Priorité : Haute · Effort : S

Analyse et idée. reporterPropagation (rc-core:38511) pousse une entrée d'historique par athlète, puis écrit et envoie à tous. Si le coach a reporté une mauvaise correction à 12 athlètes, il doit ouvrir 12 programmes et restaurer 12 fois. Le modèle lui-même garde 3 versions (MOD_VERSIONS_MAX, 38172) que personne ne peut restaurer. L'idée : chaque report reçoit un identifiant de lot, et « Annuler le report (12 athlètes) » apparaît dans le toast puis dans l'encart de propagation du modèle. Une « version précédente du modèle » devient restaurable.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. reporterPropagation (~38511) applique une correction de modèle aux athlètes cochés. Pour chacun, il appelle _pushSessionsHistory, réécrit sessions_config, pose assignedProgramVersion, puis CLOUD.pushOne. Rien ne relie ces N écritures : impossible de les annuler ensemble. modeleVersionner (~38438) garde 3 versions du modèle (p.versions) mais aucune interface ne les restaure.

À faire :

1. Un identifiant lot = 'rep_'+Date.now(). Chaque entrée d'historique créée par ce report porte {lot, motif:'report:<nom du modèle>'. currentUser.derniersReports garde au plus 5 lots : {lot, modeleId, nom, le, athletes:[ids], versionAvant}.
2. Après le report, rcAnnulable (point 2 de l'audit ; si absent, crée-le) affiche « Correction reportée chez 12 athlètes · Annuler ». Dans renderPropagationEntree, ajoute aussi une ligne « Dernier report le 6 oct. · Annuler chez les 12 », valable 7 jours.
3. annulerReport(lot) : pour chaque athlète du lot, si son sessions_config est encore exactement celui écrit par le report (comparer modSlim), restaurer l'entrée d'historique du lot et remettre assignedProgramVersion. Si l'athlète ou le coach l'a modifié depuis, ne pas écraser : le lister dans « 2 athlètes ont changé depuis : à voir un par un » avec un lien vers leur programme.

4. Sous l'éditeur du modèle, « Versions précédentes du modèle (n) » liste p.versions avec la date et « Restaurer cette version » (le contenu redevient le brouillon du modèle, et rien ne part chez les athlètes sans le report habituel).

5. Cas limites : athlète retiré du suivi entre-temps (ignoré, et le dire), envoi hors ligne (file de renvoi), double appui sur Annuler.

Tests dans app/tests.js : report sur 3 athlètes puis annulerReport rend les 3 sessions_config d'origine ; un athlète modifié entre-temps n'est pas écrasé et figure dans la liste ; la restauration d'une version de modèle ne touche aucun athlète.

Convention : nouveau build (RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne grossis pas les polices.

Fini quand : au banc, un report fait par erreur chez 3 athlètes se défait en un appui.

7. Réponse envoyée sur le mauvais bilan : impossible de la retirer — Priorité : Moyenne · Effort : S

Analyse et idée. saveReponseBilan (rc-core:78100) envoie la réponse et la notification dès le clic. Une réponse vide est refusée (« Écris ta réponse avant d'envoyer », 78107) : on ne peut donc jamais la retirer. Une correction écrase l'ancien texte sans trace. Le coach qui a répondu à Léa Martin au lieu de Léa Moreau ne peut ni annuler ni déplacer sa réponse. L'idée : la notification part après 10 s avec « Annuler l'envoi », un lien « Retirer ma réponse » sur une réponse non lue, et l'historique des versions de la réponse.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. saveReponseBilan (~78100) écrit b.reponseCoach, pousse le dossier, et à la première réponse dépose tout de suite l'événement de notification (deposerEvenement type 'reponse_bilan'). Un texte vide est refusé, donc une réponse ne se retire jamais. Une modification écrase la précédente sans garder de trace.

À faire :

1. Envoi différé de la NOTIFICATION seulement, pas de l'écriture : après l'écriture, rcAnnulable (point 2 de l'audit ; le créer s'il n'existe pas) affiche « Réponse envoyée à Léa Mo. · Annuler » pendant 10 s. Le dépôt de l'événement attend la fin du délai. Annuler remet b.reponseCoach et b.reponseDate à leur état d'avant (instantané), pousse, ne notifie pas, et remet le texte dans le brouillon (rbNoterBrouillon) pour qu'il ne soit pas perdu.
 2. Dans blocReponseBilan (~78462), sous « Ta réponse · non lue » : un lien « Retirer ma réponse ». Il supprime reponseCoach, reponseDate et reponseVue, garde le texte dans b.reponsesRetirees (au plus 3 : {t,le}) et remet le texte dans le brouillon. Si la réponse est déjà lue, le lien demande confirmation : « Léa l'a déjà lue ».
 3. Une modification garde la version précédente dans b.reponsesPrecedentes (au plus 3), visible côté coach seulement (« modifiée le ... »).
 4. Si le coach quitte l'écran pendant les 10 s, la notification part quand même à la fin (pas de perte). Si l'app est tuée, la file d'événements la rejoue au démarrage (persister l'attente dans localStorage).
- Tests dans app/tests.js : annuler dans le délai ne dépose aucun événement et restaure l'état ; retirer une réponse non lue remet le bilan dans « à lire » (hasNewBilan vrai) ; une modification garde l'ancienne version ; le contrôle d'appartenance (_estMonAthlete) s'applique aussi au retrait.
- Convention : nouveau build (RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne grossis ni police ni cible.
- Fini quand : au banc, une réponse envoyée au mauvais athlète s'annule en un appui, sans notification, et le texte reste disponible pour la bonne athlète.

8. Le coach ne peut corriger ni une séance ni une pesée aberrante de son athlète — Priorité : Moyenne · Effort : M

Analyse et idée. Testé au banc : séance de Léa avec un squat à 825 kg, faute de frappe. Sur la carte de séance du coach (_buildSessionCard, rc-core:31778), aucun bouton pour corriger, retirer une série ou l'exclure des calculs. Il n'y en a pas non plus pour les pesées. Le coach doit demander à l'élève, ou bricoler la base. L'idée : « Corriger » sur une série ou une pesée. La valeur d'origine est gardée

(`corrige:{avant,par,le}`)), l'athlète voit « corrigé par ton coach », et une option « Ne pas compter dans les calculs » ne supprime rien.

Prompt de mise en place.

Contexte : RepCore, `app/rc-core.1792.js`. Côté coach, les séances faites se lisent dans `_buildSessionCard` (~31778) et la fiche (`renderCoachSessionRecap` ~31662), sans aucune action de correction. Une faute de frappe (825 kg au lieu de 82,5) fausse records, e1RM, charges suggérées et volume. Rien ne permet non plus au coach de corriger une pesée de `weightLog`. Le mécanisme de pierre tombale (`marquerSupprime` ~6482) existe pour `sessions/bilans/videos` mais n'est appelé que pour les vidéos.

À faire :

1. Sur chaque série d'une séance faite, côté coach seulement : un appui long ou un petit lien « Corriger » ouvre une feuille avec charge, répétitions et RIR, plus une case « Ne pas compter cette série dans les calculs ». L'écriture pose `set.corrige={avant:{weight, repsDone, rir}, par:coachId, le:ts}` ou `set.horsCalcul=true`. On passe par `getOwnedClient`, `updatedAt`, `DB.set`, `CLOUD.pushOne`.
 2. Toutes les fonctions de calcul (`records`, `e1RM`, charge suggérée, volume : grep les lecteurs de `sets[].weight`) ignorent les séries `horsCalcul`. Fonction PURE `serieComptee(set)` partagée.
 3. Pesées : sur la courbe de poids côté coach, un appui sur un point ouvre « Corriger / Ne pas compter », avec le même champ `corrige`.
 4. Côté athlète, une mention discrète « corrigé par ton coach » sur la série ou la pesée, avec la valeur d'avant au survol ou à l'appui. Pas de notification.
 5. Détection : signaler au coach (une ligne, pas une alerte) une série dont la charge dépasse 2,5 fois le meilleur e1RM connu de l'exercice, avec le bouton « Corriger » à côté.
 6. Cas limites : séance modifiée en même temps par l'athlète (la fusion champ par champ de `_mergeUser` doit garder les deux : `test`), virgule décimale, annulation via `rcAnnulable`.
- Tests dans `app/tests.js` : `serieComptee` ; un record calculé ignore une série `horsCalcul` ; `corrige.avant` est conservé ; la détection signale 825 kg et pas 85 kg.
- Convention : nouveau build (`RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne grossis ni police ni cible.
- Fini quand : au banc, le coach neutralise ou corrige le 825 kg en deux gestes, et les records de Léa redeviennent justes.

9. Un « mode support » pour Kevin : voir, réparer, tracer — sans toucher à la base — Priorité : Moyenne · Effort : L

Analyse et idée. Kevin peut seulement ouvrir ou fermer un accès (`s-coach-acces`, `accesRouvrir rc-core:8836`). Les règles (`database.rules.json`, `users/$emailKey/.read`) l'empêchent de lire le dossier d'un élève d'un autre coach. Il n'y a ni « Signaler un problème », ni journal des actions coach, ni sauvegarde planifiée (aucun workflow de sauvegarde dans `.github/workflows`). L'idée : une route Worker `/support`, réservée à l'e-mail de Kevin, qui lit un dossier par e-mail et propose 6 réparations en un clic, chacune journalisée. Côté app, un bouton « Signaler un problème » joint un diagnostic.

Recoupement. La remontée automatique des erreurs JS est portée par l'expert 15 (point 5), et le formulaire de contact par l'expert 15 (point 7) : garder ici l'outil de réparation de Kevin et la sauvegarde.

Prompt de mise en place.

Contexte : RepCore. Front `app/rc-core.1792.js`, Worker `cloudflare/src` (`index.js` route les chemins), règles `database.rules.json`. Aujourd'hui Kevin (`CREATOR_EMAIL`) n'a que l'écran « Accès » (`s-coach-acces`, `accesRouvrir` ~8836, `CLOUD.poserDroits` ~5290). Il ne peut pas lire le dossier d'un élève d'un autre coach (règle `.read` de `users/$emailKey`). Toute réparation (bilan à rouvrir, programme à restaurer, file d'envoi bloquée) se fait donc à la main dans la console Firebase.

À faire :

1. Worker : route POST `/support`. Elle vérifie le jeton Firebase (même vérification que les autres routes authentifiées) ET que l'e-mail vérifié est `CREATOR_EMAIL`. Actions :
 - lire `{email}` : renvoie le dossier en lecture seule, allégé (sans photos), avec coach, statut, derniers bilans (`date`, `photos` présentes, `réponse`), `sessions_config_history` (`dates` et `motifs`), file d'événements en attente, `droits` ;

```

- rouvrirBilan {email, bilanId, vues} : pose b.aCompleter (même format que le point 1 de l'audit) ;
- restaurerProgramme {email, index} : remet une entrée de sessions_config_history, après avoir
poussé l'état courant dans l'historique ;
- retirerDoublonBilan {email, bilanId} : pose la pierre tombale (supprimes.bilans), en gardant le
bilan dans support_corbeille/{email} 30 jours ;
- rattacher {email, coachEmail} : remet coachEmailKey et coachId (retrait du suivi fait par erreur)
;
- exporter {email} : renvoie le JSON complet à télécharger.
Chaque action écrit support_log/{auto} = {le, action, email, avant (résumé), apres (résumé)}. Ce
journal est en écriture Worker seulement et en lecture Kevin seulement (ajouter la règle).
2. Front : un écran « Support », accessible depuis l'écran Accès pour Kevin seulement. Un champ e-
mail, puis une fiche résumé avec les 6 boutons, chacun avec rcConfirm qui nomme la personne. Puis le
journal des 50 dernières actions. Réutiliser les composants existants (pas de nouvelle taille de
police).
3. Côté athlète et coach : dans Profil, « Signaler un problème » ouvre une zone de texte. L'envoi part
avec un diagnostic automatique : build, écran actif, CLOUD.enAttenteDeSync(), les 20 dernières erreurs
JS gardées dans un tampon circulaire en mémoire et localStorage (window.onerror + unhandledrejection),
taille du stockage local. Le tout est envoyé à support_tickets via le Worker. Kevin voit les tickets
dans l'écran Support, et un ticket ouvre directement la fiche de la personne.
4. Plan C : un workflow GitHub planifié (cron quotidien) qui exporte la Realtime Database (avec le
compte de service déjà utilisé par les workflows) vers un stockage privé avec rétention de 30 jours.
Documenter la restauration d'UN dossier dans cloudflare/README.md.
Tests : cloudflare/test (refus si l'e-mail n'est pas celui de Kevin, refus sans jeton, chaque action
écrit support_log, restaurerProgramme garde l'état courant dans l'historique) ; app/tests.js (tampon
d'erreurs borné, contenu du diagnostic sans données de santé en clair).
Convention : nouveau build (RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3
scripts/versionner_actifs.py). Ne grossis ni police ni cible.
Finir quand : au banc, avec le Worker local, Kevin retrouve un élève par e-mail, rouvre son bilan pour
les photos et restaure son programme d'hier en moins d'une minute, et chaque geste apparaît dans le
journal.

```

Expert 9 · Bilans et suivi — 7/10

Le suivi est déjà riche : un questionnaire en 6 étapes avec reprise des mensurations, un contrôle de pose des photos par détection du corps, un minuteur 10 s, l'historique avec « Modifier », un avant/après en un geste, un brouillon de réponse chiffré, un rapport de période et la réponse vocale. Ce qui manque le plus, c'est la lecture : le coach ne voit pas ce qui a changé d'un bilan à l'autre (les ressentis qui chutent, une correction faite après sa réponse). En plus, deux graphiques de l'écran Évolution sont cassés.

1. Le bilan ne dit pas au coach ce qui a changé, et le brouillon ignore les ressentis — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc avec Léa, 6 bilans : motivation 8→8→7→7→5→4, sommeil « Bien » puis « Mauvais » deux fois, stress « Oui, beaucoup », « 4 ou plus » écarts, et la demande « Moins de jambes svp, je suis épuisée ». Le brouillon proposé (brouillonBilan rc-core:78306) ne parle que du poids et du nombre de séances, puis pose la question générique « le plus facile, le plus dur ». Aucune réponse du questionnaire ne figure dans BROUILLON_SIGNAUX (78245), et l'onglet Réponses n'affiche aucune comparaison avec le bilan précédent. L'idée : un bandeau « Depuis le bilan précédent » en tête de chaque bilan, plus une ligne « à creuser » injectée dans le brouillon.

Recoupement. Le bandeau « Depuis le bilan précédent » partage la tête du volet Réponses avec le bandeau de contexte d'entraînement de l'expert 1 (point 6) : un seul bandeau, deux lignes.

Prompt de mise en place.

```

Contexte : RepCore, app/rc-core.1792.js. Côté coach, l'onglet « Réponses » de s-coach-bilan-evo est
rendu par renderReponsesBilans(bilans,client) (≈78594). Le brouillon de réponse vient de
brouillonBilan(athlete,bilan,signaux,opts) (≈78306). Ses signaux (BROUILLON_SIGNAUX ≈78245) ne portent
que sur l'entraînement, jamais sur les réponses du bilan (bil-motivation « N/10 », bil-sleep-quality €

```

BIL_OPTS_SOMMEIL, bil-stress ∈ BIL_OPTS_STRESS, bil-cheat-meals ∈ Aucun/1/2/3/4 ou plus, bil-diff-type, bil-prog-modifs). Le coach doit lire chaque bilan seul et se souvenir du précédent.

À faire :

1. Fonction PURE comparerBilans(prec, der, u) → {lignes:[{cle,lib,avant,apres,sens:'mieux'|'moins bien'|'stable',fort:bool}], alertes:[...]}. Elle porte sur : poids en moyenne 7 jours (mm7/serieWeight, comme brouillonBilan ; masqué si aTCA(u) ou u.masquerPoids), tour de taille et de hanches (en ignorant les valeurs reportées : bmReportee), motivation (écart ≥ 2 points = fort), sommeil et stress (rang dans les listes d'options), écarts alimentaires, et la présence des 3 photos. Alertes : motivation ≤ 4 ou en baisse sur 3 bilans consécutifs ; sommeil « mauvais » 2 bilans de suite ; demande de modification du programme non vide ; photos absentes alors que le précédent en avait.
2. Dans renderReponsesBilans, côté coach uniquement (client présent), sous l'en-tête de chaque bilan non-départ : un bandeau compact « Depuis le bilan N-1 » de 3 à 6 puces (flèche + avant → après). Utilise les classes bn-* existantes. Pas de nouvelle taille de police : réutilise --fs-xs / --fs-2xs. Les alertes passent en premier, en couleur var(--orange).
3. Dans brouillonBilan, ajoute dans `accroche` au plus UNE phrase tirée des alertes (ex. « Ta motivation est passée de 8 à 4 en trois bilans et tu dors mal depuis deux semaines. »). La question devient alors ciblée (« Qu'est-ce qui pèse le plus en ce moment : le boulot, le sommeil, les séances ? »). Respecte BROUILLON_MAX_SIGNAUX : le signal de santé (douleur) reste prioritaire. Si aTCA(u) : pas de phrase sur le poids ni sur les écarts alimentaires.
4. Sur la carte de la fiche (#ccd-bilans, ≈33363), remplace la ligne « Motivation : 4/10 » par « Motivation 4/10 (↓ de 8) » quand l'écart est fort.
Cas limites : premier bilan coaching (seul le départ précède : rien à comparer, pas de bandeau) ; réponses absentes d'un des deux bilans (la puce disparaît, jamais de « undefined ») ; anciens libellés des options (passe par _texteReponse) ; un bilan modifié (modifieLe) se compare avec ses valeurs actuelles.
Tests app/tests.js : comparerBilans sur 8/10-4/10 donne fort et 'moins bien' ; sommeil Bien-Mauvais ; une taille reportée n'est pas comparée ; brouillonBilan avec alertes contient la phrase sur la motivation, et aucune phrase de poids si aTCA. Lance chargerTests().
Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. N'agrandis aucune police. Critère de fin : sur la persona de Léa, le bandeau affiche « Motivation 5 → 4 », « Sommeil Mauvais (2e fois) » et « Photos absentes », et le brouillon le mentionne.

2. Un bilan corrigé après la réponse du coach ne prévient personne — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc dans le cas réel de Kevin : bilan validé sans photos, le coach répond, puis l'athlète ajoute ses photos via « Modifier ». Le bilan reçoit bien modifieLe, mais bilanRepondu reste vrai et hasNewBilan rend false (rc-core:30413). Aucun événement n'est déposé (_bilEnregistrerModif 75074), et on ne garde pas la liste des champs changés. Les photos arrivent donc sans que le coach le sache. Il faut un signal « Bilan corrigé » qui dise ce qui a changé.

Recoupement. Point de référence pour le signal « Bilan corrigé » chez le coach (repris par les experts 7, 8 et 17).

Prompt de mise en place.

Contexte : app/rc-core.1792.js. modifierBilan(id) (≈75035) rouvre un bilan déjà envoyé, et _bilEnregistrerModif (≈75074) appelle appliquerModifBilan(b,d,maintenant) (≈75055). Cette fonction rend la liste des clefs changées et pose b.modifieLe. Côté coach, hasNewBilan(c) (≈30413) ne regarde que bilanRepondu(dernierBilan) : une correction faite APRÈS la réponse du coach (cas réel : photos ajoutées après coup) ne produit aucun signal. Le coach voit seulement « modifié le ... » s'il rouvre le bilan.

À faire :

1. Dans appliquerModifBilan, stocke b.modifs = {le:maintenant, cles:[...]} : les clefs changées, photos regroupées en 'photos', 10 au maximum. Si le bilan était déjà répondu, ajoute b.modifApresReponse = true.
2. Fonction PURE bilanCorrigeNonVu(c, vuLe) : vrai si un bilan des 60 derniers jours porte modifApresReponse et que modifs.le > vuLe. vuLe est stocké côté coach dans currentUser.bilansVus[cleAthlete] (horodatage), mis à jour quand le coach ouvre viewClientBilans pour cet athlète.
3. Dans renderTodoBlock, ligne type 'bilan_corrige', « Bilan corrigé : <prénom> a ajouté ses photos »

scripts/versionner_actifs.py). Pas de police agrandie. Critère de fin : aucune pageerror en ouvrant l'évolution d'un athlète à 7 bilans, courbes rouges, anneaux visibles, colonne Évolution juste.

4. Le poids pré-rempli est celui du bilan précédent, alors que les pesées du matin existent — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : Léa a 60 pesées quotidiennes. Pourtant, la case « Poids actuel » du bilan reprend 63 kg, le poids du bilan d'il y a 14 jours (mensurationsReprises 74742). Cette valeur reprise est ensuite retirée à l'envoi, et le banc affiche « Valider sans ton poids ? Ta courbe n'aura pas de point » sur un champ qui semblait rempli. Il faut pré-remplir avec la moyenne des pesées des 7 derniers jours, comptée comme une vraie valeur : un champ de moins à remplir, et une donnée plus juste.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. À l'ouverture d'un bilan coaching, mensurationsReprises(u) (≈74742) pré-remplit bil-weight avec le poids du DERNIER BILAN, marqué comme « reprise ». _bilSansReprises le retire au moment de l'envoi, et bilNext (≈75284) demande alors « Valider sans ton poids ? ». Or l'athlète se pèse souvent chaque jour (weightLog, lu par serieWeight / mm7, déjà utilisés par brouillonBilan ≈78316).

À faire :

1. Fonction PURE poidsPourBilan(u, maintenant) → {kg, source:'pesees', n} : la moyenne des pesées des 7 derniers jours si au moins 3 existent (arrondie à 0,1). Sinon, la dernière pesée de moins de 3 jours. Sinon null.

2. Dans la préparation du bilan coaching (là où _bilReprises est rempli), si poidsPourBilan rend une valeur : bilData['bil-weight']=String(kg), SANS l'ajouter à _bilReprises (c'est une mesure réelle). Sous la case, une ligne rc-micro : « Moyenne de tes 5 pesées de la semaine. Corrige si besoin. » Le bilan stocke bilPoidsSource='pesees'. Sinon, comportement actuel.

3. Si u.masquerPoids ou aTCA(u) : aucun pré-remplissage, comportement actuel.

4. La reprise du poids d'un ancien bilan reste en repli quand il n'y a pas de pesée.

Cas limites : pesées aberrantes (ignore les valeurs à plus de 3 kg de la médiane de la semaine) ; athlète qui corrige la valeur (bilPoidsSource retiré) ; modification d'un ancien bilan (pas de pré-remplissage : modifierBilan garde la valeur stockée).

Tests app/tests.js : 5 pesées → moyenne exacte, non comptée comme reprise, et

_bilPoidsSaisi(_bilSansReprises(...)) vrai ; 2 pesées anciennes → null ; masquerPoids → null. Lance chargerTests().

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Pas de police agrandie. Critère de fin : sur la persona du banc, la case poids montre la moyenne de la semaine et la confirmation « Valider sans ton poids ? » n'apparaît plus.

5. L'écran « Bilan enregistré » annonce « stable » sur des mesures non reprises, et ne montre jamais le chemin parcouru — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : l'athlète valide en gardant les mensurations pré-remplies, et la restitution affiche « Tour de biceps droit : stable, Tour de poitrine : stable ». restitutionBilan (86199) compare getBM sans exclure les valeurs reportées, donc ce « stable » n'a jamais été mesuré. On ne voit pas non plus le chemin depuis le début (Léa : -5 cm de taille, -3 kg). Il faut n'afficher que du mesuré, et ajouter une ligne « Depuis le début » avec la vignette avant/après déjà existante : c'est le moment « waouh ».

Recouplement. La ligne « Depuis le début » et la vignette avant/après sont portées par l'expert 13 (point 4) : appliquer ici uniquement l'exclusion des mesures reportées.

Prompt de mise en place.

Contexte : app/rc-core.1792.js, restitutionBilan(user,now) (≈86199) et ouvrirRestitutionBilan (≈86298), écran s-bilan-fait (#bf-contenu). Problèmes : (1) les mensurations reprises sans être re-mesurées (bmReportee(b,k)) sont comparées et affichées « stable » ; (2) la comparaison porte

uniquement sur le bilan précédent ; (3) aucune image.

À faire :

1. Dans `restitutionBilan`, exclus de la comparaison toute mesure reportée dans le dernier bilan (`bmReportee(der,m.k)`). Pour le bilan précédent, prends le dernier relevé réel.

2. Ajoute `out.depuisDebut` : poids (mm7 si des pesées existent) et les 2 mensurations qui ont le plus bougé depuis le bilan de départ (premier relevé réel → dernier relevé réel), avec le nombre de semaines. Respecte `user.masquerPoids`, `aTCA(user)` (pas de poids) et `_synPoidsDansLeSens`.

3. Dans `ouvrirRestitutionBilan` : un bloc « Depuis le début (N semaines) » sous les lignes existantes. Si `aaDisponible(user)`, une vignette avant/après composée hors écran avec `_dessinerAvantAprès` (premier et dernier bilan, vue face, flou du visage activé) et un bouton « Mon avant/après » qui ouvre `ouvrirAvantAprès('athlete')`. Rien n'est partagé automatiquement.

4. Si le bilan a été validé sans photos alors que le précédent en avait : une ligne « Ajoute tes photos quand tu peux » avec un bouton qui appelle `modifierBilan(id)` directement sur l'étape photos (`bilStep` = dernière étape).

Cas limites : premier bilan coaching (`depuisDebut` = celui-ci contre le départ) ; aucune mesure réelle (pas de bloc mensurations, pas de « stable » inventé) ; utilisateur qui masque son poids.

Tests `app/tests.js` : un bilan dont toutes les mesures sont reportées ne produit aucune ligne de mensuration ; `depuisDebut` donne -5 cm pour une taille 78→73 ; `masquerPoids` retire le poids des deux blocs. Lance `chargerTests()`.

Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Pas de police agrandie, réutilise les classes de l'écran. Critère de fin : sur la persona de Léa, l'écran affiche « Depuis le début : taille -5 cm, hanches -4 cm » et la vignette avant/après, et plus aucun « stable » non mesuré.

6. Un bilan plus court : mensurations clés chaque fois, mensurations complètes toutes les 4 semaines — Priorité : Moyenne · Effort : M

Analyse et idée. Mesuré au banc avec un robot (1 s de réflexion par question) : 6 étapes, 13 champs chiffrés, 31 gestes si on accepte les reprises et 56 si tout est ressaisi, soit 73 à 87 s de pure manipulation. En vrai, il faut ajouter le ruban pour 12 tours (cou, buste, mollets D/G...) et 3 photos : environ 10 à 15 min. Chez Léa, 8 tours sur 12 n'ont pas bougé en 6 bilans. L'idée : un réglage coach par athlète, « tours suivis à chaque bilan » (par défaut taille, hanches, cuisse, bras), avec le schéma complet toutes les 4 semaines. Les étapes 2 à 5, toutes courtes, se regroupent en une seule étape « Ta semaine ».

Recoupement. Les réglages valables pour toute l'équipe sont chez l'expert 14 (point 2) ; ici, le choix des tours suivis par athlète.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `BIL_STEPS` (≈78691) : 1 Mensurations (`bBodySchema('bil')` : 12 tours de MEAS + poids), 2 Difficultés & Alimentation, 3 Ressenti, 4 Sommeil & Stress, (Traitement semestriel), 5 Objectifs, (questions du coach), 6 Photos. Au banc : 6 étapes et jusqu'à 56 gestes. La plupart des tours (cou, buste, mollets) bougent peu d'une quinzaine à l'autre.

À faire :

1. Nouveau champ posé par le coach sur le dossier de l'athlète : `mesuresBilan` = {cles:[...sous-ensemble de MEAS.k], `completToutesLes`:4 (semaines)}. Par défaut (champ absent) : comportement actuel, les 12 tours. Réglage dans la fiche coach, près du calendrier des bilans (`#ccd-bil-cal`) : une liste de cases à cocher, avec un préréglage « Essentiel : taille, hanches, cuisse D, biceps D ». Vérifie que `database.rules.json` laisse le coach écrire ce champ, comme pour `questionsCoach`.

2. Fonction PURE `mesuresDemandees(u, maintenant)` : toutes les clefs si aucun bilan n'a eu le jeu complet depuis `completToutesLes` semaines, sinon `mesuresBilan.cles`. `bBodySchema` reçoit cette liste : il n'affiche que ces cases, avec un lien « + autres mesures » qui déplie le reste. Les tours masqués ne sont PAS recopiés comme reprises (pas de fausse stabilité).

3. Fusionne les étapes 2, 3, 4 et 5 en une étape « Ta semaine » : mêmes clefs, même ordre, mêmes composants (`bC`, `bEmojiScale`, `bSlider`, `bTA`). Les zones de texte « détail » ne s'affichent que si la réponse associée l'appelle (`difficultés` = Oui..., `stress` = Oui..., écarts ≥ 1). Les clefs de données restent INCHANGÉES (`bil-*`), pour l'historique, `renderReponsesBilans` et les tests.

4. Le libellé d'étape (`#bil-step-label`) passe à 3 ou 4 étapes. Le brouillon (`_bilEcrireDraft`, `bilStep`) doit rester compatible : un brouillon ancien avec `bilStep` > nombre d'étapes reprend à la dernière étape.

Cas limites : `modifierBilan` d'un ancien bilan (toutes les mesures qu'il porte doivent rester éditables) ; coach qui décoche tout (minimum : la taille) ; bilan de départ inchangé (`DEB_STEPS` hors

périmètre).

Tests app/tests.js : mesuresDemandees rend 12 clefs au 1er bilan puis 4, et de nouveau 12 après 4 semaines ; _etapesUtiles(BIL_STEPS).length ≤ 4 sans traitement ni questions du coach ; un brouillon avec bilStep=5 se recalcule. Mesure au banc le nombre de gestes avant/après et note-le dans le message de commit.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne grossis ni polices ni cibles. Critère de fin : un bilan « essentiel » se remplit en 4 étapes, avec moins de 30 gestes en ressaisie complète.

7. Photos : caler la nouvelle photo sur la précédente, pas sur une silhouette générique — Priorité : Moyenne · Effort : M

Analyse et idée. Le minuteur (bilMinuteur 76046) superpose une silhouette d'illustration (posesGenre), pas la photo précédente de l'athlète. Le contrôle (photoControle 75565) vérifie la pose, mais pas que la distance et le cadrage soient les mêmes que la dernière fois. Or l'avant/après n'est spectaculaire que si les deux images se superposent. L'idée : en option, un calque « fantôme » de la dernière photo de la même vue à 35 % d'opacité dans la caméra, plus un avertissement doux quand la taille du corps dans le cadre s'écarte de plus de 15 % (les points du corps existent déjà, voir aaReperes).

Prompt de mise en place.

Contexte : app/rc-core.1792.js. bilMinuteur(key,vue) (≈76046) ouvre la caméra (getUserMedia) avec = silhouette générique. photoControle(raw,vue) (≈75565) juge la pose à partir de 33 points (raw.pts). aaReperes(img) (≈83437) et aaModeCadrage (≈83430) savent déjà repérer épaules, hanches et chevilles pour aligner un avant/après. photoBilanSrc / aaChargerPhoto lisent la photo d'un bilan existant.

À faire :

1. Dans bilMinuteur : si un bilan antérieur porte une photo de la même vue (aaBilansAvecPhoto(currentUser,vue)), affiche par défaut sa dernière photo en calque (opacity .35, object-fit identique à la vidéo) à la place de la silhouette. Un petit bouton à bascule « Silhouette / Ma dernière photo » (mémorisé en localStorage, lecture et écriture sous try/catch).
2. Après la capture (et pour une photo ajoutée via « Ajouter »), compare la hauteur épaules-chevilles et le centre du corps entre la nouvelle photo et la précédente (aaReperes). Si l'écart de taille dépasse 15 % ou le décalage 10 % de la largeur : ajoute le code 'cadrage' (orange, jamais rouge) dans le verdict existant (_htmlPhotoVerdict), avec le message « Recule d'un pas / rapproche-toi pour retrouver le cadrage de ta dernière photo ».

3. Rien ne part au serveur : tout reste local, comme l'avant/après.

Cas limites : premier bilan (silhouette, comme aujourd'hui) ; photo précédente seulement en ligne et non téléchargeable (silhouette) ; vue de profil de l'autre côté (pas d'avertissement de décalage) ; grossesse (grossesseSuspend) : pas de calque.

Tests app/tests.js : fonction PURE ecartCadrage(ra,rb) → null sous les seuils, code 'cadrage' au-delà ; le verdict reste 'orange' au maximum pour ce code. Lance chargerTests().

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Pas de police agrandie. Critère de fin : au 2e bilan, le minuteur montre la photo précédente en transparence, et une photo prise deux fois trop loin reçoit l'avertissement de cadrage.

8. Le rapport de fin de bloc ne parle pas du corps, et le rapport de période n'a pas de pré-réglage « ce bloc » — Priorité : Moyenne · Effort : M

Analyse et idée. bilanBloc (rc-core:37462) ne rend que l'entraînement (cadre, muscles, exercices, douleurs) : ni poids, ni mensurations, ni photos, ni ressentis. Le rapport de période (RAP_BLOCS 67430), lui, a poids, mensurations et photos, mais ses pré-réglages s'arrêtent à 14 jours, 28 jours et mois précédent (RAP_PRESETS 67864). Il n'a pas non plus de bloc « ressentis » (motivation, sommeil, stress des bilans). Le coach doit donc assembler deux documents à la main. L'idée : un pré-réglage « Ce bloc », un bloc « Ressentis », et un export de fin de bloc qui réunit les deux.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. bilanBloc(u) (≈37462) et bilanBlocExportHtml/bilanBlocExporter (≈37598-37625) : bilan d'entraînement du bloc (programmeDe(u) → debut, semaines). rapportPeriode(u,debut,fin,opts) (≈67776), avec RAP_BLOCS (≈67430), RAP_PRESETS (≈67864) et _rapPresetBornes (≈67881) : rapport imprimable (rapImprimer) avec poids, mensurations (rapMensurations) et photos (rapPhotos).

À faire :

1. RAP_PRESETS : ajoute {cle:'bloc', lib:'Ce bloc'}. _rapPresetBornes('bloc') prend programmeDe(u).debut jusqu'à min(maintenant, fin prévue). Préréglage masqué si l'athlète n'a pas de bloc daté.
2. Nouveau bloc RAP_BLOCS {cle:'ressentis', lib:'Ressentis des bilans'} placé avant « Mot du coach ». Fonction PURE rapRessentis(u,debut,fin) : motivation (première → dernière, moyenne), sommeil et stress (distribution), écarts alimentaires moyens, à partir des bilans de la période. Rendu sous forme d'un tableau sobre, comme les mensurations.
3. rapMensurations : n'utilise que les relevés réels (exclure bmReportee). Aujourd'hui, une valeur reportée peut servir de borne.
4. Dans la carte « Bilan du bloc » (bilanBlocExportHtml), un bouton « Rapport complet du bloc » qui ouvre le rapport avec rapPreset('bloc') et tous les blocs cochés : un seul document à remettre à l'athlète.

Cas limites : bloc sans bilan (bloc Ressentis : RAP_INSUFFISANT) ; un seul bilan ; aTCA (pas de poids ni d'écarts alimentaires, comme rapSanteAutorisee).

Tests app/tests.js : _rapPresetBornes('bloc') suit programmeDe ; rapRessentis sur 3 bilans 8/10, 6/10, 4/10 rend debut 8, fin 4 ; rapMensurations ignore une valeur reportée. Lance chargerTests().

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Pas de police agrandie. Critère de fin : depuis la fiche, « Rapport complet du bloc » produit un PDF avec séances, poids, mensurations, avant/après et ressentis sur les dates du bloc.

9. Avant/après animé : la transformation en vidéo de 6 secondes — Priorité : Basse · Effort : L

Analyse et idée. L'avant/après actuel (_dessinerAvantAprès, aaReperes 83437) compose une image fixe entre deux bilans, déjà alignée sur les repères du corps. Léa a 7 bilans photographiés, et une seule image n'en montre que deux. L'idée : un fondu enchaîné de tous les bilans alignés, avec la date et le tour de taille qui défilent, exporté en vidéo verticale (MediaRecorder sur le canvas). C'est l'effet « waouh » le plus partagé chez la concurrence, et tout se fait sur l'appareil.

Prompt de mise en place.

Contexte : app/rc-core.1792.js, module « MON AVANT / APRÈS » (≈83245-83830) : aaBilansAvecPhoto(u,vue), aaChargerPhoto(b,vue), aaReperes(img), aaModeCadrage, _dessinerAvantAprès(d), _aaSortir (partage/enregistrement, avec avertissement de première fois et aaExportAutorise pour le coach). Tout est local, avec un flou du visage disponible.

À faire :

1. Dans la feuille avant/après, une bascule « Image / Évolution animée ». Le mode animé charge toutes les photos de la vue choisie (8 au maximum : la première, la dernière et des intermédiaires régulières), les aligne une par une sur la première avec aaReperes (échelle et décalage par épaules-chevilles), puis enchaîne des fondus de 600 ms dans un canvas au format choisi (VISUEL_FORMATS). Un bandeau bas affiche la date, « Semaine N » et, en option, le poids et le tour de taille (mêmes options que l'image : poids masqué si u.masquerPoids ou aTCA).
2. Export : canvas.captureStream(30) + MediaRecorder (webm/vp9, repli mp4 si supporté), 6 à 8 s, puis partage via navigator.share({files}) ou téléchargement. Mêmes garde-fous que _aaSortir : avertissement de première fois, flou du visage, autorisation du coach.
3. Sans MediaRecorder (certains iOS) : l'animation se joue à l'écran, et seul « Enregistrer l'image » reste proposé.

Cas limites : photos de résolutions différentes ; une photo illisible (elle est sautée, pas d'échec global) ; 2 photos seulement (simple fondu) ; mémoire (libère les ImageBitmap et révoque les URL blob).

Tests app/tests.js : fonction PURE aaSelectionAnimee(liste,8) qui garde la première et la dernière ; aaTransformeAlignement(ra,rb) rend échelle et décalage attendus sur des repères synthétiques. Lance chargerTests().

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3

Expert 10 · Intuitivité côté élève — 6/10

Chaque fonction existe et le détail est soigné : retours par `retourDe`, onglet allumé selon l'écran réellement affiché, possibilité de corriger un bilan déjà envoyé. Ce qui manque le plus, c'est la hiérarchie : 7 onglets, un accueil de 2 700 px dont l'action principale est sous la ligne de flottaison, des réglages sur 8 écrans et 4 façons de joindre son coach, si bien qu'au test des 12 tâches (annexe `audit2/10/annexe-carte-et-test.md`) il faut environ 21 taps et 18 défilements, 9 tâches font hésiter, 1 échoue (« mes records ») et 1 tend un piège.

1. L'accueil cache l'action du jour sous deux écrans de cartes — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc (390×844) : « Démarrer ma séance » est à 1 200 px, « Reprendre maintenant » (séance interrompue) à 1 640 px, sous la pesée (`index.html:3626`), et « Bilan en retard » à 1 800 px. Au-dessus, on trouve la carte coach (170 px), deux questions de sommeil d'affilée (point du jour en heures, puis check-in en note de 1 à 5) et la relance de photo. L'idée : une seule « carte du moment » dans le premier écran, choisie par une règle fixe (séance en cours > bilan en retard ou à reprendre > séance du jour > repos), et un bloc « Ton matin » qui fusionne point du jour et check-in. Le reste ne change pas, mais passe dessous.

Recoupement. La carte du moment reprend les états de séance de l'expert 16 (point 2) et de l'expert 7 (point 2).

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching). Front : app/index.html (écran #s-client-home, lignes ~3376-3700)
et app/rc-core.<build>.js (loadClientHome ~44372, _rendrePointDuJour ~81076, _rendreCheckin ~88424,
renderCartePesee ~118755, carte #clh-resume-workout, #clh-resume-bilan, #clh-echeance, bloc #clh-
demarrer = parcours d'accueil).
Problème mesuré : sur un téléphone 390×844, le bouton « Démarrer ma séance » est à 1 200 px, la
reprise d'une séance interrompue à 1 640 px et le bilan en retard à 1 800 px. Le sommeil est demandé
deux fois de suite.
À faire :
1. Fonction PURE carteDuMoment(u, maintenant, env) -> {type, titre, sousTitre, action} avec cet ordre
de priorité : 'reprise_seance' (_woLoadSnap non nul) > 'reprise_bilan' (_bilLoadDraft rempli) >
'bilan_retard' (même calcul que la carte échéance actuelle) > 'seance_du_jour' (créneau du jour avec
exercices) > 'premiere_seance' (parcours) > 'repos' (prochaine séance et date). Une seule carte,
rendue dans un nouveau conteneur #clh-moment placé juste sous le bonjour, au-dessus de la carte coach.
Elle réutilise les fonctions existantes (woResumeAndGo, openSessionPicker, openBilan, modifierBilan) :
aucune nouvelle logique métier.
2. Ne pas dupliquer : quand la carte du moment montre la reprise, masquer #clh-resume-workout plus bas
; idem pour le bilan.
3. Fusionner point du jour et check-in en un bloc « Ton matin » : si le point du jour pose la question
du sommeil, la ligne « Sommeil 1-5 » du check-in ne s'affiche pas ce jour-là (la donnée reste
saisissable dans Lifestyle). Une seule croix pour masquer le bloc entier (accueilMasquer, clés 'pdj'
et 'ci').
4. Carte coach (#clh-coach-banner) : réduire à une ligne (avatar 32 px, nom, « Écrire »), sans changer
les tailles de police.
5. Ordre ensuite : Ton matin, pesée, messages non lus, nutrition du jour, puis stats, défis et le
reste, dans l'ordre actuel.
Erreurs humaines : double tap sur la carte (garde anti-rebond comme sur openSessionPicker) ; séance
interrompue d'un autre compte (_woSnapAMoi) ; instantané périmé de plus de 24 h ; hors ligne : la
carte s'affiche à partir du dossier local.
Tests dans app/tests.js (chargerTests) : carteDuMoment rend la bonne carte pour chacun des 6 cas et
pour leurs combinaisons (reprise + bilan en retard -> reprise) ; #clh-moment est au-dessus de #clh-
coach-banner dans le DOM ; un seul champ « Sommeil » visible quand le point du jour pose la question
du sommeil ; à 390×844, #clh-moment est entièrement visible sans défiler
```

```
(getBoundingClientRect().bottom < innerHeight - hauteur de la barre d'onglets).  
Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis  
python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les cibles : on réordonne, on ne  
gonfle pas.  
Critère de fin : tests verts ; au banc, l'action du jour est dans le premier écran dans les 6 cas.
```

2. Sept onglets serrés et mal nommés : passer à 5 destinations claires — Priorité : Haute · Effort : L

Analyse et idée. index.html:5700 : Accueil, Bilan, Nutrition, Corrections, Lifestyle, Évolution, Canal. À 360 px, l'onglet Bilan mesure 32 px de large avec un libellé de 10 px. « Canal » a une icône de cloche, « Lifestyle » l'icône de profil, Bilan et Évolution deux icônes de graphique. Bilan n'est qu'un lanceur muni d'un bouton « Annuler ». La pastille Lifestyle compte les jours sans données (rc-core:77993) et affiche 7 en rouge, comme un retour du coach. L'idée : Aujourd'hui · Entraînement · Nutrition · Progrès · Coach. Toutes les fonctions sont gardées, regroupées par intention (carte complète en annexe).

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching). Barre athlète : app/index.html #client-tabbar (~ligne 5700).  
Logique : app/rc-core.<build>.js : TABBAR_ECRANS (~9185), _majTabbar (~9226), clientTab (~45255),  
pastilles (_majPastilleVideos, _majPastilleBilan ~78005, _majPastilleLifestyle ~77993,  
_majOngletCanal).  
Objectif : 5 onglets au lieu de 7, sans retirer aucune fonction.  
- 'home' « Aujourd'hui » : s-client-home.  
- 'entraînement' « Entraînement » : nouvel écran racine s-entraînement. En haut, la séance du jour et  
la semaine (réutiliser htmlCarteSeanceSlot / _renderProgExercisesInto). Puis des entrées en liste :  
Mes séances (loadSessionManager), Historique (loadHistoriqueSeances), Records (voir le lot Records),  
Corrections vidéo (loadVideos, avec la pastille des retours non vus).  
- 'nutrition' « Nutrition » : s-nutrition, inchangé.  
- 'progres' « Progrès » : s-progress, avec un sous-onglet « Pas & sommeil » qui rend le contenu de  
loadLifestyle dans s-progress (ou garde s-lifestyle mappé sur 'progres' dans TABBAR_ECRANS).  
- 'coach' « Coach » : nouvel écran racine s-coach-espace (préfixe NON « s-coach- », sinon go() le  
refuse à l'athlète ; par exemple s-mon-coach). Sections : Messages (msgOuvrirFil), Bilans (prochain  
bilan, historique, modifier : reprendre openBilanChoice et ouvrirHistoriqueBilans dans la page plutôt  
que dans une feuille), Annonces (le contenu de loadCanal). Sans coach rattaché, l'onglet reste  
visible, grisé, et mène au code coach, comme le Canal aujourd'hui (aria-disabled).  
TABBAR_ECRANS : s-videos, s-historique-seances, s-session-manager -> 'entraînement' ; s-lifestyle, s-  
steps, s-sleep -> 'progres' ; s-bilan-choice, s-canal, s-messages -> 'coach'. La barre reste ainsi  
visible sur ces écrans (aujourd'hui elle disparaît sur Pas, Sommeil et Historique).  
Pastilles : une seule couleur « à traiter » (retour du coach non lu, bilan dû, message non lu). Les  
jours sans pas ni sommeil ne font plus de pastille rouge ; au plus un point discret, gris, sur  
Progrès.  
Libellés à 10 px conservés : on gagne de la largeur (5 × 78 px à 390 px) sans agrandir la police.  
Notifications push et liens profonds : vérifier chaque appel à  
clientTab('videos'|'lifestyle'|'canal'|'bilan'|'evolution') et les remapper (grep "clientTab(").  
Erreurs humaines : retour Android (HIST_RACINES : ajouter les nouvelles racines) ; un athlète habitué  
à l'ancien onglet Corrections reçoit une fois une info-bulle « Corrections est maintenant dans  
Entraînement » (clé localStorage, try/catch).  
Tests app/tests.js : mettre à jour le test R22 (~66252), qui exige aujourd'hui 7 onglets, pour en  
exiger 5 ; chaque écran de TABBAR_ECRANS allume le bon onglet ; aucun écran s-coach-* dans la table ;  
clientTab connaît les 5 clés ; à 360 px, chaque .tab-btn mesure au moins 64 px de large ; les anciens  
appels clientTab('videos') ouvrent toujours les corrections.  
Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE de app/sw.js, python3  
scripts/versionner_actifs.py). Ne pas agrandir les polices.  
Critère de fin : tests verts ; au banc, les 12 tâches de l'annexe se font en 2 taps au plus depuis  
Aujourd'hui.
```

3. « Contacter mon coach » : 4 portes, dont une note qui ne prévient personne — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : « Un souci ? Contacte ton coach » ouvre une feuille « E-mail / Dans l'app ». « Dans l'app » ouvre la carte « À dire à ton coach », qui précise : « Ce n'est pas un message : il n'y a pas de notification » (rc-core:32008, 32060). Juste dessous, la carte « Mon coach · Lui écrire, en privé » ouvre la vraie messagerie, et l'onglet Canal (icône de cloche) est une troisième chose encore. L'idée : une seule entrée « Écrire à mon coach », qui ouvre la messagerie. La note devient un message épinglé du fil, et le Canal devient « Annonces » dans l'onglet Coach.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching), app/rc-core.<build>.js : rendu du lien de contact (~31960, construit la liste _contactBouts avec ['b', 'motOuvrirDepuisContact()', 'Dans l'app']), ouvrirContactCoach (~31985), renderMotCoach / motOuvrir (~32014-32060), msgOuvrirFil (~27586), carte #clh-coach-banner et #clh-messages sur l'accueil (app/index.html ~3420-3440).  
Problème : un élève qui veut joindre son coach choisit « Dans l'app » et écrit une note que le coach ne verra qu'en ouvrant son dossier, sans notification. C'est le même type d'erreur que le bilan envoyé sans photos.  
À faire :  
1. Dans la feuille de contact, « Dans l'app » appelle msgOuvrirFil() (la messagerie). Les canaux externes (e-mail, WhatsApp) restent, après lui, avec la mention CONTACT_SORTIE.  
2. Sur l'accueil, une seule entrée visible : « Écrire à mon coach » (la carte coach réduite, voir le lot accueil). Le lien « Un souci ? Contacte ton coach » disparaît s'il fait doublon ; garder la feuille seulement si le coach a déclaré un canal externe.  
3. La note (motCoach) devient « Note épinglée pour ton coach », éditable en tête du fil de messages (même stockage, même MOT_COACH_MAX), avec la phrase « Visible par ton coach dans ton dossier, sans notification ». Ne rien migrer : les notes existantes s'affichent telles quelles.  
4. Renommer l'onglet ou la section « Canal » en « Annonces » (le titre dit que c'est le coach qui diffuse à tous) et remplacer l'icône cloche par une icône mégaphone si elle existe dans le jeu d'icônes (sinon garder la cloche).  
Cas limites : athlète sans coach (aucune entrée, comme aujourd'hui) ; hors ligne : le message part en file d'attente (comportement existant de la messagerie), ce qui est dit à l'écran ; double tap sur Envoyer.  
Tests app/tests.js : la feuille de contact ne contient plus motOuvrirDepuisContact ; son bouton « Dans l'app » appelle msgOuvrirFil ; un seul élément de l'accueil appelle msgOuvrirFil ; la note épinglée s'affiche dans s-messages quand motCoach(u) existe.  
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.  
Critère de fin : tests verts ; au banc, depuis l'accueil, « joindre mon coach » mène à la messagerie en 1 tap, quel que soit le bouton touché.
```

4. Réglages éparpillés et Profil fourre-tout de 3 800 px — Priorité : Haute · Effort : M

Analyse et idée. Les réglages vivent sur 8 écrans : unité des macros sur l'accueil, fréquence des bilans dans Bilan, approche de la diète dans Nutrition, objectifs dans Lifestyle, masquer le poids dans Évolution, cycle, nom sur les visuels et célébrations dans Profil, kg/lb, thème et rappels dans Réglages. Le Profil mélange identité, santé, parrainage, page publique, 56 badges, multi-comptes et déconnexion (testé : 3 792 px, il faut 4 défilements pour se déconnecter). L'export existe deux fois (Profil et Réglages). L'idée : faire de Réglages l'index unique, avec des sections et des liens directs vers chaque réglage, et scinder Profil en « Mon profil » (identité, santé) et « Trophées » (badges, page publique, parrainage).

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching). Écrans : app/index.html #s-client-reglages et #s-athlete-profile (~3860-4000) ; app/rc-core.<build>.js : ouvrirReglagesAthlete (~2787), openAthleteProfile (~126806), htmlReglagesPush (~80418), setBilanFreq, ouvrirChoixDiete, sanObjectif, togglePoidsMasque,
```

```

setMacroUnite, setVisuelNom, setAtpCycleSuivi, celebrationsChoisir, themeChoisir,
_rendreUniteReglages.
Objectif : l'élève trouve TOUT réglage à un seul endroit, sans qu'aucun contrôle ne change de
comportement.
1. Réglages devient un sommaire en sections repliables :
  - Entraînement : unité kg/lb, son, écran allumé, jours et heure de séance (ouvre
_ouvrirJoursEntraînement).
  - Notifications.
  - Nutrition : approche, unité des macros, aliments évités.
  - Suivi : objectifs de pas et de sommeil, masquer le poids, cycle.
  - Bilans : fréquence, SEULEMENT si le coach ne l'impose pas.
  - Affichage : thème, célébrations, nom sur les visuels.
  - Compte : mon profil, comptes, exporter, se déconnecter, supprimer.
Chaque ligne réutilise la fonction existante ou ouvre l'écran d'origine à l'ancre du contrôle. Le
contrôle reste aussi à son endroit actuel quand il y a du sens (pas de duplication de logique, une
seule source par réglage).
2. Une seule entrée « Exporter mes données » (dans Réglages > Compte) ; retirer celle du pied du
Profil.
3. Profil : garder identité, photo, infos médicales. Déplacer badges, page publique, parrainage et
célébrations dans un écran « Trophées », ouvert depuis le rang ou les volts de l'en-tête
(rcInfoOuvrir) et depuis le Profil par une ligne. « Se déconnecter » passe dans Réglages > Compte,
visible sans défilement une fois la section ouverte.
4. Accueil : la roue « Réglages » passe dans l'en-tête, à côté de l'avatar, au lieu du lien de 12 px
sous le bonjour.
Erreurs humaines : « Supprimer mon compte » reste derrière sa confirmation actuelle
(requestAccountDeletion) et loin de « Se déconnecter » ; une section repliée garde son état
(localStorage dans try/catch, la page s'affiche correctement sans).
Tests app/tests.js : chaque fonction de réglage listée ci-dessus est appelée par au moins un contrôle
de #s-client-reglages ; exporterMesDonnees n'est appelée que depuis un seul écran ; #s-athlete-profile
ne contient plus de .bdg (badges) ni logout() ; la hauteur de #s-athlete-profile reste sous 1 800 px
avec le dossier de test.
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3
scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : tests verts ; au banc, « changer mes jours de rappel », « passer en livres », «
masquer mon poids » et « me déconnecter » se font chacun en 3 taps au plus depuis l'accueil.

```

5. Aucune recherche : ajouter une loupe qui mène directement à l'écran, la séance ou l'aliment — Priorité : Moyenne · Effort : M

Analyse et idée. Aucune recherche transverse côté élève : aucune occurrence d'une recherche globale dans rc-core.1792.js. Seuls les aliments et les exercices se cherchent, chacun dans son écran. Kevin demande que « quand je cherche les trucs », on les trouve. L'idée : une loupe dans l'en-tête d'Aujourd'hui, avec un index déterministe (sans IA). Il couvre les destinations, avec synonymes (« pesée » → poids, « PR » → records, « déco » → se déconnecter), les séances passées par nom ou date, les exercices (fiche et record), les aliments du plan et les bilans.

Prompt de mise en place.

```

Dépôt RepCore (/home/user/coaching), app/rc-core.<build>.js et app/index.html, côté ATHLÈTE
uniquement.
Objectif : une recherche locale, instantanée et hors ligne, qui ouvre directement la bonne
destination.
1. Table figée RECHERCHE_DESTINATIONS = [{id, libelle, motsCles:[...], action:'nomDeFonction(...)'}]
qui couvre chaque écran et chaque réglage élève : séance du jour, mes séances, historique, records,
corrections, plan alimentaire, compléments, caféine, évictions, traitements, poids, mensurations,
photos, bilans, pas, sommeil, messages, annonces, réglages (chaque ligne du lot Réglages), profil,
trophées, parrainage, défis, export, déconnexion. Synonymes en français courant et sans accents :
poids/pesée/balance, records/PR/max, sommeil/nuit/dodo, coach/message/écrire, kg/lb/livres, etc.
2. Fonction PURE rechercher(u, requete, maintenant) -> [{type, libelle, sous, action}], max 12
résultats, triés par score. Le score repose sur la normalisation (minuscules, sans accents), sur le
préfixe de mot puis la sous-chaîne, et privilégie les destinations ; ensuite viennent les séances
(u.sessions : nom + date « 3 oct »), les exercices déjà faits (fiche + meilleur poids via
meilleursRecordsPublics), les aliments du plan (planDe) et les bilans (historiqueBilans).

```

3. Interface : loupe dans l'en-tête de s-client-home. Elle ouvre une feuille (`_feuilleOuvrir`) avec un champ au focus. En champ vide, elle affiche les 6 destinations les plus utilisées par CET élève (compteur local, try/catch) au lieu d'une page blanche. Entrée = premier résultat. Aucun appel réseau. Erreurs humaines : faute de frappe (tolérance d'une lettre sur les mots de plus de 4 lettres) ; aucun résultat : proposer « Écrire à mon coach » et les 6 destinations ; la feuille se ferme avant l'action (pas de double écran). Tests `app/tests.js` : `rechercher('pesee')` propose la pesée en premier ; `'PR'` propose Records ; `'squat'` renvoie l'exercice et sa meilleure charge ; `'deconnexion'` renvoie Se déconnecter ; `'oct'` renvoie les séances d'octobre ; chaque action de `RECHERCHE_DESTINATIONS` est une fonction définie (`typeof window[nom] === 'function'`) ; aucun libellé de destination n'est un écran s-coach-*. Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices. Critère de fin : tests verts ; au banc, chacune des 12 tâches de l'annexe se fait en 2 taps par la loupe (loupe, puis résultat), en tapant 3 lettres.

6. Libellés ambigus : « Notes », « Lifestyle », « Mes séances » x2, tuiles Perfs muettes — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : le sous-onglet « Notes » d'Évolution contient en fait les bilans et la réponse du coach. Deux écrans portent le titre « Mes séances » (historique et gestion). Les tuiles Perfs affichent « 24 TOTAL / 0 SEMAINES / 0 % FAITES / 0 MIN DURÉE » sans unité de sens. La carte « Progression par exercice » dit « 0 record sur la dernière séance · en recul » sans nommer l'exercice. Corrections ouvre sur une bannière publicitaire « Nouveau canal » avant les retours du coach. L'idée : un passage de renommage et de réordonnancement, sans toucher au visuel.

Prompt de mise en place.

Dépôt RepCore (`/home/user/coaching`), `app/index.html` et `app/rc-core.<build>.js`. Lot de libellés côté athlète, sans changement de taille ni de style.

1. Évolution : sous-onglet « Notes » -> « Bilans » (`showProgressTab('notes')` garde sa clé interne).
2. Titres : s-historique-seances « Historique » ; s-session-manager « Mon programme ». Bouton d'accueil « Gérer mes séances » -> « Mon programme ».
3. Onglet « Lifestyle » -> « Pas & sommeil » (ou section de Progrès si le lot onglets est fait).
4. Perfs (rendu des 4 tuiles dans `loadProgress/showProgressTab('perf')`) : « 24 séances au total », « 0 semaine validée », « 0 % des séances prévues faites », « Durée moyenne : — » quand la donnée manque (jamais « 0 MIN » pour une donnée absente).
5. Carte « Progression par exercice » : nommer l'exercice sélectionné dans le titre ; si aucun exercice n'a assez de données, afficher une phrase d'état plutôt que « 0 ... en recul ».
6. s-videos : les retours du coach non vus passent EN HAUT. La bannière « Nouveau canal » ne s'affiche qu'à un élève qui n'a jamais envoyé de vidéo.
7. Onglet Bilan : retirer le bouton « Annuler » d'un écran racine (la barre d'onglets suffit).
8. Carte de correction vidéo sur l'accueil : « Ton coach a corrigé ta vidéo « sans nom » » doit utiliser le nom de l'exercice de la vidéo, ou « ta dernière vidéo » quand il manque.

Tests `app/tests.js` : les textes visibles « Notes » et « 0 MIN » n'apparaissent plus dans `#s-progress` avec le dossier de test ; les deux écrans n'ont plus le même `.topbar-title` ; dans `#s-videos`, la première carte est un retour non vu quand il en existe un ; la carte d'accueil ne contient jamais « sans nom ».

Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Critère de fin : tests verts, aucune régression de `chargerTests()`.

7. « Voir mes records » échoue : aucune page ne les rassemble — Priorité : Moyenne · Effort : M

Analyse et idée. Les records ne s'affichent qu'en fin de séance ou dans une séance passée (« Mes records », `rc-core:65229-65264`). Évolution > Perfs ne les liste pas, et la page publique en montre 3 (`meilleursRecordsPublics`, `rc-core:20310`). C'est la seule tâche du test que l'élève ne peut pas terminer. L'idée : une page « Records » dans Entraînement (et dans la recherche), une ligne par exercice, avec le meilleur poids, la date et la séance d'origine en 1 tap.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching), app/rc-core.<build>.js et app/index.html.
Objectif : une page « Mes records » trouvable en 2 taps, qui réutilise les calculs existants.
1. Fonction PURE recordsParExercice(u) : généralise meilleursRecordsPublics (~20310) sans limite n, avec pour chaque exercice {exo, kg, reps de la série, date, idSeance}. Mêmes règles : séries done, alias via resoudreAlias(exKey()), poids >0 et ≤1000. Exercices assistés (contrepoids) : utiliser la même convention que recordsDeSeance (~64915), où moins est mieux.
2. Écran s-records (non préfixé s-coach-), titre « Mes records ». Liste triée par date du record (le plus récent en haut), filtre texte, et un tap ouvre la séance d'origine (ouvrirSeanceHistorique). Rien à afficher : phrase « Tes records apparaîtront après ta première séance » et bouton vers la séance du jour.
3. Entrées : Entraînement (ou à défaut, un bouton dans Évolution > Perfs au-dessus de « Progression par exercice ») et la recherche (lot Recherche). Dans le bloc « Mes records » de fin de séance, un lien « Tous mes records ».
4. Unité : _kgAff/_uniteTxt pour respecter kg/lb.
Cas limites : séance supprimée (le lien ne casse pas : on désactive la ligne) ; exercice renommé (alias) ; haltères par main (ne pas additionner).
Tests app/tests.js : recordsParExercice sur un dossier à 3 séances rend le max par exercice ; alias fusionnés ; assisté : la plus faible assistance l'emporte ; affichage en lb quand l'unité est lb ; s-records est dans TABBAR_ECRANS sous l'onglet entraînement (ou évolution).
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : tests verts ; au banc, « voir mon record au squat » prend 2 taps depuis l'accueil.

8. Les bilans ont deux maisons, et « Récap » fait changer d'onglet — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : Bilan > Historique des bilans (feuille) > Récap emmène dans Évolution > Notes, où l'onglet allumé change sous le doigt. La feuille indique « sans photo » sur chaque bilan, mais il faut « Modifier » puis parcourir les étapes pour ajouter les photos. C'est le cas réel de Kevin. L'idée : une page Bilans unique (prochain bilan, liste, réponse du coach dépliée sur place) et un bouton direct « Ajouter mes photos » qui rouvre le bilan à l'étape des photos.

Recoupement. Le bouton « Ajouter mes photos » rejoint celui de l'expert 7 (point 1) : même fonction, deux entrées (accueil et page Bilans).

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching), app/rc-core.<build>.js : openBilanChoice (~74884), historiqueBilans / ouvrirHistoriqueBilans / modifierBilan (~74994-75060), openBilanNotes (~78058), _etapesUtiles, BIL_STEPS / DEB_STEPS, photoBilanExiste.
Objectif : relire ou corriger un bilan sans quitter l'onglet Bilan (ou Coach).
1. Dans s-bilan-choice, la liste de l'historique s'affiche dans la page (plus de feuille). Chaque bilan se déplie sur place et montre mesures, réponses et « Réponse de ton coach » (réutiliser le rendu de openBilanNotes dans un conteneur local au lieu de go('s-progress')).
2. Si photos < 3 pour un bilan : bouton « Ajouter mes photos ». Il appelle modifierBilan(id) puis positionne bilStep sur l'étape des photos (chercher son index dans _etapesUtiles(BIL_STEPS ou DEB_STEPS)). L'enregistrement suit le chemin actuel _bilEnregistrerModif (même bilan, même numéro, « modifié le », le coach voit la correction).
3. Évolution > Bilans (ex-Notes) garde les courbes et renvoie à cette page pour le détail : une seule source de lecture.
Erreurs humaines : quitter au milieu (le brouillon existe déjà : vérifier qu'une modification interrompue se reprend depuis la carte de reprise de l'accueil) ; photo trop lourde ou hors ligne (photos en attente rc_pendingphoto_, déjà gérées) ; bilan supprimé entre-temps (toast existant « introuvable »).
Tests app/tests.js : le Récap ne change plus d'écran actif (reste s-bilan-choice) ; « Ajouter mes photos » n'apparaît que si photos < 3 ; il ouvre s-bilan à l'étape dont la clé contient 'photo' ; après enregistrement, le nombre de bilans est inchangé et modifieLe est posé.
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : tests verts ; au banc, « ajouter les photos oubliées à mon dernier bilan » se fait en 2 taps puis l'envoi.

9. Écrans en double et saisies mal placées : Pas et Sommeil x2, pesée absente de la courbe — Priorité : Basse · Effort : S

Analyse et idée. Les liens du point du jour (« Voir ma semaine de pas », « Saisir mon coucher et mon lever », rc-core:80720-80724) ouvrent `s-steps/s-sleep`. Ce sont des copies des sections de Lifestyle, sans barre d'onglets ; ce sont les seuls chemins vers ces deux écrans. À l'inverse, Évolution > Poids montre la courbe mais pas de champ de saisie : la pesée n'existe que sur l'accueil (`renderCartePesee`, rc-core:118755), et le Profil a un champ « Poids » de plus. L'idée : un seul écran par sujet, et la saisie là où l'on regarde.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching), app/rc-core.<build>.js : liens du point du jour (~80711-80724, champ `ecran:'loadSleep'|'loadSteps'`), `loadSteps` (~114922), `loadSleep` (~121833), `loadLifestyle` (~122027), `renderCartePesee` (~118755), `showProgressTab('poids')`, formulaire du Profil (champ poids).

1. Les liens du point du jour ouvrent Lifestyle (ou Progrès > Pas & sommeil) et font défiler jusqu'à la bonne section (`_defiler`). `s-steps` et `s-sleep` restent définis pour la compatibilité, mais plus rien dans l'interface n'y mène. Le test le vérifie.
2. Évolution > Poids : afficher en tête la même carte de pesée que l'accueil (extraire le rendu de `renderCartePesee` dans une fonction PURE `htmlCartePesee(u, maintenant)` appelée aux deux endroits ; mêmes gardes : `aTCA`, `masquerPoids`, `paliersDeficit 'blocage'`).
3. Profil : le champ « Poids (kg) » devient une lecture seule « Dernière pesée : 63,4 kg (6 oct.) » avec un lien vers la pesée. Il n'écrit plus `currentUser.weight` directement : une seule source, le journal de pesées.

Erreurs humaines : deux pesées le même jour (comportement « Corriger » existant) ; garde TCA : aucune pesée affichée nulle part.

Tests `app/tests.js` : aucun onclick de l'interface athlète n'appelle `loadSteps()` ou `loadSleep()` sans conteneur `lifestyle` ; `htmlCartePesee` rend '' quand `masquerPoids` ; `#s-progress` contient la carte de pesée quand elle est permise ; le Profil ne contient plus d'input de poids modifiable.

Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Critère de fin : tests verts ; au banc, les points d'entrée vers pas, sommeil et poids mènent chacun à un seul écran, avec la barre d'onglets visible.

Expert 11 · Design et cohérence — 6/10

La base est solide et déjà premium : une barre de titre identique partout (Bebas 16 px, 2,5 px, majuscules, mesurée sur 27 écrans), des cartes calées à 12 px de rayon dans 110 cas sur 150, une palette noir et rouge tenue, une gouttière de 20 px sur 28 écrans sur 31. Ce qui manque le plus, c'est une grammaire unique sous la barre de titre : 77 styles de titre relevés sur 31 écrans, 7 hauteurs pour le même bouton rouge, 5 états vides différents. L'impression d'app « compressée » ne vient pas d'un gonflement général (l'échelle `--fs-*` et les `min-height` n'ont presque pas bougé depuis le build 1693, et 25 écrans sur 31 sont identiques au pixel près) : elle vient d'éléments isolés trop grands, posés à côté d'éléments petits.

Méthode : banc `run.py` (Playwright 390×844, réseau bloqué). 19 écrans athlète et 12 écrans coach capturés au build 1792, et les mêmes écrans rejoués au build 1693 (`git archive 388488d`) pour la comparaison avant/après. Pour chaque écran, relevé des styles calculés (titres, boutons, cartes, badges, gouttières), puis comptage dans `rc-style.1792.css` (hors bloc clair généré) et dans `rc-core.1792.js`. Les captures et les planches sont dans `audit2/11/` : `s2/` (1792), `s-old/` (1693), `planche-titres.png`, `pal-4.png`, `pc1-3.png`, `z-c02.png`.

1. Une échelle de titres en 4 niveaux, pas 77 — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : 77 combinaisons (police, taille, graisse, casse, interlettre) servent de titre sur 31 écrans. Le titre de section du même écran Lifestyle mesure 25,7 px (« MES PAS », `.ls-titre` en `clamp(24px,6.6vw,40px)`, `rc-style:2386`) alors que « Mes compléments » fait 16 px avec une barre rouge, « Ton coach » 15 px en bas de casse, « LA SEMAINE » 10 px blanc et « CETTE SEMAINE » 16 px gris. Dans la feuille, 286 règles en majuscules espacées donnent 173 combinaisons distinctes, et `.rc-sect-t`, le seul candidat à devenir la classe commune, n'est utilisé nulle part. Règle proposée, sans rien agrandir : **T0** titre d'écran = `.topbar-title`, inchangé. **T1** titre de page dans le contenu (au plus un par écran) = trait rouge 24x2 px, puis Bebas 20 px, graisse 400, majuscules, interlettre 1 px, blanc. C'est le modèle des écrans coach Lundi, Relances et Activité. **T2** titre de section = Montserrat 11 px, graisse 800, majuscules, interlettre 2 px, `var(--sub)`, marge basse 10 px, rouge seulement si une action est attendue (règle N5.11). **T3** titre de carte = Montserrat 14 px, graisse 800, casse normale, `var(--text)`.

Prompt de mise en place.

Dépôt RepCore (PWA). Objectif : uniformiser les titres en 4 niveaux. Ne jamais agrandir une police : on ramène les titres vers l'échelle, et les tailles ne font que baisser.

Constat (build 1792, banc 390x844) : 77 styles de titre sur 31 écrans. Exemples : `.ls-titre` (`app/rc-style.<build>.css ~l.2386`) en `clamp(24px,6.6vw,40px)`, soit 25,7 px, plus gros que le titre d'écran (16 px) ; « Mes compléments » en `<h3>` Bebas 16 px graisse 700 (`rc-core ~l.108214`) ; « LA SEMAINE » 10 px blanc (`s-client-amis`) ; « CETTE SEMAINE » Bebas 16 px gris (`s-steps`, `s-sleep`) ; « Ton coach » Montserrat 15 px (`s-messages`) ; « FRÉQUENCE DES BILANS » 11 px gris. Par ailleurs `.rc-sect-t` (`~l.1635`) existe et n'est jamais utilisé.

1. Dans le `:root` de `app/rc-style.<build>.css`, ajoute quatre classes et documente-les dans un commentaire « ÉCHELLE DES TITRES » :

`.t-page` : trait rouge `::before` 24x2 px (`margin-bottom 10px`), `font-family var(--pile-titre)`, `font-size var(--fs-xl)` (20px), `font-weight 400`, `text-transform uppercase`, `letter-spacing 1px`, `color var(--text)`, `line-height 1.1`, `margin 0 0 6px`. Au plus UN par écran, sous la barre de titre.

`.t-section`: Montserrat, `var(--fs-xs)` 11px, `weight 800`, `uppercase`, `letter-spacing 2px`, `color var(--sub)`, `margin 0 0 10px`. Variante `.t-section.is-action` : `color var(--red-text)`, réservée à ce qui attend une décision.

`.t-carte` : Montserrat, `var(--fs-md)` 14px, `weight 800`, casse normale, `letter-spacing 0`, `color var(--text)`, `margin 0 0 4px`.

La barre de titre (`.topbar-title`) ne change pas : c'est le niveau T0.

Supprime `.rc-sect-t` (mort) ou fais-en un alias de `.t-section`.

2. Applique ces classes partout où un titre de section est écrit en style en ligne ou par une classe locale. Commence par une liste : un script `scripts/titres.py --rapport` qui relève dans `app/index.html` et `rc-core.<build>.js` les `h1/h2/h3/h4` et les `span/div` dont le style en ligne combine `text-transform:uppercase` et `letter-spacing`, ou qui utilisent `var(--pile-titre)`. Il sort

fichier:ligne:texte. Remplace ensuite, écran par écran :

- les 7 `<h1 style="font-size:var(--fs-xl);...font-weight:900">` de `app/index.html` (`s-coach-lundi`, `s-coach-relances`, `s-coach-activite`, `s-coach-prospects`, `s-coach-file...`) → `<h1 class="t-page">` ;

- `.ls-titre` / `.ls-sous` (`s-lifestyle`) → `.t-page`, avec le mot en couleur gardé dans un `<span>` ; taille ramenée de 25,7 px à 20 px ;

- « Mes compléments », « CETTE SEMAINE » (pas, sommeil), « LA SEMAINE » / « CEUX QUE TU SUIS » (amis), « MES RECETTES », « FRÉQUENCE DES BILANS », « PLAN ALIMENTAIRE », « POURQUOI CET ATHLÈTE EST ICI », « ATHLÈTES ACTIFS PAR MOIS » → `.t-section` ;

- « Ton coach » (`s-messages`), « Unité des charges » (réglages), « Je reprends la main » (relances) → `.t-carte`.

Les bandeaux rouges pleins qui font titre (« MES INFORMATIONS » de `s-athlete-profile`) passent en `.t-section` dans la carte : le rouge plein reste réservé aux bannières héros (`.banner-hero`).

3. Ne touche pas aux chiffres de tableau de bord (Bebas 26-44 px des stats), qui ne sont pas des titres.

4. Relance `python3 scripts/theme_clair.py`.

5. Tests dans `app/tests.js` : (a) les quatre classes existent dans la feuille avec les valeurs ci-dessus ; (b) aucune déclaration de `.ls-titre` ne dépasse `var(--fs-xl)` ; (c) sonde au rendu : sur `s-lifestyle`, `s-supplements`, `s-client-amis`, `s-coach-relances` et `s-steps`, tout élément `.t-section` a un `font-size` calculé de 11px et un poids de 800.

6. Captures AVANT et APRÈS obligatoires. Si `scripts/captures_ui.py` n'existe pas, crée-le : Playwright Chromium, viewport 390x844, sert le dépôt sur 127.0.0.1, BLOQUE toute requête externe (`ctx.route` qui abort tout ce qui n'est pas 127.0.0.1, `data`: ou `blob`:), pose avant le chargement un faux athlète et un

faux coach dans localStorage (rc_session, rc_users, rc_cgu_ok=1, données inspirées des fixtures _ath de app/tests.js), ouvre une liste d'écrans par leurs fonctions réelles (clientTab('home'|'bilan'|'nutrition'|'videos'|'lifestyle'|'evolution'|'canal'), loadSessionManager(), loadHistoriqueSeances(), loadCharges(), openAthleteProfile(), ouvrirReglagesAthlete(), msgOuvrirFil(), loadSupplements(), loadSteps(), loadSleep(), ouvrirRecettes(), ouvrirAmis() ; côté coach go('s-coach-home'), openClientDetail(id,false,true), ouvrirLundi(), ouvrirMessages(), openCoachPrograms(), ouvrirBanque(), ouvrirRelances(), ouvrirProspects(), loadCoachActivite(), ouvrirProtocoles(), loadCanalCoach(), loadFileReprise()) et enregistre docs/captures/<build>/<écran>.png. Lance-le avant de modifier, puis après, et produis une planche avant/après des 12 écrans touchés. Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les cibles. Critère de fin : sur les 31 écrans du banc, au plus 6 styles de titre distincts hors chiffres (T0, T1, T2, T2 action, T3, et le titre de feuille modale) ; aucun titre de section plus gros que 20 px ; suite de tests à 0 échec.

2. Bebas en faux gras et mots coupés en deux sur la fiche athlète — Priorité : Haute · Effort : S

Analyse et idée. Seule la graisse 400 de Bebas Neue est chargée (app/fonts/bebasneue-400-latin.woff2, index.html:443). Pourtant 16 titres du banc sont rendus en 700, 800 ou 900 et le navigateur les épaissit en gras synthétique, baveux : les <h1 ... font-weight:900> des écrans coach (index.html:1551, 1599, 1628), « Cette semaine » et « Les règles » en 800 (Relances), « Apparence » (Réglages), .ccd-b4 en 800 (rc-style:4337). Sur la fiche athlète, la rangée .ccd-barre4 compte 5 boutons en Bebas 10 px avec overflow-wrap:anywhere (rc-style:4331) et affiche « PRATIQUE / S » et « AUTOMATI / QUES » (capture z-c02.png). Règle : Bebas toujours en graisse 400 avec font-synthesis:none. Ces cinq commandes passent en Montserrat 10 px, graisse 800, avec coupure entre mots seulement.

Prompt de mise en place.

Dépôt RepCore. Bebas Neue n'est chargée qu'en graisse 400 (app/fonts/bebasneue-400-latin.woff2, @font-face d'app/index.html ~l.443). Toute graisse > 400 sur var(--pile-titre) produit un gras synthétique flou. Au banc 390x844, 16 textes sont touchés : les <h1 style="...font-weight:900"> d'app/index.html (~l.1551, 1599, 1628 et suivants : « Qui regarder cette semaine », « Relances automatiques », « Ton portefeuille », « Les contacts de ma page », « À relancer »), des titres en 800 de s-coach-relances et s-client-reglages, « Mes compléments » (h3 en 700), et .ccd-b4 (app/rc-style.<build>.css ~l.4335, font-weight:800).

1. Dans la feuille, ajoute : h1,h2,.topbar-title,[style*="pile-titre"],.brand-font{font-synthesis:none} et, dans la règle h1,h2,.topbar-title (~l.7576), font-weight:400!important.
 2. Retire font-weight:700/800/900 de tout style (feuille, rc-core.<build>.js, index.html) qui utilise aussi var(--pile-titre) ou qui s'applique à un h1/h2. Un script de rapport (grep) liste les cas ; corrige-les à la main.
 3. Fiche athlète, rangée .ccd-barre4 (rc-style ~l.4310-4347, markup app/index.html ~l.2495-2546) : les 5 commandes (Infos pratiques, Son mot, Relances automatiques, Son numéro, Actualiser) passent en Montserrat var(--fs-2xs), graisse 800, text-transform uppercase, letter-spacing .4px, text-shadow none. Remplace overflow-wrap:anywhere par overflow-wrap:normal; hyphens:none; word-break:normal. Si un libellé ne tient pas sur deux lignes à 375 px, raccourcis-le (« Relances auto. », « Infos ») au lieu de réduire la police. Garde min-height:46px tel quel, sans l'augmenter.
 4. Relance python3 scripts/theme_clair.py.
 5. Tests dans app/tests.js : (a) sonde au rendu sur s-coach-relances, s-coach-lundi, s-coach-activite, s-client-reglages et s-coach-client : aucun élément visible dont la police calculée commence par « Bebas » n'a un fontWeight > 400 ; (b) dans .ccd-barre4, aucun bouton ne contient un mot coupé : pour chaque bouton, le nombre de lignes rendues (Range.getClientRects sur le texte) est inférieur ou égal au nombre de mots.
 6. Captures avant/après (scripts/captures_ui.py, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux coach dans localStorage) de s-coach-client, s-coach-relances, s-coach-lundi et s-client-reglages, avec un zoom x2 sur la rangée .ccd-barre4.
- Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : 0 Bebas en faux gras sur les écrans testés, plus aucun mot coupé dans la rangée de la fiche, tests verts.

3. Les réglages écrivent leurs explications en majuscules espacées — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc sur `s-client-reglages` (capture `pa3.png`) : « UN BIP COURT QUAND LE MINUTEUR ARRIVE À ZÉRO... » s'affiche en majuscules grasses espacées de 1,5 px. La règle globale `label{text-transform:uppercase;letter-spacing:1.5px;font-weight:700}` (rc-style:1088) passe par héritage aux paragraphes enveloppés dans un `<label>` (index.html:3319-3343). Juste en dessous, « Mon coach peut partager mes progrès » est en casse normale : deux lignes de réglage voisines n'ont pas la même forme. 24 `<label style="display:flex...">` sur 31 n'annulent pas cet héritage. Règle : le style en capitales s'applique au libellé de champ (`.lbl`) et à rien d'autre. Une ligne de réglage s'écrit en `.t-carte` (14 px, graisse 800, casse normale) puis en description à 11 px, graisse 400, `var(--sub)`.

Prompt de mise en place.

Dépôt RepCore. La règle globale `label{...text-transform:uppercase;letter-spacing:1.5px;font-weight:700...}` (app/rc-style.<build>.css ~l.1088) est héritée par tout le contenu des `<label>` qui enveloppent une case à cocher et sa description. Résultat au banc (`s-client-reglages`, 390x844) : les descriptions « Un bip court quand le minuteur... » et « Le téléphone ne se met pas en veille... » s'affichent en majuscules grasses espacées, alors que la ligne voisine « Mon coach peut partager mes progrès » est en casse normale. On compte 24 `<label style="display:flex...">` sans `text-transform:none` dans `app/index.html` et `rc-core.<build>.js`.

1. Crée un composant `.reg-ligne` : carte (fond `var(--surface-1)`, bordure `1px var(--border)`, rayon `var(--r-3)`, padding `14px 16px`), case à gauche `18x18` avec `accent-color var(--red)`, puis `.reg-t` (Montserrat `var(--fs-md)`, graisse 800, casse normale, `letter-spacing 0`) et `.reg-d` (`var(--fs-xs)`, graisse 400, `color var(--sub)`, `line-height 1.5`, `text-transform none`, `letter-spacing 0`), et enfin un état `.reg-etat` (`var(--fs-2xs)`, graisse 800, `uppercase`, `letter-spacing 1px`, `var(--text-faint)`). La règle `label` globale ne change pas pour les champs de formulaire, mais `.reg-ligne` et ses descendants la neutralisent (`text-transform:none;letter-spacing:0;font-weight:inherit` sur `.reg-ligne`).
2. Remplace par `.reg-ligne` les blocs de `s-client-reglages` (app/index.html ~l.3318-3345 : son de fin de repos, écran allumé, partage des progrès, etc.), puis chacun des 24 `<label style="display:flex...">` relevés (`grep -n '<label[^>]*style="display:flex' app/index.html app/rc-core.<build>.js`). Vérifie chaque écran concerné au rendu : aucune phrase de plus de 6 mots ne doit être en majuscules.
3. Relance `python3 scripts/theme_clair.py`.
4. Test dans `app/tests.js` : sonde au rendu sur `s-client-reglages`, où tout élément visible dont le texte propre dépasse 40 caractères a un `text-transform` calculé différent de `uppercase`. Ajoute aussi un test statique : la feuille contient `.reg-ligne{...text-transform:none...}`.
5. Captures avant/après de `s-client-reglages` et des autres écrans touchés (`scripts/captures_ui.py`, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux athlète dans `localStorage`). Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices : la description reste à 11 px. Critère de fin : plus aucun paragraphe en capitales dans les réglages, toutes les lignes de réglage identiques, tests verts.

4. Un seul bouton en trois tailles : le même rouge mesure de 46 à 69 px — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : `.btn-red` est rendu à 46, 48, 49, 50, 52, 64 et 69 px de haut, en 11, 12, 14 ou 23 px de texte, parfois en majuscules espacées de 2,5 px, parfois en casse normale. Sur « Mes séances », « Démarrer » a un rayon de 4 px (`.btn-sm`, rc-style:921) à côté de « PARTIR D'UN PROGRAMME » à 10 px (`.btn`, rc-style:861). Le JS combine `btn` de 82 façons. Sont trop grands : « MODIFIER LE PROGRAMME » (14 px, 52 px de haut), « DEMANDER LA VIDÉO » (Bebas 23 px, 64 px, `.dv-btn`), les choix de la feuille « Veux-tu adapter tes séances à ton cycle ? » (environ 52 px, 14 px gras). Règle : **L** = 48 px, un seul par écran, en bas de l'action principale, 12 px, graisse 800, majuscules, interlettre 1,5 px, dégradé `.btn-red`, halo réservé à **L**. **M** = 40 px, 12 px, graisse 700, casse normale. **S** = 32 px dessinés, 11 px, graisse 700, zone tactile portée à 44 px par `.hit44` (rc-style:8221) et non par `min-height`. Rayon `var(--r-3)` pour les trois tailles.

Prompt de mise en place.

Dépôt RepCore. Objectif : une seule famille de boutons en trois tailles, avec un seul rayon. Contrainte de Kevin : ne pas gonfler. Les boutons trop grands rapetissent, et la zone tactile s'obtient par `.hit44` (`app/rc-style.<build>.css ~l.8221`, un `::before` invisible), jamais par une hauteur dessinée plus grande.

Constat au banc 390x844 : `.btn-red` rendu à 46/48/49/50/52/64/69 px ; texte 11/12/14/23 px ; `.btn` (`~l.861`) `border-radius:10px` et `.btn-sm` (`~l.921`) `var(--r-1)=4px` côte à côte sur `s-session-manager` ; 82 combinaisons de classes `btn` dans `rc-core` ; « MODIFIER LE PROGRAMME » (`s-coach-client`) en 14px/52px ; `.dv-btn` « DEMANDER LA VIDÉO » en Bebas 23px/64px ; feuille du cycle (`startWorkoutSession` → « Veux-tu adapter tes séances à ton cycle ? ») avec des choix d'environ 52 px en 14 px gras.

1. Dans la feuille, définis trois tailles et documente-les :

`.btn` (taille L par défaut) : `min-height 48px`, `padding 0 20px`, `font-size var(--fs-sm)`, `weight 800`, `uppercase`, `letter-spacing 1.5px`, `border-radius var(--r-3)`.

`.btn-m` : `min-height 40px`, `padding 0 16px`, `var(--fs-sm)`, `weight 700`, `text-transform none`, `letter-spacing .2px`.

`.btn-sm` (S) : `min-height 32px`, `padding 0 12px`, `var(--fs-xs)`, `weight 700`, `text-transform none`, `border-radius var(--r-3)`, et reçoit la zone tactile `.hit44` (`position:relative + ::before inset négatif` jusqu'à 44 px).

Couleurs : `.btn-red` (dégradé actuel, halo `--glow-red` seulement sur `.btn` sans `-m` ni `-sm`), `.btn-outline`, `.btn-blanc`. Aucun autre fond rouge pour un bouton.

2. Supprime les surcharges locales de taille : `.btn.btn-red:not(.btn-sm):not(.btn-casse)` `letter-spacing 2.5px` → `1.5px` ; `.dv-btn` passe en `.btn` `.btn-red` (Montserrat, 12px) ; le bouton « MODIFIER LE PROGRAMME » de la fiche coach perd son 14px ; les choix de la feuille du cycle passent en `.btn-m` `.btn-outline`, avec le sous-texte à 11px. `.btn-2l` (« DÉMARRER MA SÉANCE DU JOUR ») garde ses deux lignes, mais à 12px.

3. Règle d'usage, à écrire en commentaire : un seul L par écran ; les rangées de choix en M ; les actions de ligne (Relancer, Reporter, Démarrer, Chercher) en S.

4. Relance `python3 scripts/theme_clair.py`.

5. Tests dans `app/tests.js` : (a) `.btn` et `.btn-sm` ont le même `border-radius` ; (b) `.btn-sm` ne déclare aucun `min-height` supérieur à 32px et porte la zone `.hit44` ; (c) sonde au rendu sur `s-session-manager`, `s-coach-client`, `s-nutrition`, `s-sleep` et `s-coach-canal` : tous les boutons `.btn` visibles ont une hauteur de 32, 40 ou 48 px (à ±1) et un `font-size` calculé ≤ 12px ; (d) `elementFromPoint`, 5 px au-dessus d'un `.btn-sm`, renvoie bien le bouton.

6. Captures avant/après (`scripts/captures_ui.py`, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux athlète et faux coach) de `s-session-manager`, `s-coach-client`, `s-nutrition`, `s-sleep`, `s-coach-canal`, `s-client-amis` et de la feuille du cycle.

Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices ni les cibles dessinées.

Critère de fin : 3 hauteurs de bouton et 1 rayon sur les écrans du banc, aucun bouton au-dessus de 12px de texte, tests verts.

5. États vides : cinq recettes, dont une à 16 px qui écrase l'écran — Priorité : Moyenne · Effort : S

Analyse et idée. `emptyState()` (`rc-core:3204`, 30 appels) n'a pas de taille de police : son texte hérite des 16 px du `body`, avec un `padding` de 48 px (`.empty-state`, `rc-style:7461`). Testé au banc, « Aucun complément noté pour l'instant... » s'affiche ainsi plus gros que tous les titres de l'écran (`pa3.png`, Compléments). À côté, `.msg-vide` est en 12 px, `.rct-vide` en 11 px et aligné à gauche, Canal affiche l'emoji 📺 en 26 px, Programmes et Banque montrent un dossier en pointillé, et 39 classes `*-vide` existent. Règle : un seul composant, avec une icône au trait de 28 px à 40 % d'opacité, un titre facultatif en `.t-carte` (14 px), un texte de 12 px en `var(--sub)` sur 2 lignes au plus, un bouton facultatif en `.btn-sm` `.btn-outline`, un `padding` de 24 px, le tout centré.

Prompt de mise en place.

Dépôt RepCore. Les états vides n'ont pas de forme commune. `emptyState(icon,message,cta,fn,style)` (`app/rc-core.<build>.js ~l.3204`, 30 appels) n'a pas de `font-size` : son texte hérite des 16px du `body` (le `body` n'en déclare aucune), avec `padding 48px` (`.empty-state`, `app/rc-style.<build>.css ~l.7461`). Au banc, « Aucun complément noté... » (`s-supplements`, `s-nutrition`) et « Aucune vidéo envoyée... » (`s-videos`) s'affichent plus gros que les titres. D'autres variantes : `.msg-vide` 12px (`s-coach-messages`), `.rct-vide` 11px aligné à gauche (`s-recettes`), `.san-vide`, `.am-vide`, un emoji 📺 en 26px sur `s-canal` et `s-`

coach-canal, un dossier en pointillé (s-coach-programs, s-coach-banque). On compte 39 classes *-vide dans la feuille.

1. Fais d'emptyState la seule fabrique. Signature inchangée, plus un paramètre facultatif titre. Rendu : `<div class="vide">` avec icône `illusIcon` 28px opacity .4, titre facultatif (`.t-carte` si elle existe, sinon 14px/800), texte 12px `var(--sub) line-height 1.5, max-width 280px`, centré, CTA facultatif en `.btn .btn-sm .btn-outline`, padding 24px 16px. Ne grossis jamais l'icône au survol.
2. Pose `font-size:var(--fs-md)` sur body. Ensuite, fais la liste au banc des textes visibles qui passent de 16px à 14px (sonde avant/après) et vérifie qu'aucun champ de saisie ne passe sous 16px sur iOS : la règle `max(16px,1em)` des inputs doit rester en place.
3. Remplace les variantes : `.msg-vide`, `.rct-vide`, `.san-vide`, `.am-vide`, `.pr-vide`, `.pc-vide`, `.fa-vide`, `.rap-vide`, `.bb-vide` et l'état « Canal injoignable » (remplace l'emoji 📶 par `icon('wifi-off')` ou une icône au trait du jeu ICONS, à ajouter si elle manque). Garde les textes métier tels quels.
4. Un état d'erreur réseau (« Les messages n'ont pas pu être chargés », « Canal injoignable », « Banque indisponible ») utilise la même forme, avec le bouton « Réessayer » en `.btn-sm`.
5. Relance `python3 scripts/theme_clair.py`.
6. Tests dans `app/tests.js` : (a) `.vide` déclare `font-size:var(--fs-sm)` ; (b) sonde au rendu sur s-supplements, s-videos, s-recettes, s-coach-messages, s-canal, s-coach-programs : l'état vide a un texte en 12px, centré, et aucun texte de l'état vide ne dépasse 14px ; (c) aucune chaîne « 📶 » dans `rc-core`.
7. Captures avant/après de ces 6 écrans (`scripts/captures_ui.py`, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux athlète et coach sans données).
Convention : nouveau build (`window.RC_BUILD` dans `app/index.html`, `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.
Critère de fin : un seul rendu d'état vide dans toute l'app, à 12px, tests verts.

6. Barre de titre : un filet tantôt rouge, tantôt gris, et un titre qui se décale — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : le filet sous la barre est rouge sur 21 écrans et gris sur 6 (Lifestyle, Canal, Mes pas, Mon sommeil, Mes amis, Mon canal), à cause de 18 surcharges en ligne du `border-bottom` de `.topbar` dans `index.html`, dont 13 `border-bottom:1px solid var(--surface-2)`. Le titre est centré vers `x=180` partout, sauf quand une action occupe la droite : il glisse alors à `x=123` (« Mon profil »), `x=113` (« Mes programmes ») ou `x=126` (« Banque d'exercices »). « Compléments » est le seul titre en rouge, avec une icône (`index.html:4816`). Règle : filet rouge de 1 px partout, titre blanc toujours centré sur l'écran (grille 44px 1fr 44px, l'action de droite étant une icône de 32 px ou un bouton S d'au plus 96 px), pas d'icône ni de couleur dans le titre.

Prompt de mise en place.

Dépôt RepCore. La barre de titre (`.topbar`, `app/rc-style.<build>.css ~l.1159` ; `.topbar-title ~l.7685`) est bien dessinée, mais 18 surcharges en ligne de son `border-bottom` la rendent incohérente : au banc 390x844, le filet est rouge sur 21 écrans et gris sur s-lifestyle, s-canal, s-steps, s-sleep, s-client-amis et s-coach-canal (13 x `style="border-bottom:1px solid var(--surface-2)"` dans `app/index.html`, plus d'autres variantes). Le titre se décale quand un bouton est à droite (`x=123` sur s-athlete-profile, `x=113` sur s-coach-programs, contre environ 180 ailleurs), parce que `.topbar` est en `justify-content:space-between` avec des côtés de largeurs différentes. Seul s-supplements (et un second écran) a un titre rouge avec une icône (style en ligne `~l.4816`).

1. `.topbar` passe en `display:grid; grid-template-columns:44px 1fr 44px; align-items:center`. `.topbar-title` va dans la colonne du milieu, `text-align center`, `overflow hidden`, `text-overflow ellipsis`, `white-space nowrap`. L'action de droite, si elle existe, occupe la 3e colonne en débordant vers la gauche (`justify-self:end`), sans élargir la colonne : icône de 32px, ou bouton `.btn-sm` d'au plus 96px. Si une action ne tient pas, elle devient une icône (« Enregistrer » de s-athlete-profile : icône coche + `aria-label` ; « + Créer » : icône plus). Écris un commentaire qui pose cette règle.
2. Retire de `app/index.html` et de `rc-core` toutes les surcharges de `border-bottom` sur `.topbar` : un seul filet, `1px var(--red)`, y compris en thème clair (le bloc généré le fait déjà). Les écrans qui ont une raison réelle de ne pas l'avoir (plein écran séance) utilisent une classe `.topbar--nue` documentée, pas un style en ligne.
3. Retire la couleur et l'icône de `.topbar-title` sur s-supplements et sur l'autre écran qui les porte. Le titre est toujours `var(--text)`.
4. Relance `python3 scripts/theme_clair.py`.
5. Tests dans `app/tests.js` : (a) aucune occurrence de `class="topbar"` suivie d'un style contenant

border-bottom dans app/index.html ; (b) sonde au rendu sur 10 écrans (dont s-athlete-profile, s-coach-programs, s-coach-banque, s-lifestyle) : le centre du .topbar-title est à 195 ± 4 px et le border-bottom-color calculé de .topbar est le même partout.

6. Captures avant/après de ces 10 écrans (scripts/captures_ui.py, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux athlète et coach).

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni la barre (hauteur 73px inchangée).

Critère de fin : un filet, un titre centré, une couleur sur tous les écrans, tests verts.

7. Rayons et cadres de carte : l'unification du premier audit n'a pas eu lieu — Priorité : Moyenne • Effort : M

Analyse et idée. Dans rc-style.1792.css (hors bloc clair), il reste 60 valeurs de border-radius distinctes : 10px x74, 14px x41, 9px x27, 11px, 7px, 13px... Les alias --r-sm, --r-md, --r-lg et --r-card existent toujours (rc-style:97, 113). On compte aussi 56 valeurs de letter-spacing et 240 box-shadow distincts. Au rendu, les cartes suivent 48 recettes, avec 17 paddings (14, 15, 16 ou 17 px de marge intérieure selon l'écran), et les boutons 6 rayons (8, 0, 4, 12, 10, 50 %). La gouttière vaut 20 px partout sauf sur Bilan (24), Banque (14) et Protocoles (16). Règle : une carte = surface-1, bordure 1 px var(--border), rayon 12, padding 16 (dense : 12), 12 px entre deux cartes, gouttière de 20 px. Rayons limités à 4 (badge), 8 (champ), 12 (carte et bouton), 18 (feuille), plein. Interlettre limité à 0, 0,5, 1, 1,5, 2 et 2,5 px.

Prompt de mise en place.

Dépôt RepCore. Objectif : rendre mécanique la forme des cartes, des boutons et des interlettres, sans changer aucune taille de texte.

Constat (app/rc-style.<build>.css hors bloc « THEME CLAIR — GENERE ») : 60 valeurs de border-radius (10px x74, 14px x41, 9px x27...), des alias --r-sm/--r-md/--r-lg/--r-card encore déclarés (~l.97, 113), 56 valeurs de letter-spacing, 240 box-shadow distincts. Au banc 390x844 : 48 recettes de carte, 17 paddings (14/15/16/17 px), une gouttière de 20px sauf s-bilan-choice (24), s-coach-banque (14) et s-protocoles (16).

1. Crée scripts/forme.py (modèle : scripts/espacements.py) avec --rapport, --appliquer et --verifier, qui traite la feuille (hors bloc clair), rc-core.<build>.js et app/index.html :

- border-radius en px : 1-5 → var(--r-1) (4), 6-9 → var(--r-2) (8), 10-15 → var(--r-3) (12), 16-24 → var(--r-4) (18), 50%/999px → var(--r-full). Les rayons composés asymétriques sont listés dans le rapport, pas modifiés.
- alias --r-sm → --r-2, --r-md/--r-card → --r-3, --r-lg → --r-4, puis suppression de leurs déclarations.
- letter-spacing : arrondi à l'échelle {0, .5px, 1px, 1.5px, 2px, 2.5px}, sauf dans les clamp() et les em.
- ombres : toute ombre noire simple est calée sur --e1/--e2/--e3 selon son flou (≤3, ≤10, >10 px). --elev-* et --el-* sont remplacés puis supprimés.

2. Composant carte : .card = fond var(--surface-1), bordure 1px var(--border), rayon var(--r-3), padding 16px, box-shadow none ; .card--dense padding 12px ; .pile-cartes>+*{margin-top:12px}. Remplace les cartes locales des écrans les plus vus (accueil athlète, s-nutrition, s-lifestyle, s-steps, s-sleep, s-coach-client, s-coach-relances, s-coach-activité) par .card / .card--dense.

3. Gouttière : .pad à 20px partout ; corrige s-bilan-choice, s-coach-banque et s-protocoles.

4. Relance python3 scripts/theme_clair.py.

5. Tests dans app/tests.js : python3 scripts/forme.py --verifier sort en 0 (ou test JS équivalent : la feuille ne contient plus de border-radius:[0-9]+px hors exceptions listées) ; plus de déclaration --elev-, --el-, --r-sm, --r-lg ; sonde au rendu sur 8 écrans : la gouttière gauche du premier bloc vaut 20px.

6. Captures avant/après des 8 écrans (scripts/captures_ui.py, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux athlète et coach). Vérifie à l'œil qu'aucune carte ne grandit. Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les espacements : quand une valeur tombe entre deux crans, on prend le plus petit.

Critère de fin : au plus 5 rayons, 6 interlettres et 4 ombres dans la feuille, forme.py --verifier à 0, tests verts.

8. Icônes : la flèche de retour est un caractère Arial, et les emojis ont augmenté — Priorité : Moyenne · Effort : M

Analyse et idée. Le jeu d'icônes ICONS (rc-core:3083, trait de 1,75) n'est appelé que 119 fois. On compte 433 emojis dans les chaînes de rc-core (416 au build 1693, la tendance monte : ✓ ×167, ⚡ ×65, 🍌 ×29, 🍌 ×13) et 73 dans index.html. Testé au banc : le bouton de retour de toutes les barres de titre est le caractère « ← » en Arial 20 px grasse 300. L'en-tête coach utilise « ⇌ » et « ← ». Canal affiche 📶. La barre d'onglets mêle les icônes au trait au cadenas emoji 🔒 (index.html:5712). Règle : icon() partout dans l'interface, en 20 px dans la barre de titre, 16 px dans un bouton et 14 px en ligne. Les emojis restent réservés aux partages et aux notifications push.

Prompt de mise en place.

Dépôt RepCore. Le jeu d'icônes ICONS (app/rc-core.<build>.js ~l.3083 ; icon() et illusIcon()) est la référence : viewBox 24, stroke 1.75, linecap square. Il n'est appelé que 119 fois. Les chaînes de rc-core contiennent 433 emojis (416 au build 1693) et app/index.html 73. Au banc 390x844 : le .back-btn des barres de titre est le caractère « ← » rendu en Arial 20px/300 (tous les écrans à barre de titre), l'en-tête coach utilise « ⇌ » et « ← » en texte, s-canal et s-coach-canal affichent 📶, et l'onglet Canal porte 🔒 (index.html ~l.5712).

1. Ajoute à ICONS, au même trait, les icônes manquantes : retour (chevron gauche), echange, cadenas, wifi-off, coche, croix, éclair, muscle, regle, flamme, cafe, the, gelule, bouclier, avertissement.
2. .back-btn : remplace le caractère « ← » par icon('retour',20), en gardant la zone tactile de 44x44 existante, sans agrandir le dessin. Fais de même pour les flèches et ⇌ de l'en-tête coach, et pour 🔒 dans la barre d'onglets (icon('cadenas',12)).
3. Écris scripts/emojis.py --rapport (fichier:ligne:contexte, commentaires exclus), puis remplace les emojis affichés dans l'interface par icon() : ✓→coche, ×→croix, ⚡→éclair, 🍌→muscle, 🍌→regle, 🔥→flamme, ☕→cafe, 🍷→the, 🍌→gelule, 🛡️→bouclier, ⚠️→avertissement, 📶→wifi-off. Tailles : 14px en ligne dans le texte, 16px dans un bouton, 20px dans la barre de titre. NE TOUCHE PAS aux textes de notification push ni aux textes de partage (Instagram, WhatsApp), où l'emoji est voulu : liste-les dans une liste blanche.
4. Tests dans app/tests.js : (a) l'ensemble des emojis présents dans les chaînes de rc-core (hors commentaires) est inclus dans la liste blanche ; (b) aucun .back-btn ne contient de texte, seulement un <svg>.
5. Captures avant/après (scripts/captures_ui.py, à créer s'il manque : Playwright 390x844, réseau externe bloqué) de s-nutrition, s-canal, s-coach-home, s-coach-client et de la barre d'onglets athlète, avec un zoom x2 sur la barre de titre.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les icônes ni les polices.

Critère de fin : 0 emoji hors liste blanche, retour en icône sur toutes les barres, tests verts.

9. Badges et bandeaux de niveau : dix recettes, et trois bandes pleines sur l'accueil coach — Priorité : Basse · Effort : S

Analyse et idée. La feuille compte 47 classes de badge, pastille, puce ou étiquette. Au banc, elles donnent 10 rendus différents : de 9,5 à 11 px, rayons de 0, 4, 8 et 12 px, interlettre de 0,4 à 2,5 px (« INACTIF », « PERFORMANCE », « COACH », « ACTIF »...). Sur l'accueil coach, « URGENT », « IMPORTANT » et « À TRAITER » sont des bandes pleines rouge, orange et vert (rc-core:28703), les surfaces les plus saturées de toute l'app, au-dessus d'un contenu parfois vide (« Rien ici »). Règle : un badge = 10 px, grasse 800, majuscules, interlettre 1 px, hauteur 18 px, padding 0 6 px, rayon 4, texte coloré sur un fond de la même teinte à 14 %. Les bandes de niveau gardent les couleurs choisies par Kevin, mais en filet gauche de 3 px avec un libellé coloré sur fond sombre, sur le modèle déjà adopté par la fiche (.cc-sect, « le filet porte le niveau »).

Prompt de mise en place.

Dépôt RepCore. On compte 47 classes de badge, pastille, puce ou étiquette dans app/rc-style.<build>.css (.badge, .badge-red/green/orange/gray, .tun-badge, .cr-badge, .sk-badge, .rc-badge-*, .supp-tag, .rir-tag, .tbk-pill, .wo-vid-badge...), pour 10 rendus différents au banc 390x844 (9,5 à 11

px, rayons 0/4/8/12, interlettre .4 à 2.5px). Sur s-coach-home, les groupes de « Mes notifications » (rc-core ~l.28703, tableau BANDES : Urgent var(--red), Important var(--orange), À traiter var(--green)) sont des bandes pleines saturées.

1. Crée .pastille (10px var(--fs-2xs), weight 800, uppercase, letter-spacing 1px, height 18px, padding 0 6px, border-radius var(--r-1), display inline-flex, align-items center) et quatre tons : .pastille--rouge/--vert/--orange/--neutre = color de la teinte, background color-mix(in srgb, <teinte> 14%, transparent). Plus .compteur (cercle de 18px, 10px, weight 800) pour les nombres.
 2. Fais de .badge et des badges locaux des alias de .pastille (ou remplace leurs usages), sans agrandir aucun texte. Les tags « PERFORMANCE » de s-protocoles, « INACTIF » de la fiche, « ACTIF » de s-session-manager et « COACH » de l'accueil athlète passent en .pastille.
 3. Accueil coach : chaque groupe de BANDES devient un en-tête sombre (fond var(--surface-1)) avec un filet gauche de 3px de sa couleur, le libellé en .t-section dans sa couleur et le compteur en .compteur. Les couleurs voulues par Kevin (vert pour À traiter) ne changent pas. Un groupe vide (« Rien ici ») se replie sur son seul en-tête.
 4. Relance python3 scripts/theme_clair.py.
 5. Tests dans app/tests.js : sonde au rendu sur s-coach-home, s-protocoles, s-coach-client et s-session-manager, où tout badge visible fait 18 ± 1 px de haut et 10px de texte ; aucun en-tête de groupe de notifications n'a un background-color plein rouge, orange ou vert.
 6. Captures avant/après de ces 4 écrans (scripts/captures_ui.py, à créer s'il manque : Playwright 390x844, réseau externe bloqué, faux coach avec au moins une alerte de chaque niveau ; inspire-toi des fixtures de app/tests.js).
- Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
- Critère de fin : un seul rendu de badge, des niveaux lisibles sans bandes pleines, tests verts.

10. La pastille « 1 envoi en attente » recouvre le contenu — Priorité : Basse · Effort : S

Analyse et idée. Testé au banc, hors ligne : la pastille de synchronisation flotte en bas à gauche sur tous les écrans. Elle recouvre des titres de section (« ...ATIONS MÉDICALES » sur Mon profil, « ...ER / MODIFIER MES DONNÉES » sur Lifestyle) et chevauche la rangée « Suivant » de la séance. Un état système ne doit pas masquer le contenu, surtout pas une action de séance. Règle : la pastille va dans la barre de titre, en point de 8 px plus un libellé de 10 px à gauche du titre, ou dans une bande de 24 px au-dessus de la barre d'onglets qui réserve sa place (padding-bottom du .scroll-area), jamais en position flottante sur le contenu.

Prompt de mise en place.

Dépôt RepCore. La pastille d'attente de synchronisation (« 1 envoi en attente », ajoutée au build 1747 avec la file rc_sync_queue ; retrouve-la avec grep -n "envoi en attente" app/rc-core.<build>.js et sa classe dans app/rc-style.<build>.css) est en position fixe en bas à gauche. Au banc 390x844, hors ligne, elle recouvre « Informations médicales » (s-athlete-profile), « Ajouter / modifier mes données » (s-lifestyle) et la rangée « Suivant » de s-workout.

1. Déplace-la dans une bande d'état de 24px collée au-dessus de la barre d'onglets (ou au-dessus de la barre d'actions sur s-workout) : fond var(--surface-1), texte var(--fs-2xs) var(--sub), point de 6px orange. Quand la bande est visible, le .scroll-area de l'écran actif reçoit un padding-bottom supplémentaire de 24px, pour que rien ne passe dessous. Garde le même geste au toucher (forcer l'envoi) et la même logique d'affichage.
 2. Sur les écrans sans barre d'onglets, la bande se place sous la barre de titre, avec la même réserve de place.
 3. Tests dans app/tests.js : avec une file non vide (fixture rc_sync_queue), sur s-athlete-profile, s-lifestyle et s-workout, le rectangle de la pastille ne recouvre aucun élément interactif ni aucun titre visible (intersection de getBoundingClientRect = 0).
 4. Captures avant/après de ces 3 écrans, en ligne et hors ligne (scripts/captures_ui.py, à créer s'il manque : Playwright 390x844, réseau externe bloqué, ce qui remplit naturellement la file).
- Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
- Critère de fin : la pastille ne masque plus rien, tests verts.

Le socle tient bien : la suite intégrée passe à **5 951 tests / 0 échec**, `cloudflare && npm test` passe à **139 / 139**, et le robot (athlète puis coach, 92 écrans, environ 450 clics, 7 valeurs absurdes par formulaire) n'a relevé **aucune erreur JS** après le démarrage. La saisie en séance est exemplaire (négatif, 9999, texte et emoji sont tous refusés avec un message clair). Ce qui manque le plus : des **bornes de saisie communes**. Le bilan, le profil, les macros du coach et le nombre de séries acceptent -5, 9999 ou « 72,5 » lu 725, et ces valeurs partent dans le dossier puis chez le coach.

Méthode : robot Playwright à 360 px, réseau externe bloqué. Il ouvre chaque `id="s-..."` par `go()`, puis visite l'accueil et les 7 onglets en cliquant un bouton sur deux à chaque niveau. Il remplit chaque formulaire avec 72, vide, -5, 9999, abc, 12,5 et 🍌🔥. Il relève les `pageerror` et `console.error`, les NaN / undefined / Infinity (à l'écran et dans `currentUser`), les débordements et les boutons sans effet. Les scripts sont dans `audit2/12/` (`robot.py`, `lib.py`, `s_*.py`).

1. Bilan : poids et mensurations absurdes enregistrés, « 72,5 » devient 725 kg — Priorité : Critique · Effort : M

Analyse et idée. Testé au banc : dans un bilan coaching, un poids de 9999, -5 ou 72,5 est enregistré sans un mot. Pour 72,5, le champ `type=number` perd la virgule (Chromium fr-FR) : on obtient `bil-weight:"725"`, `currentUser.weight="725"`, et la ligne « Poids (kg) -5 » s'affiche dans Évolution. Les champs viennent de `bQ / bMeas` (`rc-core.1792.js:75846`), et l'envoi (`bilNext`, ~75315) ne vérifie que « bilan vide ». Idée : un lecteur numérique commun (virgule acceptée, bornes par champ) et un refus en ligne, à l'étape même, avant « Suivant ».

Recoupement. Même constat que l'expert 7 (point 4), qui fait référence pour le bilan : appliquer son prompt (lireDecimal + bornes), puis étendre les bornes au profil et aux macros ici.

Prompt de mise en place.

```
Dans app/rc-core.1792.js, le bilan (openBilan / bilNext / saveBilanFinal, champs créés par bQ et bMeas vers la ligne 75846) accepte n'importe quel nombre : poids 9999, -5, et « 72,5 » saisi dans un input type=number devient « 725 » (la virgule est perdue). Fais ceci :
1) Crée une fonction PURE lireNombreFr(txt,{min,max,decimales}) qui accepte « 72,5 », « 72.5 » et « 72,5 kg ». Elle rend {ok,valeur,raison} et refuse le vide, le texte, les emoji, les valeurs négatives et celles hors bornes.
2) Passe les champs numériques du bilan (bil-weight, deb-weight, mensurations bil-*) en type="text" inputmode="decimal", comme les charges de séance, pour que la virgule ne soit plus perdue.
3) Bornes : poids 25–300 kg, tours 10–250 cm, bras et mollets 10–80 cm. Valeur hors bornes : le champ passe en orange, un message court s'affiche sous lui (« 72,5 kg ? Vérifie la virgule »), et « Suivant » reste bloqué sur l'étape. Écart de plus de 10 % avec la dernière pesée : demander une confirmation (rcConfirm), sans bloquer.
4) saveBilanFinal relit les valeurs avec lireNombreFr : aucune valeur invalide n'est écrite, même depuis un brouillon (_bilSaveDraft) déjà corrompu.
Erreurs humaines à couvrir : virgule, point, « kg » tapé, champ vidé par erreur (il ne doit pas effacer une valeur reprise), retour à l'étape précédente.
Tests dans app/tests.js : lireNombreFr sur ['72,5','72.5','725','-5','9999','abc','🍌','']; un bilan à 9999 ou -5 n'est pas enregistré ; « 72,5 » est enregistré à 72.5.
Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices. Fin : suite.mjs à 0 échec.
```

2. Erreur au démarrage « Cannot read properties of undefined (reading 'update') » — Priorité : Haute · Effort : S

Analyse et idée. Cause, testé au banc : `app/index.html:413-414`. Le code fait `navigator.serviceWorker.register('./sw.js').then(reg=>{ reg.update(); ... })`, sans garde et sans `.catch`. Quand `register` se résout sans enregistrement, `reg.update()` lève l'erreur. C'est le cas sous

Playwright (`service_workers:'block'`), dans les webviews ou chez les navigateurs qui neutralisent les workers. Si `register` est rejeté (navigation privée Firefox, `sw.js` en 404, quota), c'est une promesse rejetée sans gestionnaire. Dans les deux cas, l'écouteur `controllerchange` placé dans le `.then` n'est **jamais posé** : la mise à jour automatique ne se fait plus. L'erreur pollue aussi tous les bancs, et l'app n'a aucun `window.onerror` / `unhandledrejection` pour remonter les vraies.

Prompt de mise en place.

```
Dans app/index.html (vers les lignes 405-420), l'enregistrement du service worker fait
navigator.serviceWorker.register('./sw.js').then(reg=>{ reg.update();
navigator.serviceWorker.addEventListener('controllerchange',...) }) sans garde ni catch. Quand register
rend undefined (navigateur piloté, webview) on obtient « Cannot read properties of undefined (reading
'update') », et l'écouteur controllerchange n'est jamais posé.
Corrige :
- Pose l'écouteur 'controllerchange' AVANT et HORS du then (il ne dépend pas de reg).
- Dans le then : if(reg && typeof reg.update==='function') reg.update().catch(()=>{}).
- Ajoute .catch(e=>{ window._swEchec=String(e&&e.message||e); }) pour qu'un rejet ne fasse pas de
promesse orpheline.
- Ajoute un petit filet global : window.addEventListener('error') et 'unhandledrejection' gardent les
20 dernières erreurs dans un tableau mémoire window._rcErreurs (sans réseau, sans toast). Les bancs et
scripts/verif peuvent ainsi les lire.
Test dans app/tests.js : extrais le bloc dans une fonction rcEnregistrerSW(sw) testable avec un faux
sw dont register rend Promise.resolve(undefined), puis Promise.reject(new Error('x')). Aucune
exception, et controllerchange est bien posé.
Convention : nouveau build (window.RC_BUILD + CACHE de app/sw.js + python3
scripts/versionner_actifs.py). Fin : python3 audit2/banc.py --role coach ne montre plus l'erreur, et
suite.mjs est à 0 échec.
```

3. Macros du coach : -5 g ou 9999 g de protéines « enregistrés et transmis » — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : dans la fiche athlète, onglet NUTRITION, mode manuel, le coach tape -5 puis 9999 dans `#ccd-on-p` et appuie sur « Enregistrer et transmettre » (`enregistrerEtTransmettre`, `rc-core.1792.js:103870`, puis `saveClientNutriMacros`). Le dossier reçoit `p:-5`, puis `p:9999` avec des kcal inchangées à 2100. L'athlète voit « / 9999 g PROTÉINES » ou « 0 kcal ». La lecture (`_plSaisieCourante`, ~116818) n'est qu'un `parseFloat`. Idée : bornes par macro, cohérence kcal $\approx 4P+4G+9L$, et blocage en ligne avant l'envoi.

Prompt de mise en place.

```
Dans app/rc-core.1792.js, saveClientNutriMacros et _plSaisieCourante (~116818) lisent ccd-on-*/ccd-
off-* par parseFloat sans bornes : un coach peut transmettre p=-5 ou p=9999, ou kcal=0. Ajoute une
validation PURE validerMacrosSaisie({on,off}). Bornes : kcal 800-8000, P/G/L 0-600 g, fibres 0-100 g.
Refuse aussi les nombres négatifs et non finis. Si |kcal - (4P+4G+9L)| dépasse 15 %, ce n'est pas un
refus mais un avertissement (« Les kcal ne collent pas aux macros : 2100 saisies pour 40 600 calculées
»), avec un bouton « Corriger les kcal » qui recalcule. Une valeur invalide : champ en orange, message
sous le champ, rien n'est écrit ni transmis, et le toast « Enregistré ici... » ne s'affiche pas. Passe
les champs en inputmode="decimal" type="text" et lis-les avec la fonction commune de lecture des
nombres français (virgule acceptée).
Erreurs humaines : coach qui tape 1300 au lieu de 130, champ vidé, virgule.
Tests (app/tests.js) : -5, 9999, '' et 'abc' sont refusés ; 130/230/65 avec kcal 2005 est accepté ;
kcal incohérentes déclenchent l'avertissement sans bloquer.
Convention : nouveau build + versionner_actifs.py, sans agrandir les polices.
```

4. Profil athlète : poids -5 ou 9999 kg, prénom vidé, et le seuil de caféine suit — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : `saveAthleteProfile` (`rc-core.1792.js:126879`) fait `profileWeight=parseFloat(v)||undefined`, sans borne. Avec -5, 9999 ou « 12,5 » (lu 125), on obtient le

toast « Profil enregistré ✓ ». La fiche coach affiche alors « Poids 9999 kg » (33250). `_poidsCafeine` (109262) prend `u.weight` || `profileWeight` : avec 9999, le seuil de caféine explose. `u.weight` est aussi écrit sans contrôle par le bilan (78985). Vider le prénom affiche quand même « enregistré ✓ » alors que rien n'est changé.

Prompt de mise en place.

```
app/rc-core.1792.js : saveAthleteProfile (~126855-126932) écrit
profileWeight=parseFloat(...)||undefined sans borne. Le bilan écrit currentUser.weight=bilData['bil-
weight'] (~78985) sans contrôle. _poidsCafeine (~109262) et la fiche coach (~33250) utilisent ces
valeurs telles quelles.
1) Lis le poids du profil avec la lecture commune des nombres français, bornée à 30–300 kg. Hors
bornes : message sous le champ, et RIEN n'est enregistré (pas de toast « Profil enregistré »).
2) Prénom vidé : message « Ton prénom ne peut pas être vide », l'ancien prénom reste dans le champ.
3) Fais passer _poidsCafeine et l'affichage de la fiche coach par un seul helper poidsFiable(u) qui
ignore toute valeur hors 30–300 (repli sur la dernière pesée valide, sinon estimation signalée comme
telle).
Tests : profileWeight 9999, -5 et 'abc' sont refusés ; _poidsCafeine({weight:'9999'}) rend une
estimation, pas 9999 ; un prénom vide n'écrase pas l'ancien.
Convention build + versionner_actifs.py, sans agrandir les polices.
```

5. Les champs `type=number` perdent la virgule : 62,5 devient 625 — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc (Chromium, locale fr-FR) : « 62,5 » devient « 625 » dans la pesée de l'accueil (`#pesee-input`, `savePesee` 118842). Le message affiché est alors « Saisis un poids entre 25 et 300 kg » alors que 62,5 est dans les bornes. Même chose pour l'aliment (`#fja-qty` : 125 g ajoutés sans rien dire), le complément (`#supp-qty`), le bilan et le profil. Il y a 15 `type="number"` dans `index.html` et environ 67 dans `rc-core`. La séance a déjà la bonne solution : `type=text inputmode=decimal` et `_woChargeSaisie`. Idée : généraliser cette solution aux champs à décimales.

Recoupement. `lireDecimal`, créé par l'expert 7 (point 4) pour le bilan, sert ici à la pesée, aux aliments et aux compléments : une seule fonction pour tous les champs décimaux.

Prompt de mise en place.

```
Recense dans app/index.html et app/rc-core.1792.js les <input type="number"> qui acceptent des
décimales : poids, mensurations, quantités d'aliment, dosages, macros, km. Remplace-les par
type="text" inputmode="decimal" autocomplete="off", et lis leur valeur avec UNE fonction commune
lireNombreFr (virgule ou point, unité tolérée, bornes, raison du refus). Prends modèle sur
_woChargeSaisie (saisie des charges en séance). Garde type=number pour les entiers purs (répétitions,
séries). Dans savePesee : si la saisie brute contient une virgule, plus de message trompeur « entre 25
et 300 kg ». Pour l'aliment, demande une confirmation au-delà de 2000 g (« 9999 g, c'est bien ça ? »).
Tests app/tests.js : savePesee et la quantité d'aliment avec « 62,5 » donnent 62.5 ; un test
structurel vérifie qu'aucun champ décimal listé n'est encore en type=number.
Convention build + versionner_actifs.py, sans agrandir les polices.
```

6. Pas : « 12.500 » enregistré comme 13 pas, km sans borne et impossibles à effacer — Priorité : Moyenne · Effort : S

Analyse et idée. `sanLirePas` (`rc-core.1792.js:120571`) remplace la virgule par un point puis arrondit. « 12.500 » (séparateur de milliers français) ou « 12,5 » donnent donc **13 pas**, avec le toast « Enregistré ». Le champ km (121079) n'a pas de borne : testé au banc, 9999 km est accepté. Un km vidé n'efface jamais l'ancien (`if(km>0) e.km=km`).

Prompt de mise en place.

```
app/rc-core.1792.js, sanLirePas (~120571) et l'enregistrement des pas dans sanSaisir (~121075-121095) :
- « 12.500 », « 12 500 » et « 12,500 » (groupes de 3 chiffres) se lisent 12500. « 12,5 » ou « 12.5 » (une seule décimale, moins de 100) : confirmation « 12 500 pas ? » avec deux boutons (12 500 / 13), jamais d'enregistrement silencieux de moins de 100 pas.
- km : bornes 0,1-100, sinon message sous le champ. Un champ km vidé SUPPRIME e.km.
Tests : sanLirePas('12.500')===12500, sanLirePas('12 500')===12500, sanLirePas('8k')===8000 (déjà géré) ; km 9999 est refusé ; km vidé supprime la valeur.
Convention build + versionner_actifs.py.
```

7. Changer l'unité des macros (g / % / g/kg) affiche « Élève introuvable ou non autorisé » à l'athlète — Priorité : Moyenne · Effort : S

Analyse et idée. Reproduction (testé au banc) : en athlète, sur la carte NUTRITION de l'accueil, toucher « g », « % » ou « g/kg ». Un toast orange « Élève introuvable ou non autorisé » apparaît, une fois par appui. Cause : `setMacroUnite` (rc-core.1792.js:92598) appelle `getOwnedClient(currentClientId)` sans vérifier le rôle. Pour un athlète, `currentClientId` est vide, et `getOwnedClient` (32464) émet le toast.

Prompt de mise en place.

```
app/rc-core.1792.js ligne ~92598 (setMacroUnite) : remplace
  try{ const c=getOwnedClient(currentClientId); if(c) renderCoachNutriSection(c); }catch(e){}
par un appel réservé au coach qui a une fiche ouverte :
  if(currentUser&&currentUser.role==='coach'&&currentClientId){ ... }
Cherche les autres appels à getOwnedClient( qui peuvent tourner côté athlète (rendus partagés :
_renderStrictDiet, renderNutriAnneaux, etc.) et ajoute la même garde. Mieux : ajoute à getOwnedClient
une option {silencieux:true} et utilise-la dans les rendus.
Test app/tests.js : en athlète, setMacroUnite('pct') n'émet aucun toast (espionne toast).
Convention build + versionner_actifs.py.
```

8. Accueil athlète : « 4xundefined », et répétitions injectées sans échappement — Priorité : Moyenne · Effort : S

Analyse et idée. rc-core.1792.js:51118 écrit `${ex.series}x${ex.reps}` brut dans le résumé de la séance du jour. Testé au banc : un exercice sans `reps`, forme courante des données (fixtures de tests.js `{name, series:4}`), s'affiche « 4xundefined ». Une valeur de `reps` contenant du HTML s'exécute aussi : 12 exécutions d'un `onerror` injecté par `reps`. Même défaut à 43424 (`c.seriesxc.reps`) et 123555 (`value="${ex.reps}"`, import par photo). La ligne 45317 fait déjà bien : `String(e.series||'')`.

Prompt de mise en place.

```
app/rc-core.1792.js : aux lignes ~51118, ~43424 et ~123551-123555, les valeurs ex.series / ex.reps /
c.reps sont interpolées sans échappement ni repli. Crée un helper PURE libSeriesReps(ex) qui rend '4 x
8-10', '4 séries' si reps est vide, et '' si les deux le sont, toujours passé dans escapeHtml.
Utilise-le partout où l'on affiche séries x reps (grep 'x${', 'x', 'séries x'). Pour les attributs
value="", utilise escapeHtml.
Tests : libSeriesReps({series:4}) ne contient pas 'undefined' ; avec reps '<img src=x onerror=...>', le
HTML produit ne contient pas de balise <img>.
Convention build + versionner_actifs.py.
```

9. Programme : 9999 séries acceptées, chaque série cochée fige l'écran près de 7 s — Priorité : Moyenne · Effort : S

Analyse et idée. L'éditeur (rc-core.1792.js:43124) fait `Math.max(0,+this.value)` : pas de maximum, et 0 est accepté. « abc » donne 0, et « 12,5 » donne 125. Testé au banc : avec `series:9999`, la séance affiche « 0/10006 séries » et rend 9 999 lignes. `toggleSet` prend 6,8 s sur PC, bien plus sur téléphone.

Côté athlète, le message des répétitions (52286) dit « 1 ou plus » même quand 9999 est refusé parce que trop grand.

Prompt de mise en place.

```
app/rc-core.1792.js ~43124 (éditeur d'exercices, champ Séries) : borne entre 1 et 20. Hors bornes : message sous le champ, ancienne valeur rétablie. Applique la même borne à la lecture (startWorkoutSession / construction de woState) pour neutraliser les données déjà écrites : on joue au plus 20 séries, et le coach est prévenu sur sa fiche. Ligne ~52286 (_woRepsSaisie) : message selon le cas, « entre 1 et 999 ».  
Tests : saisir 9999 ou 0 dans Séries ne modifie pas progEx ; une séance avec series 9999 rend au plus 20 lignes ; 'abc' ne met pas 0.  
Convention build + versionner_actifs.py.
```

10. 360 px : libellés de la barre d'onglets collés, mots coupés dans la fiche coach — Priorité : Basse · Effort : S

Analyse et idée. Mesuré au banc à 360 px : les 7 libellés de `#client-tabbar` occupent exactement la largeur, avec 0 px entre eux (« ACCUEILBILANNUTRITIONCORRECTIONS... », capture `tab_360.png`). Le commentaire de `rc-style.1792.css:1225` promet « 5 px de marge ». À 320 px, Lifestyle, Évolution et Canal sont hors champ. Dans la fiche coach, `.ccd-barre4` (`css:4310`), prévue pour 4 boutons, en porte 5 : « RELANC/ES AUTOMA/TIQUES » et « PRATIQUE/ES » sont coupés en plein mot. Idée : ne pas réduire les polices mais raccourcir les libellés sous 420 px (« CORRECTIONS » devient « VIDÉOS », « Relances automatiques » devient « Relances »), avec un `gap` minimal.

Prompt de mise en place.

```
Sans changer aucune taille de police : 1) app/rc-style.1792.css ~1234 : sous 420px, ajoute gap:2px à .tab-bar et affiche un libellé court via data-court sur les onglets longs (CORRECTIONS → « VIDÉOS » ou « CORRIG. »), afin qu'à 360 px il reste au moins 3 px entre deux libellés. 2) Fiche coach (index.html ~2496-2525, css .ccd-barre4 ~4310-4330) : libellés « Relances » et « Infos », hyphens:none et overflow-wrap:normal sur .cc-sect-t>span, et passage à deux rangées (flex-basis:calc(33% - 6px)) sous 400px, pour qu'aucun mot ne soit coupé.  
Ajoute un test dans app/tests.js (ou scripts/verif) qui, à 360 px, vérifie que les rectangles de texte des onglets ne se touchent pas.  
Convention build + versionner_actifs.py.
```

Notes pour les autres agents (banc). Le dossier fictif de `banc.py` n'utilise pas le schéma de l'app. Les exercices ont `sets/rest` au lieu de `series/repos`, ce qui produit de faux « undefined×8-10 ». Les séances ont `date` en chaîne ISO et `data` en tableau au lieu d'un horodatage et d'un objet, ce qui donne une fausse fiche coach « Jamais démarré / INACTIF » malgré 24 séances. Un dossier corrigé est disponible : `audit2/12/persona_athlete.json`.

Hors bug, vérifié propre. Saisie en séance (`_woChargeSaisie`, `_woRepsSaisie`), saisie de sommeil (confirmation au-delà des bornes), objectif de pas, quantité d'aliment (1-9999 g), complément (nom et moment requis), modale « Quitter la séance » à trois issues. Aucun NaN ni Infinity trouvé dans `currentUser` après toutes ces saisies.

Expert 13 · Effet waouh — 7/10

La matière « waouh » est déjà là et souvent superbe : badges illustrés, record avec halo et « 58 → 62,5 kg », grimpeur « Ton prochain objectif », Wrapped mensuel (testé au banc : « Ton mois de septembre est prêt · 13 séances »), avant/après composé en un geste, synthèse « Sur 8 semaines : ... » dans Évolution. Ce qui manque le plus, c'est la mise en scène au bon moment : les écrans clés restent génériques (« Séance terminée », « Termine une première séance. »), le bilan ne montre jamais les

photos, le coach ne voit que les problèmes, et la fin de bloc comme les anniversaires ne déclenchent rien chez l'élève.

1. Ne plus recouvrir le record au moment où il s'affiche — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc (record squat 58 → 62,5 kg, puis premier record 40 → 45 kg) : 2,2 s après l'arrivée sur l'écran de fin, la bannière « Installe RepCore sur ton téléphone » (`setTimeout(rcBanniereInstallMontrer,2200)` à la fin de `finishWorkout`, `rc-core.1792.js:~54185`) se pose sur « Mes records », et la puce « 1 envoi en attente » chevauche le texte du record. Le seul moment où l'app « se félicite » se fait couper. Idée : la bannière attend que l'élève ait fait défiler l'écran jusqu'aux volts ou qu'il quitte l'écran de fin. La puce d'envoi se cache tant que la célébration est visible.

Recoupement. La pastille « envoi en attente » qui recouvre le contenu est traitée partout par l'expert 11 (point 10) ; ici, seul le report de la bannière d'installation.

Prompt de mise en place.

Contexte : RepCore, app de musculation. Dans `app/rc-core.1792.js`, `finishWorkout()` (vers la ligne 53790) programme à la fin `setTimeout(()=>rcBanniereInstallMontrer(),2200)` (fonction vers la ligne 124692). Testé : la bannière `#rc-ban-install` recouvre le bloc « Mes records » (`#wd-records`) de l'écran `s-workout-done` au moment où le record s'anime. La puce « 1 envoi(s) en attente » se superpose aussi au texte du record.

À faire :

1. Remplacer ce `setTimeout` par une fonction `_installApresFete()` qui montre la bannière seulement quand UNE de ces conditions est vraie : (a) l'élève a fait défiler `s-workout-done` jusqu'à ce que `#wd-volts` soit entièrement visible (`IntersectionObserver`, seuil 1), ou (b) il quitte `s-workout-done` (premier `go()` vers un autre écran, la bannière s'affiche alors sur l'écran d'arrivée). Jamais pendant qu'un calque de célébration (`#bdg-ecran`) est ouvert : dans ce cas, attendre sa fermeture.
Garder la règle existante : rien si la séance est incomplète (`incomplete===true`), et `rcBanniereInstallRaison()!='ok'` continue de tout bloquer.
2. La puce d'envoi en attente : la masquer (classe CSS, pas `display inline`) tant que `s-workout-done` est actif et que l'élève n'a pas fait défiler l'écran. Elle doit réapparaître ailleurs sans rien perdre (l'envoi continue en fond).
3. Erreurs humaines : double tap sur « Enregistrer et terminer », retour arrière depuis `s-workout-done`, animations réduites (`arcReduit()` : la bannière peut s'afficher dès le premier défilement), rechargement de la page (aucune bannière fantôme).
Tests dans `app/tests.js` : (a) après `finishWorkout()`, au bout de 2,5 s et sans défilement, `#rc-ban-install` n'a pas la classe `show` ; (b) après défilement jusqu'à `#wd-volts`, elle l'a (si `rcBanniereInstallRaison` est forcée à 'ok') ; (c) `go('s-client-home')` depuis l'écran de fin la montre une seule fois ; (d) aucun chevauchement : le rect de la puce n'intersecte pas `#wd-records`.
Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` de `app/sw.js`), puis `python3 scripts/versionner_actifs.py`. Ne pas agrandir les polices ni les cibles.
Critère de fin : sur une fin de séance avec record, rien ne recouvre « Mes records » tant que l'élève n'a pas fait défiler l'écran. Tous les tests passent.

2. Un titre de fin de séance qui raconte la séance — Priorité : Haute · Effort : S

Analyse et idée. `_htmlHeroFin` (rc-core.1792.js:65193) écrit toujours « Séance terminée », que ce soit la 1re séance, la 50e, un record ou la 7e semaine d'affilée. Le prénom de l'élève n'apparaît nulle part sur l'écran. Idée : garder le titre, mais ajouter dessous UNE phrase tirée de `_ctxFin` (déjà calculé), choisie par priorité, par exemple « Premier record, Léa : squat +5 kg en 2 jours. » ou « 50e séance. Tu as soulevé 120 t depuis juin. ». Pour cela, aucun bloc n'est ajouté.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js, écran de fin de séance s-workout-done. finishWorkout() (vers 53790)
assemble _ctxFin
{records, recordsElrm, objectif, streak, volume, volumesPrecedents, joursDepuisDerniere,
exosAmeliores, ...} puis appelle
_htmlHeroFin(_badges,_ctxFin) (vers 65193), qui rend toujours <h1 class="rcf-titre">Séance
terminée</h1>.

À faire :
1. Ajouter une fonction PURE phraseHeroFin(u,ctx,sess) qui rend UNE phrase (≤ 70 caractères) ou ''
selon cette priorité :
  a. 1re séance (u.sessions.length===1) : « Première séance, <Prénom>. Ton point de départ est posé.
  »
  b. objectif « record à portée » atteint (ctx.objectif et record battu sur cet exercice) : «
Objectif atteint, <Prénom> : <exo> <kg> kg. »
  c. premier record de l'histoire de l'élève : « Ton premier record : <exo> <avant> → <après> kg. »
  d. record : « <exo> : +<gain> kg » + (si l'ancien record a plus de 14 jours) « , ton meilleur
depuis le <date> ».
  e. jalon de séances (10, 25, 50, 100, 150, 200...) : « <N>e séance. <tonnage cumulé> soulevées depuis
<mois d'inscription>. »
    Utiliser tonnageTotalDe(u) et equivalentTonnage() si cela vaut un équivalent parlant.
  f. retour après pause (ctx.joursDepuisDerniere >= 10) : laisser la main à retourAuCombat, renvoyer
  ''.
  g. sinon : delta de volume vs la moyenne des volumesPrecedents s'il dépasse +5 % (« +8 % de volume
sur ta moyenne »),
    sinon ''.
    Le prénom vient de u.fname (nettoyé, sans majuscules forcées). Pas de prénom : la phrase sans
prénom.
2. _htmlHeroFin l'affiche dans un <p class="rcf-sous"> sous le trait, avec la taille de police
existante des sous-titres
  (var(--fs-sm) ou le jeton déjà utilisé par .rcf-nom). Ne pas créer de nouvelle taille.
3. Une phrase vide : aucun nœud, aucun espace réservé. Aucune exception ne doit casser le héros
(try/catch déjà en place).
4. Décharge (sess.deload) : pas de phrase « record » ni « volume ». Séance incomplète : seulement a, e
ou ''.
Tests dans app/tests.js, sur phraseHeroFin (pure) : chaque cas a à g, la priorité (1re séance + record
→ cas a), décharge,
prénom absent, exercice au nom très long (coupé proprement), aucune phrase qui dépasse 70 caractères.
Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js, puis python3
scripts/versionner_actifs.py).
Ne pas agrandir les polices.
Critère de fin : sur le banc, une 1re séance, un record et une 50e séance affichent trois phrases
différentes et justes.
```

3. Faire de la première séance un rituel, pas un formulaire — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc avec une élève neuve : la 1re séance ouvre le calque générique « BADGE DÉBLOQUÉ · PREMIÈRE SÉANCE · Termine une première séance. » (la règle relue, pas une félicitation), puis un écran de fin de 2 800 px. On y trouve quatre curseurs, une question sur la congestion, un paragraphe de 9 lignes sur les notifications et une carte « questionnaire ». Idée : pour la

1re séance seulement, l'écran montre « Tes 3 chiffres de départ » (meilleure charge des 3 premiers exercices, « ce sont tes premiers records à battre »), puis le jour de la prochaine séance. Le ressenti, la congestion et le texte long des notifications restent repliés derrière « Plus de détails ».

Recouplement. S'applique sur l'écran de fin raccourci par l'expert 16 (point 7) : faire le 16.7 d'abord.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. finishWorkout() (vers 53790) détecte déjà la 1re séance
((currentUser.sessions||[]).length===1,
rcm('first_workout_completed')). Le calque du badge PREMIÈRE SÉANCE passe par la file _bdgFile /
_bdgSuivant (vers 81718), et
sa description reprend la condition du badge (« Termine une première séance. »).
```

À faire :

1. Fonction PURE pointDeDepart(u, sess) → [{nm, kg, reps}] : pour les 3 premiers exercices à charge externe de la séance
(typeCharge(_exPourCharge(nm,u))==='externe'), la série validée la plus lourde. Les exercices au poids du corps donnent
« <reps> reps ». Vide si rien n'est validé.
2. Sur la 1re séance seulement, poser dans #wd-records (à la place du bloc records, vide de toute façon) une carte
« TON POINT DE DÉPART » avec ces 3 lignes et la phrase « Ce sont tes premiers records à battre. ».
Ajouter une ligne
« Prochaine séance : <jour> · <nom> », lue sur le planning existant (même source que la carte SÉANCE DU JOUR de l'accueil).
Rien si aucun créneau n'est prévu.
3. Toujours sur la 1re séance : replier le bloc ressenti (rcfPoserRessenti) derrière son « Plus de détails » existant, avec
les valeurs par défaut conservées. Dans _rendreInvitationNotif, n'afficher que la première phrase et les cases à cocher,
le paragraphe détaillé allant dans un <details>. Ne rien supprimer : tout reste accessible.
4. Calque du badge PREMIÈRE SÉANCE : remplacer la description affichée sur l'écran de célébration seulement (pas dans la vitrine) par « <Prénom>, c'est parti. » Le badge et ses visuels restent identiques.
5. Erreurs humaines : 1re séance abandonnée (incomplete) → pas de carte « point de départ » si rien n'est validé, mais aucun écran vide. Élève qui revient en arrière puis rouvre l'écran : la carte ne se duplique pas. Une 2e séance le même jour
ne réaffiche pas la carte.

Tests dans app/tests.js : pointDeDepart (charges, poids du corps, rien de validé, plus de 3 exercices) ; après une 1re finishWorkout(), #wd-records contient « TON POINT DE DÉPART » et le ressenti est replié ; à la 2e séance, plus rien de tout cela.

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : l'écran de fin de la 1re séance tient en deux écrans de téléphone au-dessus du bouton « Enregistrer et terminer ».

4. Bilan enregistré : l'avant/après apparaît tout seul — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc (3 bilans avec photos, 16 semaines) : l'écran « Bilan enregistré » (_htmlRestitutionBilan, rc-core.1792.js:~86280) montre seulement « Poids —0,8 kg · Taille —2,5 cm · Hanche —1 cm » par rapport au bilan PRÉCÉDENT. Le magnifique avant/après « TRANSFORMATION · 16 SEMAINES » n'est accessible que par un bouton de l'onglet Photos. Idée : sur cet écran, une vignette avant/après (départ ↔ aujourd'hui) avec une ligne « Depuis ton départ : —1,9 kg, —5 cm de taille », et un seul bouton « Partager ». Plan B, pour l'erreur réelle rencontrée par Kevin : si le bilan n'a pas de photo, un bouton « Ajouter mes photos à ce bilan », sans rien refaire d'autre.

Recouplement. Fusionne avec l'expert 9 (point 5) : une seule restitution « Depuis ton départ » avec vignette avant/après. L'expert 9 garde seulement la suppression des « stable » non mesurés.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Après saveBilanFinal, ouvrirRestitutionBilan() peint _htmlRestitutionBilan(user) (vers 86280) à partir de restitutionBilan(user) (vers 86199), qui ne compare qu'au bilan précédent. L'avant/après existe déjà :

aaVuesDisponibles(u) / aaDisponible(u) (vers 83268), _dessinerAvantAprès(o) (vers 83501), ouvrirAvantAprès(role) (vers 83638), et photoBilanSrc/photoBilanExiste (vers 69014) pour lire les photos. Les photos s'enregistrent via photoBilanEnregistrer(cible,bilan,vue,file).

À faire :

1. restitutionBilan : ajouter out.depuisDepart = [{libelle, valeur, sens, accord}] (poids et mensuration qui a le plus bougé, même règles de bruit que _synEcart / SYN_BRUIT_*), calculé entre le bilan de départ (bilansOrdonnes(user)[0]) et le dernier, quand il y a au moins 2 bilans et que le précédent n'est pas déjà le départ. Respecter user.masquerPoids.
2. _htmlRestitutionBilan : si aaDisponible(user) et que le dernier bilan a une photo du même angle que le départ, insérer en haut une vignette (canvas réduit de _dessinerAvantAprès, format story, largeur de l'écran moins les marges) puis la ligne « Depuis ton départ : ... » et UN bouton « Partager mon avant/après » → ouvrirAvantAprès('athlete'). Le reste de l'écran ne bouge pas.
3. Plan B : si le bilan qu'on vient d'enregistrer n'a AUCUNE photo (aucune vue où photoBilanExiste est vrai), afficher à la place une carte « Tes photos manquent à ce bilan » avec un bouton « Ajouter mes photos » qui ouvre un sélecteur des 3 vues et écrit dans CE bilan (photoBilanEnregistrer, même champ que photoBilanChamp). Pas de nouveau bilan, pas de doublon, la date du bilan est conservée. Le coach voit les photos arriver comme d'habitude.
4. Masquer la vignette si le profil le demande (même garde que la fresque photos : aTCA / masquerPoids / réglage de masquage des photos s'il existe). Rien en mode hors ligne si la photo n'est disponible qu'en URL distante : afficher la ligne de texte seule.
5. Erreurs humaines : quitter l'écran pendant l'envoi d'une photo (l'envoi continue, toast à la fin), double tap sur « Ajouter mes photos », photo refusée par photoControle (le message existant s'affiche et le bilan reste valide).

Tests dans app/tests.js : restitutionBilan.depuisDepart (2 bilans → absent, 3 bilans → présent, masquerPoids) ; rendu avec photos → vignette + bouton ; rendu sans photo → carte « Ajouter mes photos » ; l'ajout écrit dans le même bilan (même date, bilans.length inchangé).

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : sur le banc, un 3e bilan avec photos montre l'avant/après dès « Bilan enregistré » ; un bilan sans photo se complète en deux gestes.

5. Coach : « À féliciter » dans le point du lundi, avec un bravo en un geste — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc côté coach : « Le point du lundi » affiche « Léa · Rien à signaler » alors qu'elle est à 7 semaines d'affilée, avec des records sur ses 5 exercices. lundiLigne (rc-core.1792.js:27873) ne connaît que les problèmes. Les victoires (victoiresDe, :19508 ; kitVictoire, :19745) ne servent qu'au kit Instagram, et rien ne permet au coach de dire bravo (aucune occurrence de « félicité » dans le code). Idée : remplacer « Rien à signaler » par la meilleure victoire de la semaine (« Squat 58 → 62,5 kg »), avec un bouton « Bravo » qui envoie un message pré-rédigé et modifiable dans le fil privé.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. Le point du lundi : lundiLigne(c,sg,o) (vers 27873) renvoie cat:'rien' / 'Rien à signaler'
quand aucun signal ; _htmlLundiLigne et renderLundi (vers 27920) le peignent. victoiresDe(u) (vers 19508) liste les
progressions de charge par exercice ({exo,avant,apres,le,pct}). La messagerie privée envoie via
msgEnvoyer(athleteCle,brut)
(vers 27520).

À faire :
1. Fonction PURE victoireSemaine(c,maintenant) → {type:'record'|'serie'|'bilan'|'premiere', texte,
detail} ou null, dans cet
ordre : record battu dans les 7 derniers jours (victoiresDe, v.le dans la fenêtre, le plus fort
pct) ; palier de série
atteint cette semaine (SERIE_PALIER) ; bilan reçu avec un écart dans le bon sens
(_synPoidsDansLeSens) ; 1re séance de
l'athlète. Jamais pendant un drapeau santé ou une douleur.
2. lundiLigne : quand la catégorie est 'rien' ET qu'une victoire existe, cat:'victoire', signal =
texte (« Squat 58 → 62,5 kg »).
Les alertes gardent la priorité (LUNDI_RANGS : placer 'victoire' juste avant 'rien'). Couleur : le
vert existant
(var(--green)), pas de nouveau style de taille.
3. Sur une ligne 'victoire', ajouter un petit bouton « Bravo » (même hauteur que la ligne, pas plus
gros) qui ouvre une
feuille avec un message pré-rempli et modifiable, par exemple « Bravo Léa pour ton squat à 62,5 kg
🔥 On continue
comme ça. », puis « Envoyer » → msgEnvoyer. Après l'envoi : la ligne affiche « Bravo envoyé ✓ ».
Mémoriser
c.bravos[cleSemaine]=true dans le dossier coach (pas dans localStorage) pour ne pas proposer deux
fois la même victoire.
4. Côté élève : le message arrive dans le fil existant, rien de nouveau à construire.
5. Erreurs humaines : envoi hors ligne (le message part en file comme les autres, état « en attente
»), double tap
(un seul envoi), coach qui efface tout le texte (bouton désactivé), athlète sans prénom.
Tests dans app/tests.js : victoireSemaine (record dans la fenêtre, record trop ancien, série, drapeau
santé qui masque tout,
priorité record > série) ; lundiLigne renvoie 'victoire' au lieu de 'rien' ; le tri garde les alertes
en tête ; le bouton
Bravo appelle msgEnvoyer une seule fois sur un double clic.
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3
scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : sur le banc coach, Léa apparaît « Squat 58 → 62,5 kg » avec un Bravo envoyé en deux
taps.
```

6. La phrase d'accueil devient l'histoire de l'élève — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc : après 51 séances et 16 semaines, l'accueil affiche « Aucun discours n'a jamais soulevé un kilo. » (phraseDuJour , rc-core.1792.js:44336, une citation tournante générique). La synthèse d'Évolution est bloquée sur 8 semaines (SYN_SEMAINES=8 , :86068). Idée : la même zone, à la même taille, affiche une phrase personnelle quand les données le permettent, par exemple « Il y a 16 semaines, ton squat était à 40 kg. Aujourd'hui : 58. » ou « -5 cm de taille depuis ton départ ». La citation ne revient que s'il n'y a rien à raconter. Aucun écran n'est ajouté.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. phraseDuJour(d) (vers 44336) choisit une phrase de RC_PHRASES selon le
quantième, posée sur
l'accueil élève par loadClientHome (_phr.textContent=phraseDuJour(), vers 44517). Les données utiles
existent : victoiresDe(u)
(vers 19508), bilansOrdonnes(u), getBW/getBM, MEAS, u.streak, tonnageTotalDe(u).
```

À faire :

1. Fonction PURE `histoireDuJour(u,date) → string` ou null. Construire la liste des faits vrais :
- victoire de charge depuis le début (victoiresDe, gain $\geq 10\%$) : « Il y a <N> semaines, ton <exo> était à <avant> kg.
Aujourd'hui : <après>. » ;
- mensuration (départ → dernier bilan, au-delà du bruit SYN_BRUIT_MESURE, dans le sens voulu) : « <-5 cm> de tour de
taille depuis ton départ. » ;
- série ≥ 4 : « <N> semaines d'affilée sans lâcher. » ;
- tonnage cumulé ≥ 10 t : « <X> t soulevées depuis <mois>. ».
Choisir un fait par jour en tournant (quantième modulo la longueur), sans aléatoire, pour que le texte soit stable sur la journée.
Respecter `masquerPoids / aTCA` : aucun fait de poids ni de mensuration dans ce cas. null si aucun fait.
2. Sur l'accueil : `histoireDuJour(u) ?? phraseDuJour()`. Même nœud, même style, aucune taille nouvelle. Un fait personnel
s'écrit en blanc, sans guillemets (la citation garde son style actuel).
3. Évolution : la synthèse (`_htmlSyntheseProgression`, vers 86184) garde ses 8 semaines, mais quand l'élève a plus de 8 semaines
d'historique, ajouter un lien discret « Depuis le début » qui recalcule la même synthèse entre le bilan de départ et le dernier.
Un seul paramètre en plus à `_synBilansFenetre` (`depuisDebut:true`).
Tests dans `app/tests.js` : `histoireDuJour` (élève neuve → null, victoire, mensuration dans le mauvais sens → ignorée, `masquerPoids`, rotation stable sur une journée) ; l'accueil d'une élève neuve garde la citation.
Convention : nouveau build (`window.RC_BUILD`, CACHE de `app/sw.js`, `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.
Critère de fin : sur le banc, l'accueil de Léa (16 semaines) raconte un vrai chiffre à elle, celui de l'élève neuve garde la citation.

7. « Au rythme actuel » : une prévision prudente par exercice — Priorité : Moyenne · Effort : M

Analyse et idée. `recordAPorteeExo` (`rc-core.1792.js:88129`) projette seulement la séance SUIVANTE (+2,5 % au plus) sur une régression par numéro de séance, sans dates. L'onglet Perfs affiche « en progression · record sur la dernière séance » sans jamais dire où cela mène (testé au banc). Idée : sous chaque exercice en progression, une ligne « Au rythme actuel : ~66 kg × 8 vers le 17 nov. ». Elle n'apparaît que si la tendance est solide, elle est plafonnée et toujours présentée comme une estimation. Une seule ligne, pas de nouveau graphique.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js`. `recordAPorteeExo(u,nomEx,reprs,maintenant)` (vers 88129) utilise `_serieRecords(user,nomEx)` (vers 88015), `_rapElrm(sets,user)`, `tendanceLineaire(vals)` et `chargePourReps(elrm,reprs,rir)`, avec les constantes `RAP_SEANCES/RAP_MIN_POINTS/RAP_PLAFOND/RAP_FRAICHEUR_J` (vers 88090). La carte « PROGRESSION PAR EXERCICE » de l'onglet Perfs d'Évolution affiche l'état de chaque exercice.

À faire :

1. Fonction PURE `previsionExo(u,nomEx,reprs,semaines=6,maintenant) → {charge,reprs,date,elrm}` ou null. Faire une régression des `elRM` (hors décharges et hors points aberrants, comme `_serieRecords`) sur les DATES réelles des 8 dernières semaines, avec au moins 6 points, la dernière séance datant de moins de 21 jours et une pente > 0 . Plafonner le gain à 1 % de l'`elRM` par semaine (débutant : 1,5 % si moins de 12 semaines d'historique) et à 10 % au total. Convertir en charge avec `chargePourReps(elrm, reprs habituelles, RIR médian)`, puis arrondiAuPas. null si la charge prévue ne dépasse pas le record actuel.
2. Dans la carte de l'exercice (Perfs), une ligne en `var(--sub)` : « Au rythme actuel : ~<charge> kg × <reprs> vers le

<date courte> » avec rcInfo('e1rm'). Rien si null. Jamais en phase de sèche avec une pente négative, jamais pendant une suspension (suspensionEtat), jamais sur un exercice au poids du corps.

3. Ne rien afficher sur l'écran de fin de séance (le grimpeur y suffit).

Tests dans app/tests.js : progression régulière → prévision cohérente et plafonnée ; 5 points → null ; dernière séance à 30 jours → null ; pente nulle ou négative → null ; décharges ignorées ; arrondi au pas de l'exercice.

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : sur le banc, Léa voit une prévision réaliste sur 3 à 5 exercices et aucune sur un exercice qui stagne.

8. Fin de bloc côté élève : un mini-Wrapped de 3 cartes — Priorité : Moyenne · Effort : M

Analyse et idée. bilanBloc (rc-core.1792.js:37462) produit une excellente lecture (assiduité, e1RM début → fin par exercice), mais seulement pour le coach et sous forme de tableaux MEV/MRV (:37561-37628). L'élève qui finit 8 semaines de travail ne reçoit rien. Idée : réutiliser l'écran Wrapped déjà réussi (calculerWrapped, :82320 ; wrappedSlides, :82439) avec une période « bloc ». La carte d'accueil « Ton bloc est bouclé » a la même croix que la carte du mois, et les slides montrent séances/assiduité, top 3 des progressions e1RM et records, puis le profil.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Le Wrapped : wrappedPeriodes(t) (vers 82412) rend les périodes 'mois' et 'annee' ; calculerWrapped(u,debut,fin) (vers 82320) ; wrappedSlides(w,per) (vers 82439) ; ouvrirWrapped(cle) (vers 82555) ; _wrPeriodeDeCle(cle) (vers 82576) ; la carte d'accueil _rendreCarteWrapped() (vers 82747) avec sa croix (accMasquerCarte).

Le bloc : bilanBloc(u) (vers 37462) lit programmeDe(u) {debut, semaines} et rend cadre.assiduite et exercices[{nom,debut,fin,ecart}].

À faire :

- wrappedPeriodes : ajouter une période {type:'bloc', cle:'b-<debut>', debut, fin:finPrevue, titre:'TON BLOC DE <N> SEMAINES', carte:'Ton bloc est bouclé', lib} pendant les 10 jours qui suivent finPrevue d'un bloc NON interrompu, avec au moins 4 séances complètes. Priorité d'affichage sur la carte d'accueil : bloc > mois (une seule carte à la fois).
- wrappedSlides, pour per.type==='bloc' : la slide « séances » affiche l'assiduité (« 92 % d'assiduité ») ; la slide « records » affiche le top 3 de bilanBloc(u).exercices par écart e1RM (« Squat +8,5 kg ») ; les slides tonnage et profil ne changent pas. Même dessin 1080×1920 que _dessinerWrapped, sans nouveau style.
- Le coach : sur sa fiche, le bloc « Bilan du bloc » existant gagne une mention « Vu par l'athlète le <date> » quand l'élève a ouvert ce Wrapped (champ u.wrappedVus[cle]=date).
- Erreurs humaines : bloc modifié par le coach en cours de route (on lit le bloc tel qu'il est à finPrevue), décharge en dernière semaine, élève qui ferme la carte (elle ne revient pas pour ce bloc), aucun exercice fait 3 fois (la slide records retombe sur « Le prochain t'attend. » comme aujourd'hui).

Tests dans app/tests.js : wrappedPeriodes renvoie 'bloc' à J+1 et plus à J+11 ; bloc interrompu → rien ; slides du bloc (top 3 trié, assiduité) ; une seule carte d'accueil quand mois et bloc tombent ensemble.

Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

Critère de fin : sur le banc, un bloc de 8 semaines terminé hier affiche « Ton bloc est bouclé » et 5 slides justes.

9. Les jalons de la relation : 30 jours, 100 jours, 1 an — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc avec `createdAt` à 365 jours : rien ne se passe, et aucune occurrence de jalon d'inscription n'existe dans le code (seuls « mois » du 1er au 7 et « année » en décembre dans `wrappedPeriodes`). Or c'est le meilleur moment pour retenir l'élève et pour valoriser le coach. Idée : trois jalons datés à partir de la 1re séance (30 j, 100 j, 1 an), qui réutilisent le `Wrapped` avec un titre « TES 100 JOURS » et une slide avant/après si `aaDisponible`. Le coach est prévenu par une ligne dans son point du lundi.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. wrappedPeriodes(t) (vers 82412) ne connaît que 'mois' (jours 1 à 7) et 'annee' (décembre).
calculerWrapped(u,debut,fin) et wrappedSlides(w,per) sont génériques. La carte d'accueil :
_rendreCarteWrapped() (vers 82747).
L'avant/après : aaDisponible(u), _dessinerAvantAprès. Le point du lundi du coach : lundiLigne (vers 27873).

À faire :
1. Fonction PURE jalonRelation(u,maintenant) → {jours:30|100|365, debut, fin} ou null. Le point de
départ est la 1re séance
  (min des dates de u.sessions), pas createdAt (un compte créé sans s'entraîner ne fête rien). Le
jalon est actif du jour J
  à J+6, et seulement si au moins 4 séances ont eu lieu dans la période.
2. wrappedPeriodes(t,u) : quand jalonRelation renvoie un jalon, ajouter {type:'jalon', cle:'j<jours>-
<debut>', titre:
  jours===365?'TON PREMIER AN':'TES '+jours+' JOURS', carte:..., debut, fin}. Priorité de la carte
d'accueil :
  jalon > bloc > mois (une seule carte).
3. Slide en plus pour ce type, seulement si aaDisponible(u) : l'avant/après réduit (départ ↔ dernier
bilan). Sinon, les 5 slides
habituelles.
4. Coach : dans lundiLigne, quand cat vaut 'rien' ou 'victoire' et qu'un jalon est actif, signal « 100
jours avec toi cette
  semaine ». Les alertes santé gardent la priorité.
5. Erreurs humaines : élève qui fait sa 1re séance des mois après l'inscription (on compte depuis la
séance), changement de
  fuseau horaire (dates locales via localISODate), carte fermée (ne revient pas pour ce jalon).
Tests dans app/tests.js : jalonRelation (J29 → null, J30 → 30, J36 → 30, J37 → null, 100, 365, moins
de 4 séances → null,
aucune séance → null) ; priorité des cartes ; ligne coach.
Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3
scripts/versionner_actifs.py). Ne pas agrandir les polices.
Critère de fin : sur le banc, une élève à 100 jours voit « Tes 100 jours » avec son avant/après, et
son coach le voit le lundi.
```

10. Une carte du corps qui se « remplit » de records — Priorité : Basse · Effort : M

Analyse et idée. La silhouette face/dos détournée existe déjà (`htmlCarteMuscles/_dessinerCarteMuscles`, `rc-core.1792.js:83092-83145`), mais elle ne montre que le volume de la semaine (« Aucun volume cette semaine : Pectoraux, Dorsaux... », testé au banc), ce qui donne une carte surtout grise et peu flatteuse. Idée : un 2e choix dans le même sélecteur (« Semaine | 4 semaines | Depuis le début »). Avec « Depuis le début », chaque groupe s'allume à mesure que ses exercices battent des records. On ajoute « 7/11 groupes ont battu un record », sans nouvel écran.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. La carte musculaire (vers 82795) : MUSC_GROUPEs, muscDonnees(seances,o) (vers 82905), muscCouleur, htmlCarteMuscles(id,d,o), _dessinerCarteMuscles, muscEvoPeriode(p) et rendreMusclesEvolution() (vers 83226), affichée dans Évolution > Volume avec le sélecteur SEMAINE / 4 SEMAINES. victoiresDe(u) (vers 19508) donne les progressions par exercice ; la répartition des muscles d'un exercice vient du même référentiel que _volumeSeance.

À faire :

1. Fonction PURE muscRecords(u) → {groupes:{cle:{intensite 0..1, exos:[...]}, allumes, total} : pour chaque exercice avec une victoire (victoiresDe), ajouter à ses muscles PRINCIPAUX (pas les secondaires) une intensité proportionnelle au gain en %, saturée à +25 %. Le résultat a la même forme que muscDonnees pour que le dessin ne change pas.
 2. muscEvoPeriode accepte 'records' ; le sélecteur gagne « Records » (même composant, même taille). Titre : « CARTE DES RECORDS · <allumes>/<total> GROUPEs ». Le partage (story/post) fonctionne tel quel.
 3. Toucher un groupe affiche la liste de ses exercices avec « avant → après » (réutiliser la bulle d'info existante).
 4. Respecter les profils où la carte du corps est masquée (même garde que la carte actuelle).
- Tests dans app/tests.js : muscRecords (aucune victoire → 0 groupe, un squat +20 % → quadriceps et fessiers allumés, ischios non, saturation à 25 %) ; le sélecteur passe sur 'records' sans erreur et revient sur 'semaine'. Convention : nouveau build (window.RC_BUILD, CACHE de app/sw.js, python3 scripts/versionner_actifs.py). Ne pas agrandir les polices. Critère de fin : sur le banc, Léa voit sa carte des records se remplir sur les groupes réellement progressés.

Sur la gamification déjà en place (sans la repropoter) : au banc, elle est réussie. Les badges illustrés, le record « 58 → 62,5 kg +4,5 », la série en héros et le Wrapped mensuel fonctionnent. Un seul défaut d'enchaînement a été vu : le calque récapitulatif des badges (_bdgSuivant , :81718) s'ouvre aussi par-dessus « Bilan enregistré » et Évolution, parce qu'il ne regarde pas l'écran actif. Il faudrait le retenir tant qu'un écran de saisie ou de restitution est à l'écran (ECRANS_DE_SAISIE , :9640).

Expert 14 · Paramétrage pour le coach — 5/10

Beaucoup de choses sont déjà réglables, et souvent avec soin : la marque Pro avec contrôle du contraste, les relances par signal, les modèles de messages, la formule de réponse aux bilans, la cadence et les questions de bilan, le choix kg ou lb. Ce qui manque le plus : un endroit unique pour régler, et un étage « réglage par défaut du coach » entre les valeurs codées par RepCore et chaque athlète. Aujourd'hui, un coach tiers refait les mêmes réglages élève par élève, et ses documents portent les slogans de Kevin.

1. Un seul écran « Mes réglages de coach » au lieu de six endroits — Priorité : Haute · Effort : L

Analyse et idée. Testé au banc (coach, onglet PROFIL) : 4 799 px de haut, soit 7,2 écrans, avec 5 boutons « Enregistrer » distincts. Le reste est ailleurs : les relances dans un écran séparé (ouvrirRelances , rc-core:27727), la formule de réponse dans une fenêtre ouverte depuis un bilan (ouvrirFormulesReponse , rc-core:78424), les modèles de messages depuis la zone de saisie (ouvrirGestionModeles , rc-core:112749), l'échauffement par défaut via un bouton caché de l'éditeur de séance (saveWarmupTemplate , rc-core:43918), et la cadence des bilans dans le calendrier de chaque fiche (rc-core:76367). L'idée : un écran s-coach-reglages à six sections repliées (Identité et marque · Mes valeurs par défaut · Bilans · Messages et relances · Ce que voient mes élèves · Apparence et aide), avec enregistrement automatique et un « Annuler » de 10 s. Chaque ligne affiche « par défaut » ou « 3 athlètes font exception ».

Recouplement. Point de référence pour les réglages du coach (l'expert 1, point 9, y renvoie).

Prompt de mise en place.

Contexte : RepCore (app/index.html, app/rc-core.1792.js). Les réglages globaux du coach sont dispersés : onglet PROFIL (#ct-profil, index.html ~l.1964-2141, 5 boutons Enregistrer : saveCoachIdentity, saveCoachDispo, saveCoachBanners, saveCoachPhone, mqEnregistrer), écran Relances (ouvrirRelances / renderRelancesCoach, rc-core ~27727), formule de réponse aux bilans (ouvrirFormulesReponse / enregistrerFormulesReponse, ~78424), modèles de messages (ouvrirGestionModeles, ~112749), échauffement et retour au calme par défaut (saveWarmupTemplate, ~43918, champs _defaultWarmup/_defaultCooldown). Objectif : créer un écran unique « Mes réglages » (id s-coach-reglages), ouvert depuis l'onglet PROFIL et depuis un lien « Réglages » dans l'en-tête de l'accueil coach. Il contient six sections <details> repliées par défaut, dans cet ordre : 1) Identité et marque (on y DÉPLACE les blocs existants sans les réécrire), 2) Mes valeurs par défaut (vide pour l'instant, voir le lot « défauts de prescription »), 3) Bilans, 4) Messages et relances (formule de réponse en ligne, lien vers les modèles, résumé des relances avec lien vers l'écran détaillé), 5) Ce que voient mes élèves (voir le lot « affichage »), 6) Apparence et aide (htmlPrefsAide existant). Règles :

- Pas de nouveau composant visuel : on réutilise .card, .prf, .prf-t et les classes existantes. Ne pas agrandir les polices ni les cibles tactiles.
- Enregistrement automatique à la sortie du champ (blur/change) via une fonction unique reglageCoachEcrire(chemin, valeur) qui écrit currentUser, appelle saveUser() et affiche « Enregistré ✓ – Annuler » pendant 10 s ; Annuler remet la valeur précédente. Les anciens boutons Enregistrer disparaissent de ces blocs. Garder les fonctions existantes comme points d'entrée internes.
- Chaque section affiche dans son titre un résumé d'une ligne (ex. « Bilans : toutes les 2 semaines, samedi · 2 questions »).
- Erreurs humaines : un champ vidé par erreur ne doit pas effacer la donnée distante sans confirmation (nom de team, logo). Une perte de réseau garde la saisie en local et le dit (toastSync existant). Un rechargement au milieu de la saisie ne perd rien.
- L'onglet PROFIL garde la vitrine publique (bio, vision, spécialités, diplômes) et un bouton « Mes réglages ».

Tests (app/tests.js) : l'écran existe et contient les six sections ; reglageCoachEcrire écrit puis Annuler restaure ; aucun bouton « Enregistrer » ne reste dans les sections déplacées ; les fonctions historiques (saveCoachIdentity, enregistrerFormulesReponse...) restent appelables.

Critère de fin : au banc, un coach trouve tous ses réglages globaux dans un seul écran d'au plus 2 hauteurs d'écran quand tout est replié, et toute la suite de tests passe.

Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

2. Les bilans réglés une fois pour toute l'équipe — Priorité : Haute · Effort : M

Analyse et idée. La cadence et les trois questions de fin de bilan n'existent que sur le dossier de chaque athlète (_cadenceAppliquer, rc-core:30319). Un nouvel élève arrive donc en « Au choix de l'athlète, le samedi » (rc-core:30247). L'application groupée applique la cadence mais pas les questions : le texte de la fenêtre le dit lui-même, « Leurs questions de fin de bilan ne changent pas » (rc-core:30353). L'idée : un défaut au niveau du coach, defaultsCoach.bilan = {freq, jour, questions}, lu dans cet ordre : réglage de l'athlète, puis défaut du coach, puis défaut RepCore. Le changer propose « Appliquer aussi aux 12 athlètes sans réglage propre ? » avec un aperçu des noms concernés.

Prompt de mise en place.

Contexte : app/rc-core.1792.js, bloc « LA CADENCE DES BILANS » (~l.30217-30380). bilanCadence et questionsCoach vivent sur le dossier de l'athlète. bilanFreqEffective, echeanceBilan et questionsCoachDe ne connaissent que l'athlète. Le Worker applique la même règle (cloudflare/src/relances.js, echeanceBilanParis). Objectif : ajouter un étage « défaut du coach ».

- 1) Nouveau champ coach : defaultsCoach.bilan = {freq:1|2|4, jour:0..6, questions:[≤3 chaînes ≤120]}. Le classer dans CHAMPS_NON_SANTE (~l.6896) et l'autoriser dans database.rules.json.
- 2) Fonction PURE cadenceEffective(athlete, coach) → {freq, jour, questions, source:'athlete'|'coach'|'repcore'}. echeanceBilan, bilanFreqEffective, questionsCoachDe et _htmlCadenceCoach passent par elle (le coach est retrouvé côté athlète via coachAffichable / coach_public ; il faut publier defaultsCoach.bilan dans coach_public, et seulement ça).
- 3) Recopier la même règle dans le Worker (relances.js), avec un test qui compare les deux comme

relances.test.mjs le fait déjà.

4) Dans « Mes réglages » > Bilans : fréquence, jour, trois questions. À l'enregistrement, une feuille propose « Appliquer aussi aux N athlètes qui suivent ton défaut » (aperçu des prénoms) ou « Seulement aux nouveaux ». Les athlètes qui ont un réglage propre ne sont jamais touchés en silence.

5) Sur la fiche, _htmlCadenceCoach affiche « Suit ton défaut » ou « Réglage propre · revenir au défaut ».

6) selCadence (barre de sélection) propose aussi les questions, avec une case décochée « remplacer leurs questions ».

Erreurs humaines : un défaut modifié par erreur doit pouvoir être annulé (Annuler pendant 10 s, et l'ancienne valeur gardée dans defaultsCoach.bilan._precedent). Changer de cadence ne doit jamais rendre un athlète « en retard » d'un coup : afficher l'échéance recalculée de chacun dans l'aperçu avant de confirmer.

Tests (app/tests.js et cloudflare/test) : ordre de priorité athlète > coach > RepCore ; questions invalides filtrées ; un athlète avec réglage propre n'est pas modifié par « appliquer aux autres » ; même échéance côté app et côté Worker.

Critère de fin : un nouvel élève invité reçoit sans aucune action la cadence et les questions du coach.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

3. Ses valeurs de prescription par défaut (séries, reps, repos, RIR, décharge) — Priorité : Haute · Effort : M

Analyse et idée. Chaque exercice ajouté naît avec 3 × 10, 2 min de repos et un RIR vide, et cette valeur est codée en dur à trois endroits (rc-core:35982, 42389, 43827). Un quatrième endroit met « 01 min » (rc-core:42933), donc deux défauts différents selon le chemin pris. La décharge (DECHARGE_FACTEUR_SERIES=0.6, DECHARGE_RIR_PLUS=1, rc-core:14255) est présentée en commentaire comme « des réglages de coaching, pas des constantes techniques », mais aucun écran ne permet de la changer. L'idée : une section « Mes valeurs par défaut » avec séries, reps, repos, RIR visé et décharge (volume en %, RIR +n), lue par une fonction unique `exerciceVierge(coach)`.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Les exercices neufs sont créés avec

{series:3,reps:'10',repos:REPOS_DEFAULT} à trois endroits (~l.35982, ~42389, addExercise ~43827) et avec repos '01 min' à un quatrième (~42933). REPOS_DEFAULT='2 min' (~35962). La décharge utilise DECHARGE_FACTEUR_SERIES et DECHARGE_RIR_PLUS (~14255).

Objectif :

1) Champ coach defaultsCoach.prescription = {series (1-10), reps (texte ≤12), repos (texte reconnu par le parseur de repos existant), rir ('' ou une valeur de l'échelle RIR existante), dechargeSeriesPct (40-90), dechargeRirPlus (0-3)}. Le classer dans CHAMPS_NON_SANTE et l'autoriser dans database.rules.json.

2) Fonction PURE `exerciceVierge(coach)` qui rend l'objet exercice neuf. Les quatre endroits l'appellent (le '01 min' disparaît). Sans réglage, on garde exactement le comportement actuel (3, '10', '2 min', rir '').

3) `appliquerDecharge` lit `facteurDecharge(coach)` et `rirDecharge(coach)` au lieu des constantes, avec repli sur 0,6 et +1.

4) Section « Mes valeurs par défaut » de l'écran Mes réglages : 6 champs compacts sur 2 colonnes, avec un aperçu en une ligne (« Nouvel exercice : 4 × 8-10 · 2 min 30 · RIR 2 »).

Erreurs humaines : une saisie de repos illisible (« 2mn30 », « 150 ») doit être normalisée ou refusée avec un message clair, jamais enregistrée telle quelle. Changer les défauts ne modifie AUCUN exercice déjà écrit : le dire sous la section. Une valeur hors bornes est ramenée à la borne la plus proche.

Tests (app/tests.js) : `exerciceVierge` sans réglage = valeurs historiques ; avec réglage = valeurs du coach ; les 4 chemins de création produisent le même objet ; `appliquerDecharge` respecte le pourcentage du coach ; bornes respectées.

Critère de fin : au banc, un coach règle « 4 × 8-10, 2 min 30, RIR 2 », ajoute un exercice dans l'éditeur et voit ces valeurs pré-remplies.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

4. Ce que l'élève voit : au coach de décider, par élève et par défaut — Priorité : Haute · Effort : M

Analyse et idée. Le coach ne peut ni masquer les calories ni masquer le poids d'un élève.

`masquerPoids` ne se règle que chez l'athlète (`togglePoidsMasque`, rc-core:119208) ou par le questionnaire TCA (`aTCA`, rc-core:118718), et aucun indicateur de module n'existe dans le code. Testé au banc, l'accueil de Léa affiche dans tous les cas : point sommeil, check-in du matin avec 4 notes, « Ta faim, hier ? », « Défie un pote » et « 2100 / 2100 kcal ». L'idée : trois profils d'affichage, Complet, Sans chiffres (poids et kcal masqués, macros en portions) et Essentiel (séance, pesée facultative, bilan). Le coach pose un défaut global et une exception par élève. La règle la plus protectrice l'emporte toujours : un élève qui s'est masqué le poids, ou un profil TCA, ne le revoit jamais parce que le coach a choisi « Complet ».

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js. masquerPoids (blocPoids ~119155, togglePoidsMasque ~119208, utilisé aussi ~3228, ~80798, ~118763) et aTCA (~118718) masquent déjà le poids. Aucun réglage coach ne masque les calories, la pesée du jour, le check-in du matin, la question de faim ou les cartes sociales (« Défie un pote ») de l'accueil athlète.
Objectif :
1) Constante AFFICHAGE_PROFILS = {complet, sansChiffres, essentiel}. Chaque profil liste ses modules : kcal, poids, pesee, checkinMatin, faim, social, sommeil, nutritionMacros.
2) Champs : coach defaultsCoach.affichage = 'complet'|'sansChiffres'|'essentiel' ; athlète affichageCoach = même énumération ou absent (posé par le coach sur le dossier). Les classer dans CHAMPS_NON_SANTE et les autoriser dans database.rules.json (écriture d'affichageCoach réservée au coach rattaché).
3) Fonction PURE moduleVisible(athlete, coach, module) qui renvoie false dès qu'UNE source masque : réglage de l'athlète (masquerPoids...), aTCA, exception du coach sur l'athlète, défaut du coach. Jamais l'inverse : un choix du coach ne réaffiche rien que l'athlète ou le garde-fou TCA a masqué.
4) Brancher moduleVisible sur l'accueil athlète (cartes kcal, pesée du jour, check-in, faim, social) et sur l'onglet Nutrition (en sansChiffres : les grammes et les portions restent, les kcal disparaissent).
5) Côté coach : section « Ce que voient mes élèves » dans Mes réglages (3 cartes radio avec une phrase chacune), et sur la fiche athlète une ligne « Affichage : suit ton défaut · changer ».
6) Côté athlète : une phrase discrète dans ses Réglages, « Ton coach a simplifié ton affichage », sans détail clinique.
Erreurs humaines : passer un élève en « Essentiel » ne doit effacer AUCUNE donnée, seulement l'affichage. Revenir à « Complet » réaffiche tout ce que l'athlète n'a pas masqué lui-même.
Tests (app/tests.js) : la table de vérité de moduleVisible (TCA + complet = poids masqué ; masquerPoids athlète + complet = masqué ; sansChiffres = kcal masquées ; aucune combinaison ne réaffiche un module masqué par l'athlète) ; l'accueil rendu en sansChiffres ne contient ni « kcal » ni « kg ».
Critère de fin : au banc, le coach met Léa en « Sans chiffres » et l'accueil de Léa n'affiche plus ni calories ni poids, sans autre changement.
Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.
```

5. Une seule marque, appliquée partout, sans les slogans de Kevin — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc avec un coach tiers, « Sophie Durand / TEAM SOPHIE FIT » : sa fiche alimentaire imprime 4 fois « MORE THAN PROGRESS », plus « DISCIPLINE AUJOURD'HUI RÉSULTATS DEMAIN », « COACHING | NUTRITION | SUIVI » et le slogan `FA_MOTTO` (rc-core:68232, 68350-68368). Le nom affiché est « SOPHIE DURAND » et non le nom de sa team. La couleur de marque Pro n'est pas reprise. Le PDF programme ne porte pas de logo du tout (rc-core:68695). Le PROFIL demande 4 images (photo, logo, logo de marque, signature) et 2 noms (« Nom de la team » et « Nom affiché »). L'idée : un seul bloc « Ma marque », avec nom, logo, couleur, signature et slogan facultatif (celui de Kevin pré-rempli pour lui seul). Ce bloc alimente l'app, la fiche alimentaire, le PDF programme et les visuels.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Fiche alimentaire : ficheAlimDonnees (~68269) et son gabarit (~68340-68370) écrivent en dur « MORE THAN PROGRESS » (x4), « NUTRITION | PERFORMANCE | RÉSULTATS », « DISCIPLINE AUJOURD'HUI RÉSULTATS DEMAIN », « COACHING | NUTRITION | SUIVI », « DES FONDATIONS SOLIDES... » et FA_MOTTO (~68232). Le nom vient de fname+lname et ignore teamName. marqueCoachDe (~18569) lit logo||signature, alors que la marque Pro (coachs/<coach>/marque, renderMarqueCoach ~2635) a son propre logoUrl, son nom et sa couleur. Le PDF programme (ppRendre ~68707) n'a qu'une signature texte.

Objectif :

- 1) Fonction PURE marqueDocument(coach) → {nom, logo, couleur, slogan, signature}. Ordre de priorité : marque Pro (nom, logoUrl, couleur passée par couleurAccessible), puis teamName, puis prénom et nom ; logo de marque, puis logo du profil, puis signature. Slogan : nouveau champ coach `slogan` (≤60 caractères, facultatif). Pour le compte créateur (MARQUE_CREATEUR) seulement, défaut « MORE THAN PROGRESS » et les mottos actuels.
- 2) Fiche alimentaire et PDF programme : remplacer tous les textes de Kevin écrits en dur par marqueDocument. Sans slogan, les emplacements sont retirés (pas de ligne vide). La couleur d'accent de la fiche passe par une variable CSS posée sur l'article.
- 3) PROFIL / Mes réglages : fusionner « Logo » et le logo de la marque Pro en un seul champ, et « Nom de la team » avec « Nom affiché ». Migration en LECTURE seule, sans réécriture des anciens champs.
- 4) Aperçu : bouton « Voir ma fiche type », qui rend la fiche alimentaire d'un athlète fictif.

Erreurs humaines : un logo trop lourd ou non carré est refusé avec un message, l'ancien reste en place ; un slogan vide n'imprime rien ; une couleur illisible est corrigée (couleurAccessible) et la correction est montrée au coach.

Tests (app/tests.js) : pour un coach non créateur, le HTML de la fiche ne contient ni « MORE THAN PROGRESS » ni « DISCIPLINE » ; il contient teamName ; le compte créateur garde ses textes ; le PDF programme contient le logo quand il existe ; marqueDocument respecte l'ordre de priorité.

Critère de fin : au banc, la fiche alimentaire d'un coach tiers ne porte que sa marque et la petite mention « RepCore » du pied de page.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

6. Des réglages « fantômes » à exposer ou à retirer — Priorité : Moyenne - Effort : M

Analyse et idée. repereVolume (repères de volume MEV/MRV), methodesRegles (plafonds des techniques d'intensité) et poidsTechnique sont lus par l'application. Les commentaires disent « le coach les ajuste » (rc-core:41080, 71598, 41199). Pourtant aucune ligne de l'application ne les écrit : seuls les tests le font (tests.js:1983). La méthode du coach ne peut donc pas être réglée. L'idée : une section « Ma méthode », repliée sous Mes valeurs par défaut. Elle donne des repères par muscle au niveau du coach, avec une exception possible par athlète sur sa fiche Volume, et un « Revenir aux repères RepCore » à chaque niveau.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. repereEffectifs (~71610) lit REPERES_VOLUME, puis user.reperesAuto, puis user.reperesVolume. methodesRegles (~41134) lit user.methodesRegles. poidsTechnique (~41201) lit user.poidsTechnique. Aucune UI n'écrit ces trois champs : vérifier avec grep, seul app/tests.js les pose.

Objectif : ajouter un étage coach et une UI.

- 1) Champ coach defaultsCoach.methode = {repereVolume:{MUSCLE:{mev,mav,mrv}}, methodesRegles:{cle:{maxParSemaine,coutFatigue}}}. Ordre de lecture : REPERES_VOLUME, puis défaut du coach, puis repereAuto (mesure), puis surcharge de l'athlète. Le coach gagne toujours sur la mesure, comme aujourd'hui.
- 2) UI « Ma méthode » (Mes réglages > Mes valeurs par défaut, repliée) : un tableau compact muscle × MEV/MRV avec la valeur RepCore grisée en placeholder ; une liste des méthodes avec « max par semaine ». Un bouton « Revenir aux valeurs RepCore » par ligne.
- 3) Fiche athlète, écran Volume : « Ajuster pour cet athlète » ouvre le même tableau, qui écrit dans user.reperesVolume. Le champ source (déjà calculé) affiche « coach », « mesuré » ou « réf. ».
- 4) Validation existante réutilisée (bornes POIDS_TECHNIQUE_MIN/MAX, MEV < MRV).

Erreurs humaines : MEV supérieur à MRV est refusé, avec un message ; une case vidée revient au niveau supérieur et n'enregistre pas 0 ; Annuler pendant 10 s.

Tests (app/tests.js) : ordre de priorité à 4 niveaux ; écriture par l'UI puis lecture par

reperesEffectifs ; bornes refusées.

Critère de fin : chaque champ lu par l'application a un endroit pour être réglé. Sinon il est supprimé avec ses tests.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

7. « Mes modèles de plan alimentaire », comme pour les programmes — Priorité : Moyenne · Effort : M

Analyse et idée. Le programme d'un athlète peut devenir un modèle réassignable (rc-core:38924). Le plan alimentaire, lui, ne propose que les 10 modèles de Kevin codés en dur (PLAN_MODELES, rc-core:93856), choisis selon la phase et le sexe. Testé au banc : le modèle « Sèche femme » impose ses textes à Sophie, par exemple « SOUS FORME DE PANCAKES ». Un coach tiers ne peut pas garder ses propres plans. L'idée : « Enregistrer ce plan comme modèle » depuis la composition, et une liste « Mes modèles » au-dessus de « Modèles RepCore ». Les modèles RepCore peuvent être dupliqués, jamais modifiés.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. PLAN_MODELES (~93856), PLAN_MODELES_LISTE (~95670), planModeleSuggere (~94133) et planDepuisModele (~94175) composent un plan depuis des modèles figés. Les programmes ont déjà un mécanisme « transformer en modèle » (~38924) dont il faut copier le patron.

Objectif :

1) Champ coach plansModeles = [{id, nom (≤40), famille (seche|masse|recomp|maintenance|peak|autre), sexe ('H'|'F'|''), squelette, catalogues, avecComplements, createdAt}] (au plus 30). Le classer dans CHAMPS_NON_SANTE et l'autoriser dans database.rules.json. Ne stocker AUCUNE donnée de l'athlète source (ni poids ni nom) : seulement la structure, avec des quantités relatives au poids quand le plan l'exprime ainsi.

2) Bouton « Enregistrer comme modèle » sur l'écran de composition du plan (coach), qui demande seulement le nom.

3) Le sélecteur de modèle affiche « Mes modèles » (avec suggestion par famille et sexe) avant « Modèles RepCore ». « Dupliquer » sur un modèle RepCore crée une copie modifiable.

4) planDepuisModele accepte un modèle perso et recalcule les quantités selon le poids de l'athlète cible, comme pour les modèles RepCore.

Erreurs humaines : appliquer un modèle sur un plan existant demande une confirmation, et l'ancien plan reste récupérable (« Annuler » plus une copie dans l'historique du plan) ; un nom en double est suffixé « (2) » ; supprimer un modèle demande une confirmation et ne touche à aucun plan déjà posé.

Tests (app/tests.js) : aller-retour plan, modèle, plan sur un autre poids ; aucune donnée personnelle dans le modèle ; plafond de 30 ; un modèle RepCore n'est jamais modifié.

Critère de fin : au banc, un coach enregistre le plan de Léa comme modèle et l'applique à un second athlète en deux taps.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

8. Des profils de coaching pour démarrer en 30 secondes, et liés au code d'invitation — Priorité : Moyenne · Effort : M

Analyse et idée. Un coach qui démarre doit trouver chaque réglage seul, et le code d'invitation (studentCodes, rc-core:20795) ne transporte aucun réglage. L'élève arrive donc nu. L'idée : au premier passage dans Mes réglages, 4 profils en un tap (Esthétique et hypertrophie, Perte de poids grand public, Force et performance, Remise en forme et santé). Chacun remplit les défauts des points 2, 3 et 4 : prescription, cadence et questions de bilan, affichage, relances. Tout reste modifiable ensuite. Chaque code d'invitation peut porter un profil, appliqué une seule fois au rattachement.

Prompt de mise en place.

Contexte : app/rc-core.1792.js. Dépend des lots « défauts du coach » (defaultsCoach.bilan, defaultsCoach.prescription, defaultsCoach.affichage). Codes d'invitation : studentCodes (création ~20795 et ~124175), rattachement de l'athlète au code (chercher usedBy).

Objectif :

1) Constante PROFILS_COACHING (gelée), 4 entrées. Chacune a {cle, lib, phrase} et des valeurs pour prescription, bilan (freq, jour, 2-3 questions types), affichage et relances (signaux actifs et délais). Exemple « Perte de poids grand public » : 3 × 12-15, 1 min 30, RIR 2, bilan hebdo le lundi, affichage sansChiffres, relance inactivité à 7 j.

2) Mes réglages : tant que defaultsCoach est vide, une bande « Démarrer avec un profil » montre les 4 cartes et un lien « Je règle moi-même ». Choisir un profil affiche un aperçu en 4 lignes de ce qui sera posé, puis Valider. Valider écrit les défauts sans toucher aux athlètes existants.

defaultsCoach.profil garde la clé, pour la mention « Basé sur : ... (modifié) ».

3) Codes d'invitation : un sélecteur facultatif « Profil de suivi » (défaut = le profil du coach). Au rattachement, si l'athlète n'a pas de réglage propre, on pose sur son dossier l'affichage et, si le coach l'a choisi, un programme modèle (coachPrograms) à valider par le coach. On ne pose rien d'autre : le reste suit les défauts du coach.

Erreurs humaines : un profil choisi par erreur se défait (« Revenir à mes réglages d'avant », car l'ancien defaultsCoach est gardé dans _precedent) ; changer de profil ne réécrit jamais un athlète existant sans aperçu et confirmation ; un code réutilisé n'applique pas le profil deux fois.

Tests (app/tests.js) : chaque profil produit des défauts valides (passés aux validateurs des lots 2, 3, 4) ; l'application n'écrit pas les athlètes existants ; le rattachement par code applique le profil une seule fois ; l'annulation restaure l'état précédent.

Critère de fin : au banc, un compte coach neuf choisit un profil, crée une invitation, et le nouvel élève arrive avec la bonne cadence et le bon affichage sans autre action.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

9. Les formules de la vitrine aux prix du coach — Priorité : Basse · Effort : S

Analyse et idée. Le bloc « Mes formules sur ma page » n'offre que des cases à cocher. Le prix et la durée viennent du tableau de Kevin, OFFRES / TARIFS (rc-core:1077, 20657) : 99 €, 150 €, 350 €, 600 €. Testé au banc, un coach tiers ne peut ni changer un prix ni renommer une formule. L'idée : à côté de chaque case, un prix et une durée modifiables, avec ceux de RepCore en placeholder, et une formule libre en plus.

Prompt de mise en place.

Contexte : app/rc-core.1792.js, htmlReglagesProspects (~20654) et _lireReglagesProspects (~20675).

vitrineFormules ne stocke que des clés ; prixOffre et formuleDuree lisent OFFRES (~1102), qui est issu de TARIFS. La page publique (cloudflare, /coach/<slug>) lit les mêmes clés.

Objectif : un champ coach vitrineFormulesPerso = {cle:{prix (entier 0-5000), mois (1-24), lib (≤40)}} et au plus une formule libre {lib, prix, mois, inclus (≤140)}. Les prix et libellés RepCore servent de placeholder et de repli. La page publique et l'app affichent la version du coach si elle existe. Pour le compte créateur, rien ne change.

Erreurs humaines : un prix vide revient à celui de RepCore (et non à 0) ; un prix à 0 demande une confirmation (« Gratuit ? ») ; l'aperçu de la vitrine montre tout de suite le résultat.

Tests (app/tests.js et cloudflare/test) : repli sur OFFRES, bornes, rendu de la page publique avec un prix perso.

Critère de fin : un coach tiers affiche ses propres tarifs sur sa vitrine sans toucher au tableau des offres de RepCore.

Convention : nouveau build (window.RC_BUILD dans app/index.html, CACHE dans app/sw.js), puis python3 scripts/versionner_actifs.py. Ne pas agrandir les polices.

Expert 15 · Prêt à vendre — 5/10

Le produit est riche, les parcours tiennent hors ligne, la CI garde le site, et l'essentiel des chantiers du premier audit est écrit (sauvegarde, veille du serveur, déploiement du Worker, page coach, fiche Play Store, annuel remisé). Le problème, c'est que presque tout cela n'est pas en production : la PR #75 (55 lots) attend depuis le 30/09. En plus, chaque bouton « passer à Ultime » mène à un écran de paiement sans prix, et il n'existe aucune démo à montrer à un prospect.

1. La production a une semaine de retard : 55 lots bloqués dans la PR #75 — Priorité : Critique · Effort : M

Analyse et idée. `main` est au build 1792. La branche `claude/intelligent-shannon-5uut4o` est au build 1799, avec 328 commits d'écart (`git log main..origin/claude/intelligent-shannon-5uut4o`). La PR #75 est « mergeable clean » mais porte « NE PAS MERGER avant le Worker, la migration et l'UID ». Elle contient la sauvegarde (`sauvegarde.yml`), la veille (`veille-serveur.yml`), le déploiement du Worker, les tests bloquants, `coachs.html`, l'annuel à 95/249 € et la fermeture du paywall. Pendant ce temps, la production affiche encore « sans engagement » sur `/i` (`i/index.html:106`) et « engagement 12 mois » sur la landing. La fusion dépend d'une liste de 14 étapes manuelles que seul Kevin peut faire. L'idée : réduire cette liste à « coller l'UID, poser 3 secrets, cliquer 2 travaux, fusionner », avec un contrôle automatique qui dit à chaque instant ce qui manque.

Prompt de mise en place.

Contexte : dépôt RepCore. La PR #75 (branche `claude/intelligent-shannon-5uut4o`, build 1799) porte 55 lots : sauvegarde, veille du serveur, déploiement du Worker, tests bloquants, droits serveur, page coach, Play Store, annuel remisé. Elle est bloquée par une procédure manuelle de 14 étapes, décrite dans sa description : UID du créateur, ``wrangler deploy`` à la main, sauvegarde locale, ``remplir-droits.mjs``, ``migrer_boutique.mjs``, secrets SAUVEGARDE_CLE / CLOUDFLARE_API_TOKEN / CLOUDFLARE_ACCOUNT_ID. Pendant ce temps, `main` continue d'avancer (1786 → 1792) et la branche le refusionne à chaque fois.

But : que Kevin puisse mettre la PR en production en moins de 30 minutes, sans terminal local ni secret tapé dans une conversation.

Travail, SUR LA BRANCHE `claude/intelligent-shannon-5uut4o` :

- Écrire `scripts/mise_en_prod.mjs` (lecture seule par défaut). Il vérifie chaque prérequis et affiche une liste « FAIT / À FAIRE / comment » en français simple :
 - `CREATEUR_UID` renseigné dans `cloudflare/src/createur.js` ;
 - ``node scripts/verif/regles.mjs`` vert ;
 - version du Worker en ligne = dernier commit de `cloudflare/` (`GET /sante`, champ `version`) ;
 - présence des secrets (contrôlée par le travail ci-dessous) ;
 - sauvegarde complète datée de moins de 24 h ;
 - migrations jouées à blanc, avec 0 erreur.Il ne doit jamais rien écrire sans `--ecrire`.
- Ajouter `.github/workflows/preparer-mise-en-prod.yml` (`workflow_dispatch`, environnement « production » avec validation manuelle). Étapes, dans l'ordre :
 - lister les secrets reçus (nom seulement) ;
 - lancer `scripts/sauvegarde_base.mjs --complet`, chiffrer avec `SAUVEGARDE_CLE` et publier en artefact ;
 - jouer `remplir-droits.mjs` et `migrer_boutique.mjs` à blanc et publier leur résumé ;
 - si l'entrée « écrire » vaut oui, rejouer les deux avec `--ecrire`, APRÈS la sauvegarde, puis ``wrangler deploy`` avec `CLOUDFLARE_API_TOKEN`.Le travail échoue fort si une étape échoue, et n'écrit rien après un échec.
- Raccourcir la description de la PR en tête : « 3 gestes pour Kevin » (UID, secrets, lancer le travail), puis « Fusionner ». Garder le détail plus bas.
- Garde-fou contre la divergence : un travail `.github/workflows/divergence.yml` qui, à chaque push sur `main`, ouvre ou met à jour un ticket « main a N commits absents de la PR #75 ».

Cas limites : secret absent (le dire par son nom, ne rien lancer), sauvegarde qui échoue (ne jamais migrer), Worker déployé mais `/sante` en 503 (le dire, proposer le retour à la version précédente avec ``wrangler rollback``), relance du travail après un succès partiel (idempotent : une migration déjà faite rend « 0 différence »).

Tests : `cloudflare/test/` (sonde de version), et un test node de `mise_en_prod.mjs` avec une fausse base (`cloudflare/test/fausse-base.mjs`) : prérequis manquants → liste exacte ; tout présent → « Prêt à fusionner ».

Critère de fin : sur une copie de test, le travail à blanc passe vert, et `mise_en_prod.mjs` n'affiche plus que « Fusionner la PR #75 ».

Convention : pas de changement d'interface ici. Si `rc-core` est touché, le modifier dans `src/core/` puis lancer `node scripts/assembler_core.mjs`. Nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` de `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

2. Tous les boutons « passer à Uptime » ouvrent un paiement sans prix, qui débite l'annuel — Priorité : Critique · Effort : S

Analyse et idée. Testé au banc avec un compte à l'essai terminé. « Continuer avec Uptime » mène à un écran d'abonnement sans prix ni formule, avec une case de renonciation sans texte et un bouton PayPal. `_planIdChoisi()` y rend alors `P-16Y44630...`, l'Uptime ANNUEL à 298,80 € en une fois. La carte, elle, affichait « 24,90 € par mois ». La cause : `accueilChoisir` (`rc-core.1792.js:9049`) fait `go('s-subscribe')` puis `initPaypalSubscription()` sans `loadSubscribePage()`. Le même chemin sert à `rcVerrouUptime` (`l.1314`), à `garderLApp` (`l.80946`) et à la fin d'essai (`l.81011, 81022`). Le défaut est toujours présent sur la branche 1799 (`l.10133`). L'idée : toujours passer par `loadSubscribePage()`, afficher la formule et le prix sur le bouton, et proposer le mensuel par défaut, qui coûte le même total.

Prompt de mise en place.

```
Contexte : app/rc-core.1792.js (ou src/core/ si la PR #75 est fusionnée). accueilChoisir(cle,annuel)
(vers la ligne 9046) envoie un utilisateur connecté sur s-subscribe et appelle
initPaypalSubscription() SANS loadSubscribePage() (ligne ~128758). Conséquence, constatée au banc :
#sub-paliers, #sub-formule et #sub-renonc-txt restent vides, la case de renonciation n'a pas de texte,
et _planIdChoisi() retombe sur _paliersDispo()[0], c'est-à-dire l'annuel. Appelants touchés :
rcVerrouUptime (verrous Uptime), garderLApp (fin de pack), les deux boutons de l'écran de fin d'essai
(htmlEssaiBilan, « Continuer avec Uptime / Essentielle »), et les cartes de s-welcome quand un compte
existe.
À faire :
1. accueilChoisir : pour un utilisateur connecté, appeler go('s-subscribe') puis loadSubscribePage(),
et rien d'autre. loadSubscribePage rend les paliers, la renonciation et le bouton. Supprimer l'appel
direct à initPaypalSubscription.
2. loadSubscribePage lit rc_offre_choisie / rc_offre_annuel. La formule choisie (ultime, essentielle
ou ultime_demi) doit être celle des paliers affichés, et le palier présélectionné doit respecter «
annuel ». Les boutons de fin d'essai et de verrou passent annuel=false (MENSUEL par défaut) tant que
tarifs.json n'a pas d'annuel remisé (an === 12 × mois). L'annuel reste un choix visible.
3. Le bouton PayPal ne se rend que si #sub-paliers est rempli ET si le texte de renonciation est posé.
Sinon, afficher « Réessayer » et appeler loadSubscribePage.
4. Écrire la formule et le prix sur le bouton : « Souscrire à Uptime, 24,90 € par mois ».
Cas limites : double toucher sur « Continuer avec Uptime » (un seul rendu), retour arrière puis autre
formule (rc_offre_choisie remplacée, le palier suit), hors ligne (prix visible, bouton « Réessayer »),
plan Uptime absent (message de _subAnnoncerFormule), abonné déjà actif (écran « Changer de formule »,
comme sur la branche 1799).
Tests dans app/tests.js :
- après accueilChoisir('ultime',false) avec un compte à essai fini, #sub-paliers contient « 24,90 »,
#sub-renonc-txt n'est pas vide, et formuleDuPlan(_planIdChoisi()) === 'ultime' ;
- même chose via rcVerrouUptime() et garderLApp() ;
- accueilChoisir('essentielle',false) → formuleDuPlan === 'essentielle' ;
- annuel=true → plan annuel de la même formule.
Critère de fin : depuis les 4 chemins, l'écran montre la formule, le prix et la case avec son texte,
et le plan facturé est celui affiché.
Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3
scripts/versionner_actifs.py). Ne pas agrandir les polices.
```

3. Fit Pulse partage la base de production et réécrit ses règles toutes les 5 minutes — Priorité : Critique · Effort : M

Analyse et idée. `.github/workflows/fitpulse-mail.yml` tourne toutes les 5 minutes depuis `main`. Il exécute le code de la branche de travail `claude/eloquent-lamport-0qxwdd` avec le secret `FIREBASE_SERVICE_ACCOUNT` de RepCore. Ce code (`club/outils/fitpulse-serveur.mjs:171-175`) relit puis réécrit par PUT `.settings/rules.json` de `repcore-sync`. Résultat : un autre produit détient l'accès administrateur et peut casser toutes les règles de sécurité de RepCore. Il consomme aussi les 10 Go/mois du plan Spark, dont le dépassement « coupe l'app en silence » (`rc-core.1792.js:549`). L'idée : un projet Firebase séparé pour Fit Pulse (gratuit lui aussi), et plus aucun processus extérieur à RepCore qui écrit ses règles.

Prompt de mise en place.

Contexte : Fit Pulse (application de club, PR #79, branche claudelloquent-lamport-0qxwdd) utilise la base Firebase de RepCore, repcore-sync. `.github/workflows/fitpulse-mail.yml` (cron */5, sur main) fait un checkout de cette branche et lance `club/outils/fitpulse-serveur.mjs` avec `FIREBASE_SERVICE_ACCOUNT`. Ce script relit puis remplace `.settings/rules.json` pour y remettre un bloc Fit Pulse que chaque déploiement de RepCore retire. Deux écrivains se disputent donc les règles de production, et le quota Spark (10 Go/mois téléchargés, 1 Go stocké) est partagé.

À faire :

1. Sur la branche Fit Pulse : faire pointer `FIREBASE_DB_URL` vers un projet Firebase dédié (ex. `fitpulse-niort`, déjà hébergé sur `fitpulse-niort.web.app`), avec son propre compte de service, secret `FITPULSE_SERVICE_ACCOUNT`. Retirer toute écriture de `.settings/rules.json` : les règles Fit Pulse se déploient dans SON projet, par son propre `firebase.json`.
2. Écrire un script de migration à blanc (`--ecrire` pour écrire) qui copie les nœuds Fit Pulse de `repcore-sync` vers le nouveau projet, compare les comptes, puis, seulement avec `--purger` et après vérification, les supprime de `repcore-sync`.
3. Sur main : `fitpulse-mail.yml` n'utilise plus `FIREBASE_SERVICE_ACCOUNT`, et ne fait plus de checkout d'une branche de travail (ref: main, ou un dépôt séparé pour Fit Pulse).
4. Ajouter à `scripts/verif/regles.mjs` un contrôle qui échoue si `database.rules.json` contient un bloc balisé Fit Pulse, et à `firebase.yml` une étape qui compare les règles en ligne au fichier du dépôt, avec un avertissement si elles diffèrent.

Cas limites : migration interrompue (relançable, idempotente), utilisateurs Fit Pulse connectés pendant la bascule (prévenir par push, puis basculer à heure creuse), retour arrière (l'ancien nœud n'est purgé qu'après 7 jours sans erreur).

Tests : test node du script de migration avec `cloudfare/test/fausse-base.mjs` (copie exacte, purge refusée sans `--purger`) ; `regles.mjs` rouge si un bloc Fit Pulse réapparaît.

Critère de fin : plus aucun processus ne fait de PUT sur `.settings/rules.json` de `repcore-sync` en dehors de `firebase.yml` / `deploie.sh`, et Fit Pulse tourne sur son projet.

Convention : aucun changement d'app ici. Si l'app est touchée : nouveau build (`window.RC_BUILD` dans `app/index.html`, CACHE de `app/sw.js`, puis python3 `scripts/versionner_actifs.py`). Ne pas agrandir les polices.

4. Aucune démo : impossible de montrer l'app en 2 minutes à un athlète ou à un coach — Priorité : Haute · Effort : M

Analyse et idée. Il n'existe ni compte de démonstration, ni mode visite : aucune occurrence de « mode démo » ou « visite » dans `rc-core 1792` ni `1799`. Testé au banc : un coach qui vient de s'inscrire voit un tableau de bord vide (« Aucun athlète pour l'instant », « Charges articulaires 0/73 ») et ne peut rien évaluer avant d'avoir convaincu un vrai athlète. La landing parle d'un « profil de démonstration » que personne ne peut ouvrir. L'idée : un bouton « Voir l'app en 2 minutes », sur la landing, `coachs.html` et `s-welcome`. Il charge un dossier fictif local, sans compte et sans réseau, avec un bandeau « Visite » et un retour vers l'inscription.

Prompt de mise en place.

Contexte : `app/rc-core.1792.js` (ou `src/core/` après la PR #75), `app/index.html`, `index.html` (landing), `coachs.html` (branche 1799). Il n'existe aucun mode démo. Le banc d'audit montre qu'un dossier fictif injecté dans `localStorage` (`rc_session + rc_users`) suffit à faire tourner tout l'app hors ligne.

But : un prospect voit RepCore « plein » en 2 minutes, sans créer de compte, et rien de ce qu'il fait n'est écrit dans la base.

À faire :

1. Créer `app/data/visite.json`, généré par un script `scripts/visite.mjs` à partir des fixtures de `app/tests.js` (`_ath`) :
 - un athlète « Léa » : 60 pesées, 24 séances avec progression, 3 bilans avec photos d'illustration libres de droits déjà dans `app/img`, un plan alimentaire, sommeil et pas ;
 - un coach « Coach Démo » avec 6 athlètes aux états variés : en alerte, bilan à lire, programme qui se termine, silencieux, en progrès.

Dates RELATIVES, recalées au jour du lancement.

2. Ajouter une entrée `#visite=athlete` et `#visite=coach`. Charger le dossier dans un espace séparé (préfixe de clés `rc_visite_` ou `sessionStorage`), poser `window.RC_VISITE=true`, et couper toute écriture réseau :

- `CLOUD.doPushOne`, `_fbJson` en écriture, `rcm`, `Cloudinary` et `PayPal` rendent une erreur douce « Mode

visite » ;

- aucune notification n'est demandée.

3. Un bandeau fin en haut (même composant que les bandeaux existants, sans agrandir la police) : « Visite · données fictives · Créer mon compte ». Le bouton vide l'espace visite et ouvre s-register (athlète) ou l'espace coach.

4. Une visite guidée de 4 étapes au plus par rôle, avec bulles et « Passer » toujours visible.

- Athlète : la charge proposée en séance, le journal nutrition, la courbe de poids, le bilan.
- Coach : « À traiter », la fiche d'un athlète, la réponse au bilan, l'assignation d'un modèle.

5. Boutons « Voir l'app en 2 minutes » : sous le CTA principal de index.html, et dans coachs.html (coach) et s-welcome (lien discret sous les trois choix). Compteurs rcm 'visite_ouverte' et 'visite_vers_inscription' : à ajouter à RCM_EVENTS ET à la liste fermée de database.rules.json. Cas limites : un vrai compte déjà connecté sur l'appareil n'est JAMAIS touché (la visite ne lit ni n'écrit rc_session/rc_users), rechargement en pleine visite (on reste en visite), retour arrière du navigateur, sortie par « Créer mon compte » (aucune donnée fictive ne suit dans le vrai compte), double ouverture.

Tests dans app/tests.js : en mode visite, aucun fetch sortant (fetch espionné) ; rc_session inchangé avant et après ; le bandeau est présent ; « Créer mon compte » vide l'espace visite ; les dates fictives sont toujours dans les 60 derniers jours.

Critère de fin : depuis la landing, un visiteur ouvre la visite athlète ou coach en un toucher et voit des écrans remplis, sans aucune requête vers Firebase.

Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py ; visite.json doit être dans la liste du service worker). Ne pas agrandir les polices.

5. En production, on ne sait pas quand l'app casse : aucune remontée d'erreur, et le quota Spark coupe en silence — Priorité : Haute · Effort : S

Analyse et idée. Ni main ni la branche 1799 ne contiennent window.onerror ou d'écouteur unhandledrejection : une erreur JavaScript chez un athlète n'est connue que s'il écrit. Il y a 1 868 catch(e){} vides dans rc-core, contre 1 833 au premier audit. Le dépassement des 10 Go/mois « est silencieux : l'application cesse simplement de répondre » (rc-core.1792.js:549). La veille et la sauvegarde existent, mais seulement dans la PR #75. L'idée : un capteur d'erreurs agrégé (signature + build + écran, sans données personnelles) envoyé au Worker, et un courriel quotidien qui regroupe les erreurs nouvelles et la pente du quota.

Prompt de mise en place.

Contexte : app/rc-core.1792.js (ou src/core/ après la PR #75), cloudflare/src/index.js, workflow veille-serveur.yml (branche 1799). Aucune erreur client n'est remontée, et le quota Spark de 10 Go/mois coupe l'app sans prévenir.

À faire :

- Côté app : un écouteur window 'error' et 'unhandledrejection', posé dès le début de rc-core.
 - Il calcule une signature : message tronqué à 120 caractères, nom de fonction ou fichier:ligne, sans e-mail ni contenu de dossier.
 - Il la garde dans une file locale, 20 au plus par jour et par appareil, dédoublonnée par signature.
 - Il l'envoie au Worker par POST /erreur {sig, build, ecran, role, ua court}, en différé (requestIdleCallback ou sendBeacon), jamais pendant une séance active.
 - Hors ligne, il attend.
- Côté Worker : route POST /erreur avec limiteur (même API que LIMITE_ARRIVEES). Elle incrémente erreurs/<AAAA-MM-JJ>/<build>/<sig hash> {n, exemple, ecran}. Corps de 2 Ko au plus. Rien de personnel n'est stocké. Ajouter le nœud aux règles en lecture créateur seul.
- Un travail quotidien (planif.js, 7 h Paris) envoie un courriel au créateur si une signature est NOUVELLE ou si son nombre a doublé. Il ajoute l'estimation du téléchargement RTDB du mois, à partir des compteurs du Worker, avec une alerte à 70 % puis à 90 % des 10 Go.
- Un écran « Santé de l'app » dans l'espace créateur : le top 10 des signatures sur 7 jours, avec le build.
- Remplacer, dans les 154 catch vides qui entourent des écritures du dossier (grep « catch(e){} » près de DB.set, _doPushOne, localStorage.setItem), le silence par un appel à la même fonction de signalement.

Cas limites : boucle d'erreurs (plafond de 20 par jour), erreur dans le capteur lui-même (il ne lève jamais), mode visite ou banc local (rien n'est envoyé sur 127.0.0.1 ni en visite), message contenant une adresse e-mail (masquée avant envoi).

Tests : app/tests.js (signature stable, e-mail masqué, plafond respecté, rien envoyé en local) ;
cloudflare/test (route /erreur : 413 au-delà de 2 Ko, 429 au-delà du limiteur, incrément atomique,
courriel seulement pour une signature nouvelle).
Critère de fin : une erreur provoquée en production apparaît dans le courriel du lendemain avec son
build et son écran.
Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3
scripts/versionner_actifs.py). Ne pas agrandir les polices.

6. Présence en magasin : l'APK s'installe à la main, et Google Play refusera PayPal tel quel — Priorité : Haute · Effort : M

Analyse et idée. Aujourd'hui, l'APK s'installe depuis GitHub avec « Autoriser les sources inconnues » et « Installer quand même » ([aide-apk.html](#)). Il n'y a rien sur iOS en dehors de la PWA. La PR #75 prépare l'AAB, mais elle note elle-même qu'un abonnement vendu par PayPal dans une app Play « risque un refus ou un retrait ». De plus, un nouveau compte développeur exige un test fermé de 14 jours avec 12 testeurs. L'idée : une version Play qui ne vend rien dans l'app (connexion, coachs et abonnés existants), avec le paiement sur le web, et lancer dès maintenant le test fermé avec les athlètes de Kevin.

Prompt de mise en place.

Contexte : android/ (TWA, versionCode 4, apk.yml), well-known/assetlinks.json, app/rc-core (écrans s-subscribe, s-essai-bilan, verrous Ultime, boutique). La PR #75 (lot 43) ajoute la procédure Play dans android/LISEZMOI.md et signale le conflit avec Google Play Billing.
But : publier sur Google Play sans enfreindre la règle des paiements, et sans rendre l'app plus compliquée.
À faire :
1. Détecter le lancement depuis la version Play : launchUrl de la TWA avec ?src=play, mémorisé en sessionStorage et en localStorage rc_canal='play'. Exposer une fonction pure canalPlay().
2. En canal Play, aucun bouton ni lien d'achat dans l'app :
- s-subscribe, le bouton PayPal, la boutique, les verrous « Passer à Ultime » et la fin d'essai montrent une phrase neutre (« Ton accès se gère depuis ton compte RepCore ») sans lien de paiement ;
- le code coach reste disponible (l'athlète ne paie rien dans l'app) ;
- l'essai gratuit reste ouvert ;
- les abonnés existants gardent tout.
3. android/ : bundleRelease (AAB) dans apk.yml en plus de l'APK. Ajouter l'empreinte Play App Signing dans assetlinks.json (les DEUX empreintes : clé d'envoi et clé Play). Un contrôle CI vérifie que les deux sont présentes.
4. Préparer la fiche dans android/play/ : textes FR courts, 6 captures générées depuis le mode visite (point 4 de cet audit) au format Play, et la déclaration Health Connect (lecture seule, liste des types).
5. Dans android/LISEZMOI.md, la marche à suivre pour Kevin en 5 lignes : compte développeur, test fermé avec 12 testeurs (liste : athlètes actuels), 14 jours, puis production.
Cas limites : un utilisateur Play qui ouvre ensuite le site dans Chrome (le canal web reprend : les achats y sont permis), un abonnement qui expire en canal Play (message neutre plus courriel de rappel envoyé par le Worker, sans lien dans l'app), une mise à jour depuis l'APK hors store (même clé, sinon refus : garder la vérification d'apk.yml).
Tests : app/tests.js (canalPlay() vrai avec ?src=play, aucun élément contenant « PayPal », « Souscrire » ou un lien beacons/paypal dans s-subscribe, s-essai-bilan et la boutique en canal Play ; tout est inchangé en canal web) ; CI android (AAB produit, assetlinks avec 2 empreintes).
Critère de fin : l'AAB est produit par la CI, la version Play ne montre aucun achat, et la fiche est prête pour le test fermé.
Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py) et versionCode incrémenté. Ne pas agrandir les polices.

7. Le support repose sur la boîte Gmail de Kevin, et l'aide est cachée en bas des réglages — Priorité : Moyenne · Effort : M

Analyse et idée. L'aide se résume à 8 questions (FAQ_APP, rc-core.1792.js:2361) en bas de « Réglages », et à un [mailto: guellec.coachingpro@gmail.com](mailto:guellec.coachingpro@gmail.com) (l.2380). Les athlètes d'un coach tiers, même quand l'app porte sa marque Pro, sont envoyés « au créateur de RepCore (pas à ton coach) ». Il n'existe

pas de page d'aide publique, et les questions d'un coach (quota, paiement, invitation) n'ont pas de réponse. L'idée : une seule source pour les questions, servie dans l'app sous une entrée « Aide » et sur une page publique /aide, et un formulaire de contact qui joint automatiquement le diagnostic et arrive dans une boîte dédiée.

Prompt de mise en place.

Contexte : FAQ_APP et htmlPrefsAide (app/rc-core.1792.js, vers les lignes 2361-2420), rendrePrefsAide, CONTACT_CREATEUR. Pas de page /aide. Le contact est un mailto vers la boîte personnelle de Kevin.

À faire :

1. Déplacer les questions dans app/data/aide.json : {pour, theme, q, r, ecran?}. Thèmes : Démarrer, Séance, Nutrition, Bilans, Abonnement, Coach. Compléter avec au moins 10 questions coach issues du code : quota Libre/Coach/Pro et ce qui compte comme athlète actif (COACH_ACTIF_JOURS), inviter, assigner un modèle, importer une fiche, marque Pro, répondre à un bilan, changer de formule, résilier. Chaque réponse cite un écran qui existe. Un test le vérifie par l'id de l'écran.
 2. Dans l'app : un bouton « ? Aide » dans l'en-tête des réglages (athlète) et du profil (coach), qui ouvre une feuille avec recherche et thèmes. Une ligne « Ouvrir l'écran » mène à l'écran cité. Même composant que les feuilles existantes, sans police plus grande.
 3. Page publique /aide (scripts/fabriquer_site.mjs), générée depuis aide.json au déploiement, avec le JSON-LD FAQPage. Liens « Aide » dans le pied de index.html et de coachs.html.
 4. Contact : un formulaire court (sujet en 4 choix, texte, capture facultative) envoyé au Worker POST /support, avec limiteur. Diagnostic joint : build, rôle, appareil, état de la file de synchro, dernières signatures d'erreur (point 5). Le Worker l'écrit dans support/<id> et l'envoie par courriel à une adresse SUPPORT_EMAIL (variable du Worker, distincte de la boîte personnelle). Le mailto reste en repli hors ligne.
 5. Athlète d'un coach : le formulaire propose « Une question sur mon programme → mon coach » (messagerie interne existante) et « Un souci avec l'app → support RepCore ».
- Cas limites : hors ligne (brouillon gardé puis envoyé au retour), double envoi (identifiant client idempotent), texte vide (bouton inactif, avec un message), capture trop lourde (compressée à 300 Ko). Tests : app/tests.js (chaque réponse cite un écran existant, la recherche « résilier » trouve la bonne question, le diagnostic ne contient ni e-mail ni contenu de séance) ; cloudflare/test (route /support : limiteur, 413, idempotence).
- Critère de fin : la même question s'affiche dans l'app et sur /aide, et un message de support arrive avec son diagnostic sur l'adresse dédiée.
- Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

8. Le tunnel du visiteur fait passer quatre écrans avant le formulaire, et se contredit — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc en 390 et 1366 px : landing, puis « Essai gratuit », puis un mur « Installe RepCore » (un QR code sur ordinateur, avec « Continuer sans installer » en petit gris), puis s-welcome qui revend les tarifs, puis le formulaire. Le formulaire demande à un sportif autonome une photo « que ton coach verra » et annonce que le code d'un ami « t'offre ton premier mois », alors que ce mois est déjà gratuit. L'idée : le CTA de la landing ouvre directement l'inscription à l'essai, l'installation est proposée après la première séance, et le texte s'adapte au parcours.

Prompt de mise en place.

Contexte : index.html (CTA href="app/#install"), app/index.html (#s-install, #s-welcome, #s-register), app/rc-core.1792.js (routage du hash #install, accueilChoisir, rendu de s-register). Au banc : 4 écrans avant le formulaire, et un QR code bloquant sur ordinateur.

À faire :

1. Nouveau hash #essai. Les CTA « Essayer gratuitement 1 mois » de index.html y mènent. #essai ouvre s-register avec le rôle athlète et rc_offre_choisie='ultime' (l'essai ouvre Ultime), sans passer par s-install ni s-welcome. Garder #install pour les liens existants.
2. L'invitation à installer n'est plus un mur. Elle devient une carte sur l'accueil, après la première séance terminée ou au 2e lancement, avec « Plus tard ». Sur ordinateur : jamais de QR bloquant, seulement une ligne « Sur ton téléphone : scanne ce code », repliable.
3. s-register : pour un athlète sans code coach, remplacer la phrase de la photo par « Facultative. » et masquer le texte qui parle du coach. Le texte du champ ami devient « Le code d'un ami ajoute un mois offert à ton essai » (la durée vient de tarifs.json : essai_parrainage.moisEnPlus).

4. Retirer le repère « build 1792 · invitation ok » visible sous s-install (le garder derrière ? debug=1).

5. Compteurs : rcm('landing_cta_click') existe sur la branche 1799 ; ajouter 'essai_direct' pour mesurer le nouveau chemin (RCM_EVENTS et la liste des règles).

Cas limites : un visiteur déjà connecté qui touche le CTA (aller à l'accueil, pas au formulaire), l'intention d'abonnement mémorisée (rc_apres_inscription) qui doit rester prioritaire, un lien coach ? c= (garder le parcours code coach), le navigateur intégré d'Instagram (la sortie vers le navigateur reste proposée, sans bloquer l'inscription).

Tests dans app/tests.js : #essai → s-register actif, sans détour par s-install ni s-welcome ; aucun texte « ton coach verra » pour un athlète sans code ; texte du parrainage cohérent avec tarifs.json ; la carte d'installation n'apparaît qu'après la première séance.

Critère de fin : depuis la landing, le formulaire s'ouvre au premier toucher sur mobile comme sur ordinateur.

Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

9. Coach tiers : « Premiers pas » promet un code et ouvre un formulaire de 7 champs, sans tarif visible — Priorité : Moyenne · Effort : S

Analyse et idée. Testé au banc avec un coach neuf. « Inviter mon premier athlète » annonce « Un code à lui transmettre » (rc-core.1792.js:26960), mais openAddAthlete (l.123905) ouvre un formulaire : prénom, nom, e-mail, mot de passe provisoire inventé par le coach, WhatsApp, genre, durée. L'athlète recevra ensuite un mot de passe ET un code. La FAQ redit « tu obtiens un code ». L'espace coach dit « ta formule suit le nombre d'athlètes » sans donner les paliers (19 € jusqu'à 15 athlètes, 39 € sans limite). L'idée : l'invitation par lien et code, en un geste, par défaut, la création de compte par le coach en option repliée, et les trois paliers affichés là où le coach décide.

Prompt de mise en place.

Contexte : PREMIERS_PAS et _renderPremiersPas (app/rc-core.1792.js, vers la ligne 26958), openAddAthlete / createAthlete (vers la ligne 123905), le bouton « + AJOUTER DE NOUVEAUX ÉLÈVES » (app/index.html, vers la ligne 1725), l'onglet CODES (génération d'un code et _texteInvitationAthlete), COACH_PALIER (vers la ligne 1442), et l'écran s-coach-entry.

À faire :

- « Inviter mon premier athlète » et « + Ajouter de nouveaux élèves » ouvrent une feuille d'invitation courte : prénom (facultatif) et durée (3 mois par défaut), puis deux boutons, « Envoyer par WhatsApp » et « Copier le lien ». Le lien /i/?c=<code> porte le code. Réutiliser la génération de code de l'onglet CODES et _texteInvitationAthlete.
- L'actuel formulaire (création de compte par le coach) passe sous un lien replié « Créer le compte moi-même (l'athlète n'a pas d'e-mail ou ne sait pas s'inscrire) ».
- Le quota se vérifie AVANT de générer : un coach Libre qui a déjà 1 athlète actif voit les paliers, pas une erreur après coup.
- s-coach-entry et le bloc « Premiers pas » affichent les trois paliers en une ligne, lus dans COACH_PALIER et tarifs.json : « Libre 0 € · 1 athlète », « Coach 19 €/mois · 15 », « Pro 39 €/mois · sans limite ». Dire ce qu'est un athlète actif : au moins 1 séance en 60 jours.
- Corriger la FAQ coach pour qu'elle décrive exactement ce flux.

Cas limites : code copié mais jamais utilisé (il reste dans CODES, avec l'état « en attente » et « Renvoyer »), double toucher sur « Envoyer » (un seul code), athlète qui a déjà un compte (le lien le rattache après connexion), presse-papiers refusé (le texte affiché à copier à la main, comme _copierInvitationAthlete).

Tests dans app/tests.js : PREMIERS_PAS[0].action ouvre la feuille d'invitation et non le formulaire de compte ; le lien contient le code ; un coach Libre avec 1 actif voit les paliers ; les prix affichés égalent tarifs.json.coach.

Critère de fin : un coach invite son premier athlète en 2 touchers, et voit le prix de la suite avant d'en avoir besoin.

Convention : nouveau build (window.RC_BUILD dans app/index.html et CACHE de app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

10. Tout repose sur une seule personne : adresse, comptes, procédure et identité au nom de Kevin — Priorité : Moyenne · Effort : S

Analyse et idée. Tout est au nom de Kevin : - le produit vit sur `repcore-sync.web.app` ; - le compte officiel est `@kevin.gllc` (branche 1799, I.52706) ; - le support, les rapports et les alertes vont dans sa boîte Gmail personnelle ; - le WhatsApp de contact est son numéro (`RC_WHATSAPP`, I.48).

Il n'y a pas de README à la racine ni de procédure d'exploitation. Le dépôt compte 21 branches distantes et héberge deux autres produits (`vault/`, Fit Pulse). Personne d'autre ne pourrait relancer le service. L'idée : un domaine et une adresse de support propres au produit, et une fiche d'exploitation d'une page, générée et vérifiée par la CI.

Prompt de mise en place.

Contexte : dépôt RepCore. Aucun README racine ni procédure d'exploitation. Les comptes critiques (Firebase `repcore-sync`, Cloudflare Worker `repcore-serveur`, PayPal, Cloudinary, GitHub `Repcore131`, Gmail d'envoi) et les secrets sont décrits par morceaux dans les en-têtes des workflows et dans `cloudflare/README.md`. Adresse publique : `repcore-sync.web.app`. Contact : boîte personnelle.

À faire :

- Écrire `docs/EXPLOITATION.md`, d'une page au plus, généré en partie par `scripts/exploitation.mjs`. Le script lit les workflows (`.github/workflows/*.yaml`) et `cloudflare/wrangler.toml`, puis liste :
 - chaque secret attendu et où il se pose ;
 - chaque travail programmé et ce qu'il fait ;
 - les 5 gestes d'urgence : site en panne → `rollback Hosting` ; Worker muet → `/sante` puis `wrangler rollback` ; base abîmée → `restaurer_noeud.mjs` ; paiement non reconnu → écran « Remboursements et litiges » ; quota plein → que couper.Un contrôle CI (`scripts/verif/exploitation.mjs`) échoue si un secret ou un workflow n'y figure pas.
- Centraliser l'identité du produit dans `tarifs.json` ou un fichier `marque.json` : `RC_ADRESSE_AFFICHEE`, adresse de support, compte Instagram, numéro WhatsApp. Le script de recopie (comme `scripts/tarifs.mjs`) les pose dans `rc-core`, `index.html`, `coachs.html`, `aide-apk.html` et les CGV. Préparer le passage à un domaine propre (ex. `repcore.fr`) : une seule valeur à changer, plus la redirection 301 de `repcore-sync.web.app` dans `firebase.json`.
- Ménage du dépôt : un script qui liste les branches distantes fusionnées ou inactives depuis plus de 14 jours (lecture seule, suppression seulement avec `--supprimer` et confirmation), et une note qui sépare `vault/` et Fit Pulse dans leurs propres dépôts.

Cas limites : un secret renommé (le contrôle le voit), une adresse changée à moitié (le script refuse s'il reste l'ancienne valeur dans un fichier servi), une branche avec une PR ouverte (jamais supprimée).

Tests : `scripts/verif/exploitation.mjs` rouge si un secret d'un workflow manque dans `EXPLOITATION.md` ; recopie de `marque.json` idempotente.

Critère de fin : une personne qui n'a jamais vu le projet peut, avec `EXPLOITATION.md` et les accès, redéployer le site et le Worker et restaurer un nœud de la base.

Convention : si l'app est touchée, nouveau build (`window.RC_BUILD` dans `app/index.html` et `CACHE` de `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Bloqueurs avant lancement (dans l'ordre) : 1 (mettre la PR #75 en production : sauvegarde, veille, Worker en CI, paywall fermé, annuel remisé, page coach), 2 (paiement Ultime sans prix), 3 (Fit Pulse hors de la base de production), 5 (savoir quand ça casse).

Accélérateurs : 4 (mode visite pour vendre en 2 minutes), 6 (Google Play), 9 (invitation coach en 2 touches et tarifs visibles), 8 (formulaire au premier toucher), 7 (centre d'aide), 10 (domaine, adresse de support, fiche d'exploitation).

Expert 16 · Séance en salle — 6,5/10

La plupart des correctifs du 29/09 sont réels et bien faits : écran maintenu allumé (Wake Lock), virgule acceptée (« 22,5 » donne 22.5), contrôle de vraisemblance (« 152,5 kg ? »), confirmation de fin, feuille Historique, note par exercice, vidéo mise en file sans quitter la séance, repos après la dernière série d'un exercice. Il manque encore l'essentiel pour battre Hevy. Une séance réelle de 5 exercices × 4

séries m'a coûté **84 taps** (Hevy : environ 30), plus 8 défilements dans le superset. Et, téléphone en main sur une autre app, la fin du repos reste muette.

Parcours joué au banc (port 8960, Léa, « Push Pull A ») : développé couché avec une série ratée, tirage remplacé par « Rowing haltère », développé militaire avec une pause téléphone puis un rechargement forcé, et un superset curl + triceps, puis la fin de séance. Décompte : 3 taps pour lancer, 23 validations (dont 2 pour corriger une série ratée), 20 saisies de reps, 20 RIR, 4 taps pour le remplacement, 3 modales parasites, 1 tap de reprise après 1 560 px de défilement, 2 taps pour sortir.

1. La fin du repos reste muette dès qu'on regarde une autre app — Priorité : Critique · Effort : S

Analyse et idée. Testé au banc : pendant un repos, l'app passe en arrière-plan (Instagram, WhatsApp). Seule une notification **silencieuse** « Repos en cours » est posée (`_reposNotifier`, rc-core:42028, `silent:true`). À l'échéance, le tick (`_peindreRepos`, 41635) appelle `navigator.vibrate`, qu'Android ignore pour une page cachée. Il pose en même temps `reposVibre=true`, donc au retour `_reposRattraper` (42053) ne vibre plus : 0 signal perçu. La vibration fait 450 ms et le son est coupé par défaut. Il faut, à l'échéance en arrière-plan, remplacer la notification par « Repos terminé » sonore et vibrante, et ne consommer le drapeau que si la page est visible.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Le minuteur de repos repose sur une échéance `woState.reposFin`. En arrière-plan, `_woVisibilite` (≈42065) appelle `_reposNotifier` (≈42028), qui pose une notification SILENCIEUSE « Repos en cours » (tag `REPOS_NOTIF_TAG='rc-repos'`). À l'échéance, `_peindreRepos` (≈41635) met `woState.reposVibre=true` et appelle `arcHaptique('avertir')`, ignoré par Android quand `document.hidden`. Au retour, `_reposRattraper` (≈42053) ne fait rien puisque `reposVibre` est déjà vrai. Résultat : aucun signal perçu.

À faire :

1. Dans `_peindreRepos` : quand fini && !`reposVibre` && `document.hidden`, NE PAS poser `reposVibre`. Appeler à la place une nouvelle fonction `_reposNotifierFin()` : via `_swReg()`, `showNotification('Repos terminé', {body:'Série suivante : <exercice> · <charge> × <reps proposées>', tag:REPOS_NOTIF_TAG, renotify:true, silent:false, vibrate:[400,150,400,150,400], requireInteraction:false, data:{url:'./index.html#seance'}})`, puis poser `woState.reposNotifFin=true` pour ne la poser qu'une fois. Si la permission n'est pas accordée, ne rien faire (pas de demande ici).
 2. Visible : `reposVibre` ne passe à vrai que si la vibration est partie depuis une page visible. `_reposRattraper` vibre au retour si l'échéance est passée et que ni la vibration visible ni la notification de fin ne sont parties.
 3. Allonger `ARC_VIBRE.avertir` à [400,150,400,150,400] pour la fin de repos seulement (nouvelle clé 'finRepos', sans toucher aux autres usages).
 4. Pré-alerte facultative à 10 s de la fin : une vibration courte de 60 ms, seulement si la page est visible.
 5. Proposer l'autorisation des notifications AU PREMIER REPOS de la première séance, par une ligne discrète sous le cadran (« Être prévenu même hors de l'app · Activer »). Pas de modale. Si l'athlète refuse, ne plus jamais l'afficher.
- Cas limites : repos ajusté à -15 s jusqu'à passer dans le passé pendant que la page est cachée ; « Passer » puis retour ; superset (le repos part après le dernier exercice du groupe) ; écran verrouillé (le tick gelé reprend au déverrouillage et `_reposRattraper` doit vibrer une fois, pas deux) ; notification déjà fermée par l'athlète.

Tests app/tests.js (chargerTests) : simuler `document.hidden=true` et une échéance passée → aucune pose de `reposVibre`, `_reposNotifierFin` appelée une fois (remplacer `_swReg` par un faux) ; retour visible → une vibration ; double tick → une seule notification.

Critère de fin : sur Android, app en arrière-plan, la fin du repos fait vibrer et sonner le téléphone par notification, et le retour dans l'app ne vibre pas une seconde fois.

Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

2. Après une coupure par Android, la séance en cours se trouve à 1 560 px sous une modale — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : séance commencée, reps de la série 2 tapées, puis rechargement (Android tue souvent la PWA pendant une pause téléphone). Les données sont bien gardées, mais l'athlète arrive sur l'accueil. La modale « 4 semaines » s'ouvre par-dessus, et la carte « SÉANCE GELÉE · Reprendre » (`#clh-resume-workout`, `index.html:3626`) est à `y=1 560 px`, deux écrans plus bas. La notification de repos ouvre `./index.html#seance` (`rc-core:42037`), un ancre que rien ne gère. Au démarrage, il faut rouvrir la séance directement si un instantané de moins de 3 h existe.

Recoupement. À coordonner avec l'expert 7 (point 2, séance oubliée au-delà de 6 h) et l'expert 10 (point 1, carte du moment) : moins de 3 h, réouverture directe ; de 3 à 6 h, carte de reprise en tête ; au-delà, « L'enregistrer / Reprendre ».

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js. La séance en cours est sauvegardée par woPersist (instantané lu par _woLoadSnap) et reprise par woResumeAndGo (~47828). Après un rechargement, loadClientHome (~44560) se contente d'afficher la carte #clh-resume-workout (index.html:3626), placée bas dans l'accueil. Les modales d'accueil (rite « 4 semaines » via fermerRite, écran de badges #bdg-ecran) s'ouvrent par-dessus. La notification de repos porte data.url './index.html#seance' (~42037), mais aucun code ne lit location.hash=='#seance' (seul '#sante-envoyer' est géré, ~6629 et ~121554).  
À faire :  
1. Au démarrage de l'athlète, après le chargement du dossier : si _woLoadSnap() rend un instantané non terminé dont le dernier woPersist date de moins de 3 h (ajouter woState.majA=Date.now() dans woPersist s'il n'existe pas), appeler woResumeAndGo() directement, SANS passer par l'accueil, et bloquer pour cette session les modales d'accueil (rite, badges, invitations, bannière d'installation) tant que la séance est à l'écran. Elles réapparaîtront après la fin de séance.  
2. Au-delà de 3 h : ne rien rouvrir. Afficher la carte de reprise EN TÊTE de l'accueil (juste sous l'en-tête profil), avec « Reprendre » et « Terminer avec N séries » (finishWorkout(true) après confirmation) pour ne pas laisser une séance fantôme.  
3. Gérer '#seance' (au chargement et sur hashchange) : même effet que le point 1, puis nettoyer le hash avec history.replaceState.  
4. Toast discret à la reprise : « Séance reprise · repos : 01:12 » si un repos court encore (le temps restant se calcule sur l'échéance).  
Cas limites : instantané corrompu (pas de woState.exercices) → carte d'erreur et bouton « Effacer » ; deux onglets ouverts ; coach connecté en mode athlète ; séance terminée sur un autre appareil (instantané plus vieux qu'une séance enregistrée le même jour → proposer d'effacer).  
Tests app/tests.js : fonction pure doitRouvrirSeance(snap, maintenant) → vrai sous 3 h, faux au-delà, faux si snap.termine ; hash '#seance' → appelle woResumeAndGo (espion).  
Critère de fin : au banc, recharger pendant un repos ramène directement sur l'écran de séance, sans modale, avec les reps saisies et le minuteur juste.  
Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.
```

3. Trois taps par série au lieu d'un : les reps ne sont jamais proposées — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : 20 séries = 20 saisies de reps + 23 ✓ + 20 RIR. Valider en un tap est possible, mais `repsDone` reste vide : l'app compte alors en silence le milieu de la fourchette (`_perfReps` → `repsToNumber('8-10')` = 9), alors que la case affiche « 8-10 ». Hevy pré-remplit les valeurs de la dernière fois et un ✓ suffit. Il faut proposer les reps de la même série la dernière fois (sinon le haut de la fourchette), en gris « proposé », pour que le ✓ enregistre une valeur visible. La bande RIR ne doit disparaître qu'à la série suivante, pas après 4 s.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1792.js, écran de séance. renderSets (~52930) dessine chaque série : reps (input .wo-reps, vide, placeholder = fourchette), charge (pré-remplie et marquée « proposé » via isAuto), RIR (bouton), gêne, ✓ (toggleSet ~53517). toggleSet ne remplit les reps depuis prevSeries que
```

si la charge est vide. Sinon repsDone reste vide et _perfReps retombe sur repsToNumber(s.reps), le milieu de la fourchette, sans que l'athlète le voie. Après ✓, _rirBandeOuvrir (≈70332) affiche une bande RIR qui se ferme au bout de RIR_BANDE_MS=4000.

À faire :

1. Reps proposées : à la création des séries (_blocExo ≈51868) et à chaque rendu, pour une série non faite et non éditée, poser s.repsProp = les reps faites à la même série lors de la dernière séance du même créneau (prevSeries) ; à défaut, la borne haute de la fourchette (fourchetteReps) ; à défaut, le nombre seul. Les afficher dans l'input en valeur grisée (classe .wo-propose déjà existante, même taille de police), pas en placeholder.
2. toggleSet : si repsDone est vide et que repsProp existe, poser repsDone=repsProp au moment du ✓. La valeur devient visible et blanche dans la ligne validée.
3. Bande RIR : elle reste ouverte jusqu'au prochain geste sur une autre série ou jusqu'à la fin du repos (plus de minuterie de 4 s). Un tap ailleurs la ferme sans rien écrire. Garder le RIR facultatif.
4. Après ✓, mettre le focus visuel (contour) sur la série suivante sans ouvrir le clavier.

Cas limites : exercice au poids du corps (repsSuivantes existe déjà, s'en servir comme proposition) ; dégressive (P1/P2 : ne rien proposer) ; cardio ; reps prescrites par la programmation du coach (_cons.reps : proposer cette valeur) ; l'athlète efface la valeur proposée (champ vide = pas de reps, ne pas reproposer dans la même série).

Tests app/tests.js : la série 2 propose les reps de la série 2 de la dernière séance du même slot ; sans historique, la borne haute de « 8-10 » donne 10 ; ✓ sans saisie écrit repsDone=repsProp ; la bande RIR reste ouverte après 5 s (horloge simulée).

Critère de fin : au banc, une séance de 20 séries identiques à la précédente se saisit en 20 taps de ✓, et chaque série enregistrée porte des reps explicites.

Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices ni les cibles.

4. Série ratée ou mauvais RIR : la ligne est verrouillée, et la série ratée est fêtée comme RECORD — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc : la série 3 du développé couché est ratée (4 reps, « Échec »). Passé 4 s, le bouton RIR d'une série validée est disabled (classe fermee, rc-core:53147). Pour corriger, il faut dévalider, revalider (le repos repart de zéro à 02:30), puis choisir Échec, soit 3 taps de plus. La même série s'affiche « RECORD » avec l'animation foudre (_marquerRecordSiBesoin, 88977), puis « 52,5 → 61,25 kg » dans le résumé et « Tu es à 61,25 kg » dans l'objectif. Il faut un appui long sur une ligne validée pour la corriger sans perdre le repos, et aucun record sur une série en échec ou sous la fourchette.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Une série validée (s.done) rend ses boutons RIR et Gène inactifs (renderSets ≈53147, classe 'fermee', attribut disabled), et ses champs charge et reps sont disabled. Pour corriger, il faut repasser par toggleSet (≈53517), qui relance demarrerRepos à plein temps lors de la revalidation et recalcule reposReel. _marquerRecordSiBesoin (≈88977) marque un record dès le ✓, via estNouveauRecord, sans regarder rir==='echec' ni repsDone par rapport à la fourchette.

À faire :

1. Correction sans dévalider : sur une ligne validée, les boutons RIR et Gène restent actifs. Un appui long (500 ms) ou un double tap sur la charge ou les reps rouvre ces champs en édition pour cette ligne seulement, puis ils se referment au changement de ligne. Aucune modification ne relance le repos et tValid ne change pas.
2. Annuler le dernier ✓ : un toast « Série 3 validée · Annuler » de 5 s après chaque ✓. Annuler remet done=false, rend l'échéance de repos précédente et retire le record éventuel.
3. Records : ne pas marquer de record (ni badge, ni foudre, ni ligne « MES RECORDS », ni objectif) si s.rir==='echec' ou si repsDone est inférieur à la borne basse de la fourchette. Si le RIR « Échec » est posé APRÈS le ✓, retirer le badge déjà posé (_recordsVus.delete). Plafonner l'affichage à un badge RECORD par exercice et par séance (la meilleure série).

Cas limites : superset (index idx explicite) ; série dégressive ; exercice programmé par le coach (rpeCible : la colonne reste en lecture seule, ne pas l'ouvrir) ; correction d'une série d'un exercice précédent après « Suivant ».

Tests app/tests.js : modifier le RIR d'une série validée ne touche ni reposFin ni tValid ; Annuler rend reposFin d'avant ; estNouveauRecord n'est pas consulté pour une série 'echec' ; 3 séries en hausse ne donnent qu'un badge.

Critère de fin : au banc, noter « Échec » sur une série déjà validée prend 2 taps, le minuteur continue, et aucune série ratée n'apparaît comme record.

5. Superset : 663 px entre « 4a » et « 4b », sans défilement automatique — Priorité : Haute · Effort : S

Analyse et idée. Testé au banc (390×844) : la ligne 1 du curl est à y=575, la ligne 1 du triceps à y=1 238. Après le ✓ de 4a, la ligne de 4b est hors écran (top 1 264). Le bon principe est là (pas de repos entre A et B, repos après B, cadran compact), mais chaque tour demande deux défilements à une main, soit 8 pour 4 tours. Après un ✓ dans un superset, il faut faire défiler doucement vers la même série de l'exercice suivant du groupe, puis revenir à A après B. L'en-tête de 4b (charge proposée, échauffement) doit se replier une fois la série 1 faite.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Un superset (ex.ss=true, groupes via _groupesEx) affiche ses exercices l'un sous l'autre dans renderWoEx (≈51583), chacun avec son bloc _blocExo (≈51868) : en-tête, carte « Charge pour la première série », échauffement, tableau. Mesuré à 390 px : 663 px entre la série i de 4a et la série i de 4b. toggleSet (≈53517) ne fait défiler nulle part.

À faire :

1. Dans toggleSet, après une validation (pas une dévalidation), si l'exercice appartient à un groupe de plusieurs exercices : trouver la cible = même série i de l'exercice suivant du groupe ; si idx est le dernier du groupe, la série i+1 du premier exercice. La faire défiler au centre de la zone visible au-dessus du cadran de repos et de la barre du bas (scrollIntoView({block:'center',behavior:arcReduit()?'auto':'smooth'})), en tenant compte de leur hauteur), et lui poser un contour bref.
2. Replier, dans un superset, la carte « Charge pour la première série » et la ligne d'échauffement de chaque exercice dès que sa série 1 est faite (un ▼ permet de les rouvrir). C'est ce qui raccourcit la distance, sans réduire aucune police.
3. Réglage implicite : si l'athlète fait défiler lui-même dans les 600 ms qui suivent, annuler le défilement automatique (ne pas lutter contre son doigt).

Cas limites : série cible déjà validée (prendre la prochaine non faite du groupe) ; dernier tour (vers le bouton Terminer/Suivant) ; groupe de 3 exercices (4a→4b→4c→4a) ; clavier ouvert (ne pas défiler tant qu'un input a le focus).

Tests app/tests.js : fonction pure cibleSuivanteSuperset(exercices, sessionData, idx, i) → {idx,i} pour les cas 2 et 3 exercices, série déjà faite, dernier tour.

Critère de fin : au banc, un superset 4x(curl+triceps) se joue sans aucun défilement manuel.

Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

6. Remplacer un exercice efface les séries déjà faites, sans retour arrière ni suggestion — Priorité : Haute · Effort : M

Analyse et idée. Testé au banc : 1 série de tirage horizontal validée (47,5 kg), machine prise, « Remplacer » puis « Rowing haltère ». La série faite disparaît (_appliquerSubstitut, rc-core:77844, recrée sets en entier), sans confirmation ni « Annuler ». Sans alternatives posées par le coach, l'athlète tape encore le nom à la main (4 taps, plus le clavier), et rien ne l'empêche d'écrire « rowing halteres » la fois suivante et de perdre son historique. Il faut garder les séries faites sous l'ancien nom, proposer ses propres exercices déjà faits et les équivalents de la salle, et offrir « Annuler ».

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. Côté athlète, ouvrirRemplacement (≈77705) ne propose que ex.alternatives (posées par le coach) puis un champ libre ; candidatsRemplacement (≈77519) rend [] hors rôle coach ; substitutsSalle (≈72805) sait classer des équivalents par muscle, patron et matériel. _appliquerSubstitut (≈77844) renomme ex.name et RECRÉE woState.sessionData[idx].sets, ce qui efface les séries déjà validées. Aucun retour arrière.

À faire :

1. Séries faites conservées : si l'exercice a des séries done, les garder sous l'ancien nom. Insérer le remplaçant comme un nouvel exercice juste après, avec seulement les séries restantes, et marquer

l'ancien comme « interrompu » (prévu - faites). finishWorkout enregistre les deux noms. Avant d'agir, une ligne dans la feuille l'annonce : « Tes 1 série(s) de Tirage horizontal sont gardées. »

2. Suggestions sans ouvrir la banque du coach, dans cet ordre et 6 au plus : (a) ex.alternatives du coach ; (b) les exercices que l'athlète a DÉJÀ faits (noms de currentUser.sessions) qui partagent le muscle principal (résolution de volume en lecture seule) ; (c) substitutsSalle limité à 3 noms. Un tap remplace. Le champ libre reste en dessous, avec une saisie semi-automatique sur (b) pour éviter les doublons de nom.

3. Toast « Remplacé · Annuler » de 6 s, puis un lien « Revenir à Tirage horizontal » dans l'en-tête de l'exercice tant qu'aucune série du remplaçant n'est faite. Annuler retire la substitution de woState.substitutions et de ecartsSeance.

4. Le motif (_demanderMotifEcart) devient une rangée de pastilles dans la même feuille, préselectionnée sur « Matériel occupé » : un tap de moins.

Cas limites : remplaçant déjà présent dans la séance ; superset (garder ss sur le remplaçant) ; exercice demandé en vidéo (aFilmer) ; cardio ; drapeau levé (comportement actuel conservé).

Tests app/tests.js : remplacer après 1 série faite garde la série sous l'ancien nom ; Annuler rend l'état exact d'avant ; suggestions (b) triées par fréquence et sans l'exercice courant.

Critère de fin : au banc, une machine prise après une série se remplace en 2 taps, sans perte de données, et se défait en 1 tap.

Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

7. Fin de séance : 3 842 px, les chiffres en bas, et des ressentis à « 5 » enregistrés comme réels — Priorité : Moyenne · Effort : M

Analyse et idée. Testé au banc : « Terminer » ouvre d'abord un écran plein « NOUVEAU RANG » de partage (il faut « Plus tard »). Vient ensuite un résumé de 3 842 px : paliers, records, volts, trophées, rhinocéros, puis durée, séries et volume, objectif, 4 curseurs, congestion de 3 muscles. Aucun récapitulatif par exercice ni comparaison avec la dernière fois. Les curseurs démarrent à 5 et savePostSession (rc-core:67198) enregistre ces 5 comme des réponses : « Forme déclarée : 5/10 » sans aucun geste. Le résumé doit commencer par un bloc compact (durée, séries, volume, meilleure série par exercice avec flèche ↑ ou ↓), les curseurs non touchés ne doivent rien écrire, et l'écran de rang doit venir après.

Prompt de mise en place.

Contexte : RepCore, écran s-workout-done (index.html ≈4560-4650) rempli par finishWorkout (app/rc-core.1792.js ≈53790) : buildSessionComparison, _htmlStatsFin, records, volts, renderPartageBilan, objectif, curseurs de ressenti (ps-fatigue, ps-sensation, ps-energie, ps-motivation), congestion par muscle. savePostSession (≈67198) lit les curseurs, qui valent 5 par défaut, et les enregistre comme metrics, même non touchés. L'écran de rang (#bdg-ecran) s'ouvre par-dessus dès la fin.

À faire :

1. Ordre : en tête et dans le premier écran, un bloc « Ta séance » : durée, séries faites/prévues, volume, puis UNE ligne par exercice avec la meilleure série (charge × reps) et une flèche par rapport à la dernière séance du même crâneau (↑ vert, = gris, ↓ orange). Les substitutions apparaissent ainsi : « Rowing haltère (à la place de Tirage horizontal) ». Les records et les volts viennent ensuite, en hauteur réduite.
2. L'écran de rang et de badges ne s'ouvre qu'APRÈS « Enregistrer et terminer » (ou au retour à l'accueil), jamais avant le résumé.
3. Curseurs : un curseur non touché est enregistré comme absent (undefined), pas comme 5. Les afficher « non noté » jusqu'au premier geste. La congestion se replie derrière « Plus de détails ».
4. « Passer » enregistre sans métriques et sans rien inventer.
5. Bouton « Copier pour mon coach / WhatsApp » : un texte brut (date, séance, exercice : séries charge×reps@RIR) via navigator.clipboard, en plus de l'image existante.

Cas limites : séance incomplète (montrer « 12/20 » sans culpabiliser) ; aucune séance précédente (pas de flèche) ; exercice au poids du corps (reps) ; cardio (durée).

Tests app/tests.js : savePostSession sans geste n'écrit pas fatigue/energie/sensation/motivation ; fonction pure resumeParExercice(sess, precedente) → lignes avec meilleure série et sens de variation ; texte copié stable (snapshot).

Critère de fin : au banc, les chiffres de la séance et le détail par exercice tiennent dans le premier écran (390×844), et aucune métrique n'est enregistrée sans geste.

Convention : nouveau numéro de build (window.RC_BUILD dans app/index.html et CACHE dans app/sw.js, puis python3 scripts/versionner_actifs.py). Ne pas agrandir les polices.

8. Pas de séance libre, et impossible d'ajouter un exercice en cours de séance — Priorité : Moyenne · Effort : M

Analyse et idée. `openSessionPicker` (rc-core:47271) ne propose que les créneaux configurés. Il n'existe ni « Séance libre » ni « + Ajouter un exercice » sur l'écran de séance (aucun `woState.exercises.push` dans le fichier). Un ancien utilisateur de Hevy, qui démarre souvent une séance vide, ou un athlète qui veut ajouter des abdos doit d'abord passer par « Gérer mes séances ». Il faut ajouter une ligne « Séance libre » au sélecteur et un « + Exercice » en bas de séance, avec les mêmes suggestions que le remplacement. Le coach voit l'ajout comme un écart.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js. `openSessionPicker` (≈47271) liste les créneaux actifs de `sessions_config` et lance `startWorkoutSession(i)`. `launchWorkout` et `finishWorkout` (≈53790) enregistrent la séance par nom d'exercice (`data[nom]`) avec `slot`, `setsPlanned` et `substitutions`. Rien ne permet d'ajouter un exercice à `woState.exercises` pendant la séance.

À faire :

1. Sélecteur : une dernière ligne « Séance libre » (même composant `sp-ligne`, sans photo). Elle lance une séance avec `exercises=[]`, `progName='Séance libre'`, `slot:null`, et ouvre tout de suite la feuille d'ajout.
2. Écran de séance : sous le dernier exercice (et dans la vue d'une séance libre), un bouton secondaire « + Exercice ». La feuille propose : les exercices déjà faits par l'athlète (fréquence décroissante), puis les alternatives du coach, puis un champ libre avec saisie semi-automatique sur ses propres noms. Un tap ajoute `{name, series:3, reps:'8-12', repos:REPOS_DEFAULT}`, ou pour un exercice déjà fait, ses dernières séries, reps et repos. La charge est proposée par `getPrevPerf` comme d'habitude (créneau `null`, nom de séance).
3. Les exercices ajoutés portent `ajoute:true`. `finishWorkout` les enregistre et `journaliserEcart(..., motif:'ajout')` pour que le coach le voie. `setsPlanned` ne compte pas les ajouts (dénominateur du coach inchangé).
4. Un exercice ajouté peut être retiré tant qu'aucune de ses séries n'est faite (× dans son en-tête, avec Annuler).

Cas limites : séance libre terminée sans aucune série (ne rien enregistrer, comme aujourd'hui pour une séance vide) ; ajout dans un superset (non : toujours en exercice seul) ; athlète coaché avec drapeau levé (ajout permis, mais signalé) ; reprise après rechargement (`woPersist` inclut les ajouts).

Tests app/tests.js : séance libre → session enregistrée avec `name 'Séance libre'`, `slot null` ; ajout pendant une séance programmée → `setsPlanned` inchangé, `data` contient l'ajout, `ecartsSeance` a un 'ajout' ; retrait avant toute série.

Critère de fin : au banc, depuis l'accueil, une séance libre de 2 exercices se lance et s'enregistre en moins de 8 taps hors saisie.

Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

9. Lisibilité en salle : point décimal, carte de charge qui reste en tête, ligne en cours masquée — Priorité : Basse · Effort : S

Analyse et idée. Testé au banc : la grande carte affiche « 56.25kg », les champs « 58.75 », mais l'échauffement et le résumé écrivent « 27,5 kg » et « 61,25 kg » : deux formats sur le même écran. La carte « Charge pour la première série » garde ~110 px en haut même après la série 1. Pendant le repos, la bande RIR, le cadran (~190 px) et la barre du bas couvrent la série qu'on vient de valider et les suivantes (capture 12_bande.png). Il faut afficher toutes les charges à la française avec un espace avant « kg », replier la carte après la série 1 et faire défiler pour que la série suivante reste visible au-dessus du cadran.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1792.js, écran de séance. `_blocExo` (≈51868) affiche la carte « Charge pour la première série » via `_aff=kg=>kgVersAffiche(kg,currentUser)+_unite()` (≈51973), ce qui donne « 56.25kg ». `renderSets` (≈52930) remplit les champs avec `_poidsSaisie` (« 58.75 »). L'échauffement et le résumé écrivent déjà « 27,5 kg ». Le cadran de repos (`#wo-repos`, `_monterRepos`) et la bande RIR (`#wo-rir-bande`) sont fixés en bas, au-dessus de la barre de navigation. `reposCompact` (≈41546) choisit déjà

une forme compacte selon la hauteur.

À faire :

1. Un seul format d'affichage : fonction `fmtCharge(kg,user) → « 56,25 kg »` (virgule, espace insécable, sans zéros inutiles), utilisée par la carte, les étiquettes « proposé », le résumé et l'historique. Dans les CHAMPS de saisie, afficher la virgule aussi : `_poidsSaisie` rend « 58,75 », et `_woChargeSaisie` accepte déjà la virgule. Ne pas changer ce qui est stocké (nombre ou chaîne à point).

2. Replier la carte de charge de la série 1 en une ligne (« Série 1 : 56,25 kg, d'après lundi ») dès que la série 1 est faite.

3. Après chaque ✓, si la série suivante est cachée sous le cadran ou la bande, faire défiler pour la placer juste au-dessus d'eux. Ce n'est pas le même geste que celui du superset (point 5) : on réutilise la même fonction de défilement.

4. Ne rien agrandir : même tailles de police, mêmes cibles. On n'unifie que le format et la place.

Cas limites : unité lb (`kgVersAffiche`, multiples de 5 lb) ; charge par main (« /main ») ; contrepoids (texte d'assistance) ; dégressive P1/P2.

Tests `app/tests.js` : `fmtCharge(56.25)=== '56,25 kg'`, `fmtCharge(60)=== '60 kg'`, `fmtCharge` en lb ; `_poidsSaisie('58.75')=== '58,75'` et relecture par `_woChargeSaisie = 58.75`.

Critère de fin : au banc, aucun point décimal n'apparaît à l'écran de séance, et la série à faire est toujours visible pendant le repos.

Convention : nouveau numéro de build (`window.RC_BUILD` dans `app/index.html` et `CACHE` dans `app/sw.js`, puis `python3 scripts/versionner_actifs.py`). Ne pas agrandir les polices.

Feuille de route : vers une application prête à vendre

Cinq vagues, dans l'ordre. Chaque ligne renvoie au point d'expert qui porte le prompt complet (« 7.1 » = expert 7, point 1). Quand plusieurs experts parlent du même sujet, la ligne cite le point de référence en premier : les autres le complètent sans le refaire.

Vague 1 : mettre en production et ne plus perdre d'argent (cette semaine)

- **15.1** : passer la PR #75 (build 1799, 55 lots) en production avec un contrôle automatique des prérequis. Beaucoup de points « non faits » du premier audit y sont déjà.
- **15.2** : tous les boutons « passer à Ultime » doivent afficher la formule et le prix, et facturer le plan affiché (aujourd'hui l'annuel part par défaut).
- **15.3** : sortir Fit Pulse de la base de production (il réécrit les règles toutes les 5 minutes).
- **Contrôle du premier audit, section 4** : XSS stockée athlète → coach (6.2) et double abonnement prélevé (12.2), vérifiés au banc et toujours ouverts sur `main`.
- **Expert 12** : les bugs bloquants reproduits par le test exploratoire, dont l'erreur JS levée à chaque chargement.
- **15.5** : remontée automatique des erreurs et alerte de quota, pour savoir quand l'app casse.

Vague 2 : les erreurs humaines, avec plan B et plan C (semaine 2)

- **Bilan sans photos** : 7.1 (question à la validation, « Garder quand même ») → 9.2 (le coach voit « Bilan corrigé ») → 8.1 (le coach demande de compléter en un clic) → 2.4 et 2.3 (la morpho relit la photo ajoutée).
- **Séance** : 7.2 (séance oubliée jamais effacée, bien datée), 16.2 (réouverture après coupure), 16.4 (série ratée corrigeable, plus fêtée comme record), 16.6 (remplacer un exercice sans perdre les séries).
- **Saisies** : 7.4 (« 62,0 » lu 620 kg dans le bilan), 2.6 (longueurs en millimètres), 7.7 (journal alimentaire annulable et corrigeable).
- **Coach** : 8.2 (toast « Annuler » commun), 8.3 (historique du programme qui garde l'original), 8.5 (deux « Léa »), 8.6 (annuler un report de modèle), 8.7 (réponse au mauvais bilan), 8.8 (le coach corrige un 825 kg).
- **Plan C** : 7.5 (supprimer ou corriger une séance ou un bilan, avec annulation), 7.6 (déconnexion qui dit ce qui sera perdu), 8.9 (outil de réparation de Kevin + sauvegarde quotidienne).

Vague 3 : des calculs justes (semaines 3 et 4)

- **Entraînement** : 3.1 (périodisation et décharge dans la séance), 3.2 (charge suggérée qui suit reps et RIR), 3.3 (faux plateaux), 3.4 (douleur, blessure et grossesse freinent la progression), 16.1 (fin de repos sonore).
- **Nutrition** : 4.1 (un seul objectif chez l'autonome), 4.2 (cycle ON/OFF à 6 séances), 4.3 (obésité : poids ajusté écrit dans la case), 4.4 (plancher qui compte le sport), 4.5 (ajustement qui ne casse plus la fiche coach), 5.1 (garde-fou TCA sur les micros).
- **Morpho** : 2.1 (repère bras faux), 2.2 (repères par sexe), 2.7 (règle de variante appliquée à la mauvaise variante), 2.8 (asymétries en double).

- **Aliments** : 6.1 (synonymes : 20 aliments introuvables), 6.3 (génériques v2 et chaînes), 6.4 (portions), 6.5 (fautes de frappe).

Vague 4 : le coach gagne du temps (semaines 5 et 6)

- 1.1 (noms illisibles dans la liste), 1.2 (recherche toujours visible), 1.3 (un seul signal pour « À traiter » et le lundi), 1.6 + 9.1 (réponse au bilan en 3 taps, avec ce qui a changé), 1.4 et 1.5 (messages groupés dans l'app, boîte en un appel), 1.7 (cibles nutrition en tête), 1.8 (reconduire un bloc).
- Paramétrage : 14.1 (une page de réglages coach), 14.2 (bilans réglés pour l'équipe), 14.3 (prescription par défaut), 14.4 (ce que l'élève voit), 14.8 (profils de coaching), 1.9 (personnaliser l'accueil).
- Élève : 10.1 (action du jour dans le premier écran), 10.3 (une seule porte vers le coach), 10.7 (page Records), 16.3 (reps proposées : un ✓ suffit), 9.4 (poids du bilan pris des pesées), 9.6 (bilan plus court).

Vague 5 : waouh, cohérence et vente (semaines 7 et 8)

- Waouh : 13.1 à 13.5 (record non recouvert, phrase de fin qui raconte la séance, première séance rituelle, avant/après dès « Bilan enregistré », « Bravo » du coach), puis 13.6 à 13.9 et 9.9.
- Design, sans grossir les polices : 11.1 (4 niveaux de titre), 11.2, 11.3, 11.4 (trois tailles de bouton, zone tactile par `.hit44`), 11.5 à 11.10.
- Navigation : 10.2 (5 onglets), 10.4 (réglages élève), 10.5 (loupe), 10.6, 10.8, 10.9.
- Vente : 15.4 (mode visite en 2 minutes), 15.8 (formulaire au premier toucher), 15.9 (invitation coach en 2 touchers, tarifs visibles), 15.6 (Google Play), 15.7 (centre d'aide), 15.10 (exploitation sans dépendre d'une seule personne), 14.5 (marque du coach), 14.9.

Règle pour toutes les vagues. Un prompt par commit, la suite de tests à 0 échec avant chaque build, aucune police agrandie : on uniformise, on ne gonfle pas.